

System Programming: A Narrative Study Guide

Autogenerated Pedagogical Guide

June 21, 2026

Contents

1	System Programming Overview	11
1.1	What is System Programming?	12
1.1.1	The Role of the Runtime Library	13
1.1.2	Resource-Constrained Environments	13
1.1.3	Professional Responsibility	13
1.1.4	Building Knowledge: The Concept Dependency Chain	14
1.2	Language Selection: Why Not Java, Python, or Go?	15
1.2.1	The Garbage Collection Problem	15
1.2.2	The Remaining Candidates	16
1.2.3	Why This Course Uses Rust	16
1.3	Executable Programs and Their Execution	17
1.3.1	What Is an Executable Program?	17
1.3.2	The Loader: From Disk to Execution	17
1.3.3	The Fetch-Decode-Execute Cycle	17
1.4	The Address Space and the MMU	18
1.4.1	Memory as an Enormous Array	18
1.4.2	Virtual vs. Physical Addresses: The MMU	18
1.4.3	Practical Effects: Segfault, Page Fault, and Thrashing	19
1.5	Memory Hierarchy and Cache	20
1.5.1	Why the CPU Does Not Access RAM Directly	20
1.5.2	Impact on Data Structure Choices	20
1.6	The Execution Model: What the Language “Pretends”	21
1.6.1	The Entry Point: <code>main(argc, argv)</code> and the Hidden Startup	22
1.7	The Stack	23
1.7.1	Structure and Properties	23
1.7.2	Step-by-Step Code Walkthrough	24
1.7.3	The Two Constraints of the Stack	25
1.8	The Heap	26
1.8.1	Why the Stack Is Not Enough: Three Reasons	26
1.8.2	The Key Difference from the Stack	26
1.8.3	Step-by-Step C++ Code Walkthrough	26
1.9	Allocation and Deallocation Functions	27
1.9.1	In C	27
1.9.2	In C++	27
1.10	Address Space Layout	28
1.10.1	Inspecting with <code>/proc/<pid>/maps</code>	28
1.11	Variable Lifecycle: Global, Local, Dynamic	29
1.11.1	Java: Value vs. Reference Semantics	29
1.12	Pointers: Power and Peril	29
1.12.1	What Pointers Are	29
1.12.2	The Seven Ambiguities of Pointers in C/C++	30

1.12.3	The Bug Catalogue	30
1.13	Garbage Collection vs. Manual Management	31
1.14	Why Rust: Solutions to C/C++ Problems	31
1.15	Conceptual Mind Map	32
1.16	Domande ed Esercizi da Esami Passati	33
2	Architecture and Memory	35
2.1	The True Beginning: Before and After <code>main</code>	35
2.2	The Stack: Automatic and Deterministic	36
2.2.1	How the Stack Works	36
2.2.2	Trade-offs of the Stack	37
2.3	The Heap: Dynamic and Flexible	38
2.3.1	The Burden of Manual Management	38
2.3.2	Beyond the Stack and Heap: The Message Queue	39
2.4	The Danger of Pointers	39
2.5	Garbage Collection vs. Rust	40
2.5.1	The Rust Paradigm	40
2.6	Domande ed Esercizi da Esami Passati	41
3	Rust Operational Fundamentals	43
3.1	What Kind of Language Is Rust?	43
3.1.1	A Compiled Language with Cross-Compilation	43
3.1.2	Memory Safety by Design	43
3.1.3	Performance Comparable to C and C++	44
3.1.4	The Ownership Model	44
3.2	The Toolchain: <code>rustc</code> , <code>Cargo</code> , and <code>rustup</code>	44
3.2.1	The Compiler: <code>rustc</code>	44
3.2.2	Installing the Toolchain: <code>rustup</code>	45
3.2.3	The Build System: <code>Cargo</code>	45
3.3	IDE Setup and the <code>Cargo.toml</code> File	47
3.3.1	The <code>Cargo.toml</code> File	47
3.4	Rust Terminology: Crate, Module, Package	48
3.5	Project Physical Structure on Disk	49
3.5.1	Debug vs. Release Builds	50
3.6	Your First Rust Program: Functions, Macros, and Printing	50
3.6.1	The Entry Point: <code>fn main()</code>	50
3.6.2	Macros vs. C Preprocessor	50
3.6.3	Printing: <code>print!</code> , <code>println!</code> , <code>eprint!</code> , <code>eprintln!</code>	51
3.6.4	Format Placeholders: <code>{}</code> and <code>{:?}</code>	51
3.7	Variables, Mutability, and the Principle of Minimum Privilege	52
3.7.1	Introducing Variables with <code>let</code>	52
3.7.2	Immutability by Default	52
3.7.3	Deep Immutability: The Difference from Java's <code>final</code>	53
3.7.4	No <code>++</code> or <code>--</code> Operators	53
3.8	Blocks, Semicolons, and Expressions	54
3.8.1	Scoping with Blocks	54
3.8.2	The Semicolon Rule	54
3.8.3	If-Else as an Expression	55
3.9	Primitive Types and Numeric Representation	55
3.9.1	The Problem with C's <code>int</code>	55
3.9.2	Integer Types	56
3.9.3	Floating-Point Types	56

3.9.4	Special Types: <code>isize</code> , <code>usize</code> , <code>bool</code> , <code>char</code>	56
3.9.5	The Unit Type <code>()</code>	57
3.9.6	Underscores in Numeric Literals	57
3.9.7	Type Annotations and Type Inference	57
3.10	Expressions Produce Values	58
3.10.1	Loops: <code>loop</code> , <code>for</code> , <code>while</code>	58
3.11	Pattern Matching with <code>match</code>	59
3.11.1	Basic Value Matching	59
3.11.2	Destructuring with <code>match</code>	59
3.12	String Handling: Slices vs. Owned Strings	59
3.12.1	Java Comparison: <code>String</code> vs. <code>StringBuilder</code>	60
3.12.2	String Slices: <code>&str</code>	60
3.12.3	Owned Strings: <code>String</code>	60
3.12.4	Method Inheritance: <code>&str</code> Methods on <code>String</code>	61
3.12.5	Step-by-Step String Lifecycle	61
3.12.6	Converting Between <code>&str</code> and <code>String</code>	62
3.12.7	The Full Menagerie of String Types	63
3.13	Vectors: <code>Vec<T></code>	64
3.13.1	Creating and Using Vectors	64
3.13.2	Indexing and Safe Access	64
3.14	Shadowing	65
3.15	Reading from Console, Files, and Listing Directories	65
3.16	Domande ed Esercizi da Esami Passati	67
4	Ownership and Borrowing	69
4.1	The Core Concept of Ownership	70
4.1.1	What Does “Possessing a Value” Mean?	70
4.1.2	When Does Release Happen?	71
4.2	Ownership in Action: The <code>Vec</code> Example	71
4.2.1	<code>Vec::new()</code> vs. <code>Vec::with_capacity()</code>	72
4.3	Transferring Ownership: Moves	74
4.3.1	The Three Forms of Assignment	74
4.3.2	Detailed String Move Example	75
4.3.3	Reassignment After Move	77
4.3.4	Shadowing vs. Reassignment	77
4.3.5	Implicit Moves: The Operator Trap	78
4.4	The Trait System: <code>Drop</code> , <code>Copy</code> , and <code>Clone</code>	79
4.4.1	The <code>Drop</code> Trait	79
4.4.2	The <code>Copy</code> Trait	79
4.4.3	The Mutual Exclusivity of <code>Drop</code> and <code>Copy</code>	79
4.4.4	Who Decides the Trait Labels?	80
4.4.5	Copy Propagation in Composite Types	80
4.4.6	What “Move” and “Copy” Actually Do at the Machine Level	80
4.4.7	The <code>Clone</code> Trait: Explicit Deep Copy	80
4.4.8	The Relationship Between <code>Clone</code> and <code>Copy</code>	81
4.5	Heap Allocation with <code>Box</code>	82
4.5.1	<code>Box</code> with Tuples: Detailed Walkthrough	82
4.5.2	Returning a <code>Box</code> from a Function: Ownership Transfer	83
4.5.3	<code>Box</code> as a Specialized Pointer	84
4.6	Borrowing: Access Without Possession	84
4.6.1	Immutable References (<code>&T</code>)	85

4.6.2	Mutable References (<code>&mut T</code>)	85
4.6.3	The Cardinal Rule of Borrowing	86
4.7	The Borrow Checker and Lifetimes	87
4.7.1	The Compiler Proves Theorems	87
4.7.2	Lifetimes: The Concept	87
4.7.3	What Lifetimes Really Are Internally	87
4.7.4	The Core Lifetime Rule	87
4.7.5	Dangling Reference Example	88
4.7.6	Lifetime Propagation into Containers	88
4.7.7	Lifetime Annotations	89
4.7.8	Lifetimes in Structs	90
4.7.9	The <code>'static</code> Lifetime	90
4.7.10	All Checks Are Compile-Time	91
4.8	Slices: References to Sequences	91
4.8.1	Fat Pointers	91
4.8.2	Creating Slices	91
4.9	Summary: Box vs. Reference — A Comparison	93
4.10	Why Ownership Matters: Performance Without GC	93
4.11	Ownership Recap: The Professor's Summary	94
4.12	Comprehensive Ownership Tracing: Exam-Style Example	96
4.13	Domande ed Esercizi da Esami Passati	97
5	Compound Types: Structs, Enums, and Pattern Matching	101
5.1	Structs: Aggregating Heterogeneous Data	102
5.1.1	Instantiating Structs	103
5.1.2	Nominal vs. Structural Typing	104
5.1.3	Tuple Structs	105
5.1.4	Unit-Like Structs (Zero-Sized Types)	105
5.2	Memory Layout of Structs	106
5.3	Visibility, Encapsulation, and the <code>pub</code> Keyword	108
5.4	Methods: Adding Behavior with <code>impl</code> Blocks	109
5.4.1	The Separation of Data and Behavior	109
5.4.2	The Three Method Receivers	110
5.4.3	A Complete Example: <code>mirror</code> , <code>length</code> , and <code>scale</code>	111
5.4.4	Method Calls Are Syntactic Sugar	112
5.4.5	Constructors: The <code>new</code> Convention	112
5.4.6	Associated Functions (Static Methods)	113
5.5	Why Rust Rejects Inheritance	113
5.6	The <code>Drop</code> Trait: Destructors and RAII	114
5.6.1	How <code>Drop</code> Works	114
5.6.2	The RAII Pattern	115
5.6.3	Implementing <code>Drop</code>	115
5.7	Structs Are Product Types	116
5.8	Enums: Expressing Alternatives (Sum Types)	116
5.8.1	Simple Enums: Labels with Optional Values	116
5.8.2	Enums with Data Payloads	116
5.8.3	Memory Layout of Enums: Tagged Unions	117
5.9	Pattern Matching with <code>match</code>	119
5.9.1	The <code>if let</code> Construct	120
5.9.2	Destructuring Structs	120
5.10	Summary: Product Types vs. Sum Types	120

6	Error Handling	125
6.1	The Taxonomy of Failures: Recoverable vs. Unrecoverable	125
6.2	Unrecoverable Errors and Panics	126
6.3	Enums, Structs, and Pattern Matching	127
6.3.1	Selective Destructuring with <code>if let</code> and <code>while let</code>	127
6.3.2	Struct Destructuring	128
6.4	Recoverable Errors: <code>Result</code> and <code>Option</code>	128
6.4.1	Extracting Values and Handling Errors	129
6.5	The Happy Path: The <code>?</code> Operator	131
6.6	Handling Heterogeneous Errors	132
6.6.1	Trait Objects: <code>Box<dyn Error></code>	132
6.6.2	Custom Enums	133
6.6.3	Ecosystem Solutions: <code>thiserror</code> and <code>anyhow</code>	133
6.7	Domande ed Esercizi da Esami Passati	135
7	Modules and Visibility	137
7.1	Introduction: The Need for Organization	137
7.2	The Rust Project Structure: Packages and Crates	137
7.2.1	Building Multiple Executables	138
7.2.2	Versioning	138
7.3	Defining Modules and Visibility	139
7.3.1	Modules and Source Files	140
7.3.2	The <code>mod</code> and <code>pub</code> Keywords	140
7.3.3	Navigating Paths and Re-exporting	141
7.4	External Dependencies	142
7.5	Foreign Function Interface (FFI)	143
7.5.1	Building a Dynamic C-Library	144
7.5.2	Calling Rust from Python	145
7.6	Domande ed Esercizi da Esami Passati	145
8	Polymorphism and Traits	147
8.1	Domande ed Esercizi da Esami Passati	157
9	Testing and CI	161
9.1	The Philosophy of Testing	161
9.2	The Testing Pyramid	162
9.3	Unit Testing: The Foundation	164
9.3.1	Mechanism: The Test Module	165
9.3.2	Assertions and Usage Guidelines	165
9.3.3	Advanced Testing Mechanics	166
9.3.4	Testing Edge Cases and Failures	166
9.4	How Tests Drive API Design	166
9.5	Integration Testing	167
9.5.1	Structure of Integration Tests	167
9.6	Parametric Testing with <code>rstest</code>	168
9.7	The Limits of Testing: Concurrency and Proofs	169
9.8	Domande ed Esercizi da Esami Passati	170

10 Collections and Persistence	171
10.1 Introduction to Dynamic Containers	171
10.1.1 Shared Collection API and Complexity	171
10.2 Sequences: Vectors (<code>Vec<T></code>)	172
10.2.1 The Problem of Contiguous Dynamic Data	172
10.2.2 Mechanism and Memory Layout	172
10.2.3 Concrete Example: Building a Vector	173
10.2.4 Trade-offs	174
10.2.5 Heterogeneous Sequences via Enums	174
10.3 Sequences: Double-Ended Queues (<code>VecDeque<T></code>)	175
10.3.1 Mechanism and Memory Layout	175
10.4 Sequences: Linked Lists (<code>LinkedList<T></code>)	176
10.4.1 Mechanism and Memory Layout	176
10.4.2 Trade-offs	176
10.5 Text Collections: <code>String</code> vs <code>&str</code>	177
10.5.1 Building Intuition	177
10.5.2 Concrete Example: Conversions	177
10.6 Maps, Sets, and Priority Queues	178
10.6.1 The Problem of Fast Lookup	178
10.6.2 Mechanism: Hashing and Buckets	178
10.6.3 Map Initialization and Filtering	179
10.6.4 Concrete Example: The Entry API	179
10.6.5 Sets: <code>HashSet</code> and <code>BTreeSet</code>	180
10.6.6 Priority Queues: <code>BinaryHeap</code>	180
10.7 Persistence: <code>Path</code> , <code>PathBuf</code> , and <code>File</code>	180
10.7.1 Paths: <code>PathBuf</code> vs <code>Path</code>	180
10.7.2 File System Utility Functions	181
10.7.3 File System Operations and I/O Traits	181
10.7.4 Error Handling in I/O	183
10.7.5 Structured Data Serialization with <code>Serde</code>	183
10.8 Closures and Functional Traits	183
10.8.1 Function Pointers (<code>fn</code>)	183
10.8.2 Closures and State Capture	184
10.9 Domande ed Esercizi da Esami Passati	184
11 Smart Pointers	185
11.1 Exclusive Ownership on the Heap: <code>Box<T></code>	185
11.2 Shared Ownership: <code>Rc<T></code> and <code>Arc<T></code>	187
11.3 Safely Mutating Shared Data	190
11.4 Breaking Cycles with <code>Weak<T></code>	191
11.5 Domande ed Esercizi da Esami Passati	192
12 Closures and Iterators	195
12.1 Introduction to Closures	195
12.1.1 Building Intuition: Capturing the Environment	195
12.1.2 The Closure Traits: <code>FnOnce</code> , <code>FnMut</code> , and <code>Fn</code>	196
12.1.3 Passing and Returning Closures	197
12.2 Iterators: The Core Concept	199
12.2.1 The <code>Iterator</code> Trait	200
12.2.2 Iteration Modes	200
12.3 The Iterator Pipeline: Adapters and Consumers	201
12.3.1 Lazy Evaluation	201

12.3.2	Adapters	203
12.3.3	Consumers	204
12.3.4	Understanding <code>fold</code>	205
12.4	Implementing Custom Iterators	205
12.5	Domande ed Esercizi da Esami Passati	207
13	Asynchronous Programming	209
13.1	Introduction: The Waiting Problem	209
13.2	Strategies for Handling Concurrency	210
13.3	From Callbacks to State Machines	211
13.3.1	The <code>async/await</code> Transformation	211
13.4	Formalizing the Mechanism: The <code>Future</code> Trait	212
13.4.1	Understanding <code>Pin</code> and <code>Context</code>	212
13.4.2	Ownership Across Time	213
13.5	Executors and the Runtime	213
13.6	Working with Tokio	214
13.6.1	Concurrent Task Spawning	215
13.6.2	Coordinating Tasks: <code>join!</code> and <code>select!</code>	216
13.7	Sharing State and Communicating	217
13.7.1	Shared State: <code>Arc<tokio::sync::Mutex<T>></code>	217
13.7.2	Message Passing: Channels	217
13.8	Domande ed Esercizi da Esami Passati	218
14	Asynchronous Concurrency	219
14.1	The Limits of Standard Threads	219
14.2	The Rust Async Model: Cooperative Scheduling	220
14.2.1	The <code>Future</code> Trait and State Machines	220
14.2.2	The Executor and the Waker	221
14.3	Building Async Programs with Tokio	221
14.3.1	Spawning Independent Tasks: <code>tokio::spawn</code>	221
14.3.2	Combining Futures: <code>join!</code> and <code>select!</code>	222
14.3.3	Avoiding Executor Starvation	223
14.4	Shared State and Synchronization	223
14.4.1	Synchronous Coordination: <code>Mutex</code> and <code>Condvar</code> (Bounded Buffer)	223
14.5	Message Passing: Channels in Tokio	226
14.5.1	Backpressure and the MPSC Channel	227
14.5.2	Common Channel Patterns	228
14.6	Choosing Between Shared State and Channels	229
14.7	Domande ed Esercizi da Esami Passati	230
15	Unsafe Rust and FFI	235
15.1	Breaking the Rules (Carefully): The Need for Unsafe Rust	235
15.2	Raw Pointers: Bypassing the Borrow Checker	236
15.2.1	Creating and Dereferencing Raw Pointers	236
15.3	Unsafe Functions and Safe Abstractions	238
15.3.1	The Philosophy of Safe Abstractions	238
15.3.2	Unsafe Traits: <code>Send</code> and <code>Sync</code>	239
15.3.3	Demonstrating Undefined Behavior: Forcing a Data Race	239
15.4	Foreign Function Interface (FFI)	241
15.4.1	Calling C from Rust	241
15.4.2	Calling Rust from C	242
15.5	Trade-offs: Performance vs. Correctness	243

15.6 Domande ed Esercizi da Esami Passati	243
16 Exam Strategies and Past Exercises	245
16.1 Mastering the Final Exam	245
16.2 Type 1: Memory Layout & Sizing Analysis	245
16.3 Type 2: Theoretical Concurrency Concepts	246
16.4 Type 3: Low-Level Concurrency Implementation	247

Chapter 1

System Programming Overview

The world of software development can be broadly divided into two major domains: *application programming* and *system programming*. When writing an application—be it in Python, Java, or C#—you implicitly rely on a massive underlying infrastructure. You assume the presence of an interpreter, a virtual machine, and an operating system that seamlessly handles the hardware on your behalf.

Consider what happens when you write a simple script in Python that prints “Hello, world!”. You execute it, and it works. But *why* does it work? Who provides the Python interpreter? Who handles the fact that the CPU cannot natively understand Python? Who allocates the memory for your variables, interacts with the filesystem, and connects to the network? The answer is an entire hidden software ecosystem: the operating system kernel, device drivers, compilers, linkers, loaders, and runtime libraries.

System programming is the discipline of building that exact infrastructure. It is the software that acts as the intermediary between raw hardware (CPU, memory, peripherals) and high-level applications. As the professor put it: “*The amount of system code involved in allowing even a simple application to run is astonishing.*”

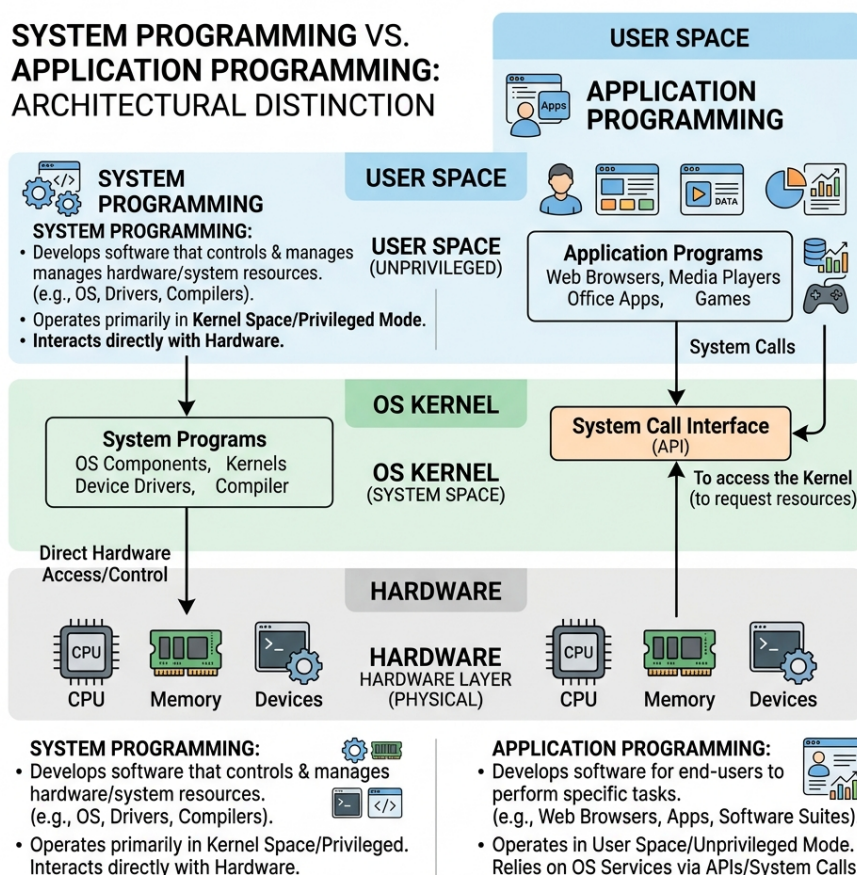


Figure 1.1: The typical software stack, illustrating how system programming bridges the gap between application software and raw hardware.

1.1 What is System Programming?

The scope of system programming is far wider than most students initially imagine. It encompasses every piece of software that must be *particularly efficient*, interact directly with hardware, or provide infrastructure upon which other software depends. Specifically, it includes:

- The **operating system kernel** and all of its internal components;
- **Peripheral drivers**—the code that translates abstract instructions into physical signals for network cards, disks, GPUs, and other hardware;
- **Compilers and interpreters**—the programs that transform source code into something the processor can actually execute;
- The **linker and loader**—tools that “stitch together” your code with libraries written by others and load it into RAM when you execute it, without you even noticing;
- **Runtime libraries**—invisible foundations that support every construct of the language you use daily;
- **File systems, the network stack, DBMS engines, virtual machines, cryptographic libraries, and embedded systems firmware.**

In short: if a software component must be efficient, interact with hardware, or serve as a foundation for others, it belongs to system programming.

1.1.1 The Role of the Runtime Library

A concept that deserves special attention is the **runtime library**. High-level programs are not transformed by the compiler into pure machine instructions alone. The compiler also generates calls to a set of support functions—the runtime library—that coordinate the execution environment. Within the runtime library, you find:

- **Completely hidden functions** that are essential for correct operation but invisible to the programmer—for example, functions that check for stack overflow when too many nested calls accumulate;
- **Wrapper functions** that make operating system services accessible—for example, `fopen()` does nothing more than translate your request into a *system call* that asks the kernel to open a file. The file is managed by the operating system in its kernel; you do not have direct visibility, and the runtime library provides the bridge;
- **Initialization and cleanup code** (the `_start/CRT0` routines discussed later) that prepares the execution environment before `main()` and tears it down afterward.

In essence, the runtime library is the invisible substrate that makes the abstractions of a high-level language actually *work* on real hardware, and it is itself a piece of system software.

1.1.2 Resource-Constrained Environments

A defining characteristic of system programming is operating under **strict resource constraints**. The professor illustrated this with the example of a home Wi-Fi router: a small device containing a low-cost, low-power CPU, limited RAM, and at least two network interfaces (the ISP-facing one and the local one for your apartment—possibly doubled with Wi-Fi and Ethernet). The software inside that router must guarantee continuous, reliable packet routing, operating within its meager resources.

Insight

The Router Example

“If you’re building a domestic router, there is a processor inside, there is RAM, there are network interfaces, and there is software. This software must guarantee that the router does its job—routing packets according to the logic you configured—in the context of relatively little RAM. The processor must be cheap so the router can be mass-produced. So we need to avoid waste of memory, CPU cycles, and memory locations. Knowing exactly how many bytes we use and how many CPU cycles we consume *makes sense* in this context.”

In such environments, every byte of memory and every processor cycle counts. You cannot afford expensive abstractions that “work” but waste resources. You must know exactly what happens under the hood.

1.1.3 Professional Responsibility

The professor dedicated significant attention to a concept that transcends pure technique: professional responsibility. An engineer is someone who *designs, builds, and deploys systems while*

assuming responsibility for their correctness and safety. This is not an abstract concept—it is a concrete, personal commitment.

When asked to define what an engineer is, the core answer is: someone who **designs, builds, and puts systems into the field**—and **assumes personal responsibility** for their correct and safe operation. This responsibility rests on two pillars: *theory* (the accumulated knowledge that prevents us from reinventing the wheel or attempting perpetual motion) and *craft* (the practical skill of knowing *how* to use tools effectively and safely, just as soldiers learn to keep the safety on their rifle).

Warning

Real-World Responsibility

When you enter an elevator, it says “maximum capacity: 400 kg.” Someone signed their name to certify that up to that limit, the elevator is safe. If it breaks and someone is hurt, that engineer is personally accountable. The same principle applies to software: if you write the ABS braking controller for a car, and someone dies because of a bug rooted in your superficiality, you carry that weight. If you write ATM software and a retiree loses their entire pension because of a security flaw, that responsibility is yours.

Every time you sit down, you trust that whoever built the chair designed it to hold your weight. Every time you walk under a video projector mounted on the ceiling, you trust that the person who installed it did so correctly. We all live embedded in a web of trust—and as software engineers, others will trust our code with their safety, their data, and their money.

The professor also referenced the Morandi bridge collapse in Genoa as a stark example: *“The people who left the bridge without performing proper maintenance were murderers. Over 40 people died. Because someone said ‘Yes, but what do you want? It will hold up.’ The same happens to those who write software with superficiality instead of precision.”*

Insight

The Web of Trust

“All of us live by trusting that others have built things that are suitable for their context. And others will trust *us* in the same way. What you are doing here—your studies—is very important to focus on well. We are here to become adults in a society that, with all its contradictions and limits, seeks to create a better world. And this goes through assuming responsibilities.”

1.1.4 Building Knowledge: The Concept Dependency Chain

The professor stressed that the topics in this course build *vertically*—each concept depends on the previous ones being solid, unlike courses where concepts are explored horizontally and can be studied in any order.

Insight**The Concrete Floors Analogy**

“You can’t build the roof of a house and then work downward to the foundations. And when you pour a new concrete floor, you must wait 28 days for it to solidify before you can build on top. The same applies to these concepts: if you don’t understand how memory is managed, it is useless to talk about ownership. If you don’t understand ownership, it is useless to talk about lifetimes. If you don’t understand lifetimes, it is useless to talk about concurrency. They build one on top of the other. And they assume that the bottom floor is solid.”

This is why the course is structured to alternate lectures week by week, giving students time to *assimilate*—the professor compared it to eating regularly rather than fasting for a month and then gorging: “*Study must be dosed so you can actually digest it. Cramming the night before an exam is like stuffing yourself after a month of fasting—you’ll just vomit it all back up.*”

1.2 Language Selection: Why Not Java, Python, or Go?

To write high-performance system software, you need a language that gives you total control over how memory, CPU, and I/O are used. This criterion immediately disqualifies languages with automatic **Garbage Collection (GC)**: Java, JavaScript, Python, C#, Go, and similar languages.

1.2.1 The Garbage Collection Problem

Languages with garbage collection work on a simple principle: you request memory when you need it, but when you no longer need it, you simply *abandon it*. At some point, a garbage collection cycle will reclaim the abandoned memory and make it available again. This convenience comes at a steep price.

Insight**The Washing Dishes Analogy**

“Languages with a garbage collector are like someone who never washes dishes after a meal. You eat lunch, leave the dish in the sink. You do it for one day, two, three, four, five. When the pile is high, you decide to wash them all at once. It works—somehow—besides the fact that bacteria proliferate. But the problem is that you need an enormous number of dishes! If you work in a resource-constrained context, you must wash the dishes *every time*—which means releasing memory explicitly.”

The global memory footprint of a garbage-collected application is always significantly larger than the strictly necessary minimum, because memory is released in “waves” rather than immediately when no longer needed.

Stop-the-World, Mark & Sweep, Reference Counting

To determine which blocks of memory can be freed, the Garbage Collector must “stop the world”—**suspending the entire program’s execution** (freezing all threads) to examine the heap in a static snapshot. Starting from all variables on the stack and in global variables, it recursively traverses all pointers. Everything not reachable from this root set can be freed. This is the essence of the **Mark & Sweep** family of algorithms, the most famous GC approach. An alternative approach is **Reference Counting**, which tracks how many references point to each object.

The “stop-the-world” pause is **non-deterministic**—it can last from milliseconds to entire seconds, depending on how much heap must be analyzed. The professor made this vivid: *“In one second, your CPU performs about a billion operations. If the CPU’s clock cycles were heartbeats, a billion heartbeats would be centuries. That’s how expensive a GC pause can be.”*

In a real-time system—a laser cutter, a car’s ABS brake controller, or an industrial press—such pauses are completely unacceptable. You cannot stop controlling the laser for a millisecond to do garbage collection.

1.2.2 The Remaining Candidates

With GC-based languages excluded, few options remain for system programming:

- **C**—the historic “battle horse” on which operating systems and most legacy system software were built;
- **C++**—offering a higher level of abstraction than C, with more structured programming capabilities;
- **Rust**—the modern contender, designed to solve C/C++’s safety problems;
- **ZIG** and **Carbon**—niche languages without significant adoption yet;
- **Swift**—relevant primarily in the Apple ecosystem, with an uncertain broader future.

1.2.3 Why This Course Uses Rust

Rust is not simply “another system language.” It was designed to solve a specific class of problems that have plagued C and C++ for decades, by encoding memory management responsibilities directly into the type system.

Insight

The Mental Model Quote

“Rust forces us to mature a coherent mental model of how things actually work. Once we understand it, whether we then write in Rust, Python, or Java, it doesn’t matter anymore—because the underlying principles are clear to us. Rust is like Italian in this lecture: it shapes how we *think* about the problems, not just how we express solutions.”

The critical point is that **Rust’s compiler refuses to compile code that violates memory safety rules**. In C, the compiler says “ok” and your program—with its bug—goes into execution, maybe apparently works, and then crashes in production at 3 AM. In Rust, the error surfaces at compile time, when it is still cheap to fix.

Insight

The Drunk Driving Analogy

“The difference between catching a bug at compile time versus at runtime is like being caught drunk *before* causing an accident versus being discovered the day after you’ve killed someone because you couldn’t brake in time. A compile-time error is infinitely more valuable than a runtime crash.”

Rust is not an absolute guarantee—it cannot detect infinite loops or deadlocks in general—but it eliminates the vast majority of memory-related bugs that cause catastrophic damage in C programs. The professor emphasized: *“In C, the program compiles—but that doesn’t mean it works. If you’ve tried building a linked list in C, you know the enormous gap between ‘it compiles’ and ‘it actually works.’ Segmentation faults tell you that story.”*

1.3 Executable Programs and Their Execution

To take responsibility for how software behaves, we must deeply understand how hardware executes it.

1.3.1 What Is an Executable Program?

An executable program is, at its essence, a **sequence of binary numbers**. It contains machine instructions, data values, configuration information, and control structures. Understanding this representation is crucial:

- The **meaning of machine instructions** is hardwired into the specific processor's hardware. An opcode like 7B5F on an x86 processor means one precise thing; on an ARM processor, the same bit pattern might mean something completely different or be entirely invalid. Programs are therefore *architecture-specific*—compiled for a particular processor family.
- The **meaning of other data** (constants, configuration structures, OS-specific sections) depends partly on the instruction semantics (e.g., the byte following a “load immediate” instruction is the value to load) and partly on the operating system that will load and execute the program.

1.3.2 The Loader: From Disk to Execution

How a program gets into memory depends on the type of system.

Embedded systems (e.g., Arduino): The compiled program is transferred directly into the microcontroller's flash memory via a hardware programmer (USB-serial connection). When the device resets, the processor begins executing from a predefined address. There is no file system, no operating system—the program runs on bare metal. Importantly, an Arduino **has no MMU**—it is too simple internally—so the addresses your program uses are the physical addresses directly.

General-purpose systems (PC, server, smartphone): Programs reside as files on disk. To execute them, the operating system uses a component called the **loader**:

1. It reads the executable file (**.exe** on Windows, an **ELF** file on Linux);
2. It identifies which **dynamic libraries** the executable depends on (**DLL** on Windows, **.so** on Linux) and loads them as well;
3. It transfers all necessary code and data from disk to RAM;
4. It configures internal structures and **creates a process with a thread** dedicated to executing instructions sequentially.

The operating system then schedules this thread, letting it execute for a time slice (a few milliseconds), then freezing it and letting other programs run. This is how your PC can have 27 open programs, each seemingly running simultaneously.

1.3.3 The Fetch-Decode-Execute Cycle

Once the program is in RAM, the CPU operates on a continuous, elementary cycle:

1. **Fetch:** The CPU reads the next instruction from RAM at the address stored in a dedicated register called the **Program Counter** (or **Instruction Pointer** on x86). This register always points to the next instruction to execute.

2. **Decode:** The fetched instruction is analyzed internally. The CPU determines what kind of operation it is: an addition, a multiplication, a data transfer, a jump, etc.
3. **Execute:** The hardware activates the necessary logical units to perform the operation. Effects may include updating internal registers, setting flags, writing to memory, or activating I/O lines that command external electronics.

Modern processors operate in **pipeline**: while executing one instruction, they simultaneously decode the next and fetch the one after that. This allows CPUs to be particularly efficient. But conceptually, the model remains this simple cycle: *Fetch-Decode-Execute*, *Fetch-Decode-Execute*, endlessly.

1.4 The Address Space and the MMU

1.4.1 Memory as an Enormous Array

From a program's perspective, memory can be imagined as a **huge array of bytes**, each identified by a numerical index called an *address*. The CPU can read or write individual bytes or groups of 2, 4, or 8 bytes at a time, depending on the data width.

The maximum number of theoretically addressable cells depends on the processor architecture. A 64-bit CPU like Intel x64 would nominally have 2^{64} addresses—an astronomical number. However, for practical implementation reasons, the **effective address space is limited to 2^{48} cells** (approximately 281 terabytes). Obviously, your PC does not have that much physical RAM.

Memory Model Analysis

Address Space Sizes

- A 32-bit processor has indices from 0 to $2^{32} - 1$, giving approximately **4 GB** of addressable space.
- A 64-bit processor nominally has 2^{64} addresses, but current implementations use only **48 address lines**, yielding $2^{48} \approx$ **281 TB** of virtual address space.
- Your actual physical RAM (e.g., 16 GB) is a tiny fraction of this virtual space.

This leads to a critical distinction:

- The number of **potentially addressable cells** depends on the processor architecture;
- The number of **actually usable cells** is limited by the installed physical RAM and controlled by the operating system.

Consequently, a program's address space is not a uniform, contiguous block—it is a **sparse space**, where only certain zones are mapped and accessible.

1.4.2 Virtual vs. Physical Addresses: The MMU

The addresses your program uses are **not** the actual physical hardware addresses of the RAM chips. They are **virtual addresses**. The CPU's hardware **Memory Management Unit (MMU)** translates these virtual addresses into physical addresses on the fly, using a **per-process translation table** maintained by the operating system.

This is not a universal feature: if you have an Arduino, there is no MMU—it is too simple internally—and the addresses you use are the physical addresses directly. But on any modern PC, server, or smartphone, the MMU is always present.

Insight

The Parking Zones Analogy

“The operating system manages memory zones just like parking zones in a city. White stripes mean you can park freely. Blue stripes mean you must pay. Yellow stripes are reserved. The operating system marks each memory zone with permissions: this zone can be read but not written; this zone can be read and executed; this zone is completely off-limits. These ‘stripes’ tell the hardware what operations are allowed.”

The MMU provides three crucial benefits:

1. Isolation between processes: If two programs—say, Notepad and your application—both use virtual address 0x7F799000, the MMU maps each to a *different* physical location. They cannot interfere with each other’s memory. Without this mechanism, two programs would constantly corrupt each other.

2. Permission control: The MMU, directed by the OS, can mark each memory page as readable, writable, and/or executable, independently. The executable code segment is typically marked read+execute but not write—any attempt to modify running code triggers an immediate error.

3. Virtual memory exceeding physical: The OS can make a program *believe* it has access to more memory than physically exists. If you have 16 GB of RAM and request 20 GB, the OS grants it. It maps the 16 GB to physical RAM and marks the rest as “not currently backed.” When you access an unbacked page, the OS intervenes, finds physical RAM (possibly by saving another program’s unused page to disk—the *swap* or *page file*), updates the translation table, and lets you proceed. This is entirely transparent to your program.

1.4.3 Practical Effects: Segfault, Page Fault, and Thrashing

Normally, all this machinery is invisible. But three situations make it painfully visible:

Segmentation Fault: When a program accesses a virtual address that is *not mapped* in its address space, or that is mapped with insufficient permissions, the MMU generates a hardware interrupt. The OS intercepts it and brutally terminates the process. The “Segmentation fault” message you have surely seen in C programs is precisely this.

Page Fault: When a program accesses a virtual address that is mapped but whose physical backing has been temporarily saved to disk. The MMU generates an interrupt, the OS loads the data from disk back into RAM, updates the translation table, and resumes execution. It is transparent to the program—but enormously expensive:

Memory Model Analysis

Page Fault Cost Analysis

A normal RAM access costs approximately **28 nanoseconds**. A page fault can elevate this to **3 milliseconds**—about **100,000 times slower** (the professor says “50,000 times,” but the order of magnitude is the key point). Once the 4K page is loaded, subsequent accesses within that page are fast again until the next page fault occurs.

Thrashing: The extreme case where the system has so many simultaneous memory demands that it spends more time swapping pages between RAM and disk than actually executing programs. The machine practically freezes—you can hear the disk growling, but nothing

progresses. The professor’s advice: *“In that situation, the only cure is usually to kill everything and start over.”*

1.5 Memory Hierarchy and Cache

1.5.1 Why the CPU Does Not Access RAM Directly

Between the CPU and RAM sits the **cache**—a small but incredibly fast memory hierarchy (L1, L2, L3). This exists because there is an enormous speed discrepancy: the CPU is thousands of times faster than RAM. If the CPU had to wait for RAM on every access, it would waste almost all its time waiting.

The cache exploits the **principle of locality**: if a program has just accessed address I , it is very likely to access $I + 1$, $I + 2$, $I + 3$ soon (spatial locality), or address I again (temporal locality). When the cache reads address I , it pre-fetches an entire block of adjacent addresses, keeping them ready for subsequent accesses.

Insight

The Supermarket Analogy

“When you go to the supermarket, you don’t buy one bottle of water at a time. You buy a whole case, bring it home, and take bottles one by one from the pantry. When the case is empty, you go back to the supermarket. The CPU cache works exactly the same way: it fetches a whole block of data and keeps it close, so the next accesses are almost free.”

1.5.2 Impact on Data Structure Choices

This principle has concrete consequences for data structure selection. A `Vec<T>` (Rust’s dynamic array, equivalent to Java’s `ArrayList`) stores elements in **contiguous memory positions**. When you access element 0, the cache automatically loads elements 1, 2, 3, ..., N nearby. Subsequent iterations find data already in cache: extremely fast.

A `LinkedList<T>`, by contrast, stores each node at an **arbitrary location in the heap**. The next node can be hundreds or thousands of addresses away. Each traversal step potentially causes a **cache miss**—the cache must fetch new data from RAM, costing vastly more time.

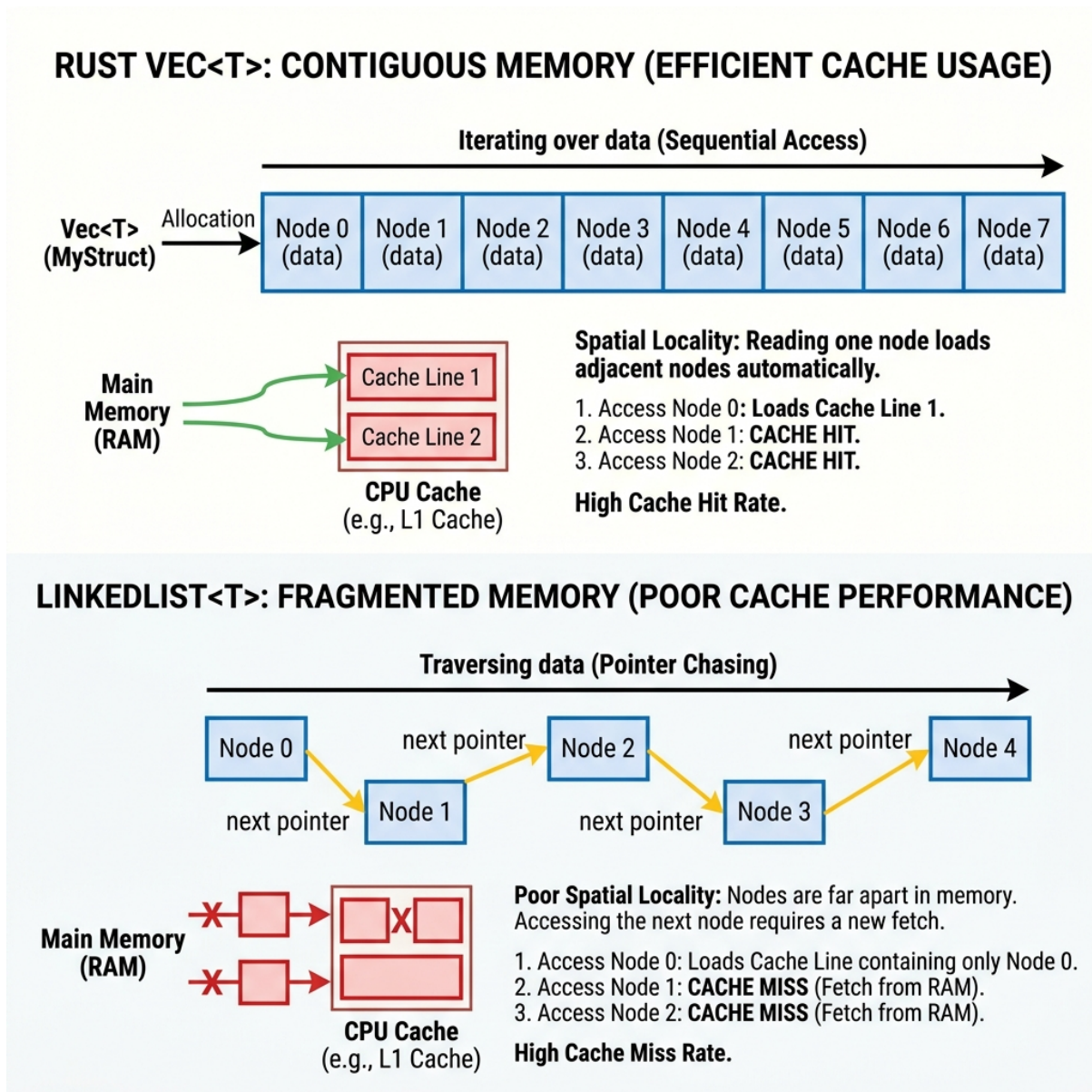


Figure 1.2: Visualizing how Contiguous Memory (Vec) maximizes CPU Cache efficiency compared to Fragmented Memory (LinkedList).

The professor used another vivid image: “With a *LinkedList*, it’s like having a glass of cold water, then wanting a natural one—back to the supermarket! Then you want a Coke—back again! You accomplish very little. The *LinkedList* benefits very little from the cache.”

The linked list does have algorithmic advantages ($O(1)$ insertion/removal with node access), but awareness of the cache cost is essential: in practice, `Vec<T>` is often faster even for operations where the linked list has better theoretical complexity.

1.6 The Execution Model: What the Language “Pretends”

The C/C++ (and Rust) execution model makes several simplifying assumptions that do not correspond to physical reality but allow the programmer to reason more easily:

1. The program is the **sole user** of the computer (the only evidence of other programs comes

from shared resources like the file system);

2. It can access **any cell** in the address space (a pointer is a number valid from zero to the maximum integer);
3. Execution is **sequential**—one instruction after another, with no inherent parallelism;
4. There are **no limits** to the number of instructions, time, or memory available.

Obviously these assumptions are all false—limits exist and condition how programs must be written—but the compiler and OS work together to make them true *to the extent possible*. When reality exceeds the illusion (memory exhaustion, stack overflow, illegal access), the illusion shatters and errors appear.

The professor elaborated on each failure mode: memory is not infinite—as much as the operating system can make the virtual space appear larger than physical RAM, even the disk eventually fills up. The stack has a finite depth, so purely recursive programs may be infeasible (the stack overflow). And even though modern processors have 4 or 8 cores capable of true parallelism, the C/C++ execution model does not inherently expose this—concurrency must be explicitly programmed.

1.6.1 The Entry Point: `main(argc, argv)` and the Hidden Startup

Every C/C++ program has a well-defined entry point: the function `main()`, which accepts two parameters:

```
int main(int argc, char* argv[])
```

`argc` is an integer indicating the number of command-line arguments. `argv` is an array of pointers to strings (the arguments themselves), terminated by a null pointer. This is the *first convention* of the C language: computation starts here.

But `main()` is **not the true starting point**. Before `main()`, the **runtime library** executes hidden initialization code—often called `_start` or `CRT0` (C Runtime Zero)—that sets up the stack, initializes global variables, prepares the heap allocator, and performs other housekeeping. After `main()` returns, additional cleanup code runs. The programmer never sees this, but it is always there.

Within the execution flow, two fundamental data structures manage memory:

- The **Stack**—for local variables, function parameters, return addresses, and the history of function calls;
- The **Heap**—for dynamically allocated memory whose lifetime is not tied to any single function.

In C++, there is an additional element: the stack also carries data needed for **structured exception handling**—the mechanism for handling situations where computation cannot proceed (disk broken, network down, disk full). In JavaScript/TypeScript, there would also be a **message queue** for event handling—the structure that allows the program to receive mouse clicks, timer events, network data arrivals, and other asynchronous notifications.

Warning**The Processor Does Not Know About Stack or Heap**

It is important to understand that the *processor itself* is entirely unaware of the stack and heap as concepts. From the CPU's perspective, they are simply memory addresses like any other—it reads and writes wherever it is told to. The stack and heap are abstractions maintained by the compiler, the runtime library, and the operating system, not by the hardware. The CPU has no knowledge of whether it is reading a local variable from the stack or a dynamically allocated block from the heap—it only sees addresses.

1.7 The Stack

1.7.1 Structure and Properties

The Stack is an automatically managed block of memory, typically around **1 MB** in size, allocated when the program starts. It operates on a **Last-In, First-Out (LIFO)** basis—like a pile of plates. When a function is called, a new frame is pushed onto the top; when it returns, the frame is popped off.

The stack stores:

- **Local variables** declared within functions;
- **Parameters** passed to function calls (copied by value);
- **Return values** (space reserved by the caller);
- **Return addresses** (where to resume after the function completes);
- In C++: **exception handling data** for structured exception management.

The professor summarized its key properties: “*Automatic, extremely fast, deterministic lifetime.*” The stack supports the principle of locality very well, and its management is handled entirely by the compiler-generated code—the programmer does not need to write a single line to manage it.

Step 1 — Startup: Before `main()` even begins, the hidden `_start/CRT0` code runs. It initializes the stack and reserves space for the return value of `main()` (the “...” or “res” slot).

Step 2 — `int v = 9`: The stack grows by 4 bytes. The value 9 is written into this new space. The local variable `v` now exists on the stack.

Step 3 — Preparing the call `f(v)`: Before jumping to function `f`, the caller prepares the stack:

1. Space is reserved for `f`’s return value (another “...” slot);
2. The *value* of `v` is **copied** as parameter `a`—note: this is a *copy*, not a reference; `a` is completely independent of `v`;
3. The **return address** is pushed—the address of the instruction in `main()` where execution should resume after `f` completes (the assignment `v = ...`).

Step 4 — Inside `f()`: The function begins executing. It declares `int i = 5`—the stack grows further to hold this local variable. At this point, the stack contains: `main`’s res, `v=9`, `f`’s res, `a=9`, return address, `i=5`.

Step 5 — `return a + i`: Since `a` is 9 and `i` is 5, the result is 14. This value is written into the reserved return-value slot. Then `f`’s local variables (`i`) and parameters (`a`) are invalidated as the stack contracts. The Program Counter jumps to the saved return address.

Step 6 — Back in `main()`: The value 14 is read from `f`’s return slot and assigned to `v`. Now `v = 14`. All traces of `f`’s execution on the stack have been erased.

Step 7 — `return 0`: The value 0 is written into `main`’s return-value slot. The stack contracts further. Execution returns to the hidden startup library (`_start/CRT0`), which performs final cleanup before terminating the process.

Insight

The Airbnb Analogy

“The stack’s memory is like an Airbnb apartment. You check in, stay three days, use it for your vacation, then check out and leave. The apartment is not demolished—it stays there, ready for the next guest. If after calling `f()` you call `g()`, function `g` will reuse the exact same memory that `f` just used for its own calculations. It’s always the same memory, used and regularly recycled.”

1.7.3 The Two Constraints of the Stack

This enormous efficiency comes with two fundamental constraints:

Temporal Constraint: Data on the stack *cannot last longer than the function (or block) in which it was declared*. When the function returns, the stack pointer moves back down and that memory is logically gone. You **cannot return a pointer to a local variable**—it will become a dangling pointer the instant the function exits.

Dimensional Constraint: The stack has a finite size (typically ~1 MB). You cannot use it for large data or data whose size is unknown at compile time. Even without massive data, this limits **recursion depth**: a recursive factorial implementation makes the stack grow with each recursive call. At some point, the stack hits its limit—a **stack overflow**—even though the computation has not finished.

The professor noted: “*Since the stack starts at 1 megabyte, after putting a single very large local variable, there might be no room for anything else. That’s why we need the heap.*”

1.8 The Heap

1.8.1 Why the Stack Is Not Enough: Three Reasons

There are three situations where the stack is insufficient:

1. **The data must outlive the function that created it**—for example, you allocate a buffer inside a function and want to use it from another;
2. **The data's size is unknown at compile time**—for example, an array whose size depends on user input;
3. **The data is too large** for the stack—a 10 MB image cannot live in a stack frame.

For these cases, the **Heap** (also called *Free Store* in C++) exists: a much larger memory area where blocks can be allocated and released dynamically, with lifetimes not tied to any specific function.

1.8.2 The Key Difference from the Stack

On the stack, declaring `int i = 5;` is sufficient: the compiler knows exactly where `i` will go (relative to the stack pointer), and the generated machine code handles everything automatically.

On the heap, there is no such automatism. You must:

1. **Explicitly request** a block of memory using a dedicated function (`malloc` in C, `new` in C++);
2. **Receive a pointer** to the beginning of the allocated block—this is the *only* way to access it;
3. **Use the block** through that pointer (using the dereference operator `*`);
4. **Explicitly release** the block when you no longer need it, using the dual function (`free` in C, `delete` in C++).

1.8.3 Step-by-Step C++ Code Walkthrough

Let us trace a heap allocation example:

```
int* f(int a) {
    if (a > 0) {
        return new int[a]; // allocate an array of 'a' ints on the heap
    } else {
        return nullptr;
    }
}

int main() {
    int v = 2;
    int* buffer = f(v); // buffer points to the heap block
    // ... use buffer ...
    delete[] buffer; // explicitly release
    buffer = nullptr; // discipline: prevent dangling pointer
    return 0;
}
```

`int v = 2`: A local variable on the stack. Normal.

`int* buffer = f(v)`: The variable `buffer` is a **pointer**—it occupies 8 bytes on the stack (on 64-bit systems), but its value (an address) is not yet initialized (the “three dots”). Then `f` is called.

Inside `f()`: The function receives `a = 2`, verifies it is positive, and executes `new int[2]`. If this had been C, we would write `malloc(2 * sizeof(int))`. The `new int[a]` tells the runtime: “prepare a contiguous space of as many integers as the value in variable `a`.” The runtime finds space somewhere in the heap—it is irrelevant whether it is a bit lower or higher—marks it as occupied, and returns a pointer to the first byte.

The function returns the pointer: The stack frame of `f` is dismantled, but the heap block **remains intact**—it does not belong to any stack frame. The variable `buffer` in `main` receives this address.

`delete[] buffer`: This is the act of *restitution*. It informs the runtime that the block is no longer needed. The runtime marks it as available for future allocations. **But the pointer `buffer` still contains the old address!** That block no longer belongs to the program, but the pointer doesn’t know that.

`buffer = nullptr`: This is the disciplined programmer’s safety measure. By explicitly setting the pointer to null after deletion, any future accidental access generates a controlled segfault instead of silently corrupting memory. *The language does not force you to do this*—you must have the discipline to do it yourself.

Warning

The `delete[]` Danger

After `delete[] buffer`, the pointer `buffer` still holds the old address. It has become a *dangling pointer*. The owner (the runtime/OS) is now free to give that memory to someone else, or even “demolish the building.” Accessing `*buffer` after deletion is **Undefined Behavior**. Always set pointers to `nullptr` after freeing.

1.9 Allocation and Deallocation Functions

1.9.1 In C

The C standard library provides three allocation functions:

- `void* malloc(size_t s)` — allocates `s` bytes, *without initializing them*. Contents are whatever was previously in that memory. Returns `NULL` on failure.
- `void* calloc(int n, size_t s)` — allocates `n * s` bytes, initializing them all to zero. Safer but slightly slower.
- `void* realloc(void* p, size_t s)` — resizes a previously allocated block. If it succeeds, the first portion retains the old data; additional space contains garbage. It might move the block to a new location (the old pointer becomes invalid). Returns `NULL` on failure (original block remains intact).

Deallocation: `void free(void* p)`.

1.9.2 In C++

C++ adds the keywords `new` and `delete`:

- `new T{args}` — allocates memory and calls the constructor;

- `new T[n]` — allocates an array of `n` elements;
- `delete p` — calls the destructor and deallocates;
- `delete[] p` — calls destructors for all elements and deallocates.

Warning

Fundamental Rule: Dual Functions

Every block must be released with the *dual* of the function that allocated it: `malloc` $\leftrightarrow$ `free`, `new` $\leftrightarrow$ `delete`, `new[]` $\leftrightarrow$ `delete[]`. Mixing these pairs corrupts the runtime's internal data structures and produces unpredictable behavior.

1.10 Address Space Layout

When a program executes, its virtual address space is divided into distinct zones, each with specific purposes and permissions:

- **Executable Code (Text Segment):** Contains the processor's machine instructions. Mapped as *read + execute only*—not writable. Any attempt to modify running code triggers a segfault.
- **Constants (Read-only Data):** Contains string literals (e.g., the "Hello world" in a `printf` call), global constants, etc. Mapped as *read-only*. Attempting to write here causes a segfault.
- **Global Variables (Data Segment):** Contains variables declared outside any function. Readable and writable. Initialized by the loader *before* `main()` is called.
- **Stack:** Grows from high addresses downward (on many architectures). Contains function frames with parameters, local variables, and return addresses.
- **Heap:** Grows from low addresses upward. Contains dynamically allocated blocks, managed by the allocator (part of the runtime library).
- **Runtime Libraries:** Memory segments mapped from dynamic libraries (DLL / `.so`) used by the program.

1.10.1 Inspecting with `/proc/<pid>/maps`

On Linux (or WSL2 on Windows), you can directly observe your program's address space. For any running process with PID `<pid>`, the file `/proc/<pid>/maps` (a pseudo-file generated dynamically by the kernel each time you read it) shows every mapped zone with its addresses, permissions, and origin.

```
$ cat /proc/4742/maps
00400000-00452000 r-xp /usr/bin/myprogram # executable code
00652000-00653000 r--p /usr/bin/myprogram # read-only data
00653000-00654000 rw-p /usr/bin/myprogram # global variables
01a3e000-01a5f000 rw-p [heap] # heap
7f8a1c000000-... r-xp /lib/x86_64.../libc.so.6 # shared library
7ffd4bc9e000-... rw-p [stack] # stack
```

You will clearly see that of all 2^{48} potential addresses, only tiny fragments are actually mapped. The numbers will differ each time—the OS randomizes locations for security through a mechanism called **ASLR** (Address Space Layout Randomization)—but the structure is always the same. **Any attempt to access addresses outside these ranges causes a segmentation fault.**

1.11 Variable Lifecycle: Global, Local, Dynamic

The C/C++ execution model distinguishes three categories of variables, each with fundamentally different properties:

Property	Global	Local	Dynamic
Location	Data segment	Stack	Heap
Address	Fixed (compile/link time)	Relative to stack top	Absolute (runtime)
Lifetime	Entire program	Block/function scope	Until explicit release
Initial value	Loader-initialized	Random garbage	Uninitialized (C) / Constructed (C++)
Access	By name, anywhere	By name, in scope	Only via pointer

Global variables have a fixed address determined by the compiler and linker. They exist from program start to program end. If initialized (e.g., `int x = 27;`), the loader “paints” that value into memory *before* `main()` is called—it is not an executed instruction; the data is pre-loaded.

Local variables exist on the stack, at addresses relative to the current stack top. The compiler cannot say where variable `i` will be in absolute terms—it only knows at runtime. Their lifetime is tied to the block in which they are declared: they start to exist when execution enters the block, and cease to exist when execution exits it (even if the block is a `for` loop or an `if` body, not just an entire function).

Insight

Random Garbage in Local Variables

“What do you find when you enter an Airbnb apartment? The mess the previous guest left behind. If the previous guest was clean, great. If not, you’re in trouble. Local variables are exactly the same: they contain whatever was written in that stack position by the *previous function call*. You must always initialize them before reading.”

Dynamic variables live on the heap, at addresses determined at runtime by the allocator. They can only be accessed through pointers. In C, they are uninitialized (`malloc`); in C++, the constructor is called (`new`). Their lifetime extends from explicit allocation to explicit deallocation—entirely under the programmer’s control.

1.11.1 Java: Value vs. Reference Semantics

In Java, the situation is subtly different. Primitive types (`int`, `boolean`, `char`, `float`, `double`) have **value semantics**: when you write `int j = i;`, `j` receives a *copy* of `i`’s value. Modifying `i` afterward does not affect `j`.

Everything that is not a primitive type—every object, every `String`, every `ArrayList`—has **reference semantics**. The variable is actually a pointer to the heap. When you write `String s = "hello";`, `s` is a pointer to a heap-allocated string. When you write `ArrayList<Integer> list = new ArrayList<>();`, `list` is a pointer. All objects in Java are heap-allocated and accessed exclusively through references (hidden pointers). The garbage collector is responsible for freeing them when they are no longer reachable.

1.12 Pointers: Power and Peril

1.12.1 What Pointers Are

A pointer is simply an **integer representing a memory address**. Declaring it uses the asterisk syntax: if an integer is declared with `int i;`, an integer pointer is declared with `int*`

`p`; To access the value it points to, you use the dereference operator `*p`.

Pointers are necessary: they are the only way to access heap memory, to implement complex data structures (linked lists, graphs, trees), and to pass large data to functions without copying. But they are also **extremely dangerous** because they are intrinsically ambiguous.

1.12.2 The Seven Ambiguities of Pointers in C/C++

Given a pointer `int* p`, the language provides **no answer** to any of the following questions:

1. **Is the address valid?** It could be `NULL`, an uninitialized value, or the address of a block that has already been freed. The convention in C is that zero is invalid (the macro `NULL` is defined as zero, easily recognizable). But just because a pointer is non-zero does not mean it is valid: `0x3B7F` might point to unmapped memory.
2. **How large is the block?** A single integer? An array of 100? The type says “pointer to `int`” but nothing about the extent of the allocation.
3. **For how long is access guaranteed?** The pointer does not carry an “expiration date.” The professor: *“If you buy food at the supermarket, there’s a ‘best before’ date. Pointers have no such date. If you save a pointer for later use, how long can you safely use it? The language gives you no help at all.”*
4. **Can I modify the contents?** Only through the `const` qualifier, and only if the programmer was disciplined enough to use it.
5. **Must I release it?** The pointer might refer to a stack variable (must not be freed) or a heap allocation (must be freed). The type does not distinguish.
6. **Can I release it?** Others might hold copies of the pointer and still need the data, unaware of the release.
7. **Does it express optionality?** Sometimes `NULL` means “no data available,” but the type does not distinguish this from normal usage.

The professor summarized the situation starkly: *“In C/C++, the programmer is responsible for all of these questions. In Rust, the responsibility is codified in the type system—Rust offers a number of different pointer types, each removing specific ambiguities, so that it is evident which pointer must be released, which must not, which one lasts briefly, and which one lasts long.”*

1.12.3 The Bug Catalogue

These ambiguities lead directly to a catastrophic catalogue of bugs:

Bug	Description	Consequence
Dangling pointer	Accessing memory after it has been freed	Undefined Behavior (UB), crash, silent corruption
Wild pointer	Using a pointer that was never initialized	UB, crash, corruption
Double free	Releasing the same block twice	Corruption of the allocator’s internal structures
Memory leak	Never releasing allocated memory	Progressive exhaustion of address space
Buffer overflow	Accessing beyond the allocated boundary	UB, corruption, security vulnerabilities
Data race	Multiple threads accessing the same data without synchronization	Random results, non-reproducible crashes

The professor stated: “About 90% of system bugs are born from these problems—from illicit access to memory areas. 70% of the security vulnerabilities detected in Windows are due to memory management issues.”

Insight

The Garage Key Analogy (Dangling Pointer)

“It’s like renting a garage. You have the keys and can enter freely. At some point, the lease expires and you return the keys to the owner. But you made a copy. The owner is now free to rent it to someone else, demolish the building, or do whatever he wants. One day you use your copy—the fridge you thought was there is gone, because the new tenant has already moved their things in, or worse, the owner demolished the wall and the roof collapses on you. Returning `free(p)` is returning the keys. But unlike the Airbnb key box, pointers are immaterial—you still have a copy. And the language doesn’t force you to destroy it.”

Insight

The Red Light / Private Parking Analogy (Undefined Behavior)

The professor used additional analogies for undefined behavior: “The language tells you: *you must not do this. It’s like saying you must not park in a private parking space. If you do, will you automatically get a ticket? No. You might be fine, you might not. You must not run a red light. If you do, will you automatically have an accident? No. But you might. The problem is that the language leaves the responsibility to you. It tells you the rule but does not enforce it.*”

1.13 Garbage Collection vs. Manual Management

Characteristic	C/C++ (Manual)	Java/Python/C# (GC)
Who releases memory	The programmer	The garbage collector
When	Controlled by programmer	Controlled by GC, non-deterministic
Real-time compatible	Yes	No (stop-the-world pauses)
Memory usage	Minimum necessary	Significantly higher
Risks	Leak, dangling, double free, UB	Almost none on memory
Destructors	Supported (C++ RAII)	Not guaranteed (Java deprecated finalizers)

1.14 Why Rust: Solutions to C/C++ Problems

Rust adopts a radically different approach: it uses **no garbage collection**, yet it does not leave the programmer free to mismanage memory as C does. Instead, it introduces a sophisticated type system that reasons about *ownership* and *lifetimes* at **compile time**—with zero runtime cost.

! Exam Priority

Rust's Solutions to C/C++ Problems

Rust systematically eliminates pointer ambiguities through its type system:

C/C++ Problem	Rust Solution
Dangling / Wild pointer	Borrow checker + Lifetimes : the compiler verifies that references never outlive the data they point to
Double free	Ownership system : every block has exactly one owner; memory is automatically dropped when the owner goes out of scope—once and only once
Memory leak	RAII pattern : deallocation happens automatically in the owner's destructor
Null pointer	Option<T> : absence of value is explicitly represented in the type; you cannot forget to handle the None case
Shared ownership / Release management	Box<T> , Rc<T> , Arc<T> : smart pointers that encode ownership and sharing semantics directly in the type

Expected Mastery Level: Very High. You must deeply understand how virtual memory, the Stack vs. Heap, and manual memory allocation interact to appreciate why Rust's compiler rules exist.

The fundamental point: if a Rust program **successfully compiles**, the compiler guarantees the absence of the memory management bugs described above. It is not an absolute guarantee of logical correctness (infinite loops, deadlocks, and algorithmic errors remain possible), but it eliminates an entire *category* of bugs that in C/C++ can remain hidden for years.

1.15 Conceptual Mind Map

To consolidate the chapter's concepts, here is the logical hierarchy to keep in mind:

```

ADDRESS SPACE (virtual)
|
+-- Executable Code      -> Read + Execute only
+-- Constants            -> Read only
+-- Global Variables     -> Read + Write, lifetime = entire program
|
+-- STACK
|   +-- Function frames (params + locals + return addr)
|   +-- Lifetime = duration of block/function
|   +-- Automatic allocation, extremely fast
|   +-- Size limit (~1 MB) -> stack overflow risk
|
+-- HEAP
|   +-- Blocks of arbitrary size
|   +-- Lifetime = controlled by programmer (alloc/free)
|   +-- Accessible only via pointers
|   +-- Risks: leak, dangling, double free, UB
|
+-- Runtime Libraries    -> DLL/.so mapped segments

```

Why cache matters: Contiguous data structures (arrays, vectors) exploit the principle of locality → benefit from cache → are faster in practice than fragmented structures (linked lists),

even when the abstract algorithmic complexity would be equal.

Why Rust is the right learning tool: Not because you will program in Rust for the rest of your life, but because its compiler transforms every violation of the correct mental model into a compile-time error. Rust is a merciless mirror that reveals immediately when your mental model is wrong—before the 3 AM production crash does.

Exam Focus

Exam Relevance and Recurrence

Score: High

While this chapter is introductory, these concepts are foundational for every subsequent topic. Common exam question patterns include:

- **Memory Layout & Sizing Analysis:** Given a Rust/C program, identify where each variable resides (stack, heap, data segment) and explain why.
- **Theoretical Concepts:** Compare cache efficiency of `Vec` vs `LinkedList`; explain why garbage collection is unsuitable for real-time systems; describe the three benefits of the MMU.
- **Implementation Questions:** Trace stack growth/contraction during a function call sequence; explain what happens after `free(p)` if the pointer is reused.

Understanding stack vs. heap is **absolutely prerequisite** for grasping Ownership (Chapter 4), Lifetimes (Chapter 5+), and Concurrency (Chapter 14). If these foundations are not solid, everything built on top will be unstable—as the professor says: *“You can’t build the roof of a house and then work downward to the foundations. And when you pour a new floor, you need 28 days for the concrete to solidify before you can build on it.”*

1.16 Domande ed Esercizi da Esami Passati

Exam Focus

Domanda:

In un sistema operativo conforme alle specifiche Posix, cosa succede se in un processo in cui sono in esecuzione più thread, viene invocata la chiamata di sistema `fork()`? Al suo ritorno, quanti thread saranno presenti nel processo figlio e quanti nel processo padre? Che tipo di controllo può esercitare il programmatore per gestire questo tipo di situazione?

Risposta:

Poiché questo capitolo si concentra sull’introduzione alla programmazione di sistema e sull’architettura della memoria, non tratta esplicitamente la chiamata di sistema `fork()` o le specifiche POSIX.

Tuttavia, sulla base dei concetti introdotti nel capitolo per quanto riguarda i processi e il modello di esecuzione: il sistema operativo, tramite il loader, crea un processo inizialmente dotato di **un singolo thread** dedicato all’esecuzione sequenziale delle istruzioni. Inoltre, il modello di esecuzione del C/C++ assume di default che l’esecuzione sia unicamente sequenziale, non esponendo intrinsecamente il parallelismo dell’hardware. Di conseguenza, il programmatore è costretto a gestire **esplicitamente** la concorrenza e il controllo dei flussi multipli (thread o processi), in quanto l’astrazione di base del linguaggio non prevede meccanismi nativi per queste situazioni.

Chapter 2

Architecture and Memory

The execution model associated with a high-level program does not mirror the actual underlying processor hardware. Instead of directly writing raw instructions for a CPU's fetch-decode-execute cycle, modern programming languages provide an abstraction layer. This layer relies on structured data, intermediate representations, and a runtime library to manage the execution environment. The high-level code we write is translated by the compiler into machine instructions, but its execution is supported by data structures managed behind the scenes.

When a program runs, the operating system and the runtime library conspire to give it the illusion of being the sole user of the machine, with a vast, continuous memory space at its disposal. However, this virtual address space is not a single infinite void. It is carefully segmented to handle different types of data with different life cycles.

Insight

The Illusion of Infinite Memory

Languages often abstract away memory limits. A program requests memory, and the system complies. However, physical memory is finite. The operating system uses virtual memory and paging to extend available space, but bounds always exist. Recursion limits and out-of-memory errors are stark reminders of the physical hardware beneath the abstraction.

2.1 The True Beginning: Before and After `main`

While we often consider the `main` function to be the starting point of our program, it is merely the user's entry point. Behind the scenes, the runtime library inserts a hidden startup function (often called `_start` or `CRTmain`). This function prepares the memory layout, sets up the `argc` and `argv` parameters, and ultimately calls `main`. When `main` finishes, the runtime takes back control to properly dismantle the execution environment before handing control back to the operating system.

To understand how programs manage state, we must examine the internal layout of the program's address space. Typically, this space is divided into several areas:

- **Executable Code:** The actual machine instructions.
- **Constants:** Immutable data, such as string literals (e.g., `"Hello, World"`), mapped as read-only.
- **Global Variables:** Variables that exist before the program starts and remain until termination, holding fixed addresses determined at compile time.
- **The Stack:** Used for managing function calls and local variables.

- **The Heap:** Used for dynamic memory allocation.

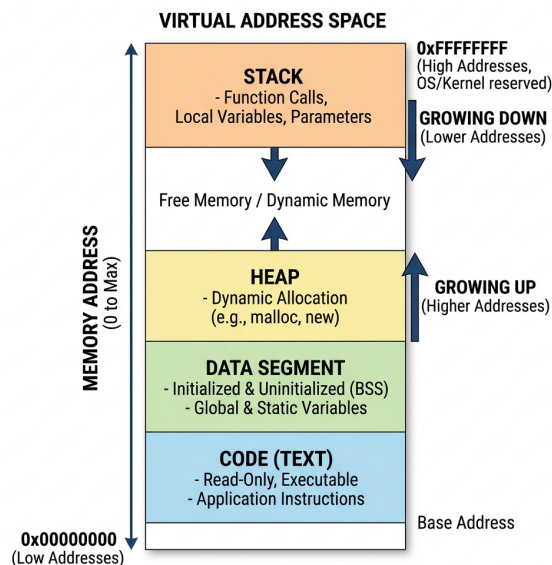


Figure 2.1: Typical Memory Layout of a Process.

Insight

Inspecting the Address Space

On Linux or WSL2, you can directly inspect the segmented address space of a running process. Every process is assigned a Process ID (PID). By reading the virtual file `/proc/<PID>/maps`, you can see exactly which memory ranges are mapped to the Heap, the Stack, executable code, and shared libraries. Attempting to access an address outside these mapped ranges, or violating their access permissions (e.g., trying to write to the read-only constants area), immediately triggers a **Segmentation Fault**.

The most critical distinction for a systems programmer is understanding how and when to use the Stack versus the Heap.

2.2 The Stack: Automatic and Deterministic

The Stack is a predefined block of memory provided by the operating system when the program starts. It is designed to manage the execution of functions, keeping track of where we are, what variables we have, and where we must return when a function completes.

2.2.1 How the Stack Works

Consider a simple function call. When `main` calls a function `f(v)`, several things must happen in a highly structured order:

1. **Preparation:** Space is reserved for the return value of `f`.
2. **Parameters:** The arguments (e.g., `v`) are copied into the stack so `f` can access them.
3. **Return Address:** The address of the next instruction in `main` is pushed onto the stack so execution can resume correctly.

4. **Local Variables:** As `f` executes, any local variables it declares are pushed onto the top of the stack.

Beyond managing function variables, the Stack in languages like C++ also holds crucial metadata for structured exception handling. If a critical error occurs (such as a full disk or an interrupted network connection), the program uses this information to "unwind" the stack, safely skipping remaining operations and jumping to an appropriate error handler.

```
int f(int a) {  
    int i = 5;           // Local variable pushed onto the stack  
    return a + i;        // Value is computed and placed in the return slot  
}  
  
int main() {  
    int v = 9;           // Local variable in main's stack frame  
    int result = f(v);    // Triggers a new stack frame  
    return 0;  
}
```

Deep Memory Model Analysis: In the snippet above, `v` is allocated on `main`'s stack frame. When `f(v)` is called, the value of `v` is copied into the parameter `a` on `f`'s new stack frame. The local variable `i` is also stack-allocated within `f`'s frame. Because these are purely value types (`int`), no heap allocation occurs. When `f` returns, its entire stack frame—including `a` and `i`—"evaporates", returning the computed value to `main`'s pre-allocated return slot.

When `f` finishes, it executes a `return`. The result is deposited in the reserved space, and the stack pointer is forcefully pulled back to its position before `f` was called. All the local variables and parameters of `f` instantly "evaporate." The memory is not physically erased, but it is released and will be overwritten by the next function call.

Note

The AirBnB Analogy

Think of the stack like an AirBnB rental. You arrive, use the space for your stay, and then leave. The next guest will occupy the exact same room and overwrite whatever "mess" you left behind. This is why local variables in C/C++ contain unpredictable garbage values if not explicitly initialized—they simply hold the leftover data from previous stack frames!

2.2.2 Trade-offs of the Stack

The Stack is incredibly efficient. Allocation and deallocation are instantaneous, requiring only the adjustment of a single pointer. It guarantees perfect deterministic lifetimes: a variable's life is strictly bound to its lexical scope. Furthermore, because it operates sequentially in contiguous memory, it is highly cache-friendly.

When and Why to Use It: Prefer stack allocation whenever possible. It provides zero-cost automatic memory management and excellent cache locality. Use it for local variables, small fixed-size arrays, and values that do not need to outlive their declaring function scope.

However, the Stack is fundamentally limited. First, its size is fixed (often around a megabyte). Deep recursion will exhaust this space, resulting in a Stack Overflow. Second, the strict life cycle means a function cannot create data on the stack and pass it upwards to live longer than the function itself.

2.3 The Heap: Dynamic and Flexible

When a program needs memory that outlives the function that created it, or requires a dynamically determined size that is too large for the stack, it must turn to the Heap. The Heap is a vast, unorganized area of memory managed by the OS and the runtime library.

Unlike the Stack, memory on the Heap is not given out automatically. You must explicitly request it using an allocation function (e.g., `malloc` in C, or `new` in C++). The runtime searches for an available chunk of memory, marks it as reserved, and returns a *pointer* to its starting address.

```
int* create_array(int size) {
    if (size > 0) {
        // Requesting dynamic memory on the Heap.
        // C equivalent: int* buffer = malloc(size * sizeof(int));
        int* buffer = new int[size];

        // Ownership of this memory is passed back to the caller.
        // The data will survive even after create_array returns!
        return buffer;
    }
    return nullptr;
}

int main() {
    int* my_array = create_array(2);

    if (my_array != nullptr) {
        // ... use my_array across different functions ...

        // When done, we MUST explicitly release it.
        // C equivalent: free(my_array);
        delete[] my_array;
    }
    return 0;
}
```

Deep Memory Model Analysis: Here, `create_array` allocates memory on the heap via `new int[size]`. The pointer `buffer` itself is stored on the stack, but the array it points to resides on the heap. Ownership of this heap allocation is implicitly transferred back to `main` when the pointer is returned. In `main`, `my_array` holds this pointer. Because C/C++ does not automatically track or enforce ownership semantics, it is strictly up to the programmer to ensure `delete[] my_array` is called exactly once. Failure to do so leads to a memory leak.

2.3.1 The Burden of Manual Management

When and Why to Use It: Use heap allocation when the size of the data is unknown at compile time, when the data is too large to fit safely on the stack (risking a Stack Overflow), or when the data must outlive the function that created it.

The great flexibility of the Heap comes with a heavy responsibility. The compiler and runtime do not track when you stop using a dynamically allocated block. It is entirely up to the programmer to return the "key" to the runtime by calling `free()` or `delete`.

If you forget to release the memory, it remains marked as "in use." If this happens repeatedly, the program will consume more and more memory until the OS refuses to provide any more, causing an application crash. This phenomenon is known as a **memory leak**.

2.3.2 Beyond the Stack and Heap: The Message Queue

While the Stack and Heap form the core of C and C++ memory management, other execution environments (such as JavaScript or TypeScript) rely on a third fundamental structure: the **Message Queue**. This queue operates on a First-In, First-Out (FIFO) basis, intercepting asynchronous events like mouse clicks, network responses, and timers, dispatching them to the execution thread when it is free.

2.4 The Danger of Pointers

Pointers are the only way to interact with the Heap, yet they are intrinsically ambiguous and dangerous. A pointer is simply a numerical memory address (e.g., `0x3B7F`), but the language provides almost no metadata about it.

! Exam Priority

The Ambiguity of Pointers in C/C++

When a C++ programmer looks at an `int* ptr`, they cannot answer fundamental questions:

- **Validity:** Is the address valid? By convention, 0 is a null pointer, but if it is `0x3B7F`, it might point to a valid integer, or it might point to unowned garbage.
- **Ownership:** Am I responsible for freeing this pointer, or is another function managing it?
- **Multiplicity:** Is this pointer an array of integers, or just a single integer?
- **Lifetime:** How long is this memory guaranteed to remain valid? The professor often compares this to a carton of milk lacking an expiration date.

Common Pitfall: Assuming a non-null pointer is safe to dereference or free without verifying ownership.

Expected Mastery Level: High. You must explicitly recall these four missing metadata elements, as they are central to understanding why Rust's type system is designed the way it is.

To understand the lifetime ambiguity, consider an analogy: when you buy a carton of milk at the supermarket, it comes with a clear expiration date. You know exactly when it will go bad. Pointers, unfortunately, do not have an expiration date stamped on them. If you save a pointer to be used later, the language provides no built-in mechanism to check if the underlying memory has been released by another part of the program in the meantime.

Consider the notorious **Use-After-Free** bug. Because pointers are immaterial copies of addresses, releasing the memory does not destroy the pointer variables that hold that address.

```
int* ptr = new int(42);
delete ptr; // Memory is returned to the OS.

// 'ptr' is now a Dangling Pointer.
// It still holds the address, but we no longer own the memory!
*ptr = 100; // Undefined Behavior! Writing to unowned memory.
```

To avoid this, a disciplined programmer must manually track pointer lifecycles, which is notoriously difficult in large, complex codebases.

2.5 Garbage Collection vs. Rust

To solve the pointer crisis, many modern languages (Java, Python, C#) abandoned manual memory management entirely, introducing a **Garbage Collector (GC)**.

In Java, elementary types (like `int`) live on the stack with value semantics, but objects live on the heap with reference semantics.

```
// Java GC Example
int i = 5; // Value on the stack

// Semantic of reference
String s = "ciao_mamma"; // Allocated on the Heap
ArrayList<Integer> list = new ArrayList<>(); // Allocated on the Heap

// Notice: No explicit free() or delete!
// The Garbage Collector will eventually clean them up.
```

With a GC, the programmer never calls `free()`. Instead, the runtime periodically pauses the program to scan the heap, look for objects that no longer have any active pointers referencing them, and reclaims their memory. The most renowned algorithms used for this task belong to the **Mark and Sweep** family, often coupled with **Reference Counting** strategies.

! Exam Priority

The Cost of Garbage Collection

While GCs eliminate memory leaks and use-after-free bugs, they introduce non-deterministic pauses ("Stop The World" events). When the GC runs, the execution of the main program halts. As emphasized in lectures, this pause can last seconds, which for a CPU doing a billion operations per second is an eternity.

Common Pitfall: Believing GC solves all memory issues without side effects. In real-time systems (e.g., medical devices, aerospace), this lack of predictability is entirely unacceptable.

Expected Mastery Level: High. Be prepared to contrast GC latency with Rust's compile-time memory management.

2.5.1 The Rust Paradigm

Rust enters the landscape with a bold proposition: *Safety without Garbage Collection*. Systems programmers want the determinism and extreme performance of C/C++, but without the undefined behavior and pointer bugs.

Rust achieves this by embedding memory management rules directly into its type system. Instead of having a single, ambiguous `int*` pointer, Rust distinguishes between different types of ownership and borrowing. By forcing the programmer to explicitly state who owns the memory and how long it lives, the Rust compiler can insert the necessary `free` calls automatically at compile time, eliminating both the GC overhead and manual memory bugs.

Exam Relevance and Recurrence

Score: Very High

Notes: The fundamental differences between stack and heap, and the intrinsic ambiguities of C/C++ pointers, frequently appear in exams. The "Stop The World" problem of Garbage Collection and how Rust fixes this via ownership rules is a recurring short-essay question.

2.6 Domande ed Esercizi da Esami Passati

Exam Focus

Domanda 1: Si illustrino le differenze tra stack e heap. Insieme alle differenze, indicare per i seguenti costrutti Rust, in modo dettagliato, dove si trovano i dati che li compongono: `Box<T>`, `RefCell<T>` e `&[T]`.

Soluzione basata sul capitolo: Lo **Stack** è una memoria efficiente e deterministica fornita dal sistema operativo, utilizzata per la gestione delle chiamate a funzione e delle variabili locali. L'allocazione e deallocazione avvengono spostando un singolo puntatore (stack pointer) in modo quasi istantaneo. I dati hanno un ciclo di vita limitato allo scope della funzione in cui sono definiti e la dimensione totale dello stack è fissa. L'**Heap** è un'area di memoria vasta e non organizzata, utilizzata per l'allocazione dinamica, essenziale quando la dimensione dei dati non è nota a compile time o quando i dati devono sopravvivere alla funzione che li ha creati. L'heap richiede un'allocazione e deallocazione esplicita (rischio di memory leak e dangling pointer in assenza di Garbage Collection o del type system di Rust). *Nota sui costrutti Rust:* Il capitolo si focalizza sui principi generali della memoria in C/C++ e sul superamento del Garbage Collector in Rust, ma non dettaglia l'implementazione in memoria di `Box`, `RefCell` o `&[T]`. In base ai concetti del capitolo, si deduce unicamente che eventuali puntatori alle risorse risiedono sullo stack locale, mentre i dati gestiti dinamicamente risiedono nell'heap.

Exam Focus

Domanda 2: Si definiscano le principali aree di memoria associate ad un eseguibile e si mostri, attraverso opportuni esempi di codice, in quale situazione ciascuna di esse viene utilizzata.

Soluzione basata sul capitolo: Lo spazio di indirizzamento virtuale di un processo è diviso nelle seguenti aree principali:

- **Executable Code:** contiene le istruzioni macchina da eseguire.
- **Constants:** area a sola lettura per i dati immutabili, ad esempio i letterali stringa (es. "Hello, World").
- **Global Variables:** variabili i cui indirizzi sono determinati a tempo di compilazione e persistono per tutta la durata del programma.
- **Stack:** utilizzato per la gestione automatica delle chiamate a funzione e delle variabili locali. *Esempio:* In `int f(int a) { int i = 5; return a + i; }`, le variabili `a` e `i` sono collocate sullo stack del rispettivo frame.
- **Heap:** area usata per l'allocazione dinamica. *Esempio:* In `int* buffer = new int[size];`, l'array la cui dimensione è calcolata a runtime viene allocato nello heap, mentre il puntatore `buffer` che ne mantiene l'indirizzo si trova nello stack.

Exam Focus

Domanda 3: Si considerino le seguenti strutture dati e rispettive porzioni di codice. Per ciascuna di esse si indichi la dimensione di memoria allocata nello stack e nello heap, ipotizzando un'architettura a 64 bit.

```
let boxed: Box<[u32]> = vec![10, 20, 30, 40,
```

Soluzione basata sul capitolo: Il Capitolo 2 non include le nozioni per calcolare la dimensione in byte per architetture a 64 bit o il dettaglio dei tipi `Box` e `vec!`. Applicando esclusivamente la teoria del capitolo: la variabile locale `boxed` occupa memoria nello **stack** (poiché conterrà il puntatore o il descrittore del dato), mentre l'allocazione dinamica dei valori forniti a runtime avverrà nell'**heap**.

Exam Focus

Domanda 4: Sia data la struttura `LinkedList<T>` definita come:

```
pub struct LinkedList<'a, T> {
    pub val: Option<T>,
    pub next: Option<&'a Box<LinkedList<'a, T>>>,
}
```

Si definisca l'occupazione di memoria di un elemento della lista e si indichi come sia possibile definire il fine lista.

Soluzione basata sul capitolo: Il testo affronta il layout di memoria generico (Code, Data, Stack, Heap) e i concetti di ownership senza calcolare il footprint in memoria di strutture Rust specifiche. Pertanto, l'occupazione in memoria dell'elemento non è ricavabile. Il fine lista, basandosi sul parallelismo con l'assenza di riferimenti validi discussa per i puntatori nulli (es. 0 in C/C++), si modellerà in Rust indicando la non presenza di un puntatore successivo (ovvero con la variante `None` dell'`Option`, anche se non descritto esplicitamente nel capitolo).

Exam Focus

Domanda 5: Data la struttura dati definita come

```
struct Data { Element: AsVector, next: Rc<Data> }
enum AsVector { AsVector(Box::<Rc<i32>>), None }
```

Indicare l'occupazione di memoria di un singolo elemento in termini di: 1. numero di byte complessivi (caso peggiore, architettura a 64 bit) e 2. posizionamento dei vari byte in memoria (stack, heap, ecc.).

Soluzione basata sul capitolo:

1. **Numero di byte:** Non calcolabile, poiché il capitolo non specifica la grandezza in byte di puntatori, `enum`, `Box` o `Rc`.
2. **Posizionamento in memoria:** Sulla base della logica introdotta nel testo, l'elemento allocato all'interno di una funzione occuperà memoria sullo **Stack** per le sue componenti di base (come per i puntatori `int* ptr` e le variabili locali). Tuttavia, la memoria sottostante effettivamente incapsulata e gestita dinamicamente da tipi come `Box` o `Rc` risiederà nell'**Heap**.

Chapter 3

Rust Operational Fundamentals

The transition from languages like C, C++, or Java to Rust requires adopting an entirely new set of mental models. This chapter covers the operational foundations you need before writing any serious Rust code: what kind of language Rust is, how to set up and use its toolchain, how to structure projects, and how Rust’s fundamental types—variables, numbers, strings, and vectors—work at the memory level. Every concept here was presented live by the professor during the lecture, with concrete demonstrations in the IDE.

Insight

The professor opened the lecture by stating four explicit objectives:

1. Understand what type of language Rust is.
2. Know how to write a minimum program that can be compiled and executed.
3. Understand what the compiler guarantees and why this is useful.
4. Prepare the conditions to go ahead and do everything that awaits in subsequent lectures.

These four objectives provide a roadmap for everything in this chapter.

3.1 What Kind of Language Is Rust?

The professor opened the lecture by establishing four defining characteristics of Rust that shape everything that follows.

3.1.1 A Compiled Language with Cross-Compilation

Rust is a **compiled language** that generates specific executables for the target machine. By default, the target machine is your own PC. On a Mac with an ARM M2 processor, the compiler generates ARM executables; on a Linux x86 machine, it generates x86 executables. But the compiler is also capable of *cross-compilation*: you can install more complex toolchains and say “I want to generate code for Linux x86,” and the compiler will produce the appropriate binary. Clearly, to execute it you would need to transfer that binary to an actual Linux x86 machine.

3.1.2 Memory Safety by Design

Rust guarantees, *by design*, the correctness of accesses to memory. It is not possible—at least when using the “safe” subset of Rust, which is the default—to write programs that access

memory that no longer exists, that was valid in the past but is no longer valid now. The compiler tracks not only the validity of references but also the time intervals, the so-called **lifetimes**, during which it is possible to access a given reference. Beyond that interval, the reference is no longer guaranteed, and the compiler refuses to compile. This completely eliminates all problems related to **dangling pointers**, **double frees**, and similar memory bugs.

Insight

The professor emphasized: “There is no `null` in Rust, simply because there is nothing. Unlike languages like C or Java, where you can mark a pointer as invalid using `null`, in Rust there is no such concept—so there is no danger of null pointer exceptions.”

3.1.3 Performance Comparable to C and C++

Rust tries to guarantee performance similar to what you can achieve with C and C++. The professor noted that C is *slightly* faster because it has fewer abstraction layers, but Rust gives higher semantic expressiveness and costs only marginally more—“a really negligible thing, because the language is strongly oriented toward optimization and gives us everything that is possible.”

3.1.4 The Ownership Model

All the abstractions that will be encountered in subsequent weeks are based on a foundational concept called **ownership** (possession). Every value in Rust is owned by one and only one variable. This has very important consequences that will be explored in later chapters. For now, the professor asked students to simply accept this as the base model.



Exam Priority

The four pillars introduced here—compiled language, memory safety by design, C-like performance, and ownership—form the conceptual skeleton of Rust. Exam questions frequently ask you to explain *why* Rust is suitable for systems programming, and the answer always traces back to these four properties.

Expected Mastery Level: High. Be able to articulate each pillar and how they interrelate.

3.2 The Toolchain: `rustc`, Cargo, and `rustup`

From an operational point of view, you need specific tools installed on your machine to develop in Rust.

3.2.1 The Compiler: `rustc`

The basic piece required to write a Rust program is the compiler, called `rustc`. You *could* invoke it directly—nobody would prevent you—but normally you would never call it directly. Why? Because you use `cargo`, which does everything for you in a cleaner and more convenient way.

The professor showed that `rustc` can be called by passing it the name of the `.rs` file to compile, specifying what it should generate, and so on—“nothing ambiguous, just like compiling with the C compiler.” However, this manual invocation is almost never done in practice.

3.2.2 Installing the Toolchain: rustup

To get `cargo` and `rustc` on your PC, go to the website <https://rustup.rs>. There you will find instructions tailored to your platform. The page automatically detects your operating system and provides the appropriate installer. Setup time is usually about 5 minutes.

The professor described the experience: “The page automatically says ‘I see you have a wonderful Windows 11 PC. Click here to download.’ Or ‘You have a Linux PC with X86, click here,’ etc. It gives you detailed instructions on what to do.”

Insight

The professor noted that on Windows, the installer may ask whether you want to use the Microsoft compilation chain (MSVC—the linker used by Visual Studio) or the GNU toolchain (GCC). Both are quite equivalent. If you already have the Microsoft ecosystem installed, choose MSVC. The GNU-GCC environment on Windows works well in some cases, less so in others. The professor’s advice: “Do whatever you want, as long as you arrive with a working environment.”

3.2.3 The Build System: Cargo

Cargo is Rust’s project manager and build system. It is analogous to what `npm`, `yarn`, or `pnpm` are for JavaScript (as used in Web Application courses), or what `Maven` and `Gradle` are for Java. Cargo is a powerful, articulated tool that handles:

- Creating project scaffolding
- Downloading libraries automatically when you declare them as dependencies
- Managing versions
- Compiling the project
- Running tests
- Formatting code

The professor described it vividly: “You simply write ‘I need these libraries,’ and Cargo says ‘Don’t worry, I’ll fetch them, put them in the right place, and do everything you need.’”

Essential Cargo Commands

Command	Description
<code>cargo new <name></code>	Creates a new project folder with the minimum scaffolding needed: a <code>src/</code> directory containing <code>main.rs</code> , and a <code>Cargo.toml</code> file. Be careful: if a folder with that name already exists, Cargo will refuse.
<code>cargo build</code>	Compiles the project and produces the artifact (executable or library) in <code>target/debug/</code> .
<code>cargo build --release</code>	Compiles with optimizations, producing the artifact in <code>target/release/</code> . The release version strips debugging information and unnecessary overhead for optimal performance.
<code>cargo run</code>	Builds the project (if necessary) and immediately executes the binary. If sources have changed since the last build, Cargo detects the discrepancy and recompiles automatically.
<code>cargo check</code>	Verifies that the code would compile without actually generating the final binary. Useful for large projects where full code generation is slow—gives you a quick idea of whether your code is correct.
<code>cargo test</code>	Runs the project’s tests: unit tests (testing individual functions), integration tests (testing broader interactions), and documentation tests (code examples embedded in doc comments using <code>///</code>).
<code>cargo fmt</code>	Reformats the entire project according to the community’s standard style guidelines: 4-space indentation, opening braces on the same line (not on the next line), etc.

Documentation Tests in Detail

The professor elaborated on documentation tests: within documentation comments (written with `///` or similar notation), beyond describing “this method does X, parameter 1 means Y, parameter 2 means Z”—everything you normally write in documentation—you can also embed runnable code examples. These code examples serve a dual purpose: they help people reading the documentation understand how to use the API, and they are also executed by `cargo test` to ensure the examples remain correct as the codebase evolves.

Insight

The professor demonstrated `cargo fmt` live by deliberately messing up the formatting of a file, then running the command to instantly restore it. He explained the rationale: “The opening brace goes on the same line, not on a new line, because vertical space is limited. I can keep my code more compact, which means I can see and understand more lines at once. When you put the opening brace on a new line, you eat a row for nothing.”

Warning

`cargo run` only works for executable projects. If your project is a library (created with `cargo new --lib`), `cargo run` will fail because there is no `main` function—“someone else needs to call it.”

Live Terminal Workflow Demonstration

The professor demonstrated the complete build-and-run cycle in a terminal session. This workflow is essential to internalize:

```
# Step 1: Compile the project
$ cargo build

# Step 2: Inspect the output
$ ls -l target/debug/
# ... many temporary files and folders ...
# ... r1 (the executable, named after the project)

# Step 3: Run the executable directly
$ ./target/debug/r1
Hello, Rust!

# Step 4: Or, more conveniently, use cargo run
$ cargo run
Hello, Rust!
```

The key insight from the demonstration was how `cargo run` detects staleness: “If I change my source file and then rewrite `cargo run`, it says: ‘Oh wait. There’s a discrepancy. There’s already an executable, but it’s older than your source file. Wait a minute. I’ll recompile the source, redo the executable, and then launch it.’ This is the reason why we don’t call the compiler by hand, because it’s much more convenient.”

3.3 IDE Setup and the Cargo.toml File

The professor demonstrated using **RustRover**, a JetBrains IDE available free to Politecnico di Torino students through the JetBrains academic license (renewable annually while enrolled). Alternatively, **Visual Studio Code** with the `rust-analyzer` plugin works well.

Insight

The professor noted that if using Visual Studio Code, you need to install the Rust plugin (`rust-analyzer`) which provides syntax highlighting, error detection, and other essential features: “If you use Visual Studio Code, you have to install the relative plugin for Rust compilation, which colors the syntax correctly and does all these things.”

The project he created was called `r1` (“the first Rust project we’re doing”). After creation, the interface shows:

- The `src/` folder with `main.rs`
- The `target/` folder (created after the first build)
- The `Cargo.toml` file describing the project

3.3.1 The Cargo.toml File

In its minimal form, `Cargo.toml` contains:

```
[package]
name = "r1"
version = "0.1.0"
edition = "2024"
```

```
[dependencies]
```

Rust Editions

Rust releases **macro editions** every three years: 2015 (the first), 2018, 2021, and now 2024 (which will last until 2027). Each edition can introduce syntactic changes. Separately, the compiler itself receives a new release **every six weeks**. The professor noted with amusement: “In the entire history of Rust, from 2015 to now, there was only one time when it wasn’t exactly six weeks—because of some political mess in a country, there was a difference of one day. But otherwise, for 11 years, every six weeks, a new version.” Each new compiler version is fully backward-compatible and adds features that have been incubated and stabilized over a long period.

Insight

The professor emphasized the stability guarantee: new features added to Rust are “pieces that have been incubated for a long time before being approved. This is to guarantee to the developers that when they start using a new feature, that one is a stable feature.” This long incubation period is what allows Rust to maintain backward compatibility while still evolving.

Adding Dependencies

The `[dependencies]` section is where you declare external libraries. For example, to use `clap` (a library for command-line argument parsing):

```
[dependencies]
clap = "2.34.0"
```

The moment you save and run `cargo build`, Cargo automatically fetches the library from `crates.io` (the official Rust package registry) and places it in the project. The professor demonstrated this live, and the IDE immediately underlined the dependency with a warning: “You’ve added it, but you’re not using it. What did you add it for?” This illustrates how the Rust compilation ecosystem actively helps you keep your project consistent—unused dependencies produce warnings.

The professor also showed that when typing the dependency name, the IDE auto-suggests available versions: “Starting to write, he suggests to me: ‘Look, we are at version 2.34.0.’ Wonderful.” The library is downloaded from `crates.io` and placed in the project’s “external libraries” folder.

3.4 Rust Terminology: Crate, Module, Package

Working with Rust introduces terminology that differs from what you may be used to in Java or Python.

Term	Meaning
Crate	The unit of compilation. A crate can be a standalone executable binary or a library. It is fundamentally “a compilable module.” An executable crate is standalone—you can run it as-is. A library crate offers methods that someone else will call. Both fall under the generic term “crate.”
Module	A confined portion of Rust code within a crate. Analogous to a Java package. Modules serve two purposes: (1) partitioning the codebase into simpler, organized parts, and (2) acting as the unit of encapsulation . Everything inside a module is visible to other items in the same module. To make something accessible <i>outside</i> the module, you must prefix it with the pub keyword.
Package	Your entire project. Created by cargo new , described by Cargo.toml , and containing one or more crates.

Insight

The professor compared modules to Java packages: “When you create a package in Java, you partition your classes into subpackages for convenience, to keep related things close together. In Rust, the module does the same, but it is also the unit of encapsulation. Everything not preceded by **pub** is visible only inside the module.”

Modularity in Rust is expressed in the file system. Each source file in the **src/** directory can become a module. You can also create modules directly inside a single source file, but the professor noted “it’s not a great idea” for larger projects. When you want to divide your codebase into multiple files, each file becomes a module that must be named and imported—similar to how in Java you use **import** to access classes from another package.

3.5 Project Physical Structure on Disk

When you execute **cargo new my_package**, the following structure is created:

```
my_package/
|-- Cargo.toml
|-- src/
    |-- main.rs
```

The professor described the minimal contents: “Inside the **src** folder, you find **main.rs**, which is the entry point of our program. And there is a file **Cargo.toml**, which describes the project. You can eventually add other files in **src/** that will become your modules.”

After running **cargo build**, a **target/** folder appears:

```
my_package/
|-- Cargo.toml
|-- src/
    |-- main.rs
|-- target/
    |-- debug/
        |-- my_package      (executable on Mac/Linux)
        |-- my_package.exe  (executable on Windows)
        |-- ... (temporary compilation files)
```

On Mac or Linux, the executable has no extension; on Windows, it has **.exe**. If instead you run **cargo build --release**, a **target/release/** folder is created with the optimized binary.

3.5.1 Debug vs. Release Builds

	Debug Build	Release Build
Command	<code>cargo build</code>	<code>cargo build --release</code>
Output directory	<code>target/debug/</code>	<code>target/release/</code>
Binary size	Larger (debug symbols included)	Smaller (stripped)
Runtime checks	Includes extra internal checks for diagnostics during development	All unnecessary overhead removed
Performance	Not optimized	Fully optimized
Use case	Development and debugging	Distribution to users

The professor summarized the workflow: “You develop using debug builds, always, until you have enough confidence that your program meets the required characteristics. At that point, you build the release version to distribute the executable in its optimal format.”

3.6 Your First Rust Program: Functions, Macros, and Printing

3.6.1 The Entry Point: `fn main()`

Functions in Rust are introduced by the keyword `fn`, followed by a name and optionally parameters. The entry point of every executable Rust program is `main`:

```
fn main() {
    println!("Hello, Rust!");
}
```

In this case, `main` has no parameters and returns nothing. In C you would write `void main()`. In Rust, if you want to be explicit, you can write the return type as `-> ()`, which reads as “arrow unit”—a type that contains nothing, representing the absence of value. When a function does not return anything, you can either write `-> ()` or simply omit the return type entirely; they are equivalent.

```
fn main() -> () { // Explicit "returns nothing"
    println!("Hello, Rust!");
}
```

The return value of a function is expressed with the arrow notation: `-> ReturnType`. For example, if `main` needed to return an `i32`, you would write `-> i32`. However, the professor immediately noted two problems with this: (1) the function body is not returning an `i32`, and (2) `main` has special constraints on what it can return.

Warning

The professor explained that `main` is special: “This confuses the compiler with the fact that it *knows* `main`, because `main` is the entry point of our program.” The `main` function has restrictions on its return type—it can return `()`, `Result<(), E>`, or certain other specific types, but not arbitrary types like `i32`.

3.6.2 Macros vs. C Preprocessor

The `println!` call has an exclamation mark `!` after the name. This signals that it is a **macro**, not a regular function. The professor drew a detailed comparison with C:

In C, you can write `#define NULL ((void*)0)`. The C preprocessor, before the compiler processes anything, finds the symbol `NULL` and replaces it with its expansion—raw text substitution. The professor described this limitation: “The C preprocessor operates at the level of

character sequences. It searches for words and replaces them. It’s like doing Find and Replace in your text editor—if the resulting sentence doesn’t make sense, too bad.”

Rust macros are fundamentally different. They have **full visibility of the syntax tree** and can only generate **syntactically correct code**. The professor emphasized: “The substantial difference is that macros in Rust have full visibility of the syntax and can only generate syntactically correct code. The C preprocessor operates at the level of character sequences and can replace characters with others without understanding what they mean.”

The professor added a reassuring note for students: “The code we write is not necessarily what the compiler compiles. This should not surprise any of you who have done a minimum of things in C”—referring to how the C preprocessor already transforms code before compilation, but Rust macros do it in a much safer, syntax-aware manner.

Warning

Do not confuse Rust macros (invoked with `!`) with C preprocessor macros. Rust macros are hygienic and type-aware; C macros are blind text substitution. This is a frequent exam concept.

3.6.3 Printing: `print!`, `println!`, `eprint!`, `eprintln!`

Every program has two output streams available:

- **Standard output** (`stdout`): for messages destined for the user.
- **Standard error** (`stderr`): for communicating errors.

Macro	Stream	Newline?
<code>print!</code>	<code>stdout</code>	No
<code>println!</code>	<code>stdout</code>	Yes
<code>eprint!</code>	<code>stderr</code>	No
<code>eprintln!</code>	<code>stderr</code>	Yes

The professor explained why both streams exist: “When we automate our program in a Bash script, we have the possibility to look at what happens in `stdout` versus `stderr`, to decide whether to continue the pipeline or not. This is part of the general conventions of how a program behaves in the context of an operating system with scripting.”

When running from the console, `stdout` and `stderr` are both displayed on the same terminal, so they appear indistinguishable. But if you redirect output with pipes, you would see them separately.

Insight

The professor clarified the behavior of `print!` (without “`ln`”): “If I call `print!` multiple times, things appear one next to the other”—i.e., without newlines. This is useful when you need to build output incrementally on the same line.

3.6.4 Format Placeholders: `{}` and `{:?}`

Inside the format string, curly braces `{}` act as placeholders that are filled by subsequent arguments:

```
fn main() {
    let i = 42;
    println!("{}", i);           // Prints: 42
}
```

```
println!("{}", 3.14, i); // Prints: 3.14 42
}
```

The placeholders are positional: the first `{}` is filled by the first argument after the format string, the second `{}` by the second argument, and so on. The professor demonstrated the macro’s syntax awareness: “If you put two expansion points `{}` but only provide one value after the format string, the macro immediately marks it with a red error—‘two expansion points but only one value.’ This is the macro enforcing our syntax.”

Insight

The professor compared Rust’s two formatting families to Java’s `toString()` mechanism. In Java, all objects inherit `toString()` from `Object`, but most implementations produce unhelpful output like `ClassName@hashCode`. Rust divides values into two families:

- **Display** (`{}`): A clean, user-facing representation. Used for types that offer a polished way to show themselves.
- **Debug** (`{:?}`): A programmer-facing representation. Used for inspection and debugging, available for types that may not have a user-friendly display.

The professor noted: “The number doesn’t change—it’s the same. But I can use different notation inside the braces.” For simple types like numbers, both `Display` and `Debug` produce the same output. For complex types, the distinction becomes critical.

3.7 Variables, Mutability, and the Principle of Minimum Privilege

3.7.1 Introducing Variables with `let`

Variables in Rust are introduced by the keyword `let`:

```
let i = 42;
```

If you provide an initialization value, the compiler can **infer the type** automatically. In this case, `i` is inferred to be `i32` (the default integer type). You can also specify the type explicitly:

```
let i: i32 = 42;
```

3.7.2 Immutability by Default

Variables introduced with `let` are **immutable by default**. Attempting to reassign them produces a compile-time error:

```
let i = 42;
i = i + 1; // ERROR: cannot assign twice to immutable variable 'i'
```

The professor demonstrated this live: “I wrote `i = 42` and then tried `i = i + 1`. It becomes underlined with red. The error says: ‘Cannot assign a new value to an immutable variable.’ Since we said just `let`, for Rust, `i` is immutable. It’s worth 42, always. Don’t change it.”

To allow mutation, you must explicitly add the `mut` keyword:

```
let mut i = 42;
i = i + 1; // This is fine
```

! Exam Priority

The Principle of Minimum Privilege: The professor stated this principle clearly: “If you don’t need to do something, don’t do it. And to be sure you don’t do it, make it impossible by default.” Variables start at the minimum privilege level (immutable). If you need more power (mutability), you must ask for it explicitly with `mut`. This prevents you from having powers you don’t exercise with awareness, because “powers you wield without awareness are dangerous—you use them without realizing it.”

Expected Mastery Level: Very High. You must reflexively know when to use `let` versus `let mut`.

3.7.3 Deep Immutability: The Difference from Java’s `final`

The concept of immutability in Rust is **much deeper** than what Java’s `final` provides. The professor used a concrete comparison:

In Java, `final` means **non-reassignable**, not immutable. You can write:

```
final ArrayList<Integer> al = new ArrayList<>();
// al = new ArrayList<>(); // ERROR: cannot reassign
al.add(27);                // FINE: can modify contents!
```

The `final` keyword prevents you from assigning a different object to `al`, but it does *not* prevent you from calling `al.add(27)` to modify the contents.

In Rust, if you declare the equivalent (a `Vec`) without `mut`:

```
let v: Vec<i32> = Vec::new();
// v = Vec::new(); // ERROR: cannot reassign
// v.push(42);      // ERROR: cannot mutate - v is immutable!
```

The professor demonstrated this: “Not only can you not reassign `v` to a different vector—you’ve frozen it to be *this* vector—but you also cannot tamper with it, cannot do things that change its internal content, because `v` is immutable and frozen there.”

To be able to push elements, you must declare the vector as mutable:

```
let mut v: Vec<i32> = Vec::new();
v.push(42); // Now this is fine
```

Warning

Rust’s `let` $\neq$ Java’s `final`. Java’s `final` prevents reassignment but allows content modification. Rust’s `let` (without `mut`) freezes the entire data structure—neither reassignment nor internal mutation is allowed. This distinction is a classic exam pitfall.

3.7.4 No `++` or `--` Operators

Rust deliberately omits the pre-increment (`++i`) and post-increment (`i++`) operators. The professor explained why: historically, the confusion between pre-increment and post-increment has been a significant source of bugs in C/C++ codebases. “People forget it after a while. When they do `a = i++` they start making mistakes, because in a superficial reading you say ‘yes, it’s one more’ but you don’t think carefully about whether the increment happens before or after the use.” This has been shown to be a significant source of bugs in historically analyzed codebases.

Rust’s decision: remove them entirely. To increment a value:

```
let mut i = 0;
i += 1; // Clear, explicit, unambiguous
```

The professor noted this “makes it obvious what you’re doing, and especially prevents you from accidentally assigning the result to someone else.” He specifically advised writing the increment on a separate line to keep code clear and avoid confusion.

3.8 Blocks, Semicolons, and Expressions

Rust is an **expression-oriented language**. Almost everything evaluates to a value. This includes constructs that in C or Java are mere statements: `if`, `loop`, and even bare blocks.

3.8.1 Scoping with Blocks

You can enclose code in curly braces to create a new scope. Variables declared inside the inner block are limited to that block’s visibility, as in most other languages:

```
fn main() {  
    let x = 10;  
    {  
        let y = 20;  
        println!("{}", x + y); // Fine: both x and y visible  
    }  
    // println!("{}", y); // ERROR: y is not visible here  
}
```

3.8.2 The Semicolon Rule

The semicolon has a crucial semantic role in Rust. The professor explained it this way:

- **With semicolon:** The value produced by the expression is **discarded**. The line becomes a statement.
- **Without semicolon** (on the last line of a block): The value produced becomes the **return value of the block**.

```
fn square(a: i32) -> i32 {  
    a * a    // No semicolon: this value IS the return value  
}
```

The professor walked through this: “The body is simply `a * a`, without a semicolon and without `return`. It recalculates this multiplication and considers this value—since there is no semicolon—as the value of the block. And what is this block? The block of the function. Therefore, the value of the function.”

Insight

The return keyword is optional. The professor deliberately highlighted that this function uses “no semicolon *and* no `return`.” In Rust, the idiomatic way to return a value from a function is to make it the last expression without a semicolon. The `return` keyword exists but is typically reserved for early returns (exiting a function before the last line). Writing `return a * a;` would also work but is not considered idiomatic Rust style.

Warning

The semicolon can only be meaningfully omitted on the **last line** of a block. If you had more instructions after it, those wouldn't execute (the function would already have a return value). So the semicolon is used on most lines; on the last line, it depends on whether you want to return a value.

3.8.3 If-Else as an Expression

Since `if-else` produces a value, you can use it directly in assignments:

```
let mut v: Vec<i32> = Vec::new();
v.push(5);

let i = if v.is_empty() { 0 } else { 1 };
println!("{}", i); // Prints: i = 1
```

Note that the values 0 and 1 inside the branches do *not* have semicolons, because they are the return values of the respective blocks. The overall `if-else` expression produces a value that gets assigned to `i`.

The professor demonstrated this live, building the example step by step: “I created the vector, I put something in it, then I have an `if`. And I say: if the vector is empty, put 0; else, put 1. Note that 0 and 1 don't have the semicolon, because I need them as a return value and, entering this pattern, I will assign the result to `i`.”

3.9 Primitive Types and Numeric Representation

3.9.1 The Problem with C's `int`

The professor motivated Rust's type system by contrasting it with C. In C, the standard defines `int` but does not define its size precisely—it depends on the platform. “You don't know how short it is, how long it is. This has brought big problems in writing compatible programs across platforms.” To work around this, C programmers often use `int32_t` from `<stdint.h>`, which is a `typedef` that guarantees exactly 32 bits. The professor gave the full picture: “Often, programmers don't use the basic `int` type. They replace it with `int32_t`, which is a define that easily allows you to modify the architecture and say ‘I want an 8-bit integer, I want an unsigned 8-bit integer.’ This is how you do things in C to make them multi-compilable on a different platform without going crazy.”

Rust solves this problem at the root: all numeric types have **explicit, fixed sizes**.

3.9.2 Integer Types

Type	Size	Range
i8	8 bits	−128 to 127
u8	8 bits	0 to 255
i16	16 bits	−32,768 to 32,767
u16	16 bits	0 to 65,535
i32	32 bits	≈ −2 billion to +2 billion
u32	32 bits	0 to ≈ 4 billion
i64	64 bits	very large range
u64	64 bits	very large range
i128	128 bits	enormous range
u128	128 bits	enormous range
isize	architecture-dependent	32 or 64 bits
usize	architecture-dependent	32 or 64 bits

The default integer type, when no annotation is given, is `i32`—“enough for most operations.” You can force a specific type with a suffix:

```
let a = 42;           // i32 (default)
let b = 42u32;        // u32 (unsigned 32-bit)
let c = 42u8;         // u8  (0 to 255)
```

The professor demonstrated the suffix notation live: “If we want, we can add a suffix. `u32` only—and it automatically converted it into a 42 unsigned. You want to do it as a byte? `u8`. Okay, this one goes from 0 to 255.”

3.9.3 Floating-Point Types

```
let x = 1.32;          // f64 (default): 64-bit, ~16 significant digits
let y: f32 = 1.32;     // f32: 32-bit, ~5 significant digits
```

The professor noted: “`f64` represents numbers up to about 10^{306} with 16 significant digits after the decimal point. `f32` occupies 4 bytes and represents numbers up to about 10^{38} with only 5 significant digits in exponential notation.”

3.9.4 Special Types: `isize`, `usize`, `bool`, `char`

`isize` and `usize`: These indicate the default dimension of the current architecture. On a 64-bit machine, `isize` is equivalent to `i64`; on a 32-bit machine, to `i32`. The professor explained why they’re useful by connecting to a fundamental concept from C: “A pointer is an integer to all effects, whose dimension depends on the dimension of the machine’s architecture. So when we need alignment or memory occupation optimized for a certain machine, we can refer to `usize` or `isize` to represent the optimal data for the current machine.” This **pointer–integer equivalence** is the reason these architecture-dependent types exist.

`bool`: The boolean type (`true` or `false`).

`char`: A character type that occupies **four bytes**. The professor emphasized: “Why four bytes? Because it can represent all Unicode values. The Unicode symbols are millions, and therefore need a 32-bit representation.”

Warning

The fact that `char` is 32 bits does **not** mean that each character in a string occupies 32 bits! Strings in Rust use **UTF-8 encoding**, which is a variable-length encoding. An ASCII character like 'A' occupies 1 byte (value 65 or 0x41). An accented letter like 'è' occupies 2 bytes. A Chinese ideogram occupies 3 bytes. An emoji occupies 4 bytes. So a string of three characters can be 3 bytes long or 12 bytes long, depending on which characters are inside.

Insight

The professor explained the UTF-8 encoding mechanism in detail: “In a single byte, there are 128 symbols of ASCII coded, plus some extension symbols. In Unicode, a first prefix is used to say ‘Look, this character is longer, the real value is in the second byte.’ And then you can find accented letters there. Chinese ideograms need three bytes. If you want to put a smiley or a cake emoji, those need 4 bytes.” This is why Rust strings have a length in bytes that is *not identical* to the length in characters.

3.9.5 The Unit Type ()

The empty parentheses () represent the **unit type**—equivalent to `void` in other languages. It represents the absence of value.

3.9.6 Underscores in Numeric Literals

For readability, you can use underscores as visual separators in numeric literals:

```
let big = 1_000_000;           // Same as 1000000
let weird = 1_0_0_0_0_0_0;    // Also valid, but don't do this
```

The underscore is simply ignored by the compiler. You can place it anywhere within the number, not just in groups of three. “You can put it 27 times in a row if you want. It’s useful just for readability.”

3.9.7 Type Annotations and Type Inference

Variables can have their type explicitly annotated with : `Type` after the name:

```
let v: i32 = 123;
let mut w = v;    // w inferred as i32 (same as v), mutable
```

The compiler can usually infer the type from the initialization value. However, in some cases inference is impossible. The professor gave the example of an empty vector: “All empty vectors look the same. How can the compiler know if it’s an empty vector of integers, or strings, or something else? In that case, you must tell it.”

```
let v: Vec<i32> = Vec::new();    // Must specify type
let v2 = vec![5, 6, 7];        // Inferred as Vec<i32>
```

The professor elaborated on the inference for initialized vectors: “If I had initialized the vector with numbers, say `vec![5, 6, 7]`, the compiler automatically sees that I put in 5 and 6 and 7, and says ‘oh well, it’s an integer vector, fine.’ ”

Insight

The professor showed an important subtlety about type annotations and immutability working together:

```
let v: i32 = 123;
// v = -5;           // ERROR: v is immutable
let mut w = v;       // w gets value 123, but w is mutable
w = -5;              // Fine: w is mutable
```

The type of `w` is inferred from `v` (i.e., `i32`), and since `w` is declared with `mut`, its value can change. This demonstrates how type inference and mutability are orthogonal concepts.

3.10 Expressions Produce Values

The professor emphasized that in Rust, “all expressions produce a value.” This is different from C and Java, where many constructs (like `if` and `while`) do not produce values. In Rust, even `loop` is an expression that can return a value via `break`:

```
let mut counter = 0;
let result = loop {
    counter += 1;
    if counter == 10 {
        break counter * 2; // Loop returns 20
    }
};
println!("result = {}", result); // Prints: result = 20
```

The professor explained the `break` mechanism: “That `break` there not only interrupts the infinite cycle, but allows me to indicate what the value is that the loop is returning.”

3.10.1 Loops: `loop`, `for`, `while`

`loop` creates an infinite loop. You exit with `break`, optionally returning a value.

`for` iterates over iterable objects:

```
let array = ["a", "b", "c"];
for s in array {
    println!("{}", s);
}
```

An iterable object is something that can be converted into an **iterator**—“something that allows you to go through pieces one after the other.” The professor noted: “We need an iterable object. That is, an object that can be converted—whether it is already an iterator or convertible into an iterator. What is an iterator? We will learn it, but intuitively, it is something that allows us to go through the pieces one after the other.” He promised to explain iterators in depth in a later lecture.

`while` works as in other languages, looping while a condition holds:

```
let mut n = 0;
while n < 10 {
    n += 1;
}
```

3.11 Pattern Matching with match

The professor introduced `match` as the most distinctive and powerful control-flow construct in Rust. He started from what students might already know: “If you know the `switch` statement in C or Java—`switch` allows you to choose based on a value. `match` does something much more powerful: it lets you choose based not only on *value* but also on *shape*.”

He expanded on this: “When I have the possibility to ask for a match on a value, I can say: if it matches a value with one element, or two elements, or five elements, or three elements in which the second is two. All these things I can write as a condition. And manage them.”

3.11.1 Basic Value Matching

```
let item = 15;
let s = match item {
    0 => "zero",
    10..=20 => "between 10 and 20",
    40 | 80 => "40 or 80",
    _ => "other",
};
println!("{}", s); // Prints: between 10 and 20
```

This is like an enhanced `switch`: you can match on exact values (0), ranges (10..=20), alternatives (40 | 80), and a wildcard catch-all (_).

The professor described this as “a version of the Switch, a little more powerful. But the most interesting is the structural matching.”

3.11.2 Destructuring with match

The most interesting use is structural pattern matching, where you can “disassemble” a value:

```
let val: Vec<i32> = vec![5, 10];
match val.as_slice() {
    [] => println!("empty"),
    [a] => println!("one element: {}", a),
    [a, b] => println!("two elements: {} and {}", a, b),
    _ => println!("more than two elements"),
};
```

The professor explained: “If there’s nothing inside, I tell you it’s empty. If there’s only one element, not only do I tell you there’s one element, but I also tell you what it is, because I read it into the variable `a`. And if there are two, I tell you what they are.” This is pattern matching not just on values but on the *structure* of data.

Exam Focus

`match` is Rust’s most powerful control-flow construct and replaces both `switch` and many `if-else` chains. Expect exam questions asking you to write `match` expressions that destructure tuples, slices, or `Option<T>` values. The wildcard `_` and the exhaustiveness requirement (all cases must be covered) are frequently tested.

3.12 String Handling: Slices vs. Owned Strings

The professor introduced strings by connecting to students’ prior experience: “I hope you made enough effort during your algorithms course wrestling with C strings and the null terminator, and all the problems that arise when you want to insert one string into another. Only if you

went through that struggle can you understand the complexity behind this, and the need to treat it differently.”

3.12.1 Java Comparison: String vs. StringBuilder

Starting from Java (“which you might have more recent memories of”), the professor drew a parallel. In Java:

- **String** objects are **immutable**—you cannot insert or remove characters within them. You can only derive new strings.
- For mutable text manipulation, Java offers **StringBuilder** (introduced in Java 1.2) with **insert**, **delete**, **replace** methods. (There’s also the older **StringBuffer** from Java 1.0, which is synchronized and slower.)

In Rust, there is something similar, but structured differently.

3.12.2 String Slices: &str

Immutable strings in Rust are called **string slices**. Their type is **&str** and they are written as string literals in double quotes:

```
let hello: &str = "hello,";
```

A **&str** is a **fat pointer**—it contains two pieces:

1. A **pointer** to where the bytes of the string start.
2. A **length** indicating how many bytes are in the string (not characters—bytes).

Memory Model Analysis

Memory layout of `let hello: &str = "hello,";`

The string literal `"hello,"` is known at compile time and will never change. The compiler places it in a **read-only static memory segment**—a segment with read-only access, no writing allowed.

On the stack, the variable `hello` is a fat pointer consisting of two fields:

- **ptr**: pointer to the first byte of `"hello,"` in static memory
- **len**: 6 (six bytes)

Unlike C strings, there is **no null terminator**. The length is explicitly stored in the fat pointer.

The professor contrasted this with C: “In C, you have the pointer to the first character, and then you have to dig through memory to find the last one, assuming the string ends with a zero. Here, the fat pointer contains both the pointer *and* a number telling me how many bytes are inside—which is essential for efficient operations.”

3.12.3 Owned Strings: String

Modifiable strings have the type **String**. They can be:

- Created empty with `String::new()`
- Created from a literal with `String::from("hello")` or `"hello".to_string()`

- Manipulated with methods like `push_str`, `insert`, `replace`, etc.

A `String` object is stored on the stack but manages a buffer on the **heap**. Its stack representation contains **three** fields:

1. A **pointer** to the heap buffer
2. The current **length** (bytes in use)
3. The total **capacity** (bytes allocated)

3.12.4 Method Inheritance: `&str` Methods on `String`

The professor made an important point about the relationship between the methods available on these two types:

Insight

“The slice type `&str` has a plurality of methods to inspect the content of a string. All these methods are automatically inherited—they are available for `String` as well. And `String` adds to them its own unique methods, which are those for insertion, cancellation, and substitution.”

In other words, every operation you can perform on a `&str` (such as `len()`, `contains()`, `starts_with()`, etc.) can also be performed on a `String`, because `String` automatically *dereferences* to `&str`. On top of those, `String` adds mutating methods like `push_str()`, `insert()`, `replace()`, and others. This is why you can think of `String` as a superset of `&str` in terms of available operations.

3.12.5 Step-by-Step String Lifecycle

The professor walked through the complete lifecycle of a `String` with vivid personification. Let us trace it:

```
{  
    let hello: &str = "hello,";  
    let mut s = String::new();  
    s.push_str("hello,");  
    s.push_str("␣world");  
    // ... end of scope  
}
```

Memory Model Analysis

Step 1: `let mut s = String::new()`

The `String` object is created on the stack with: `ptr = (invalid)`, `len = 0`, `capacity = 0`. No heap allocation happens yet.

Step 2: `s.push_str("hello,")`

The string says (as the professor personified it): “You want to place things in me? Hmm, I don’t have any space. How many bytes do you want to put in? Six bytes. That’s not a power of two—let me round up. What’s the next power of two? Eight.” So the `String` requests 8 bytes from the heap. The operating system provides the allocation. Now:

- `ptr`: points to 8 bytes on the heap
- `capacity`: 8

The six characters of “hello,” are copied into the buffer:

- `len`: 6

Step 3: `s.push_str(" world")`

“How many bytes is ‘ world’? It’s 6 bytes. I already have 6 in use, that makes 12 total needed, but my capacity is only 8—not enough! Operating system, what’s the next power of two? 16. Operating system, do you have 16 bytes for me?” The OS provides 16 bytes on the heap. The `String` copies the old content (“hello,”) into the new buffer, appends “ world”, and frees the old 8-byte buffer. Now:

- `ptr`: points to 16 bytes on the heap
- `len`: 12
- `capacity`: 16

Step 4: End of scope (closing brace)

When the scope ends, Rust automatically calls the `drop` method on `s`. The `String` “takes the trouble to go to the operating system and say: ‘Dear operating system, you gave me 16 bytes—I don’t need them anymore, I’m returning them.’” The heap memory is freed. Then `hello (&str)` goes out of scope. It only held a static pointer—no dynamic resources to clean up. “No problem. We can throw this away without doing anything. Flup!” The stack is cleaned (obvious), and the heap is also cleaned (less obvious). **Automatically.**

Insight

The professor described the `drop` mechanism with a metaphor of a testament: “The `String` knows it has taken memory from the operating system and knows it must restore it before it dies. In its little testament, it says: ‘I leave instructions to return to the operating system whatever it previously gave me.’” This is the Rust equivalent of C++’s RAII pattern—deterministic, automatic resource cleanup without garbage collection.

3.12.6 Converting Between `&str` and `String`

Direction	Method	Notes
<code>&str</code> → <code>String</code>	<code>String::from("text")</code> or <code>"text".to_string()</code>	Creates a new heap-allocated copy
<code>String</code> → <code>&str</code>	<code>s.as_str()</code> or implicit <code>&s</code>	Borrows as immutable slice, no copy

The professor noted: “It is possible to view a mutable `String` as an immutable `&str`—you freeze it temporarily and say ‘don’t touch it, just look at it.’ But you cannot do the reverse: from an immutable string, you can only create a *new* mutable copy.”

The professor also highlighted that the two conversion methods from `&str` to `String` are equivalent: `String::from("rust")` and `"rust".to_string()` are equivalent—in both cases, starting from this sequence of characters, I generate a `String` object that contains a copy of those characters, but it is a mutable copy, a copy in which I can insert, cancel, and replace pieces.”

3.12.7 The Full Menagerie of String Types

The professor warned that strings in Rust are “even more elaborate” than what he showed. Beyond `&str` and `String`, Rust also has:

Type	Purpose
<code>&[u8]</code>	A byte slice. Not quite a string, but very close.
<code>&[u8; N]</code>	A fixed-size byte array reference.
<code>Vec<u8></code>	An owned vector of bytes.
<code>OsStr</code> / <code>OsString</code>	Strings as the operating system expects them. On Windows, for example, the OS wants 16-bit characters internally (UTF-16), so when calling Windows system calls you must convert to <code>OsStr</code> .
<code>Path</code> / <code>PathBuf</code>	File system paths. Always appear to be strings but with special semantics—segmented by <code>/</code> (Unix) or <code>\</code> (Windows).
<code>CStr</code> / <code>CString</code>	Null-terminated C-style strings, needed when calling C libraries via FFI.
<code>&'static str</code>	String slices with a lifetime guaranteed for the entire duration of the program.

The professor also mentioned `&u8`—a simple reference to a single byte—as yet another related type in the string/byte ecosystem.

The professor’s conclusion: “What seems like a banality is not a banality. Behind it is a whole series of distinctions. We’ll encounter these as we go deeper into lifetimes and FFI.”

Insight

The professor specifically explained the need for `Path/PathBuf`: “It is always apparently a string, but a string with a particular semantic and segmentable by slash or backslash, or in other operating systems, by vertical bars or other things.” And for `CStr/CString`: “The strings coded in the way that C codes them, with the final zero. I need them when I have to call a C library.” Understanding *why* each type exists is more important than memorizing them all.

! Exam Priority

The distinction between `&str` (immutable fat pointer: `ptr + len`, pointing to static or borrowed data) and `String` (owned heap data: `ptr + len + capacity`) is **almost guaranteed to appear on the exam**. You must be able to draw the memory layout of both and trace the lifecycle of a `String` through allocation, growth, and deallocation.

Common Pitfall: Confusing byte length with character count. A string of 3 characters can be 3 to 12 bytes long depending on the characters.

Expected Mastery Level: Very High.

3.13 Vectors: `Vec<T>`

The other “fast object” the professor introduced is `Vec<T>`—a generic, dynamically-sized sequence of elements.

3.13.1 Creating and Using Vectors

`Vec<T>` is a generic type: the `T` is a placeholder for any type. The professor illustrated this versatility: “I can make number vectors, string vectors, vector vectors, list vectors, file vectors, etc. That `T` becomes the type of element that the vector contains.”

A critical property of `Vec<T>` is **homogeneity**: the vector encloses within itself a sequence of **homogeneous elements of type `T`**. All elements must be of the same type.

```
let mut v: Vec<i32> = Vec::new(); // Empty vector of i32
v.push(42); // Add to the end
v.push(99);
let last = v.pop(); // Remove from end: Some(99)
```

If the vector is mutable, it can grow and shrink. `push` adds to the end; `pop` removes from the end.

3.13.2 Indexing and Safe Access

You can access elements by index with square brackets:

```
let v = vec![10, 20, 30, 40];
let third = v[2]; // 30 (zero-indexed)
// let oob = v[10]; // PANIC! Index out of bounds
```

The professor explained indexing: “`v[3]` means ‘I assume that vector has at least four elements, and I access the fourth element.’ `v[0]` accesses the first element. If the element is not there, it generates a so-called panic condition.”

If the index is out of bounds, this generates a **panic**—the program crashes immediately.

To avoid panics, use the `get` method (for read-only access) or `get_mut` (for mutable access). These return an `Option<T>`:

```
let v = vec![10, 20, 30];
match v.get(5) {
    Some(val) => println!("Found: {}", val),
    None => println!("Index out of bounds!"),
}
```

Insight

The professor explained `Option`: “`get` and `get_mut` don’t give you the element immediately. They wrap it in an `Option`. What is an `Option`? It’s something that can contain either the value (`Some`) or nothing (`None`). And it’s your job to check which one it is.” This is Rust’s replacement for `null`—instead of risking a null pointer, you are *forced* to handle the case where the value might be absent.

The distinction between `get` and `get_mut` reflects whether you want to **read** or **write** the element at that position. The professor made this explicit: “So I can distinguish what I want to do. I want to read or I want to write in that vector. `get` or `get_mut`.”

3.14 Shadowing

Occasionally you may want to reuse a variable name while changing its type or making an immutable copy. Rust permits **shadowing**:

```
fn main() {
    let x = 5;
    let x = x + 1;    // Shadows the previous x

    {
        let x = x * 2; // Shadows in inner scope
        println!("Inner_x: {}", x); // 12
    }

    println!("Outer_x: {}", x); // 6
}
```

Memory Model Analysis

Each `let x` allocates a **new** stack slot. The previous `x` is effectively hidden (shadowed) but still exists in memory until its scope ends. The inner scope creates yet another stack slot. When the inner block closes, the inner `x` is dropped, and the outer `x` (value 6) becomes visible again. No heap allocation or ownership transfer occurs here—these are primitive integers on the stack.

Insight

Shadowing is **distinct from mutability**. When you shadow a variable, you are declaring a completely new variable that happens to share the same name. This allows you to change the type or enforce immutability after a transformation. It is commonly used in Rust to avoid inventing arbitrary names like `x_str` or `x_num`.

3.15 Reading from Console, Files, and Listing Directories

The professor briefly mentioned that the lecture slides contain concrete examples of:

- Reading from the console (standard input)
- Reading a file line by line
- Writing to a file

- Listing all files in a directory

These examples would be used in the laboratory exercises the following day. While the professor did not walk through them in detail during the lecture, he indicated that they provide the starting point for hands-on exercises: “In the slides that follow, you have a series of concrete examples—how I read from the console, how I read a file line by line, how I write to a file, how I list all the files of a directory. Tomorrow you will use them. So here you have the examples from which to start to be able to do the exercises.”

Exam Focus

While the file I/O examples may not be heavily tested on their own, understanding how Rust handles `Result<T, E>` (for error handling during I/O) and how iterators work with file lines is foundational. These concepts build upon the `Option<T>` pattern introduced with vectors.

Summary: Key Comparisons

Concept	C / Java	Rust
Integer sizes	C: <code>int</code> has platform-dependent size. Java: <code>int</code> is always 32-bit.	Explicit sizes: <code>i8</code> , <code>i16</code> , <code>i32</code> , <code>i64</code> , <code>u8</code> , etc.
Null values	C: <code>NULL</code> (a <code>(void*)0</code> macro). Java: <code>null</code> reference.	No <code>null</code> . Uses <code>Option<T></code> (<code>Some/None</code>).
Immutability	C: <code>const</code> (advisory). Java: <code>final</code> (prevents reassignment only).	<code>let</code> = deep immutability. <code>let mut</code> = explicit opt-in to mutability.
Increment operators	C/Java: <code>++i</code> , <code>i++</code> (source of subtle bugs).	Removed. Use explicit <code>i += 1</code> .
String termination	C: null-terminated (<code>'\0'</code>).	Fat pointer: pointer + length. No null terminator.
String mutability	Java: <code>String</code> (immutable) vs. <code>StringBuilder</code> (mutable).	<code>&str</code> (immutable slice) vs. <code>String</code> (owned, mutable).
String methods	C: manual functions (<code>strlen</code> , etc.). Java: separate method sets on <code>String</code> and <code>StringBuilder</code> .	<code>&str</code> methods are inherited by <code>String</code> ; <code>String</code> adds mutating methods.
Macros	C: preprocessor text substitution (<code>#define</code>).	Syntax-tree-aware, hygienic, type-checked macros (<code>!</code>).
Switch / match	C/Java: <code>switch</code> on values only.	<code>match</code> : value + structural pattern matching with exhaustiveness.
Return values	C/Java: explicit <code>return</code> keyword required.	Last expression without semicolon is the return value; <code>return</code> is optional.

Exam Relevance and Recurrence

Score: Very High

Notes: This chapter covers the foundations upon which *every* subsequent Rust topic is built. Exam questions from this material typically fall into these categories:

1. **Memory Layout questions:** Draw the stack/heap layout for a `&str` and a `String`. Trace the lifecycle of a `String` through push operations and scope exit.
2. **Theoretical Concept questions:** Explain the Principle of Minimum Privilege. Compare Rust immutability with Java's `final`. Explain why Rust has no `null`. Describe the difference between Rust macros and C preprocessor macros.
3. **Code Prediction questions:** Given a snippet with `let` vs `let mut`, predict whether it compiles and what it prints. Given a `match` expression, determine the output.

3.16 Domande ed Esercizi da Esami Passati

Nessuna domanda d'esame passata mappata direttamente su questo capitolo.

Chapter 4

Ownership and Borrowing

The way a programming language manages memory fundamentally defines its character, performance, and safety guarantees. In languages like C and C++, memory management is manual: the programmer must explicitly request and release memory. While this offers maximum control, it inevitably leads to complex bugs such as *dangling pointers* (using memory after it has been freed), *double frees* (freeing the same memory twice), and memory leaks. On the other end of the spectrum, languages like Java or Python use a Garbage Collector (GC), which automatically tracks and frees unused memory at runtime. This removes the burden from the programmer but introduces performance overhead and unpredictable pauses.

Rust introduces a paradigm-shifting alternative: **Ownership**. By shifting memory management checks to compile-time via a component known as the *Borrow Checker*, Rust achieves the performance of manual memory management without its inherent dangers. The programmer is forced to explicitly consider the lifespan and ownership of every value during the writing phase. While this may initially feel like a “difficulty gradient,” it acts as a massive safety net. As the professor emphasized: ownership, borrowing, and lifetimes are concepts that intrinsically belong to programming—they exist in every language—but Rust is the first to make them *explicit*. Other languages allow the programmer to describe things that break at runtime because they never force this awareness. Rust’s compiler *rejects* programs that do not satisfy these rules.

Insight

The Red Light Analogy

The professor compared ignoring ownership rules to running a red light: *“It doesn’t automatically mean getting hit by a truck. Sometimes you get lucky. And so you develop the false awareness that your program works—but in reality, it is intrinsically broken. It will break at the worst possible moment.”* In Rust, the code simply will not compile if memory safety rules are violated, preventing you from ever reaching that dangerous intersection.

Insight

The Hypercorrectivity Analogy

The professor explained why bugs escape programmers’ attention with a linguistic analogy: *“We are naturally hypercorrective. When we read a text, we give it a quick look, intuit what should be written, and if it mentally ‘wires’ correctly, we are happy. People don’t notice missing accents or misplaced apostrophes—their brains auto-correct. The same happens with code: we glance at it, we intuit that it should work, and we move on. We don’t have the mental strength to pay attention to every detail.”* The Borrow Checker, unlike the human mind, does not suffer from hypercorrectivity—it meticulously checks every single path.

4.1 The Core Concept of Ownership

The foundation of Rust’s memory model rests on three deceptively simple rules:

1. Every value in Rust has a variable that is called its *owner*.
2. There can only be one owner at a time.
3. When the owner goes out of scope, the value will be dropped (its memory and resources are freed).

Note the choice of subject and object: *the value is possessed by a variable*. If a value exists, it must reside somewhere in memory—a value without representation ontologically cannot exist. The memory in which that value resides *is* the variable. A particular component of the compiler, the **Borrow Checker**, verifies this at every point of your program during compilation. It reads your code with one specific objective: is there any point where the programmer behaves as if a value is not possessed by exactly one variable? If it finds such a point, it blocks compilation and explains exactly why.

4.1.1 What Does “Possessing a Value” Mean?

Possessing a value means being **responsible for its release**. The professor made this crucial distinction: the problem is not what you do with a value while you are using it—you do whatever you want. The problem is: *when you no longer need it, you must ensure that the memory (and any associated resources) returns to its legitimate owner*.

Sometimes the cleanup goes beyond simply releasing memory. Consider opening a file: the operating system builds internal data structures—tables that map the file handler to disk locations, buffers for I/O operations, and so on. The value you hold is just a small number (a file handler). Beyond releasing the memory where that small number lives, you must also ensure the operating system cleans up all the information it is keeping associated with that handler. The file must be *closed*. The same principle applies to network sockets, database connections, and any other system resource.

Insight**The House Ownership Analogy**

During a Q&A session, the professor illustrated ownership with a vivid analogy: *“The house in which you live—whose owner is it? The landlord’s. But you can live in it. Why? Because you have a contract. You have permission to be there. You don’t own the house—you are borrowing it.”* This maps directly to Rust’s reference system: variables can grant temporary access (references) to their values without transferring ownership. But the owner remains the one who is responsible for the “house” when the “tenant” leaves.

Insight**The Differentiated Waste Collection Analogy**

Possessing a value in Rust means being responsible for its disposal. Think of it like differentiated waste collection: *“What do I do with paper? I put it in the yellow bin. What do I do with glass? In the blue bin. What about electronics, or hazardous materials like paint residues? Those require a trip to the eco-center.”* Based on the typology of what must be disposed of, different operations are required. Similarly, some data types have trivially simple disposal (just pop the stack), while others require complex cleanup procedures. When we introduce a type in Rust, we can declare specialized disposal instructions by implementing the `Drop` trait.

4.1.2 When Does Release Happen?

The release of a value occurs in three situations:

1. **End of scope:** When the variable goes out of scope—that is, when program execution reaches the closing brace of the block in which the variable was declared—the value is released.
2. **Reassignment:** When a variable is assigned a new value while still in scope, the *previous* value must be cleaned up first. Only then is the new value assigned. Failing to clean up the previous value would result in a resource leak.
3. **Explicit drop:** You can accelerate the release by calling `drop(variable)`. This tells Rust: “from a syntactic point of view, this variable would still be usable for the next 25 lines, but I no longer need it—drop it now.” After the call, Rust marks the variable as unusable for the remainder of its scope.

Warning**Why release matters**

Resources occupied by a variable must become available to others. If a variable occupies memory, that memory could be reused by others doing something entirely different. If a file handle is not released, the operating system accumulates garbage in its internal tables. Release is not optional—it is *mandatory* for correctness.

4.2 Ownership in Action: The Vec Example

Let us trace a complete ownership lifecycle through the professor’s detailed example of a vector.

4.2.1 `Vec::new()` vs. `Vec::with_capacity()`

Before diving into the example, the professor distinguished two ways to construct a vector:

- `Vec::new()` — creates an empty vector with *no* initial heap allocation. It starts with capacity 0 and length 0. No memory is requested from the operating system until the first element is pushed.
- `Vec::with_capacity(n)` — creates an empty vector but *pre-allocates* space for `n` elements on the heap. It starts with capacity `n` and length 0. The heap block is ready, even though no elements have been inserted yet.

The professor chose `with_capacity(4)` for the example to illustrate the reallocation mechanism clearly.

```
{  
    let mut v = Vec::with_capacity(4);  
    for i in 1..=5 {  
        v.push(i);  
    }  
    println!("{}", v);  
} // 'v' goes out of scope here
```

Step-by-step memory walkthrough:

Memory Model Analysis

Step 1 — Declaration: `let mut v = Vec::with_capacity(4);`

The stack grows by 24 bytes to accommodate the variable `v`. The `Vec` structure contains three 64-bit fields on the stack:

- An 8-byte **pointer** to a heap-allocated buffer
- An 8-byte **capacity** field (currently = 4)
- An 8-byte **length** field (currently = 0, the vector is empty)

Because we used `with_capacity(4)`, the operating system has already been asked to provide a block of 4 slots on the heap. At this moment, `v` is the *possessor* of this composite structure: a bit of stack and a bit of heap.

Step 2 — Pushing elements 1 through 4:

Each `v.push(i)` places the value `i` into the next available heap slot and increments the length counter. After four pushes: `capacity = 4`, `length = 4`. All slots are occupied.

Step 3 — Pushing element 5 (reallocation):

When we try `v.push(5)`, the vector discovers it has no more room. It *doubles* its capacity: it asks the operating system for a new block of 8 slots, copies the existing 4 elements into the new block, returns the old block of 4 to the OS, then inserts the 5th element. The stack pointer is updated to point to the new block. Capacity becomes 8, length becomes 5. If further pushes filled slots 6, 7, 8, the next push would trigger another doubling to 16 slots, and so on.

Step 4 — End of scope (the closing brace):

The stack needs to contract by 24 bytes. But `v` is not a simple variable—it has data on the heap. The `Vec` type is labeled with the `Drop` trait: there are “dismantling instructions on its box.” Rust reads the type, sees the drop label, and *before* contracting the stack, executes the specialized drop procedure. The drop logic checks: if the capacity is non-zero, then the internal pointer is valid and the heap block must be freed. In this case, capacity is 8, so 8 slots worth of heap memory are returned to the OS. Only then does the stack contract.



Figure 4.1: Memory layout of a `Vec`: stack metadata (pointer, capacity, length) and heap buffer, including the reallocation process when capacity is exceeded.

Exam Focus

Understanding the Drop Trait for Vec

The compiler knows what to do because the `Vec` type is *decorated* with drop instructions. The compiler knows the type of every variable and, consequently, when it arrives at the point where a `Vec` must be released, it applies the appropriate rules. The drop trait for `Vec` checks: “if my capacity is different from zero, the pointer in my belly is valid and must be freed.” This is the essence of **RAII** (Resource Acquisition Is Initialization) in Rust.

4.3 Transferring Ownership: Moves

Because a value can have only one owner, assigning an existing variable to a new one fundamentally changes the state of the program. This is the concept of **move**.

4.3.1 The Three Forms of Assignment

The professor identified three syntactic forms of assignment, all operationally identical:

1. **Explicit assignment:** `let b = a;` — the value moves from `a` to `b`.

2. **Passing a parameter:** `f(a)` — the value is copied from `a`’s position to the function’s parameter slot on the stack.
3. **Returning a result:** `return a;` — the value is passed from the function’s local scope to the caller.

In all three cases, when the type is not `Copy`, the *possession transfers* from the source to the destination. The source becomes inaccessible. The compiler tracks this: it will do nothing special when the source reaches its end of life (no drop instructions), and will instead execute the specialized drop on the *destination* when the destination’s life ends.

The professor observed an asymmetry in how surprising these moves feel: *“The move on return is not surprising—we don’t even get surprised that the function disappears. We get more surprised on the parameter. If I call `f(a)`, on the next line I can’t read `a` anymore.”* This surprise fades with experience, but it is the most common stumbling block for new Rust programmers.

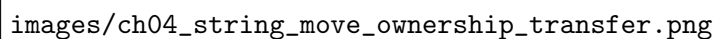
4.3.2 Detailed String Move Example

```
let mut s1 = "Hello".to_string(); // s1 owns the string
println!("{}", s1);              // Works fine
let s2 = s1;                     // Ownership MOVES to s2
// println!("{}", s1);           // COMPILER ERROR: value used after move
println!("{}", s2);              // Works fine: s2 is the new owner
```

Memory Model Analysis

Step-by-step memory trace:

1. `let mut s1 = "Hello".to_string();` — The stack grows by 24 bytes for `s1`. A `String`’s internal structure is very similar to a `Vec`: a pointer to the heap, a capacity, and a length. The `to_string()` method allocates exactly 5 bytes on the heap (“Hello” has 5 characters), initializes them, and sets `capacity = 5`, `length = 5`.
2. `let s2 = s1;` — The stack grows by another 24 bytes for `s2`. What goes into these 24 bytes? A *bitwise copy* of everything that was in `s1`—the pointer, the capacity, the length. This is a “dumb copy” (*copia beccera*), and it is extremely cheap: just a `memcpy` of 24 bytes.
3. Without the move rule, we would be in an extremely uncomfortable situation: two variables pointing to the same heap block. Who releases it? If both try, it is a double free. If neither does, it is a leak. The solution: the compiler marks `s1` as **inaccessible** from this line forward. Any attempt to read `s1` results in a compile error.
4. When `s2` reaches the end of its scope, the compiler knows it is the owner: it executes the `Drop` procedure (freeing the heap block), then contracts the stack. When `s1` reaches the end of scope, the compiler *omits* the specialized drop—`s1` still physically exists on the stack (the stack must still contract by 24 bytes), but it no longer “has a value” in the ownership sense. It is inaccessible.



images/ch04_string_move_ownership_transfer.png

Figure 4.2: Ownership transfer of a **String**: the bitwise copy of stack data, invalidation of the source, and how drop is executed only by the new owner.

Insight

How this resolves C’s pointer ambiguity

The professor emphasized that this mechanism removes one of the essential ambiguities of C pointers: *“We said pointers are ambiguous—we don’t know if they are valid, if we have to release them, or if someone else already released them. Here the compiler takes awareness and tells us: ‘Calm down, I’ll track it.’ One and only one pointer is the possessor of that block. The other one can’t touch it anymore.”*

Insight**How this prevents double frees**

The professor explicitly connected single ownership to double-free prevention: *“Since ownership belongs to one and only one variable, and only the owner releases, there is no possibility of a double release—which is another of the problems of C. Because if I have a pointer, a copy of that pointer, and someone has already freed it, and I don’t know this, I could call `free` a second time. But if we call `free` twice, in the meantime that memory has been given to others—I’ll break everything.”* In Rust, the compiler statically guarantees that exactly one entity performs the release, exactly once.

4.3.3 Reassignment After Move

A moved variable is not “dead forever.” If you assign a *new* value to it, it comes back to life:

```
let mut s1 = "Hello".to_string();
let s2 = s1;                                // s1 is moved, now inaccessible

// Read access to s1 here would fail:
// println!("{}", s1);                      // ERROR: value used after move

s1 = "World".to_string(); // s1 is REBORN with a new value
println!("{}", s1);        // Works: prints "World"
println!("{}", s2);        // Works: prints "Hello"
```

After the reassignment, `s1` has a completely new value—a fresh heap allocation containing “World”. The old “Hello” heap block belongs entirely to `s2`. When both variables reach the end of scope, `s2` will free the “Hello” block and `s1` will free the “World” block. Each pointer releases its own block, exactly once.

The professor noted: *“After the move, read access to the source will cause a compilation error, but write access—assigning a new value—will succeed and result in the variable acquiring ownership of the new thing.”*

The professor also clarified an important detail about the heap: *“When you reassign `s1` with a new value, the old heap block is not reused. A completely new arrow is born, pointing to a different location on the heap. The only way to modify the variable is to use one of the creation functions of the type, which paints all the data structure—the three fields of the `String`—with something valid. The old block was separated, so it won’t bother us.”*

4.3.4 Shadowing vs. Reassignment

The professor carefully distinguished between `let s1 = s2;` (shadowing with `let`) and `s1 = s2;` (reassignment without `let`):

- `let s1 = s2;` introduces a *new* variable with the same name. The old `s1` still physically exists on the stack but becomes inaccessible—not because it lost its value via a move, but because it is *hidden* (*shadowed*) by the new declaration. The new `s1` occupies additional space further on the stack.
- `s1 = s2;` (without `let`) performs the *reverse movement*: the value moves back from `s2` into the existing `s1` variable. Now `s2` loses possession and becomes inaccessible, while `s1` regains ownership.

Warning**Shadowing creates a new stack slot**

When you write `let s1 = s2;` inside the same scope, you are not reusing `s1`'s stack slot. The new `s1` occupies *additional* space on the stack. The old `s1` is still there, hidden. When the block ends, the stack contracts and the old one becomes syntactically visible again (if the block was an inner block). This is a subtle difference from simple reassignment.

4.3.5 Implicit Moves: The Operator Trap

Moves are not limited to direct assignments. Any operation that syntactically looks like an operator may actually be a function call that takes ownership:

```
let s = String::from("Hello");
let b = s + " world"; // IMPLICIT call to concat function!
// s is NO LONGER accessible here - it was moved into concat
println!("{}", b);    // Works: prints "Hello world"
// println!("{}", s); // COMPILE ERROR: s was moved
```

The `+` operator on strings is actually an implicit call to a concatenation function (internally `add/concat`). Because you are passing `s` as a parameter, ownership transfers to the function. From that point on, `s` is inaccessible—the heap block it was using may have been reused by the result `b`.

The professor traced the chain of possession explicitly: *“The possession of that block has potentially passed into the hands of `b`—that is, passed into the hands of `concat`, which could have been transferred to `b` in the last analysis. So `s` has no right to look inside anymore.”* Inside `s` the pointer still physically exists on the stack, but it is *inaccessible* because the heap memory it was pointing to may have been reused by the result.

Contrast this with numeric types:

```
let i = 42;
let j = i + 1; // Also an implicit function call (add)
println!("{}", i); // Works! i is still 42
println!("{}", j); // Works! j is 43
```

The difference? Integers implement the `Copy` trait. They have no special disposal instructions, no heap allocations, no ambiguity. Both `i` and `j` remain independently usable.

**Exam Priority****Implicit Moves and Reassignment**

Moves occur in three ways: direct assignment (`let s2 = s1;`), passing arguments to a function (`f(s1)`), and returning a value from a function (`return s1;`). Furthermore, implicit function calls through operators (e.g., `let b = s + " world";`) also trigger moves. Once you pass a non-copyable variable to a function, you yield ownership to that function's parameter, rendering the original variable inaccessible in the calling scope.

- **Common Pitfall:** Students often assume variables remain usable after passing them to a function or using them in an operator expression. In Rust, this causes a “value used after move” compile error.
- **Expected Mastery Level:** Very High. You must statically trace the ownership of every value. Recognize that `Copy` and `Drop` traits are mutually exclusive. `Copy` types are the *exception*, not the rule.

4.4 The Trait System: Drop, Copy, and Clone

Types in Rust carry “labels” that describe how they should be treated. These labels are called **traits**. Three traits are central to the ownership system:

4.4.1 The Drop Trait

The **Drop** trait indicates that a type requires **specialized cleanup** when it is released. For a **Vec**, this means freeing its heap buffer. For a **File**, this means closing the file handle and releasing OS resources. When a variable of a **Drop** type goes out of scope, Rust automatically calls its **drop** method before contracting the stack.

The professor noted a naming subtlety: the **Drop** *trait* (capital D) is the label/interface; the **drop()** *method* (lowercase d) is the actual destruction operation that gets called. They are related but distinct—the trait declares the capability, the method implements it.

4.4.2 The Copy Trait

The **Copy** trait indicates that a type can be **freely duplicated**—when an assignment is made, the original remains accessible. Why? Because the value has no special meaning beyond its raw bit representation. There are no heap pointers to manage, no OS resources to clean up. The copy is harmless.

The professor explained: *“The numeric types—**i32**, **f64**, understood as integers and floats on which I do multiplications and divisions—can be easily copied. There are no particular dismantling instructions. They are among those things I can toss directly into the regular trash bin. No differentiated collection needed.”*

The professor further clarified why strings cannot be **Copy**: *“The copy of the pointer bothers me because that pointer, even though it is a bit sequence like all other values, has a particular meaning—it tells me something about the heap. It possesses the block to which it points. When should I destroy the heap block? When **s1** ends its life, or when **s2** ends its life? Both seem to have the right to that thing, and it’s not okay. So the **String** is not copyable—it’s only movable.”*

4.4.3 The Mutual Exclusivity of Drop and Copy

Warning

Drop and Copy are mutually exclusive

If a type implements **Drop**, it **cannot** implement **Copy**. If it implements **Copy**, it **cannot** implement **Drop**. The compiler enforces this constraint. The logic is straightforward: if a type needs specialized cleanup (**Drop**), then having two independent copies creates ambiguity—which copy is responsible for the cleanup? If a type is trivially copyable (**Copy**), it by definition requires no special cleanup.

There is also a third category: types that are **neither Drop nor Copy**. These types have no specialized cleanup instructions, but they are not freely copyable either—for other semantic reasons. Assignment of these types still results in a move.

Type Category	Drop?	Copy?	On Assignment
i32 , f64 , bool , char	No	Yes	Bitwise copy (original accessible)
String , Vec<T> , File	Yes	No	Move (original inaccessible)
Neither Drop nor Copy	No	No	Move (original inaccessible)

4.4.4 Who Decides the Trait Labels?

The professor stressed an important point: *“A type is not Copy because the compiler decides—it is because the programmer labels it. The compiler merely verifies that the label is compatible.”* When using existing standard library types (`String`, `Vec`, `i32`, `File`, etc.), the labels are already set by the library authors. But when you introduce your own types—which happens starting from the next week of the course—it will be *you* who decides “what label to use,” and you must do it knowing what you are putting inside the type. The compiler enforces consistency (e.g., you cannot label a type both `Drop` and `Copy`), but the initial decision is the programmer’s.

4.4.5 Copy Propagation in Composite Types

The `Copy` property propagates through composite types:

- **Tuples:** A tuple is `Copy` if *all* its elements are `Copy`. For example, `(i32, f64)` is `Copy` because both components are `Copy`. But `(i32, String)` is *not* `Copy` because `String` is not `Copy`.
- **Arrays:** A fixed-size array `[T; N]` is `Copy` if `T` is `Copy`. Arrays are homogeneous and fixed-size (known at compile time).
- **Tuples vs. Arrays:** Tuples can be heterogeneous (`(i32, f64, bool)`), arrays are homogeneous. Both propagate the `Copy` trait from their elements.

The professor gave the example of 3D coordinates: *“If I create a triple of numbers to represent a point in space, the numbers are Copy, and consequently the triple representing the three coordinates x , y , z is automatically Copy.”*

The professor also introduced the general term: *“A tuple is the generic name to indicate a pair, a triple, a quadruple, an n -tuple. A triple always has three things. Those three things can be potentially heterogeneous.”* This heterogeneity is what distinguishes tuples from arrays—arrays are always homogeneous.

4.4.6 What “Move” and “Copy” Actually Do at the Machine Level

At the hardware level, both a move and a copy are *identical*: a `memcpy` of `N` bytes from source to destination. The only difference is a **compiler annotation**: when a type implements `Copy`, the compiler does *not* add the “data is now inaccessible” mark to the source. That is the entire distinction. No different machine instructions, no runtime cost difference—just a compile-time bookkeeping difference.

4.4.7 The Clone Trait: Explicit Deep Copy

Sometimes you genuinely need an independent copy of a non-`Copy` type. The `Clone` trait provides this through an explicit method call:

```
let s1 = "Hi".to_string();
let s2 = s1.clone(); // Deep copy: new heap allocation
// Both s1 and s2 are independently usable
println!("{}", s1, s2);
```

Memory Model Analysis**Clone vs. Move in memory:**

When we wrote `let s2 = s1;` (move), only the 24 stack bytes were copied. The heap block remained singular—and the source was invalidated.

When we write `let s2 = s1.clone();`, the clone operation:

1. Allocates a *new* block on the heap, as large as the original
2. Copies all the characters from the original heap block to the new one
3. Stores the pointer to the new block in `s2`, along with the appropriate capacity and length

Now `s1` and `s2` each have their own independent heap block. Modifying one does not affect the other.

```
let mut s1 = "Hi".to_string();
let s2 = s1.clone();           // s2 gets independent copy
s1.push('!');                  // Modifies only s1's heap data
println!("{}", s1);           // "Hi!"
println!("{}", s2);           // "Hi" (unchanged)
```

The professor traced the push operation in detail: when `s1.push('!')` is called, `s1` only has a 2-byte heap block (for ‘H’ and ‘i’), fully occupied. Since `s1` is mutable, it requests a larger block from the OS (e.g., 4 bytes), copies the existing 2 characters, adds the exclamation mark in the third slot, and frees the old 2-byte block. Meanwhile, `s2`’s heap block remains untouched. The professor concluded: *“So when I clone, I’m separating the objects. When I had the movement, instead, they were the same object with one of the paths disabled.”*

Warning**Clone can be expensive!**

The professor used a vivid example: *“While a string move always copies exactly 24 bytes (fixed cost), cloning a string depends on its length. If the string contained the entire Divine Comedy, the clone would require allocating a huge heap block and copying every single byte. The cost is proportional to the data size.”* Always consider performance implications when using `clone()`.

4.4.8 The Relationship Between Clone and Copy

There is a logical dependency:

- Only **Clone**-able types can possibly be **Copy**. If a type cannot be cloned, it means its semantics fundamentally exclude replication—so there is no reason it could be copyable.
- When a type is both **Clone** and **Copy**, their implementations are generally identical (but this is not mandatory).
- **Copy** is used implicitly on assignment/passing. **Clone** requires an *explicit* `.clone()` call.
- Non-cloneable types are relatively rare but exist for specific semantic reasons.

The professor added: *“It is not obligatory to be cloneable, just as it is not obligatory to have particular recycling rules, and just as it is not obligatory to be copyable. A good portion of the*

types are cloneable, and that's fine. Some are not—for serious reasons which we will understand better later.

The professor stressed: *“Copy types are the exception, not the rule. They are a very small subset: numbers, booleans, single characters. Most types in Rust are not Copy. And a type is not Copy because the compiler decides—it is because the programmer labels it. The compiler merely verifies that the label is compatible.”*

4.5 Heap Allocation with Box

All local variables live on the stack. This creates three situations where the stack is insufficient:

1. **The data is too large** for the stack (stacks are typically ~1 MB).
2. **The data must outlive** the current function—when the function returns, the stack contracts and the data would evaporate.
3. **The data size is not known at compile time**—the stack mechanism requires knowing exactly how many bytes to reserve at compilation.

In these three situations, data must live on the **heap**. Rust provides `Box<T>` for this purpose.

Memory Model Analysis

Why the stack requires compile-time known sizes:

The professor explained the technical reason behind the third constraint in detail: *“The compiler, looking at how the function works, sees variable 1, variable 2, variable 3, and must immediately say: ‘variable 1 is located at stack pointer plus delta_1, variable 2 at stack pointer plus delta_2...’ with offsets calculated in the compilation phase. But if the data dimension were only known at runtime, these fixed offsets could not be stabilized.”* This is why dynamically-sized data *must* go on the heap—the stack’s addressing scheme is fundamentally based on fixed, compile-time-known offsets.

Insight

Stack Overflow from Large Data

The professor gave a concrete example of the “data too large” case: *“A stack is typically one megabyte. If I need to place 20 GB—or even just 3 MB—I simply cannot put it on the stack. It would immediately get stuck. It would generate a stack overflow situation.”* This is why large data structures like big arrays or buffers must be heap-allocated using `Box`.

```
let i = 5;                // On the stack, no issues
let b = Box::new(27);    // 27 is allocated on the heap
println!("{}", *b);      // Dereference with * to access content
```

`Box` is a generic smart pointer: it holds a pointer to heap-allocated data and implements `Drop`. When the `Box` goes out of scope, it automatically frees the heap allocation.

4.5.1 Box with Tuples: Detailed Walkthrough

```
{
    let i = 5;                // i on the stack (4 bytes)
    let mut b = Box::new((5_i32, 2_i32)); // Tuple on the heap
    (*b).1 = 7;              // Modify second field via dereference
```

```
println!("{:?}", b);           // Prints (5, 7)
} // b dropped: heap freed, then stack contracts
```

Memory Model Analysis

Step-by-step:

1. `let i = 5;` — 4 bytes on the stack for `i`. No special behavior.
2. `let mut b = Box::new((5, 2));` — On the heap, a block of 8 bytes (two 4-byte `i32` values) is allocated, containing (5, 2). On the stack, `b` occupies 8 bytes (a pointer to the heap block).
3. `(*b).1 = 7;` — Dereference `b` with `*` to reach the heap, then access field `.1` (the second element of the tuple). Replace 2 with 7. The heap now contains (5, 7).
4. The professor noted an important shorthand: the explicit `*` is needed only when you are not already accessing the content through the dot operator. In practice, `b.1 = 7;` also works because Rust automatically dereferences through `.` (auto-deref). The `println!` macro, for example, calls `b.debug()`, and the dot already implies dereferencing.
5. At the closing brace: `b` has drop instructions. The heap block (8 bytes) is freed first. Then the stack contracts to remove both `b` (8 bytes) and `i` (4 bytes).

4.5.2 Returning a Box from a Function: Ownership Transfer

```
fn make_box(a: i32) -> Box<(i32, i32)> {
    let r = Box::new((a, 1));
    return r; // Ownership of the Box transfers to the caller
}

fn main() {
    let b = make_box(5);
    println!("{:?}", b); // Prints (5, 1)
} // b is dropped here: heap freed
```

Memory Model Analysis

What happens internally:

1. In `main`, space is prepared on the stack for `b`, and then `make_box(5)` is called.
2. Inside `make_box`: the parameter `a` receives the value 5. The local variable `r` creates a `Box`: a heap block is allocated for the tuple (5, 1), and `r` holds the pointer.
3. `return r;` — This is a return assignment. Ownership of `r` is ceded to the caller's destination (`b`). Because ownership has transferred, `r` does *not* execute its drop instructions—everything has passed to `b`.
4. `b` is now the owner of the heap block that was allocated by someone else (inside `make_box`). The program proceeds; when `b` reaches the end of `main`, it frees the 8-byte heap block and the stack contracts.



Figure 4.3: Returning a `Box` from a function: the heap allocation survives the function’s stack frame because ownership transfers to the caller.

4.5.3 `Box` as a Specialized Pointer

The professor placed `Box` within the broader context of Rust’s pointer types during the recap lecture: *“For the pointers, there are many types, each with different characteristics. The `Box` is a pointer that necessarily points to the heap and possesses its data. It’s a pointer that, when it comes out of scope, guarantees the release of data. It’s a pointer that is moved on assignment. A single variable is the owner of the `Box`. When that variable assigns it to someone else, the `Box` passes by property.”*

This makes `Box` one specific case in Rust’s family of smart pointers. Others (like `Rc`, `Arc`, and references) have different ownership semantics, which we will explore in later chapters.

4.6 Borrowing: Access Without Possession

Constantly transferring ownership in and out of functions is tedious and unergonomic. The professor put it plainly: *“When I want to pass data to a function, it’s not always that I really want to ‘send it away’ forever. If I just hand it over directly, Rust says ‘that’s a transfer for me.’”*

To solve this, Rust introduces **references**—pointers that grant access *without* ownership. This mechanism is called **borrowing**.

Insight**The Library Book Analogy**

A reference is like borrowing a book from the library. You can read it, take notes from it, but you *must return it*. You cannot burn it, sell it, or claim ownership. When the loan period ends, you have nothing to “release”—the book continues to exist, controlled by its owner (the library). The reference allows access to a value *without acquisition of the property*.

Technically, a reference is still a pointer—an 8-byte number indicating a memory address. The difference from Box is entirely semantic: a reference carries **no ownership responsibility**. When a reference goes out of scope, nothing is released because the possession was someone else’s and remains someone else’s.

The professor positioned references in the broader taxonomy of pointers during the recap: “A reference is a pointer that does not possess the data—it is a loan. It means ‘I let you look inside, you can temporarily use it.’ It is a pointer to a value that is possessed by another variable. When the reference comes out of scope, I don’t have to do anything because the possession was of another and remains of the other.”

4.6.1 Immutable References (&T)

An immutable reference grants **read-only** access to the underlying data. You can create as many immutable references simultaneously as you wish—multiple readers do not conflict.

```
fn print_string(s: &String) {
    println!("{}", s);           // Read access through reference
}                                // Reference dropped, nothing released

fn main() {
    let my_string = String::from("Hello");
    print_string(&my_string);    // Borrow, don't move!
    println!("{}", my_string);  // Still valid! Ownership never left.
}
```

The `&` symbol creates a reference: `&my_string` borrows the string without transferring ownership. Inside `print_string`, the parameter `s` is a reference—it can look at the data but cannot modify or release it. When `print_string` returns, the reference is simply dropped (no cleanup needed), and `my_string` remains perfectly accessible.

Owner Freeze During Immutable Borrows

While immutable references exist, the underlying data is *frozen*: the original owner is temporarily prohibited from modifying the data or transferring ownership. This guarantees that references always point to valid, stable values.

The professor elaborated on this freezing during the recap lecture: “While read-only references exist, the variable that possesses the value is also accessible—but only in read-only mode, even if we had potentially declared it mutable. It will return to full possession of its value only when the program reaches a line where none of the immutable references are still in use.” This means the owner temporarily *restricts its own power* because it has lent the data to others.

4.6.2 Mutable References (&mut T)

Sometimes you need to modify borrowed data in place. A mutable reference (`&mut T`) grants this power, but under a strict condition: **exclusivity**.

```
fn add_world(s: &mut String) {
    s.push_str("world"); // Modify through mutable reference
}

fn main() {
    let mut s = String::new();
    add_world(&mut s);      // Exclusive mutable borrow
    println!("{}", s);      // Prints " world"
}
```

Owner Blocked During Mutable Borrows

The professor detailed an even stricter rule for mutable references during the recap: *“With mutable references, it is possible to get a mutable reference only from a mutable variable—otherwise I would give a permission that I don’t have. And while there is a mutable reference, there cannot be any immutable references, nor is it possible to access the variable that actually possesses the value.”* The owner is completely locked out while the mutable borrow is active. *“When I finally reach the line that goes beyond the last access to my mutable reference, the compiler unblocks the thing and leaves me to access it again.”*

This is more restrictive than immutable borrows: with immutable borrows, the owner can still *read* the data. With mutable borrows, the owner cannot do *anything* with the value until the borrow ends.

4.6.3 The Cardinal Rule of Borrowing

! Exam Priority

The Readers–Writer Rule

At any given time, you can have **either**:

- **One mutable reference** (&mut T) — the single writer, or
- **Any number of immutable references** (&T) — multiple readers

But never both simultaneously.

The professor explained the reasoning: *“If the writer is alone, no one gets hurt. If many are reading, no one gets hurt because nothing changes. But if while the writer is writing, someone reads, they could read something wrong—the writer wrote ‘ca’ but is actually writing ‘casa.’ The reader misinterprets. It must be certain that when I read, the data is stable.”*

This is the same principle found in databases (multiple readers / single writer locks) and concurrent programming. It prevents *data races* at compile time.

The professor further explained during the recap: *“The presence of the mutable reference guarantees that, for the time interval coinciding with the program interval in which the mutable reference is valid, no one can use another path to access that data. This is fundamental because if there is an in-progress transformation, readings made concurrently might read something wrong—intermediate states that tell me the data hasn’t reached its stability yet.”*

- **Expected Mastery Level:** Very High. You must identify violations of this rule in exam code snippets.

4.7 The Borrow Checker and Lifetimes

The *Borrow Checker* is the subsystem of the Rust compiler responsible for enforcing ownership and borrowing rules. It scrutinizes every code path—including all branches of `if` statements and all iterations of loops—to ensure that no reference outlives the data it points to.

4.7.1 The Compiler Proves Theorems

The professor described the Borrow Checker’s work in formal terms during the recap lecture: *“The compiler proves a theorem, basically. It proves many theorems. For any reference, it proves the theorem that, given that code, there is no path—no possible path within the various `if`, `while`, function calls, or such things—that allows contemporary access violating one of the rules we explained.”* This means the Borrow Checker is not performing simple pattern matching—it is conducting a formal proof over all possible execution paths.

4.7.2 Lifetimes: The Concept

Every value has a **lifetime**—the span of the program during which it is valid. A variable comes into existence when it is declared and ceases to exist when its scope ends. The professor drew a parallel to human life: *“Before you are born, you don’t exist. After you die, you no longer exist. In the middle is a piece of life. Programs revolve around time just the same.”*

Crucially, program execution is not simply linear. In a loop, line 23 may execute *after* line 25 (because we loop back). So if line 25 gave something away (moved it), we cannot use it again when we return to line 23. We must follow the **logic of program execution**, tracing with our finger: first this, then this, then that, go back, this, this, go back...

The professor noted: *“Programs break not because programmers are idiots. Simply because when complexity grows—nested ifs, loops, branching paths—the ability of the mind to track all ramifications becomes overwhelming. Some pieces will escape our attention. Rust’s Borrow Checker does this tracking for us, following all paths, however complex, and verifying that in every possible execution path, the rules are never violated.”*

4.7.3 What Lifetimes Really Are Internally

The professor gave a more technical description of lifetimes during the recap lecture: *“To be able to say whether there is concurrent access, the compiler must keep track, for each existing reference, of the set of lines in which that reference appears—in which it is valid, in which it is accessed. This set of lines generally takes the name of ‘lifetime.’ It is information that lives inside the compiler’s tables. The compiler uses it to do its math.”*

Lifetimes are fundamentally a compile-time bookkeeping mechanism. They have no runtime representation whatsoever. The compiler uses them to prove that the **lifetime of a reference is completely contained within the lifetime of the value it points to**—and if it cannot prove this, compilation fails.

4.7.4 The Core Lifetime Rule

The lifetime of a reference must be **entirely contained within** the lifetime of the value it points to. The professor used a body analogy: *“If your liver is destined to be destroyed tomorrow, how long can you live? At most until tomorrow—because an essential part of you ceases to exist. You might die sooner (a brick falls on your head), but never later.”*

This propagation works recursively: if a reference is inserted into a larger structure, the lifetime of the entire structure is constrained to the lifetime of the reference. And if that structure is part of an even larger one, the constraint propagates upward.

4.7.5 Dangling Reference Example

```
let r;
{
    let x = 1;
    r = &x;           // r borrows x
}                    // x is dropped here!
// println!("{}", r); // ERROR: x does not live long enough
```

Memory Model Analysis

What happens internally:

1. `r` is declared on the stack (8 bytes for a pointer), but not yet initialized.
2. A new block opens. `x = 1` is placed on the stack (4 bytes).
3. `r = &x`; — `r` now points to the stack location of `x`.
4. The closing brace contracts the stack, removing `x`. But `r` still holds the old address—now pointing to deallocated stack memory. If an interrupt fires, it could overwrite that memory. If we read `*r`, we might get garbage.
5. In C, this compiles and “works” 49 out of 50 times. The 50th time, an interrupt happens at the wrong moment and you get corrupted data. **In Rust, the compiler catches this at compile time** and says: “`x` does not live long enough—it is dropped at the closing brace while still borrowed by `r`.”

The professor used the rental analogy: “It’s like when your lease on a storage unit ends and you return the keys. But you made a copy. The landlord rents it to someone else. You walk in with your copy of the key—but the fridge you expected to find is gone, replaced by someone else’s furniture. That’s a dangling pointer.”

Dangling Pointers in C: A Detailed Walkthrough

The professor gave a concrete walkthrough of a dangling pointer in C during the recap lecture: “A dangling pointer is something I create in C with `malloc`. I do a 10-byte `malloc`. I get the pointer: `0x7B5F4N`. I save it in a variable. At this moment, it exists. I use `*ptr` to access the object. Then at some point I `free` that pointer and return the memory. But I don’t cancel the pointer—it remains written `0x7B5F4N` somewhere. In the next few lines, if I hadn’t noticed that I had freed it and I still use it, the compiler would let me do it. It’s a shame that my house is no longer there. It became someone else’s house—that piece of memory is now used by something else. You go inside and do things. That’s a dangling pointer—a pointer that is used outside of the time in which it is valid.”

“Since C doesn’t track the lifetime, the compiler doesn’t know how long a value lasts. That awareness is entirely the programmer’s responsibility. If the programmer does well, the problem doesn’t arise. The real problem is that the programmer often doesn’t do well—not because they are bad, but because the details are many, and something will escape.”

4.7.6 Lifetime Propagation into Containers

```
let mut v = Vec::new();
{
    let data = 1;
    v.push(&data);    // Reference to data stored in vector
}                  // data is dropped!
// println!("{:?}", v); // ERROR: v contains a dangling reference
```

The vector `v` contains a reference to `data`. When `data` is dropped, that reference becomes dangling. Therefore the *entire* vector becomes invalid. The compiler reports: “*You took a reference to `data` at line 4, but `data` does not live long enough—it was dropped at line 5 while still borrowed.*”

The professor generalized: “*If I had put five references in the vector, the vector’s lifetime would automatically be the shortest among all the things I put in it. The lifetime of the weakest part propagates to the container.*”

The professor further explained during the recap: “*If this reference becomes part of a wider structure, the constraint is automatically propagated to the wider structure. And if that structure is still part of another thing, it goes up recursively. The same for functions: if a function receives a reference as a parameter, the lifetime of that reference becomes part of the function’s signature. The function is usable only while that reference exists. The time of life of the reference conditions the time of life of the function.*” When you choose a specific reference to pass to a function, you are conditioning the usability of the function itself.

4.7.7 Lifetime Annotations

In most cases, the Borrow Checker can deduce lifetimes automatically (**lifetime elision**). However, when functions receive multiple references and return a reference, the compiler cannot always determine which input lifetime the output depends on. In these cases, **explicit lifetime annotations** are required:

```
fn longest<'a>(s1: &'a str, s2: &'a str) -> &'a str {
    if s1.len() > s2.len() { s1 } else { s2 }
}
```

The annotation `'a` is a “lifetime parameter”—it tells the compiler that the returned reference lives *as long as both inputs*. The professor explained: “*I could have a function ‘max’ that takes references to two numbers and returns a reference to the larger one. How long does the result last? It depends on which input was selected—and the compiler can’t know that statically. So we must tell it: the result lasts as long as both inputs.*”

Lifetime Annotations as Purely Symbolic Names

The professor gave a particularly illuminating explanation of what lifetime annotations actually represent during the recap lecture:

“*We don’t give a concrete representation to our lifetime. We’re not going to say ‘you go from January 25th to January 27th’—that doesn’t make any sense. We give the references’ lifetimes a purely symbolic name: `'a`, `'b`, `'c`, where `a`, `b`, and `c`, in themselves, have no meaning, other than that within the same function, all things marked with `'a` mean the same thing, and they are different from things marked `'b`.*”

“*So the lifetime criterion is only a way to say ‘this is the same as that’ or ‘this is different from that.’ The name is purely arbitrary.*”

This leads to an important consequence for shared lifetimes: “*If one reference could go from line 1 to line 11, and the other from line 3 to line 13, giving both the same lifetime `'a`, automatically—even though one could individually be valid for a greater number of lines—because I’m forcing them to use the same lifetime, I’m tightening the usage to only the intersection. Trying to access one or the other outside the intersection will cause a compilation error.*”

Exam Focus**Lifetime Intersection Rule**

When two references share the same lifetime parameter `'a`, their valid usage window becomes the **intersection** of both lifetimes. If reference A is valid from line 3 to line 13 and reference B is valid from line 1 to line 11, then both are usable only from line 3 to line 11. Accesses outside this intersection are compile errors.

Lifetime rules for functions:

- If there is a *single* reference parameter, the compiler automatically assumes the output lifetime equals the input lifetime.
- If there are *multiple independent* references, the compiler may require explicit annotations.
- The presence of a lifetime parameter in a function signature conditions the *usability* of the function: you can only call it while all the referenced data is alive.
- If a function receives more references, it could be necessary to indicate explicitly whether those references must last at the same time or if they are independent.

4.7.8 Lifetimes in Structs

When a struct holds a reference, the struct's lifetime is bounded by the reference's lifetime:

```
struct User<'a>{
    id: u32,
    name: &'a str,    // Reference with lifetime 'a
}
```

The `User` struct can only live as long as the `name` reference is valid. The `id` field is owned by the struct (it is just a number), so it poses no constraint. But `name` is borrowed—and this borrowed piece makes the entire container perishable. If `User` is placed inside yet another struct, the lifetime constraint propagates recursively upward.

4.7.9 The 'static Lifetime

The special lifetime `'static` means the data lives for the *entire duration of the program*. String literals are `&'static str` because they are stored in the read-only constants segment of the binary, which is never freed:

```
let s: &'static str = "Hello"; // Lives forever
```

This is different from taking a reference to a `String` created at runtime, which would have a limited lifetime.

The professor elaborated during the recap: “When we write a string literal like `"ciao"`, it automatically becomes a compile-time constant that exists for the entire duration of the program because it is written directly into the executable, somewhere, and it will be there the entire time. So we can, at any moment in our program's life, refer to that particular string—but exactly that one, which will be at some fixed address. `&'static str` denotes a sequence of characters that exists for the duration of the program.” It is fundamentally different from `&str` obtained by borrowing a `String`: the latter is `&'a str` where `'a` is the lifetime of the `String`, which “lasts little” by comparison.

4.7.10 All Checks Are Compile-Time

Insight

Zero Runtime Cost

The professor emphasized: *“All of this is done at compile time. There is no trace of it at runtime. It doesn’t affect execution time. We don’t pay for it. It’s not like Java’s null pointer check, where every time Java has a pointer it asks ‘is this null?’ and burns four CPU cycles just for that—and pointers are everywhere. Here, the control is done a priori. References can never be null: if I can only create a reference from data that exists, null references are ontologically impossible.”*

The professor further explained the null-impossibility argument: *“ $r = \&a$. If a is not there, I can’t take the reference. So the null reference doesn’t exist. If there are references, it means the data existed. But that reference is accessible only while the data exists.”*

4.8 Slices: References to Sequences

In C, a pointer to an array decays into a pointer to the first element, losing all dimension information. This is the root cause of buffer overflows and the notorious complexity of C string handling.

Rust addresses this with **slices**: a reference to a contiguous sequence of elements, represented as a **fat pointer**.

4.8.1 Fat Pointers

A regular reference ($\&T$) occupies 8 bytes—just a memory address. A slice ($\&[T]$) occupies **16 bytes** (128 bits):

- First 8 bytes: pointer to the first element
- Second 8 bytes: the number of elements (length)

This explicit length allows Rust to perform **bounds checking at runtime**: if you attempt to access an index beyond the length, the program will **panic** (a controlled crash) rather than silently corrupting memory.

4.8.2 Creating Slices

```
let a = [3, 4, 5, 6, 7]; // Fixed-size array on the stack
let s: &[i32] = &a[1..3]; // Slice: elements at index 1 and 2
// s points to [4, 5], length = 2
println!("{:?}", s);      // Prints [4, 5]
```

The notation $\&a[1..3]$ creates a reference to elements from index 1 (inclusive) to index 3 (exclusive): values 4 and 5. The slice s is a fat pointer containing the address of element at index 1 and the count 2.

Warning**Slices and Panic**

Accessing a slice out of bounds triggers a **panic**—the program terminates in a controlled manner. This is dramatically safer than C’s silent buffer overflow. However, you should still write defensive code: check the slice length before indexing, use iterators when possible, and avoid raw index access when the bounds are uncertain. The slice index type is **usize** (unsigned), so negative indices are syntactically impossible.

The professor noted: *“The slice encapsulates the length, so I am able to verify that we’re not going beyond our responsibility. The only way to guarantee bounds safety is at runtime (since indices depend on runtime values), and slices make this possible by carrying their length with them.”*

Slices can be taken from arrays, vectors, strings (**&str** is a string slice), and even from **Box** allocations:

```
let v = vec![10, 20, 30, 40, 50];
let slice: &[i32] = &v[1..4]; // [20, 30, 40]
println!("{}", slice);
```

Since a slice is a reference, all borrowing rules apply: while the slice exists, the vector is borrowed and cannot be modified.



images/ch04_slice_fat_pointer.png

Figure 4.4: A slice as a fat pointer: the pointer addresses the first included element, and the length field enables runtime bounds checking.

4.9 Summary: Box vs. Reference — A Comparison

The professor presented a clear comparison between the two main pointer types:

Property	Box<T>	&T / &mut T
Ownership	Owns the data	Borrows the data
Points to	Always the heap	Stack or heap
On drop	Frees the heap allocation	Does nothing
Duplicable?	No (moves on assignment)	Yes (&T can be copied freely)
Mutable access?	If the Box is mutable	Only with &mut T (exclusive)
Can be null?	No	No
Binary representation	8 bytes (pointer)	8 bytes (pointer)

The professor noted: *“Both Box and Ref are pointers. If I look at them in binary representation, they are indistinguishable—8 bytes with a number. The difference is entirely in semantics. Box is responsible for release. Ref is not—the data exists before the reference and will continue after it.”*

The professor also framed this comparison during the recap: *“I can copy a reference—there can be many in read-only mode, or one in write mode. I can’t copy a Box—I can move it, but there cannot exist two Boxes that point to the same data.”*

Key distinctions from C pointers:

- Neither Box nor &T/&mut T can be null. There is no possibility of null pointer dereferencing.
- C pointers allow aliasing (multiple pointers to same data) without any control, creating validity and double-free problems. With &T, many can read simultaneously but the double-free problem cannot arise because references don’t release. With &mut T, exclusivity prevents data races. With Box, there is always a unique owner.
- In C, a pointer might represent a single value or an array (the type doesn’t distinguish). Rust uses slices (&[T]) as fat pointers to represent arrays, encoding the length explicitly.

Insight

The Great Ambiguity of C, Resolved

The professor concluded the pointer comparison with a summarizing observation: *“Despite all this complexity, it is actually to protect us. The effort we have to make to squeeze awareness, to disambiguate situations—the great ambiguity of the C pointers—here we fight with different pointers, each of which has powers and limits. This allows the compiler to do a control that it would not otherwise be able to do, and to generate only the necessary operations, giving us access to what we need.”* The “complexity” of Rust’s pointer system is the *solution* to C’s dangerous simplicity.

4.10 Why Ownership Matters: Performance Without GC

The professor concluded the lecture by connecting ownership to performance guarantees. Rust’s ownership system provides all benefits of a garbage collector *without one*:

- **No non-deterministic pauses:** Programs have performance homogeneous with respect to the same input. Garbage-collected programs depend on factors outside your control, introducing non-deterministic pauses.

- **Minimal memory usage:** GC languages tend to hold memory until a collection cycle runs (“spray, spray, spray, and eventually I’ll recover something”). Rust releases memory deterministically the moment it’s no longer needed.
- **Cloud cost implications:** The professor noted that in cloud environments where memory usage is billed, writing programs that use a tenth of the memory means paying significantly lower prices.
- **Resource cleanup beyond memory:** Ownership and Drop guarantee correct release of file handles, sockets, hardware devices—not just memory.

Insight

Reading the Compiler Messages

The professor urged students to *read* the Rust compiler’s error messages carefully: “*Students arrive, write their code, run `cargo build`, it breaks. ‘Fix it.’ ‘I don’t know how.’ ‘How can you not know? It’s written right there!’ The compiler explains in detail what’s wrong and why. It’s verbose, yes, but extremely analytical. It says exactly: ‘Look, the problem is here.’ The solution is ours to find, but the diagnosis is precise. You must not be intimidated by the number of lines—force yourself to read them.*”

The professor added: “*The compiler can’t tell us how to adjust it, but it explains in detail why something doesn’t compile. Either we eliminate the assignment, or we eliminate the print line. The solution is for us to decide—but the problem is precisely identified.*”

4.11 Ownership Recap: The Professor’s Summary

At the beginning of the second lecture session, the professor provided a structured recap of all ownership concepts. This summary is valuable as a mental checklist:

Exam Focus**The Ownership System in Summary**

1. Every value is possessed by one and only one variable. That variable is responsible for its release.
2. The **type** of the value indicates what must be done at release time. Simple types require only the elimination of the variable (stack contraction). More articulated types require specialized instructions (**Drop** trait).
3. When I assign a variable to another, pass a variable as a parameter, or return a variable as a result—in all three cases, the **possession transits** from the source to the destination through the operation called *move*. The source becomes inaccessible.
4. Some particular types are marked as **Copy**. This means: since there are no particular recycling instructions and the meaning of what is written inside has no special implications, the information remains accessible after a copy.
5. When a pointer **possesses** the data it points to, use the **Box** type. Box is useful for variable-length data, very large data, or data that must outlive its function.
6. When I need to **access without owning**, use **references** (**&T** or **&mut T**). References are pointers without possession—loans.
7. The Borrow Checker is the compiler component that manages references. It enforces that **&T** and **&mut T** are mutually exclusive: many **&T**, or one **&mut T**, never both.
8. While mutable references exist, even the original owner cannot access the data.
9. All checks are done at compile time with zero runtime cost.

4.12 Comprehensive Ownership Tracing: Exam-Style Example

Exam Focus

Mental Model for the Exam

When analyzing Rust code, mentally track who *owns* each value and trace the *lifetimes* of any borrows. The professor walked through this exact pattern:

```
let s1 = String::from("Hello"); // s1 owns heap data
let s2 = s1;                     // MOVE: s1 -> s2
// s1 is now inaccessible
let num1 = 3;                    // num1 on stack (Copy type)
let num2 = num1;                 // COPY: num1 still valid
// Both num1 and num2 are independently usable
```

The stack trace:

1. `s1` allocated (24 bytes), heap block for “Hello”
2. `s2` allocated (24 bytes), receives bitwise copy of `s1`’s stack data. `s1` marked inaccessible.
3. `num1` allocated (4 bytes), value = 3
4. `num2` allocated (4 bytes), value = 3 (copy, not move—`num1` remains valid)

At end of scope:

- `num2`: no special drop, stack contracts
- `num1`: no special drop, stack contracts
- `s2`: Drop trait invoked → heap freed, then stack contracts
- `s1`: Drop *omitted* (no longer owns anything), stack contracts

The professor noted an important observation: “*num1* can increase freely and *num2* goes for its road. They are two independent copies and there are no particular links between them.”

Exam Relevance and Recurrence

Score: Very High — Tested in EVERY exam

Notes: This topic forms the absolute core of the exam. You will consistently find questions asking you to:

- Trace ownership through a code snippet, identifying which variables own which values at each point
- Identify compilation errors caused by moves, dangling references, or borrowing violations
- Explain the difference between a bitwise copy (Move) and a deep copy (Clone)
- Explain why Copy and Drop are mutually exclusive
- Determine the lifetime of references and identify dangling pointer situations
- Distinguish between shadowing (`let x = ...`) and reassignment (`x = ...`)

- Explain how the owner is frozen/blocked during borrows (immutable: read-only; mutable: completely blocked)
- Describe why lifetime annotations use symbolic names ('a, 'b) and what shared lifetime parameters imply (intersection)

4.13 Domande ed Esercizi da Esami Passati

Exam Focus

Domanda: Si definiscano i concetti di Dangling Pointer, Memory Leakage e Wild Pointer, facendo esempi concreti, usando dello pseudocodice, che possono generare questi fenomeni.

Risposta: In base al capitolo, un *dangling pointer* è un puntatore a un'area di memoria non più valida o deallocata. In C, si verifica ad esempio se si usa `malloc`, si salva il puntatore, si esegue la `free` per restituire la memoria, ma non si cancella il puntatore originale, continuando ad usarlo (come avere la copia della chiave di un magazzino già affittato ad altri). Il *memory leakage* avviene invece quando la memoria allocata non viene mai rilasciata (nessun proprietario chiama le istruzioni di rilascio o `Drop`). I *wild pointer* non sono direttamente citati nel testo, ma rientrano nella problematica generale dell'ambiguità dei puntatori privi di validità garantita a compile-time. Rust previene tutto ciò assegnando ad ogni dato un solo *owner* e assicurando che la *lifetime* dei puntatori (reference) non superi mai quella del valore originario.

Exam Focus

Domanda: Completa questa funzione in modo che compili correttamente, se ritieni che ci sia un errore di compilazione (descrivendolo), altrimenti dichiara il motivo per cui la funzione è scritta correttamente:

```
fn get_prefix(s: &str, len: usize) -> &str {
    &s
```

Risposta: La funzione è incompleta. Per compilare correttamente e restituire un prefisso (ovvero una *string slice*), deve essere completata usando la sintassi delle *slice*:

```
fn get_prefix(s: &str, len: usize) -> &str {
    &s[..len]
}
```

Non ci sono errori di compilazione legati alle *lifetime* perché il compilatore applica la regola di *lifetime elision*: essendoci un solo parametro di tipo reference in ingresso (`s: &str`), assume automaticamente che la lifetime del valore restituito sia identica alla lifetime dell'input.

Exam Focus

Domanda: Si consideri il seguente frammento di programma Rust:

```
1 fn main() {  
2   let mut s = String::from("esame");  
3   let r1 = &s;
```

Risposta: In questo frammento, `s` è il proprietario (*owner*) di una stringa allocata nell'heap e `r1` è una *immutable reference* (prestito o *borrow* in sola lettura) a `s`. Secondo le regole del *Borrow Checker*, fintanto che `r1` è in uso, il proprietario originale `s` viene "congelato" (*owner freeze*): non può modificare la stringa, né trasferirne la proprietà, per garantire che `r1` continui a puntare a dati validi e stabili. Questo previene la possibilità di avere modifiche impreviste mentre vi sono lettori attivi (regola dei *Readers-Writer*).

Exam Focus

Domanda: Si descriva la gestione della memoria in Rust e si spieghi come vengono evitati i problemi di sicurezza comuni come le violazioni di accesso o la presenza di puntatori nulli.

Risposta: Rust gestisce la memoria tramite il concetto di *Ownership* valutato a tempo di compilazione (zero runtime cost). Ogni valore ha un solo *owner* e viene rilasciato automaticamente all'uscita dello scope, prevenendo *double free* o *memory leak*. I prestiti (*borrowing*) evitano copie non necessarie tramite reference (`&T` o `&mut T`); le violazioni di accesso o le data race sono impedito assicurando la mutua esclusione tra `&T` (molti lettori) e `&mut T` (un solo scrittore esclusivo). I puntatori nulli non esistono perché ogni reference deve essere sempre valida (*lifetime* verificata staticamente), mentre per identificare i limiti degli array Rust usa i *fat pointer* (*slice*), garantendo l'assenza di buffer overflow tramite runtime bounds checking.

Exam Focus

Domanda: Si descriva il concetto di "ownership" in Rust e come contribuisca a prevenire errori come le race condition e i dangling pointer rispetto ad altri linguaggi come C++.

Risposta: L'*ownership* stabilisce che ogni valore ha uno e un solo proprietario, responsabile della sua deallocazione (*Drop*). Questo previene problemi come *double free* perché solo l'owner rilascia la risorsa. Nel concedere accessi temporanei (*borrowing*), il compilatore verifica che una reference non viva più a lungo del suo proprietario (*lifetime core rule*), eliminando alla radice i *dangling pointer*. Le *race condition* sono prevenute dalla rigida regola dei Readers-Writer: a compile-time non è possibile avere reference mutabili (`&mut T`) contemporaneamente ad altre reference alla stessa memoria, garantendo stabilità dei dati.

Exam Focus

Domanda: Si spieghino le differenze tra costruzione di copia e costruzione per movimento nel linguaggio C++11 in termini di comportamento e di prestazioni, e si descriva un caso d'uso nel quale il movimento risulta vantaggioso rispetto alla copia.

Risposta: Applicando i principi descritti nel capitolo al concetto di "movimento", una *move* in Rust è una copia a basso costo (*bitwise copy* o `memcpy` dello stack, tipicamente 24 byte per una Stringa) che contemporaneamente invalida l'accesso alla variabile sorgente, evitando di dover clonare o gestire i dati nell'heap. La costruzione di *copia* completa (in Rust rappresentata da `Clone`) genera invece una nuova allocazione nell'heap, implicando un costo di prestazione proporzionale ai dati (es. ricopiare l'intera "Divina Commedia"). Il movimento è estremamente vantaggioso ovunque sia necessario trasferire il possesso di un oggetto "pesante" (es. `String` o array dinamico) senza sostenere l'onere in termini di tempo e memoria richiesto dalla duplicazione dei dati heap associati.

•

Chapter 5

Compound Types: Structs, Enums, and Pattern Matching

When modeling real-world domains, primitive types like integers and booleans are rarely sufficient on their own. To represent complex entities—such as a player in a game, a geometric shape, or an HTTP response—we must aggregate simpler data into cohesive, logical units. In Rust, we achieve this using two fundamental building blocks: **Structs** and **Enums**.

Structs allow us to group heterogeneous fields together into a single composite value—a “product” of its parts. Enums allow us to express that a value can be exactly one of several possible variants—a “sum” of alternatives. Together with **pattern matching**, they form the core of data modeling in Rust and represent one of the language’s most powerful features, far exceeding what is available in C or Java.

This chapter traces the professor’s lecture precisely, from the declaration and instantiation of structs, through memory layout analysis and encapsulation, to Rust’s powerful enums with data payloads and the `match` construct that safely extracts their contents.



images/ch05_structs_enums_overview.png

Figure 5.1: Structs combine fields with AND semantics (product types), while Enums offer OR semantics (sum types). Pattern matching is the safe key to open both.

5.1 Structs: Aggregating Heterogeneous Data

A **struct** (short for *structure*, also called a *record* in other languages) is a way of putting a set of potentially heterogeneous fields—a string, a number, a list, something else—together as a single, cohesive piece of information. As the professor put it: “Record is basically a way of putting a set of fields, potentially heterogeneous, together as a distinctive collective information.”

Consider implementing a game. You might need to represent a player through their nickname, their remaining health points, and perhaps their current level of ability. You could build a struct of this kind:

```
struct Player {  
    name: String,  
    health: i32,  
    level: u8,  
}
```

The syntax is straightforward: you use the keyword **struct**, give it a name (**Player**), and then list the fields it must contain inside curly braces, each with a name and a type.

Insight

[The Compiler Registers a New Type] When the compiler encounters a **struct** declaration, something very important happens: it registers a **new type** alongside all those it already knows. We are introducing a **user-defined type** because we have chosen to call it **Player**. The compiler immediately calculates its minimum size: 24 bytes for a **String**, 4 bytes for an **i32**, plus 1 byte for a **u8**—a total of 29 bytes at minimum. But 29 is an “ugly” number (not a multiple of 4 or 8), so the compiler will likely round it up to 32 bytes for alignment efficiency. We will explore exactly why later in this chapter.

5.1.1 Instantiating Structs

Once a struct is declared, we populate it by providing a value for each of its fields. The basic syntax uses the pattern `field.name: value`:

```
let s = Player {
    name: "Mario".to_string(),
    health: 25,
    level: 1,
};
```

Here we have filled every field with a concrete value. Within `s`, we now have a complete **Player** structure. We can access its fields using dot notation: `s.name`, `s.health`, `s.level`.

Field Init Shorthand. If you already have variables of the right type whose names coincide with the field names, you can use a shorthand notation:

```
let name = String::from("Mario");
let health = 25;
let level = 1u8;

// Shorthand: no need to write name: name, health: health, etc.
let p = Player { name, health, level };
```

This shorthand allows us to avoid writing `name: name, health: health, level: level`—a convenient syntactic sugar that reduces redundancy.

Struct Update Syntax (Spread). If you want to create a new struct instance that is mostly the same as an existing one, but with a few fields changed, you can use the **struct update syntax**:

```
let s1 = Player {
    name: "Paolo".to_string(),
    ..s // take the remaining fields (health, level) from s
};
```

The operator `..s` means: “for all the fields I have not mentioned here, go and take them from inside `s`.” In this case, `health` and `level` are taken from `s`, while `name` is overridden with “Paolo”.

Insight

[Struct Update vs. TypeScript Spread] The professor drew an important comparison with TypeScript. In TypeScript, there is a similar spread operator (`...`), but there is a critical difference in position. In TypeScript, the spread goes **before** the modifications: you first spread the original object, then override specific fields. In Rust, the operator `..`, if present, can **only be at the bottom**. Fields written in advance have precedence, and whatever is missing is then taken from the spread source. This ordering difference is important to remember!

Warning

[Ownership and Struct Updates] When using the struct update syntax (`..s`), Rust **moves** the fields from the original struct if they do not implement the `Copy` trait. For instance, `String` does not implement `Copy`. If we had written:

```
let s1 = Player { ..s }; // complete copy via spread
```

then the `name` field (a `String`) would be **moved** from `s` into `s1`. After this, `s.name` is no longer accessible—ownership has transferred. The `i32` and `u8` fields are fine because they implement `Copy` (they are simply bit-copied). If we need to keep `s` usable, we would need to `clone()` the `name` field explicitly. The professor emphasized: “The rule of the ownership is always valid. I’m disassembling `s` to populate `s1`, but I’m taking away the name, which at this point would remain property of `s1` and no longer accessible on the other side.”

To summarize, there are three main ways to populate a struct:

1. **Explicit field assignment:** `field: value` for each field.
2. **Field init shorthand:** when a variable with the same name and type as the field already exists.
3. **Struct update syntax:** provide some fields explicitly and spread the remaining ones from an existing instance with `..existing_instance`.

Accessing and Modifying Fields. Fields are accessed using dot notation. If the struct variable is declared as `mut`, we can modify individual fields:

```
let mut s = Player {
    name: "Mario".to_string(),
    health: 25,
    level: 1,
};

s.level = 2; // modification requires 'mut'
```

5.1.2 Nominal vs. Structural Typing

The professor drew a very important distinction between Rust’s type system and that of TypeScript.

! Exam Priority Nominal vs. Structural Type Equivalence

In **TypeScript**, types use **structural equivalence**. If you create a type `Point` with fields `x: number` and `y: number`, and then create a type `Beta` also with fields `x: number` and `y: number`, TypeScript considers `Point` and `Beta` to be **the same type**. The fact that you have called them different names is incidental—what matters is the structure. In TypeScript’s notation, you write `type Something = { x: number, y: number }`, which is essentially a **type alias**.

In **Rust**, types are **nominal**. The names *count*. A struct `Player` with fields `name: String`, `health: i32`, `level: u8` has **nothing to do** with a struct `SomeOtherThing` that has exactly the same three fields. They are completely separate, incompatible types.

Expected Mastery Level: High. You must understand that in Rust, two structs with identical field layouts are **not** interchangeable. The name of the type is part of its identity.

The professor noted that the *usage syntaxes* for populating structs are similar between Rust and TypeScript—curly braces, field-colon-value—but the type semantics are fundamentally different.

5.1.3 Tuple Structs

In addition to named-field structs (with curly braces), Rust also allows us to express structs where the fields are **anonymous**—they are represented as a tuple. These are called **tuple structs**:

```
struct Playground(String, i32, i32);

let f = Playground("Central_Park".to_string(), 100, 50);
println!("{}", f.0); // "Central Park"
println!("{}", f.1); // 100
println!("{}", f.2); // 50
```

With the tuple notation, the fields have no explicit names. They are accessed by **positional index**: `f.0`, `f.1`, `f.2`. This syntax is useful when the semantics of the data is obvious and does not need explicit labels. A classic example is a point in 3D space:

```
struct Point(f64, f64, f64);
```

Instead of writing `x:`, `y:`, `z:`, it could be sufficiently clear to say that the three numbers that compose the point are naturally in order: `x`, `y`, and `z` (or depth). The professor stressed: “For the rest, there is no difference. They occupy exactly the same memory, they work the same way. The real distinction is in the readability for the programmer. The tuple struct is less readable, but when the semantics is obvious, it’s faster—you have to write fewer things. The struct with named fields helps to understand the semantics more.”

5.1.4 Unit-Like Structs (Zero-Sized Types)

Rust can also represent types that **do not occupy any memory at all**. These are called **Zero-Sized Types** (ZSTs):

```
struct Empty;
```

`Empty` is a type that occupies zero bytes. It is value-free. What sense does it make to have a type that contains nothing inside? The professor explained: “It can serve in certain situations

where I only need a **marker**—its presence itself tells me something. At the moment, we are happy to take awareness that this exists.”

Insight

[Zero-Sized Types You Already Know] The professor pointed out that we have already used ZSTs without knowing it. The **unit type** `()`, written as open/close parentheses, is another zero-sized type—it is the return type of functions that don’t return anything. But why would we also need **struct Empty**? Because we might need **distinct markers** in some situations. These are particularly interesting because they give us the possibility to compile computations **at compile time**, without having to allocate any memory at runtime. More advanced examples will come in later chapters.

5.2 Memory Layout of Structs

! Exam Priority Struct Memory Layout

Let us go back to our **Player** struct and analyze how it is laid out in memory. The professor traced this step by step:

- **name:** **String** occupies **24 bytes**. Being a **String**, there are 8 bytes that point to the heap where the sequence of characters lives, plus 8 bytes that define the capacity (how many more characters could fit), plus 8 bytes for the length (how many characters are actually there).
- **health:** **i32** occupies **4 bytes**.
- **level:** **u8** occupies **1 byte**.

This totals 29 bytes at minimum. But 29 is a “stupid number” (the professor’s words!) because it is not a multiple of 4 or 8. Modern CPUs do not read memory byte-by-byte—for example, x86 processors read 8 bytes per block, but **only if those 8 bytes start at an address that is a multiple of 8**. You cannot read in a “distorted way, a little bit here, a little bit there.” Therefore, the compiler decides to round up: probably to **32 bytes**, because it is much more comfortable to manage.

Expected Mastery Level: High. You must understand that the sum of field sizes does not equal the struct size due to alignment and padding.

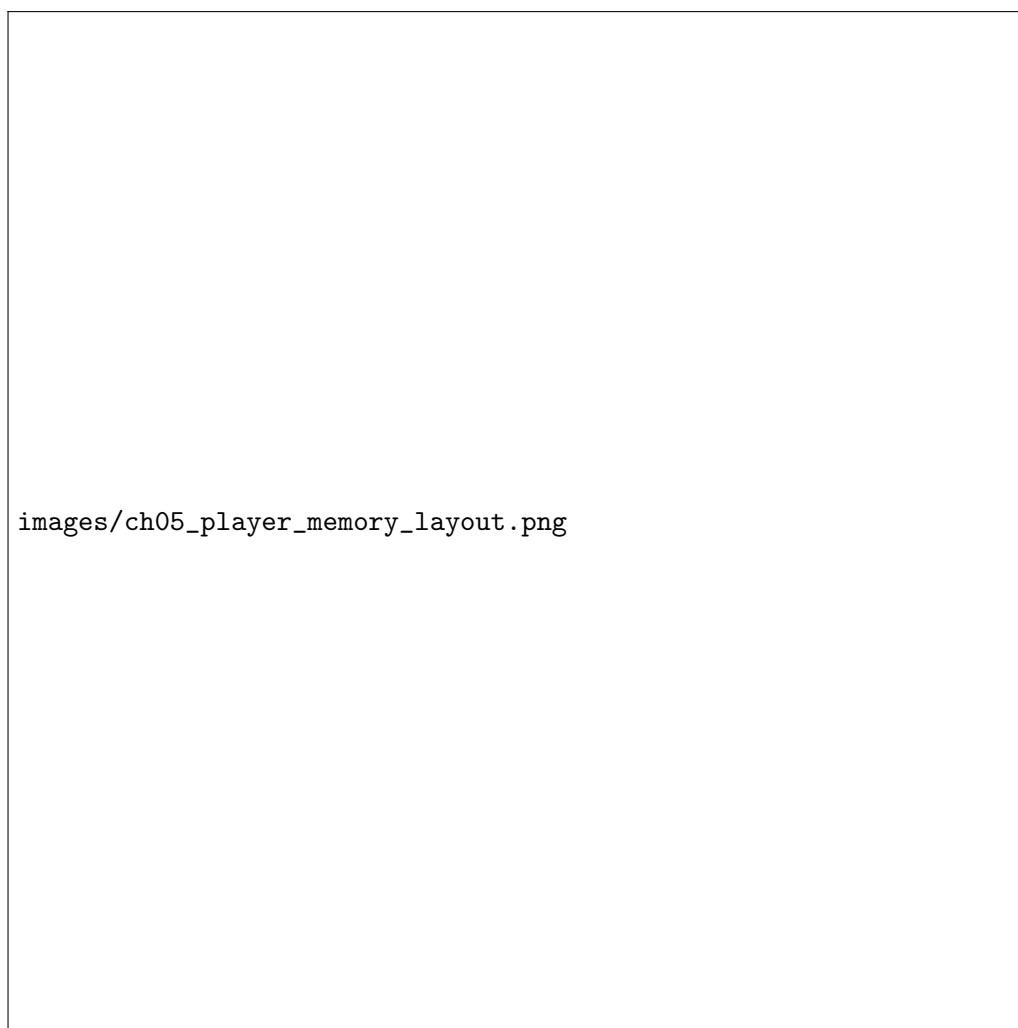


Figure 5.2: Memory layout of the `Player` struct. The 24-byte `String` header (pointer, capacity, length) is followed by a 4-byte `i32` and a 1-byte `u8`, with padding bytes added by the compiler for alignment.

No Guaranteed Field Order. A crucial point the professor made is that the compiler is **free to reorder the fields internally**. The order in which you write `name`, `health`, `level` in your source code does **not** guarantee that they appear in that same order in physical memory. The compiler chooses the layout that is optimal for the particular architecture you are compiling for:

- Compiling for **x86**: one arrangement might be optimal.
- Compiling for **ARM**: a different arrangement might be chosen.
- Compiling for **PowerPC**: yet another arrangement.

We have **no guarantees** on the field ordering. The compiler guarantees that the information we put in is certainly there—but *how* it is arranged is the compiler’s decision, optimized for the target architecture.

Forcing the Layout with `#[repr(C)]`. In some situations, we *need* to force the memory layout. The professor identified two key scenarios:

1. **Interoperability:** The data must be shared with code written in other languages, typically C or C++.
2. **Hardware representation:** The data itself is a representation of hardware information—for example, accessing a processor register that must be read in an exact sequence.

In these situations, we can prepend the attribute `#[repr(C)]` to the struct definition:

```
#[repr(C)]
struct Player {
    name: String,
    health: i32,
    level: u8,
}
```

This tells the compiler: “use the representation of the C language.” This guarantees that the fields are laid out in memory in the **exact order they are declared**, following C’s alignment rules. This gives us interoperability with other languages. If we don’t need this guarantee, we can leave it off and let the compiler freely choose the optimal disposition.

Inspecting Memory at Runtime. Rust provides low-level functions in `std::mem` to inspect how values are laid out:

```
use std::mem;

let p = Player {
    name: "Mario".to_string(),
    health: 25,
    level: 1,
};

// How many bytes does this value occupy?
println!("Size: {} bytes", mem::size_of_val(&p));

// What alignment does it require?
// Returns 1 if any address, 2 if even addresses only,
// 4 if multiples of 4, 8 if multiples of 8, etc.
println!("Alignment: {} bytes", mem::align_of_val(&p));
```

The professor explained: “`align_of_val` returns 1 if that value can start at any address, 2 if it can only start at an even address, 4 if it can only start at a multiple-of-4 address, 8 if it can only start at a multiple-of-8 address. Usually it stops there, because there are no alignment requirements above 8 on most architectures.” And `size_of_val` tells us exactly how many bytes a particular value occupies.

5.3 Visibility, Encapsulation, and the `pub` Keyword

When we introduce a struct with named fields, we have the possibility to control the visibility of each field individually.

Default Visibility. By default, the fields of a structure are visible **only to code present in the current module and its sub-modules**. This is a private-by-default policy.

Making Fields Public. To grant wider visibility, we prefix a field with the keyword `pub` (short for “public”). The struct itself must also be public if external modules need to know about it:

```
pub struct Player {
    pub name: String,    // visible to everyone
    health: i32,         // private (current module only)
    level: u8,           // private (current module only)
}
```

Insight

[Public Struct, Private Fields] Note that it is perfectly valid to have a struct that is **public** without its fields being public. This means that external code can know the type exists and can work with instances of it, but **cannot see or directly access what is inside**. The professor gave an example: “What happens with `Vec`? `Vec` is a public struct, where its fields (which we know are a pointer, a capacity, and a size) are **not** public. `Vec` manipulates them internally.” This is the foundation of encapsulation in Rust.

The general pattern is to define the struct in a module and control what is visible:

```
mod something {
    pub struct MyStruct {
        // fields are private by default
        internal_data: Vec<u8>,
    }

    impl MyStruct {
        pub fn new() -> Self {
            MyStruct { internal_data: Vec::new() }
        }
        // ... public methods that control access ...
    }
}
```

This implements the **principle of encapsulation**: everyone at home does their own thing, and others don’t need to peek inside. If there is a struct with fields I cannot manipulate directly, I can interact with it through **methods** that have internal visibility, just as happens in object-oriented programming when we build a class with private fields and public methods.

Exam Focus

[Encapsulation in Rust] The professor made a key observation about Rust’s relationship with object-oriented programming: “Rust is not an object language. It has many characteristics of object languages. In particular, it implements the concept of **encapsulation**, implements the concept of **polymorphism**, but **rejects** and has no concept of **inheritance**.” We will explore the reasoning behind this rejection shortly.

5.4 Methods: Adding Behavior with `impl` Blocks

If a struct has private fields, how do we interact with its contents? By linking **methods** to the struct. Methods are functions defined within an `impl` block that have visibility of the struct’s internal state.

5.4.1 The Separation of Data and Behavior

In many object-oriented languages (Java, C++, TypeScript), the class declaration contains *both* the data fields and the methods in one block. Rust takes a deliberately different approach:

- The `struct` block declares **only the data**.
- A separate `impl` block attaches **the methods**.

The professor contrasted this explicitly: “In our object-oriented language, we have written the class with the `class` keyword, and inside the block there are the various fields and the various methods, even mixed. The choice of Rust, instead, is to have two separate parts: I introduce the struct, I declare only the data. Then I can have an `impl` block which takes the name of the structure and adds the methods.”

```
pub struct Point {
    x: f64,
    y: f64,
}

impl Point {
    // Constructor (associated function - no self parameter)
    pub fn new(x: f64, y: f64) -> Self {
        Point { x, y }
    }

    // Method: immutable borrow of the receiver
    pub fn length(&self) -> f64 {
        ((self.x * self.x) + (self.y * self.y)).sqrt()
    }
}
```

Notice that `Point` is public, but `x` and `y` have no `pub` prefix—they are private. Outside the current module, nobody has direct access to `x` or `y`. The `impl` block is where we provide controlled access.

5.4.2 The Three Method Receivers

Methods are recognized because in their parameter list they have `self`. The `self` parameter represents the **instance** on which the method operates (the “receiver”). Unlike languages like Java or JavaScript, where `this` is implicit and handled by the compiler, in Rust we **make it explicit**. Why? Because we must specify *what type of access* we are taking on that `self`:

! Exam Priority The Three Types of *self*

1. **&self (immutable borrow)**: “This method has the right to read all fields of my struct, but cannot modify them.” It takes the receiver as a **shared reference**. The usual borrowing rules apply: multiple **&self** borrows can coexist.
2. **&mut self (mutable borrow)**: “This method can access all the fields of the struct **and can modify them**.” It takes the receiver as an **exclusive reference**. For the duration of the method’s execution, the original variable is inaccessible. Only one mutable borrow can exist at a time, and it cannot coexist with immutable borrows.
3. **self (ownership transfer)**: “This method **consumes** the receiver, takes ownership, and transforms it into something else.” After calling such a method, the original variable is **destroyed**—it can no longer be used.

The **self** parameter, if present, **must be the first** in the parameter list. Its type is implicitly the type being implemented (e.g., in `impl Point`, `self` is of type `Point`).

Expected Mastery Level: Very High. You must intuitively know which receiver to use and what happens to the original instance after each kind of method call.

5.4.3 A Complete Example: *mirror*, *length*, and *scale*

Let us see all three receivers in action:

```
pub struct Point {
    x: f64,
    y: f64,
}

impl Point {
    // Takes ownership of self, destroys the original, returns a new Point
    pub fn mirror(self) -> Self {
        Point { x: self.y, y: self.x }
    }

    // Borrows self immutably: reads but does not modify
    pub fn length(&self) -> f64 {
        ((self.x * self.x) + (self.y * self.y)).sqrt()
    }

    // Borrows self mutably: modifies the coordinates in place
    pub fn scale(&mut self, factor: f64) {
        self.x *= factor;
        self.y *= factor;
    }
}
```

Now let us trace through the usage step by step, exactly as the professor did:

```
let p1 = Point::new(3.0, 4.0);    // create point (3, 4)

// mirror() takes 'self' (ownership) -> p1 is consumed!
let p2 = p1.mirror();             // p2 = Point(4, 3). p1 no longer exists.

// length() takes '&self' (immutable borrow) -> p2 is temporarily borrowed
let l1 = p2.length();             // l1 = 5.0. p2 is returned after the call.
```

```
// scale() takes '&mut self' (mutable borrow) -> p2 must be 'mut'
let mut p2 = p2;           // rebind as mutable
p2.scale(2.0);             // p2 is now (8, 6). Exclusive access during call

// After scale returns, the mutable borrow ends. p2 is accessible again.
let l2 = p2.length();      // l2 = sqrt(64+36) = sqrt(100) = 10.0
```

Memory Model Analysis Tracing the Borrow Dance

1. `p1.mirror()`: The compiler sees that `mirror` takes `self` (ownership). `p1` is **moved** into the method. After this call, `p1` no longer exists. The method creates and returns a new `Point`.
2. `p2.length()`: The compiler sees that `length` takes `&self`. It automatically creates an immutable reference `&p2`, passes it to the method, and the reference is destroyed when the method returns. `p2` remains fully usable.
3. `p2.scale(2.0)`: The compiler sees that `scale` takes `&mut self`. It automatically creates a mutable reference `&mut p2`, which grants exclusive read-write access for the duration of the call. No other code can access `p2` during this time. The coordinates are multiplied by 2 in place. The mutable borrow ends when the method returns.
4. `p2.length()`: After the mutable borrow has ended, we can borrow `p2` immutably again. The new length reflects the scaled coordinates.

5.4.4 Method Calls Are Syntactic Sugar

The professor made a very important theoretical point: “Methods don’t really exist. They are just functions.” The call:

```
let l1 = p2.length();
```

is automatically translated by the compiler into:

```
let l1 = Point::length(&p2);
```

`length` is a function inside the `Point` namespace. When we write `p2.length()`, the compiler translates this into `Point::length(&p2)`—the function `Point::length` receiving an explicit reference to `p2`.

Insight

[Methods Are Just Functions — In Every Language] “This is valid in all languages—C++, Java, TypeScript. The concept of method is only a syntactic sugar, because compilers know only variables and functions. This is their job. All other abstractions are just more comfortable ways to give us more readable, easier to manipulate notations. But internally, in the assembler, there is the `call` instruction and the `move` instruction, and that’s it.”

5.4.5 Constructors: The new Convention

In Java, a constructor is a special method identified by the `new` keyword. In Rust, there is **no special constructor syntax**. Any method inside an `impl` block that returns `Self` (with capital S, meaning “the type we are implementing”) can serve as a constructor:

```
impl Point {
    fn new(x: f64, y: f64) -> Self {
        Point { x, y }
    }
}
```

The name `new` is not a keyword—it is a convention, chosen because it evokes the idea from other languages. The professor noted: “You noticed when we created vectors, we always wrote `Vec::new()`. We were explicitly invoking the `new` method of `Vec`.”

Multiple Constructors. Since Rust does **not have function overloading** (you cannot have two methods with the same name but different parameters), if you need multiple constructors, you give them different names. The convention is:

- `new()`: the simplest constructor, often no parameters or minimal parameters.
- `with_capacity(n)`: a constructor that pre-allocates space.
- `with_something(...)`: names that make it clear what the parameter represents.

For example, for `Vec`:

```
let v1 = Vec::<i32>::new();           // empty vector
let v2 = Vec::<i32>::with_capacity(100); // empty vector with 100 slots pre-allocated
```

5.4.6 Associated Functions (Static Methods)

Functions inside an `impl` block that do **not have self** as a parameter are called **associated functions**. They serve two purposes:

1. **Constructors**: if they return `Self`, they act as constructors (like `new`).
2. **Utility functions**: generic functions related to the type but not operating on a specific instance.

Associated functions are called using the double colon syntax: `Point::new(3.0, 4.0)`.

5.5 Why Rust Rejects Inheritance

The professor devoted considerable time to explaining why Rust deliberately omits inheritance, a cornerstone of many object-oriented languages.

Insight

[The Rise and Fall of Inheritance] “After the fall of the wave in the 1990s around the concept of inheritance—which seemed to solve all the problems of the world, at least of programming—we understood that inheritance is a bit of a double-edged sword.”

The problems with inheritance include:

1. **Unwanted coupling:** In Java, single inheritance means everything derives from `Object`. But `Object` contains methods like `equals()`, `toString()`, `wait()`, `notify()`, `notifyAll()` (three versions of `wait()`). Why does every object need `wait()`? Because that object *could* be used for concurrency synchronization. But should a `Point` know about threading?
2. **The Diamond of Death:** When a class inherits from two superclasses that themselves share a common base class, the derived class receives the base class’s members through two different paths, potentially modified in different ways. “It’s like a child whose mother has green eyes and father has blue eyes—what color does the child have? Purple? Both green and blue—how?” This is called the **Diamond of Death**, and it creates many more problems than it solves.
3. **Forced declarations:** Inheritance forces you to declare a parent class, and building healthy hierarchies is extremely difficult in practice.

Insight

[The Origin of Rust: Graydon Hoare’s Elevator] The professor shared the colorful origin story of Rust: “Graydon Hoare, the Rust inventor, worked at Mozilla and programmed in C++. One day, he came home to his high-rise apartment building to find the electricity was out. He had to climb 28 floors on foot. In his frustration and the tedium of climbing, he began thinking about a better language. The thing kept going for about two years—he kept refining the idea—and then Rust started to emerge, first as a personal project, then Mozilla saw it as useful, put money into it, and eventually it became a foundation.” Rust was born from a very concrete frustration: a bug-prone C++ codebase and 28 flights of stairs. Hoare’s key decision was: “It was better to get rid of classes and inheritance.” This does not mean getting rid of *methods*—only getting rid of *inheritance*.

5.6 The Drop Trait: Destructors and RAII

Within a data structure, sometimes the values have a meaning that goes beyond their memory representation. The professor explained: “If I have a number 42, it could be just an integer, or it could be a **file descriptor**. If it’s an integer, just throw it away and release the memory. If it’s a file descriptor, I probably want to **close that file** before discarding the number.”

Rust doesn’t automatically know what our values *mean*. To handle cleanup responsibilities, Rust provides the **Drop trait**—a mechanism for defining a destructor.

5.6.1 How Drop Works

The `Drop` trait defines a single method, `drop(&mut self)`, which is **automatically called by the compiler** when an object is about to cease to exist:

```
struct Shape {
    name: String,
    sides: u32,
```

```

}

impl Drop for Shape {
    fn drop(&mut self) {
        println!("Dropping shape: {}", self.name);
        // Perform cleanup actions here (close files, release resources, etc.)
    }
}

```

Insight

[The Drop Method Is Like a Will] The professor used a vivid analogy: “It’s a bit like writing a will. You write it, but it’s not *your* turn to execute it—someone else does. ‘I leave my bicycle to this person, my figurine collection to that person.’ If you have something to bequeath, you have the possibility to write the will. You will never call `drop` yourself. But you are sure that the compiler will call it right before releasing the memory that the object is using. It’s the final moment that allows you to release, or make all necessary actions, to guarantee a correct transfer of property.”

5.6.2 The RAI Pattern

The idea that in the **constructor** we prepare resources we need, and in the **destructor** we free them, is known as the **RAII pattern**—*Resource Acquisition Is Initialization*.

The professor connected this to the file exercise: “If you think about the exercise you did last week, you opened files, you wrote to a new file, but you never *closed* the file. This is because the `Drop` trait handles it automatically: when you arrive at the bottom of your function where the file variable goes out of scope, the compiler releases it, but immediately *before* releasing it, it calls the `drop` method, inside which is written ‘tell the operating system to perform `file_close`’, after which it throws away the memory. This is the mechanism.”

5.6.3 Implementing Drop

While normal methods are defined in a plain `impl Type` block, the `Drop` trait uses a special syntax:

```

// Normal methods
impl Shape {
    fn new(name: &str, sides: u32) -> Self {
        Shape { name: name.to_string(), sides }
    }
}

// Destructor: special syntax
impl Drop for Shape {
    fn drop(&mut self) {
        println!("Cleaning up shape: {}", self.name);
    }
}

```

The block is written as `impl Drop for Shape`—“implement the `drop` method *for* this type.” Inside, you describe what you think is necessary to do before the value is destroyed. When a `Shape` object reaches the closing curly brace that establishes the end of its lifetime, `drop()` will be invoked automatically.

Warning

[Drop and Copy Are Mutually Exclusive] The `Drop` trait **can only be implemented by structs that do NOT implement the `Copy` trait**. `Copy` and `Drop` cannot coexist. Why? Because `Copy` implies that a simple bitwise duplication is sufficient and safe. But `Drop` implies that the value holds a resource requiring special cleanup. If a value could be both copied (bit-for-bit) and dropped (with cleanup), you would get **double-free errors**—the cleanup code would run on multiple copies of the same resource.

5.7 Structs Are Product Types

Before moving to enums, the professor introduced an important piece of type theory terminology.

Structs are called **Product Types**. Why? Because their **domain**—the set of all possible values—is the **Cartesian product** of the domains of all their fields.

```
struct Point {
    x: i32,
    y: i32,
}
```

The domain of `Point` is the Cartesian product of all 32-bit integers times all 32-bit integers: all possible pairs of integers from approximately -2 billion to $+2$ billion. Hence, it is a **product type**.

5.8 Enums: Expressing Alternatives (Sum Types)

While structs combine fields with “AND” semantics (a `Player` has a name *and* health *and* a level), enums express “OR” semantics: a value is one variant *or* another.

Enums are **Sum Types** because their domain is the **union** (or sum) of the domains of the things that represent them.

5.8.1 Simple Enums: Labels with Optional Values

At their most basic, enums are a way of representing named constants:

```
enum HttpResponse {
    Ok = 200,
    NotFound = 404,
    InternalError = 500,
}
```

These are three values: `Ok`, or `NotFound`, or `InternalError`. The sum of the three domains is just these three constants. The professor explained: “If I hadn’t written `= 200`, `= 404`, `= 500`, internally these would become 0, 1, 2. So the enum, in its elementary version, becomes small integers numbered in sequence. I have the possibility of forcing these numbers to be something different—so I choose the HTTP status codes—but in the end, it’s a way to not manipulate raw numbers that I might confuse with arithmetic values. They are only **distinctive codes** for a semantic purpose.”

5.8.2 Enums with Data Payloads

Here is where Rust’s enums become far more powerful than those in C or Java. Each variant can carry its own **distinct data payload**:

```
enum HttpResponse {
    Ok, // no additional data
    NotFound(String), // carries the name of the missing resource
    InternalError(String, Vec<u8>), // carries a description AND raw data
}
```

The professor walked through the semantics: “If it’s `Ok`, there are no other details. If I say `NotFound`, I add a string which is probably the name of the thing I didn’t find. And if I say `InternalError`, I give you a more articulated structure: not just the description (a string), but also data in the form of a byte vector that allows me to describe better why things went wrong internally.”

The `HttpResponse` type can hold `Ok`, *or* `NotFound` encapsulating a string, *or* `InternalError` encapsulating a string composed with a vector of bytes. This is a kind of sum: the total domain is the union of all three variant domains.

Insight

[Enums Can Also Have Methods] Just like structs, enums can have `impl` blocks with methods:

```
impl HttpResponse {
    fn is_success(&self) -> bool {
        matches!(self, HttpResponse::Ok)
    }

    fn description(&self) -> &str {
        match self {
            HttpResponse::Ok => "Success",
            HttpResponse::NotFound(_) => "Not_Found",
            HttpResponse::InternalError(_, _) => "Internal_Error",
        }
    }
}
```

“Not only do I have the possibility of representing alternative structures, but I also have the possibility of linking methods that allow me to do some work on top.”

5.8.3 Memory Layout of Enums: Tagged Unions

How does the compiler represent an enum with heterogeneous variants in memory? The professor explained this with a concrete example:

```
enum Shape {
    Square(f32), // tag 0, payload: 4 bytes
    Point(f64, f64), // tag 1, payload: 16 bytes
    Empty, // tag 2, payload: 0 bytes
}
```

! Exam Priority Enum Memory Layout: Tagged Union

The compiler examines all the variants that can end up inside the enum and allocates a memory space that is the **largest of the possible alternatives**, plus one or more bytes to use as a **distinctive tag** (discriminant).

For the `Shape` enum above:

- The compiler assigns tag 0 for `Square`, tag 1 for `Point`, tag 2 for `Empty`.
- The largest variant is `Point(f64, f64)` which needs $2 \times 8 = 16$ bytes.
- Plus the tag (1 byte).
- Total: 17 bytes minimum, but the compiler will likely round to a comfortable alignment, probably **24 bytes**.

The tag is always at the beginning. For `Square(5.0)`: the first byte(s) contain the tag value 0, followed by the 4 bytes of the `f32`, followed by padding (unused bytes, because the slot is sized for the largest variant). For `Empty`: the first byte(s) contain the tag value 2, followed by **all padding**—no payload data at all.

Expected Mastery Level: High. You must be able to calculate the approximate size of an enum and explain why it equals the size of the largest variant plus tag plus alignment padding.

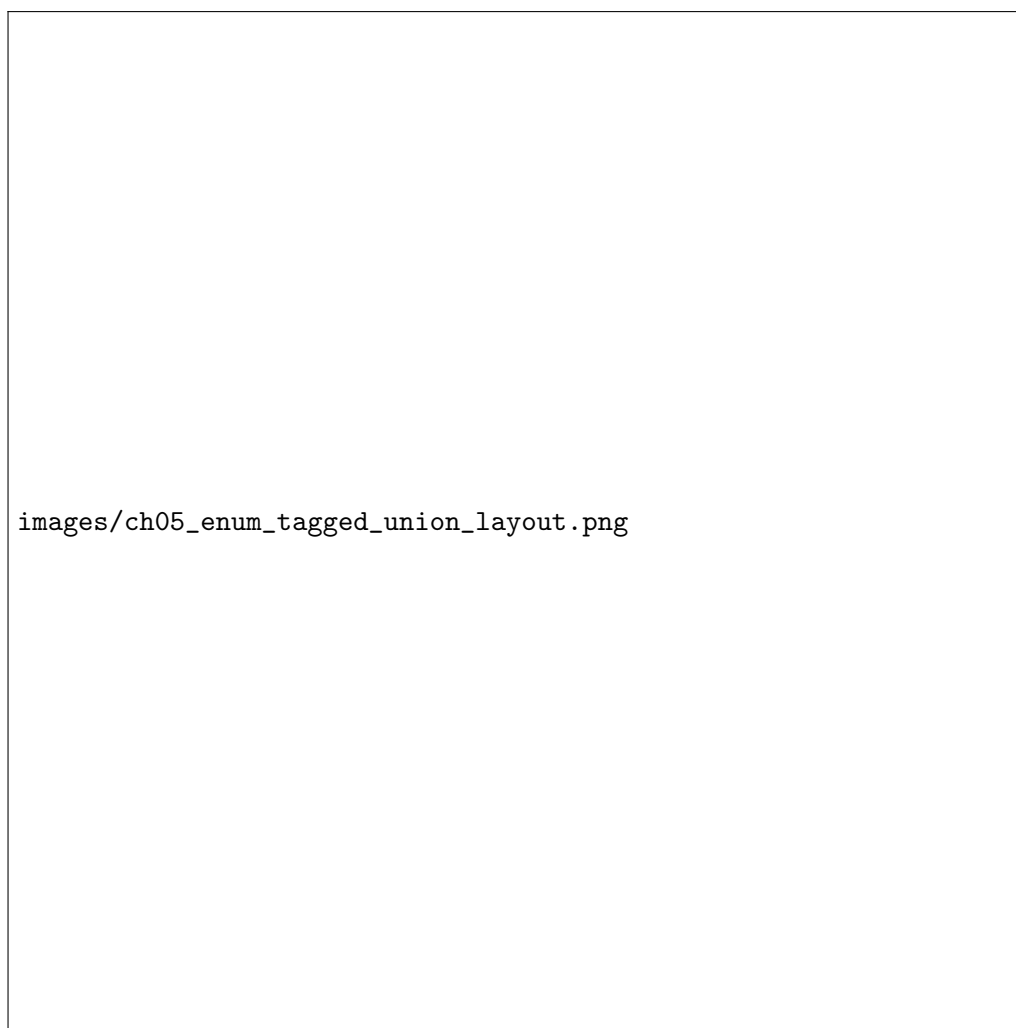


Figure 5.3: Memory layout of `enum Shape`. All variants occupy the same total space. The tag byte identifies which variant is active, and the remaining space holds the payload (or padding if the variant's data is smaller than the largest variant).

5.9 Pattern Matching with match

Enums work **magnificently** with the `match` construct, which allows us to safely inspect which variant is active and extract its payload:

```
enum Shape {
  Square(f64),
  Circle(f64),
  Rectangle(f64, f64),
}

let s = Shape::Circle(5.0);

match s {
  Shape::Square(side) => {
    println!("Square with side {}", side);
  }
  Shape::Circle(radius) => {
    println!("Circle with radius {}", radius);
  }
}
```

```

Shape::Rectangle(w, h) => {
    println!("Rectangle_{}_x{}", w, h);
}
}

```

The `match` construct is **exhaustive**: the compiler refuses to compile if you do not handle every possible variant. If you add a new variant to `Shape` later (say `Triangle(f64, f64, f64)`), the compiler will force you to handle it in every `match` block. This statically eliminates an entire class of runtime errors.

5.9.1 The `if let` Construct

Sometimes, using a full `match` statement is overkill because you are only interested in **one specific variant** and want to ignore the rest. For these scenarios, Rust provides `if let`:

```

if let Shape::Square(side) = s {
    println!("Found_a_square_with_side:{}", side);
}

```

The professor explained: “`if let` is like a `match` in which instead of having to split all the clauses, we just check a single case. If `s` is a `Square`, we do certain things. This is what I need when I’m interested in trying a single alternative, instead of having to handle all 27.”

5.9.2 Destructuring Structs

The professor also briefly mentioned that `match` can be used with structs for destructuring—extracting internal fields directly:

```

struct Point {
    x: f64,
    y: f64,
}

let p = Point { x: 3.0, y: 4.0 };

let Point { x, y } = p; // destructure: x = 3.0, y = 4.0
println!("x={},y={}", x, y);

```

This is similar to destructuring in JavaScript/TypeScript—a convenient way to extract multiple fields into individual variables in a single statement.

5.10 Summary: Product Types vs. Sum Types

Concept	Structs (Product Types)	Enums (Sum Types)
Semantics	<i>A and B and C</i>	<i>A or B or C</i>
Domain	Cartesian product of field domains	Union of variant domains
Example	<code>Player { name, health, level }</code>	<code>HttpResponse { Ok, NotFound, ... }</code>
Memory	Sum of field sizes + padding	Largest variant + tag + padding
Access	Dot notation (<code>s.field</code>)	Pattern matching (<code>match</code>)
Type Theory Name	Product Type	Sum Type / Tagged Union

Exam Relevance and Recurrence



Exam Priority Overall Exam Focus for This Chapter

Score: Very High

Key areas frequently tested:

- **Nominal vs. Structural Typing:** Explain why two structs with identical fields are not interchangeable in Rust.
- **Memory Layout:** Calculate the size of a struct/enum considering padding, alignment, and the tagged union model. Know what `#[repr(C)]` does and when to use it.
- **Method Receivers:** Distinguish `self` vs `&self` vs `&mut self` and predict what happens to the original variable after each kind of method call.
- **Ownership with Struct Update Syntax:** Understand when the `..` operator moves vs copies fields.
- **Drop Trait and RAII:** Explain why `Drop` and `Copy` are mutually exclusive. Describe the RAII pattern.
- **Enum as Tagged Union:** Compute the memory footprint of an enum (largest variant + discriminant + padding).
- **Exhaustive Pattern Matching:** Explain why `match` prevents a class of bugs that `if-else` chains cannot.

Domande ed Esercizi da Esami Passati

Exam Focus

[Domanda 1] All'interno di un programma è definita la seguente struttura dati

```
struct Bucket { data: Vec<i32>, threshold: Option<i32> }
```

Usando il debugger si è determinato che, per una istanza di `Bucket`, essa è memorizzata all'indirizzo `0x00006000014ed2c0`.

Osservando la memoria presente a tale indirizzo, viene mostrato il seguente contenuto (per blocchi di 32bit):

```
308a6e01 00600000 03000000 00000000 03000000 00000000 01000000 0a000000
```

Cosa è possibile dedurre relativamente ai valori contenuti dei vari campi della singola istanza?

Risposta:

In base ai concetti di **Memory Layout** discussi nel capitolo:

- La struttura è composta da un `Vec<i32>` e da un'enum `Option<i32>`. Il compilatore alloca la memoria affiancando i campi (con possibile riordino e padding).
- Il `Vec` occupa 24 byte: 8 byte per il puntatore all'heap (come spiegato per la struttura `String`), 8 byte per la capacità e 8 byte per la lunghezza.
- L'enum `Option` (essendo un *Tagged Union* o *Sum Type*) occupa lo spazio del variant più grande (i 4 byte di `i32`) più un tag per distinguere le varianti, raggiungendo con il padding 8 byte. In totale, la struttura occupa 32 byte.
- I primi due blocchi da 32 bit (`308a6e01 00600000`) formano il puntatore a 64 bit all'heap dove risiedono i dati.
- I successivi due blocchi (`03000000 00000000`) formano un intero a 64 bit di valore 3, che rappresenta la **capacità** del `Vec`.
- I successivi due blocchi (`03000000 00000000`) formano un altro intero a 64 bit di valore 3, che rappresenta la **lunghezza** attuale del `Vec`.
- Il settimo blocco (`01000000`) è il **tag** dell'enum `Option` (indica la variante con valore, ovvero `Some`).
- L'ottavo blocco (`0a000000`) è il **payload** dell'enum, ovvero il valore esadecimale `0x0a` (10 in decimale) rappresentato come `i32`.

Deduciamo quindi che il `Vec` ha 3 elementi (con capacità pari a 3) e che la soglia è impostata a `Some(10)`.

Exam Focus

[Domanda 2] Dato il seguente frammento di codice Rust (ogni linea è preceduta dal suo indice)

```
1. struct Point {  
2. x: i16,  
3. y: i16,  
4. }  
5.
```

Risposta:

In base ai concetti discussi nel capitolo per quanto riguarda le strutture (*Product Types*):

- **Type System Nominale:** Il compilatore registra un nuovo *user-defined type* chiamato `Point`. Essendo il typing in Rust nominale (e non strutturale come in TypeScript), un'altra struct con gli stessi identici campi (`x` e `y`) ma con un nome diverso sarà considerata un tipo completamente incompatibile.
- **Product Type (Struct):** `Point` è un *Product Type* perché aggrega campi eterogenei (in questo caso omogenei) con semantica "AND" (possiede una coordinata *x* e una coordinata *y*).
- **Memory Layout e Padding:** I due campi di tipo `i16` occupano 2 byte ciascuno. La struct richiederà almeno 4 byte. Non vi sono garanzie assolute sull'ordine in cui `x` e `y` saranno disposti fisicamente in memoria, poiché il compilatore è libero di ottimizzare e riordinare il layout (a meno di non forzarlo tramite l'attributo `#[repr(C)]`).

Chapter 6

Error Handling

Software systems constantly interact with unpredictable environments: networks drop, disks fill up, files are locked by other processes, and users provide invalid input. Robust professional software is not defined merely by its ability to execute the "happy path," but by its resilience and predictability when failures occur.

In this chapter, we explore how Rust models and manages errors. By treating errors as standard data types and leveraging the type system, Rust forces developers to acknowledge failures without sacrificing code readability.

6.1 The Taxonomy of Failures: Recoverable vs. Unrecoverable

When an operation fails, we must determine if the program can gracefully recover. Failures generally fall into two categories:

- **Unrecoverable Errors:** These occur when a fundamental invariant of the program is broken, or a catastrophic resource constraint is hit (e.g., Out of Memory, Stack Overflow). For instance, accessing an array out of bounds implies a logic bug. In these scenarios, the safest action is to immediately halt execution to prevent data corruption.
- **Recoverable Errors:** These arise from temporary or external conditions, such as attempting to read a missing configuration file, dealing with a locked file, or failing to connect to a database. The program should anticipate these. Common recovery strategies include:
 - **Exponential Backoff:** When a network server is unreachable, wait 100ms, then 200ms, 400ms, 800ms, up to a sensible maximum to gracefully retry without overwhelming the system. Be careful to avoid infinite loops.
 - **Skip and Continue:** If reading a file and a specific line (e.g., line 27) is corrupted, log the error and continue processing the next lines.
 - **Default Values:** If a configuration file is missing, fallback to sensible default settings.
 - **User Intervention:** If the application is interactive (attached to a console), prompt the user for alternative input. If running in batch mode (e.g., standard input via a pipe), abort or log the failure instead.

! Exam Priority

The Database Analogy (ACID Properties)

Consider how a Database Management System (DBMS) handles failures. If a disk fills up mid-transaction, the DBMS does not corrupt the data; it rolls back to a valid, consistent state. This is governed by ACID properties:

- **Atomicity:** The entire transaction either succeeds or fails.
- **Consistency:** The database moves from one valid state to another (e.g., referential integrity is maintained, non-null and unique constraints are respected).
- **Isolation:** Intermediate states during a transaction are invisible to other operations.
- **Durability:** Once committed, the changes remain permanently.

Professional software should mimic this resilience, managing side-effects and ensuring predictable state transitions even when individual operations fail.

Common Pitfall: Treating file operations like guaranteed database transactions without manually handling partial writes or state corruption on failure (e.g., a file write fails halfway through because the disk is full).

Expected Mastery Level: Deep conceptual understanding; you should be able to explain how error handling in Rust provides similar guarantees to database ACID properties.

6.2 Unrecoverable Errors and Panics

When an unrecoverable error occurs, Rust provides the `panic!` macro. Invoking a panic immediately halts the current thread's normal execution flow.

```
fn main() {
    let device_id = 42;
    panic!("A critical hardware failure occurred on device {}. Aborting.", device_id);
}
```

Just like `println!`, the `panic!` macro accepts formatting placeholders to build detailed, context-rich error messages.

By default, panicking initiates *stack unwinding*. Rust walks back up the call stack, cleaning up variables, calling destructors, and freeing memory cleanly. Alternatively, a panic can be configured at compile time to immediately *abort* the process, leaving the cleanup to the operating system.

Warning

Panic vs. Process Exit

There is a crucial difference between panicking and forcefully exiting via `std::process::exit(code)`. Invoking `exit()` immediately terminates the program without running any destructors, akin to sending a `kill -9` signal to the process from the OS. In contrast, `panic!` safely unwinds the stack. Avoid `exit()` unless you specifically intend to bypass cleanup.

Another useful macro is `unreachable!()`. We use it to mark execution paths that we, the developers, know are logically impossible, even if the compiler cannot prove it. If an

`unreachable!()` is ever executed, it immediately triggers a panic.

Warning

Do not use `panic!` for standard control flow or predictable environmental errors. A library function should never panic on bad user input; it should return a structured error instead, leaving the decision to halt up to the application developer.

6.3 Enums, Structs, and Pattern Matching

Before diving into Rust's primary error structures, we must briefly revisit Enums and pattern matching, as they form the foundation of error handling.

In Rust, Enums are *Sum Types*. From a mathematical perspective, the domain of an enum is the union of the domains of its variants. They can represent entirely distinct alternatives and encapsulate data within those alternatives.

```
enum Shape {
    Square(f64),
    Circle(f64),
}

impl Shape {
    fn area(&self) -> f64 {
        match self {
            Shape::Square(s) => s * s,
            Shape::Circle(r) => r * r * std::f64::consts::PI,
        }
    }
}
```

Memory Model Analysis

Memory Layout of Enums:

Internally, an enum requires memory for a *discriminant* (a tag indicating which variant is currently active, typically at least 1 byte) plus the space required for its largest variant. For instance, if an enum has a `Square(f64)` variant requiring 8 bytes, the enum will take 1 byte for the tag plus 8 bytes for the data. Due to memory alignment rules, this usually pads the total size to 16 bytes.

When deconstructing an enum via `match`, the enum is consumed. If passed by value, its stack memory is reclaimed once the function returns.

When and why to use it: Use `match` when you need exhaustive checking over all possible variants of an Enum. The compiler will enforce that every variant is accounted for, making refactoring much safer.

6.3.1 Selective Deconstructing with `if let` and `while let`

When we only care about extracting data from a single variant, writing a full `match` block with a catch-all `_ => ()` arm can be verbose. Rust provides the `if let` construct for targeted deconstructing.

```
let shape = Shape::Square(5.0);
if let Shape::Square(s) = shape {
    println!("Found a square with side: {}", s);
}
```

This code attempts to assign the value inside `shape` to the variable `s`. If `shape` is indeed a `Square`, the block executes. Because this operation involves a move assignment (unless references are used), the original `shape` is dismantled and becomes inaccessible after this block.

Insight

Enums as Monads

In the functional theory of types, an Enum in this context acts as a *monad*. It is a container that safely encapsulates varying internal details depending on the specific variant. By providing controlled mechanisms to “unpack” this container (like `if let`), Rust forces you to handle the encapsulated state safely.

Similarly, `while let` can be used to continuously pull values out of an Enum until it fails. This is highly useful when reading lines from a file sequentially:

```
while let Ok(line) = reader.read_line() {
    // Process line
}
```

6.3.2 Struct Destructuring

While enums represent alternative variants requiring conditional destructuring (hence `if let`), Structs do not have ambiguity. We know exactly what fields a struct possesses, so destructuring never fails:

```
struct Point { x: f64, y: f64 }
let p = Point { x: 1.0, y: 2.0 };

// Infallible destructuring
let Point { x, y } = p;
```

This syntax avoids the repetitive boilerplate of writing `p.x` and `p.y`. Note that destructuring a struct follows standard move semantics: if the struct implements the `Copy` trait, the original remains usable; otherwise, ownership of the fields transfers to the new variables, rendering the original struct inaccessible. (Note: Modern Rust 2024 editions simplify retaining references during destructuring).

6.4 Recoverable Errors: Result and Option

Historically, languages like C handled recoverable errors by returning special integer codes (e.g., returning 0 for success or -1 for failure). This approach is flawed because the compiler cannot force the developer to check the return code.

Languages like Java and C++ introduced Exceptions, which enforce error handling but hide the control flow. In Java, exceptions were split into `Error` (unrecoverable catastrophic failures like `OutOfMemoryError` or `StackOverflowError` where the JVM breaks) and `Exception` (recoverable). *Checked exceptions* forced developers to handle errors, but often led to unreadable boilerplate (such as deep `catch` blocks), pushing many towards unchecked `RuntimeExceptions`. Furthermore, the superstructure required to manage the call stack and context switching for exception handling is costly. This is a primary reason why C++ exceptions cannot be used when writing Linux kernel modules, as they break the kernel’s execution assumptions.

Rust avoids implicit exception throwing, solving this instead using standard Enums. The two primary types for representing optionality and errors are `Option<T>` and `Result<T, E>`.

```
enum Option<T> {
    Some(T),
```

```

    None,
}

enum Result<T, E> {
    Ok(T),
    Err(E),
}

```

The `Option` enum represents the presence (`Some`) or absence (`None`) of a value. It is ideal when the *reason* for absence is obvious or irrelevant.

The `Result` enum is more expressive. It represents either a successful outcome containing a value of type `T`, or a failure containing an error of type `E`. Because `Result` is a standard return type, it explicitly signals in the function signature that an operation can fail.

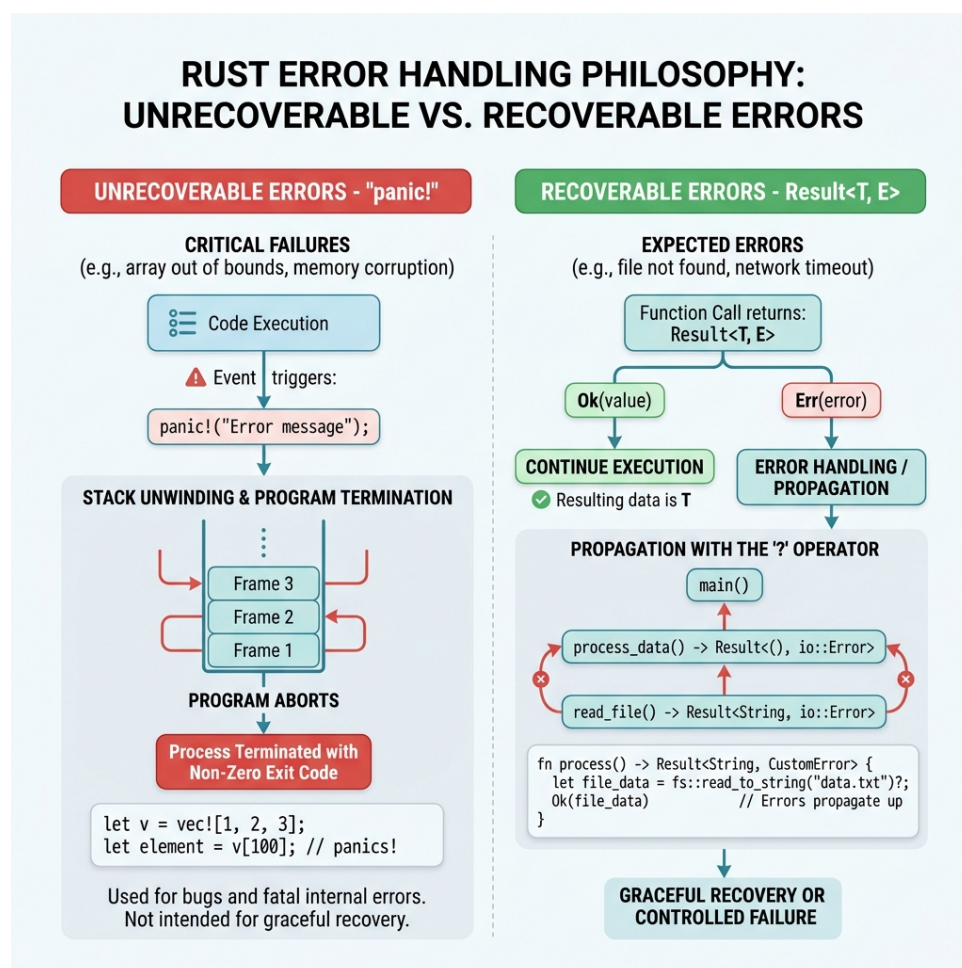


Figure 6.1: Conceptual model of Error Handling with `Result` in Rust.

6.4.1 Extracting Values and Handling Errors

When a function returns a `Result`, you must explicitly handle it. The most exhaustive method is using a `match` block.

```

use std::fs::File;

let file_result = File::open("config.txt");

let file = match file_result {
    Ok(f) => f,

```

```

Err(e) => {
    println!("Failed to open file: {}", e);
    return;
}
};

```

Memory Model Analysis

The `File::open` call asks the OS to allocate a file descriptor, and it returns a `Result<File, std::io::Error>` enum allocated on the stack. The `match` expression consumes this `Result` by value. If it is `Ok`, ownership of the `File` struct is moved into the `file` variable. If it is `Err`, the error `e` is captured by value, printed, and the function returns early. Since `Result` transfers ownership, `file_result` is invalid and inaccessible after the `match` block.

Rust also provides a suite of helper methods on `Result` and `Option`:

- `is_ok()` / `is_err()`: Returns a boolean indicating the variant.
- `map()`: Applies a function to the inner value if it is `Ok` or `Some`, leaving errors untouched.
- `contains(&x)`: Returns true if the result is successful and contains a value that exactly matches the provided argument `x`.

Note

The `ok()` and `err()` methods are extremely useful for converting between `Result` and `Option`. Calling `ok()` on a `Result` returns `Some(value)` if it was successful, and `None` if it was an error, effectively discarding the error details when you no longer need them. Conversely, you can convert an `Option` into a `Result` using `ok_or()`.

For rapid prototyping, you might see `unwrap()` or `expect()`.

```

// Panics if the result is an Err
let file = File::open("config.txt").unwrap();

// Panics with a custom message
let file = File::open("config.txt").expect("config.txt must exist!");

```



Exam Priority

The Danger of `unwrap()` and `expect()`

Using `unwrap()` explicitly converts a recoverable error (`Result`) into an unrecoverable error (`panic!`). In professional code, `unwrap()` should be avoided unless you are absolutely certain the operation cannot fail (e.g. proving an invariant). Panicking initiates stack unwinding or aborts the process entirely, potentially leaving external resources or shared state in an inconsistent state.

Common Pitfall: Using `unwrap()` in production code for convenience, resulting in unexpected application crashes when an environment changes (like a missing file or a network drop).

Expected Mastery Level: High. You must confidently distinguish between a recoverable failure (to be handled via `Result` and `?`) and an unrecoverable failure (where `unwrap` or `panic!` is semantically correct).

6.5 The Happy Path: The ? Operator

Handling every error manually with `match` drastically increases cyclomatic complexity. Our code becomes deeply nested, obscuring the primary logic (the "Happy Path").

Often, a function doesn't know how to handle an error; it simply needs to propagate it up to the caller. Rust provides the `?` (question mark) operator to automate this boilerplate.

Consider this verbose function:

```
use std::fs::File;
use std::io::Read;

fn read_file_verbose(name: &str) -> Result<String, std::io::Error> {
    let mut file = match File::open(name) {
        Ok(f) => f,
        Err(e) => return Err(e), // Early return on error
    };

    let mut s = String::new();
    match file.read_to_string(&mut s) {
        Ok(_) => Ok(s),
        Err(e) => Err(e),
    }
}
```

The `?` operator can be placed after any expression returning a `Result`. If the value is `Ok`, it unwraps the inner value and execution continues. If the value is `Err`, it *immediately returns* from the entire function, passing the error to the caller.

Here is the exact same logic written idiomatically:

```
fn read_file(name: &str) -> Result<String, std::io::Error> {
    let mut file = File::open(name)?;
    let mut s = String::new();
    file.read_to_string(&mut s)?;
    Ok(s)
}
```

Memory Model Analysis

`name` is a borrowed string slice (`&str`), meaning it points to string data managed elsewhere without taking ownership. `File::open` returns a `Result` on the stack. The `?` operator consumes it: on success, the `File` struct's ownership is moved to `file`. `String::new()` allocates a string struct on the stack with a pointer to an initially empty heap buffer. `read_to_string` mutably borrows `s` (`&mut s`), enabling it to dynamically resize the heap buffer to fit the file contents. Finally, `Ok(s)` takes ownership of `s` and transfers it back to the caller wrapped in a `Result`, seamlessly preserving the heap-allocated buffer.

When and why to use it: The `?` operator should be your default choice for propagating errors. It flattens the "Happy Path" and delegates error handling to higher levels in the call stack where more context is available.

Insight**The Power of the ? Operator**

The ? operator is not just syntactic sugar; it defines the visual flow of Rust code. It keeps the core logic flat and readable while maintaining strict type safety. Notice that any function using the ? operator must itself have a return type compatible with the error being propagated (e.g., returning a `Result`).

6.6 Handling Heterogeneous Errors

A significant challenge arises when a single function performs operations that produce *different* error types. For example, reading a file produces an `std::io::Error`, but parsing its string contents into an integer produces an `std::num::ParseIntError`.

```
use std::fs::File;
use std::io::{self, BufRead};

// What should the Return Type be?
// We have two incompatible Error types!
fn sum_file(name: &str) -> Result<i32, ???> {
    let file = File::open(name)?; // io::Error
    let reader = io::BufReader::new(file);
    let mut sum = 0;

    for line in reader.lines() {
        let line = line.unwrap(); // Using unwrap to avoid type mismatch
        let num: i32 = line.parse()?; // ParseIntError
        sum += num;
    }

    Ok(sum)
}
```

We cannot place two different types in the `E` generic parameter of a standard `Result`. There are three primary ways to solve this gracefully.

6.6.1 Trait Objects: `Box<dyn Error>`

We can use polymorphism by returning a pointer to *any* type that implements the `Error` trait.

```
use std::error::Error;

fn sum_file(name: &str) -> Result<i32, Box<dyn Error>> {
    let file = File::open(name)?;
    let reader = io::BufReader::new(file);
    let mut sum = 0;

    for line in reader.lines() {
        let line = line?; // Works automatically!
        let num: i32 = line.parse()?; // Works automatically!
        sum += num;
    }

    Ok(sum)
}
```

Memory Model Analysis

Returning `Box<dyn Error>` triggers a heap allocation if an error occurs. The specific error struct (like `std::io::Error` or `std::num::ParseIntError`) is allocated and moved onto the heap. A *fat pointer* (comprising a data pointer and a vtable pointer for dynamic dispatch) is returned on the stack inside the `Result::Err`. While this provides extreme flexibility for polymorphic error handling, it incurs the runtime cost of memory allocation and pointer indirection, unlike generic `Result<T, E>` objects which reside entirely on the stack.

The `?` operator automatically converts specific errors (like `io::Error`) into a heap-allocated `Box<dyn Error>` by leveraging the standard library's implementation of the `From` trait. While highly flexible, the caller loses compile-time knowledge of exactly which errors can occur.

To handle specific errors dynamically, the caller must use *downcasting*:

```
fn handle_sumfile_errors(name: &str) {
    match sum_file(name) {
        Ok(sum) => println!("The sum is: {}", sum),
        Err(e) => {
            if let Some(io_err) = e.downcast_ref::<io::Error>() {
                println!("Disk/Network failure: {}", io_err);
            } else if let Some(parse_err) = e.downcast_ref::<std::num::ParseIntError>() {
                println!("Invalid number format: {}", parse_err);
            } else {
                unreachable!("Unexpected error type!");
            }
        }
    }
}
```

6.6.2 Custom Enums

For rigorous domains, creating a custom Enum to unify the errors is the most robust approach.

```
enum SumFileError {
    Io(std::io::Error),
    Parse(std::num::ParseIntError),
}
```

This gives the caller exhaustive, type-safe matching without needing dynamic dispatch. However, manually writing the conversion boilerplate to allow the `?` operator to seamlessly cast `io::Error` into `SumFileError::Io` can be tedious.

6.6.3 Ecosystem Solutions: `thiserror` and `anyhow`

Because managing heterogeneous errors is so common, the Rust ecosystem relies heavily on two community crates:

- **thiserror**: Used when writing *Libraries*. By simply adding the `#[derive(Error, Debug)]` attribute, and annotating Enum variants with `#[error("...")]` and `#[from]`, it automatically implements the `Error`, `Display`, and `From` traits. This allows you to elegantly expose strict, structured errors to your users as part of your library's API contract.
- **anyhow**: Used when writing *Applications*, where code simplicity takes priority over strict error type inspection. It provides a smart `anyhow::Result` type that transparently absorbs any error implementing the `Error` trait. It allows you to attach custom contextual strings to explain what the code was trying to do when the failure occurred (e.g.,

```
file.read().context("Failed to read config") or .with_context(|| format!("Missing  
path {}", path))), and handles stack traces gracefully.
```

! Exam Priority

Handling Heterogeneous Errors

Be prepared to identify why a function containing multiple `?` operators fails to compile due to mismatched error types. Use custom Enums (or `thiserror`) when you need the caller to meticulously inspect and recover from specific errors; use `Box<dyn Error>` (or `anyhow`) when you simply want to propagate a formatted error trace and abort or log it.

Common Pitfall: Misunderstanding the performance and API consequences of `Box<dyn Error>`. While easy to write, it hides error types from the compiler, making strict error recovery tedious and requiring manual downcasting.

Expected Mastery Level: Very High. You must be able to recognize heterogeneous errors and implement both a custom enum wrapper and a dynamic dispatch approach.

Further Reading

To deepen your understanding of error handling in Rust, the following resources (recommended in the lectures) are highly valuable:

- Rust Error Handling In Practice: A solid overview of basic error management concepts with practical examples.
- The simplest guide to error handling in Rust: A hands-on guide focusing heavily on how to structure errors using the `thiserror` and `anyhow` crates.
- Idiomatic Error Handling in Rust: Another practical approach to applying Rust's patterns idiomatically.
- Error Handling In Rust - A Deep Dive: An excerpt from the book *"Zero To Production In Rust"* that explores advanced error management in real-world scenarios.

Exam Relevance and Recurrence

Score: Very High

Notes: Error handling is guaranteed to appear in exams. You will likely be asked to refactor verbose code using `match` into idiomatic code using the `?` operator. Questions frequently cover the implications of using `unwrap()`, memory models and cost differences between `Box<dyn Error>` and custom Enums, and identifying missing implementations to allow the `?` operator to seamlessly cast errors.

6.7 Domande ed Esercizi da Esami Passati

Exam Focus

Domanda 1

1. Si descriva il meccanismo di gestione degli errori fornito dal linguaggio RUST, differenziando tra quelli recuperabili e irrecuperabili
2. Si spieghi il funzionamento dell'operatore ?
3. Si mostri un esempio di applicazione dell'operatore ? scrivendo una funzione (avendo cura di scriverla completamente) che deve cercare una stringa (`my_string`) e restituire il suo offset in un file, se presente.

Soluzione:

1. In Rust, la gestione degli errori distingue tra due categorie principali:
 - **Errori irrecuperabili:** si verificano quando vengono violate invarianti fondamentali (es. accesso fuori dai limiti di un array) o per vincoli insuperabili. Vengono gestiti tramite la macro `panic!` (o `unreachable!()`), che interrompe immediatamente l'esecuzione del thread corrente ed esegue lo *stack unwinding* per liberare la memoria in modo sicuro.
 - **Errori recuperabili:** sono fallimenti prevedibili legati all'ambiente (es. file mancante). Rust non usa eccezioni implicite, ma si affida ai tipi `Result<T, E>` e `Option<T>`. Il compilatore forza lo sviluppatore a gestire esplicitamente questi scenari, ad esempio tramite costrutti di pattern matching come `match` o `if let`.
2. L'operatore `?` è uno strumento idiomatico per propagare gli errori, riducendo la verbosità del codice (*boilerplate* dei blocchi `match`). Posizionato dopo un'espressione che restituisce un `Result`, se il risultato è `Ok(T)`, l'operatore estrae il valore interno `T` permettendo all'esecuzione di continuare; se il risultato è `Err(E)`, la funzione termina immediatamente restituendo l'errore al chiamante. L'operatore gestisce anche conversioni automatiche di tipi di errore compatibili.
3. Esempio di applicazione dell'operatore `?` per la ricerca di una stringa in un file:

```
use std::fs::File;
use std::io::{self, Read};

fn find_string_offset(name: &str, my_string: &str) -> Result<Option<usize>, i
    let mut file = File::open(name)?; // Propaga l'errore se il file non esis
    let mut contents = String::new();
    file.read_to_string(&mut contents)?; // Propaga l'errore se la lettura fa
    Ok(contents.find(my_string)) // Restituisce l'offset avvolto in Result::O
}
```

Exam Focus**Domanda 2**

Si illustri come sia possibile gestire correttamente le situazioni di errore in Rust, distinguendo tra `Option` e `Result`.

Soluzione:

In Rust, gli errori recuperabili vengono gestiti evitando le eccezioni implicite e utilizzando due Enum standard:

- **`Option<T>`:** rappresenta la presenza (`Some(T)`) o l'assenza (`None`) di un valore. È la scelta ideale quando il motivo del fallimento o dell'assenza del dato è ovvio o irrilevante.
- **`Result<T, E>`:** è un'astrazione più espressiva che rappresenta un esito di successo con il valore associato (`Ok(T)`) o un fallimento con l'errore associato (`Err(E)`). Includendo esplicitamente il potenziale fallimento nella *signature* della funzione, il compilatore costringe a gestire entrambi i casi.

Queste strutture devono essere gestite in modo esplicito (es. con `match`, `if let` o l'operatore `?`). Esistono inoltre vari metodi di utilità come `ok()` ed `err()` che permettono di convertire agevolmente un `Result` in un `Option`, scartando i dettagli dell'errore. L'uso di funzioni come `unwrap()` è invece sconsigliato in produzione, poiché converte tali errori recuperabili in errori irrecuperabili causando un `panic!` e arrestando l'esecuzione del programma.

Chapter 7

Modules and Visibility

7.1 Introduction: The Need for Organization

When writing simple academic exercises, a single monolithic source file is often sufficient. However, as projects evolve into systems that solve real-world problems, complexity naturally grows. A file with thousands of lines becomes unwieldy: it is difficult to navigate, hard to understand, and almost impossible for multiple people to work on simultaneously.

To manage this complexity, we must divide our code into logical, bite-sized components. This practice provides several core benefits:

- **Maintainability:** Dividing code into smaller pieces makes it easier to comprehend, modify, and extend over time, reducing the risk of unintended side effects.
- **Encapsulation:** By hiding implementation details, we ensure that other parts of the program interact only with a well-defined public interface. Think of a car: you interact with the steering wheel and pedals, but the internal engine mechanics remain safely enclosed under the hood. You don't need to touch them to drive, and keeping them closed off prevents you from accidentally breaking the engine.
- **Safety at Compile Time:** Rust's visibility system allows the compiler to detect logical errors and API abuse during compilation, increasing the reliability of the software.
- **Reusable Libraries:** Structuring code into well-designed modules and crates allows us to reuse components across projects, reducing duplication.

Rust provides a powerful module system designed specifically to handle these software engineering challenges, allowing us to build scalable and robust applications.

7.2 The Rust Project Structure: Packages and Crates

To understand how Rust organizes code, we must define its hierarchical project structure, which consists of **Packages**, **Crates**, and **Modules**.

Insight

Packages vs. Crates A **Package** is the highest level of organization. It is a container managed by Cargo (defined by a `Cargo.toml` file) that groups together one or more Crates. A **Crate** is the fundamental unit of compilation and versioning in Rust. When you invoke `cargo build`, the compiler processes a crate to produce either an executable binary or a library. The process of combining these crates is called *linking*, which can be *static* (the executable includes a full copy of the library code) or *dynamic* (external code is loaded at runtime via `.dll`, `.so`, or `.dylib` files).

Memory Model Analysis

Memory and Process Separation with Libraries

The professor emphasizes a crucial operating system concept regarding libraries: "The library, in total, does not allow me to have a variable divided between two processes. Two processes are and remain separate." Even if multiple executables load the same dynamic library, they do not share memory or variables. The operating system might share the *read-only code pages* in physical memory to save RAM, but the data segments remain strictly private to each running process.

Furthermore, regarding linking:

- **Static Linking:** The linking is done directly in the compilation phase. The resulting executable is larger but self-contained and stable.
- **Dynamic Linking:** The linking phase only injects markers (or stubs) into the executable. When the program is launched, the OS locates the DLL/SO/DYLIB and maps it into the process's memory space. This makes the executable smaller, but relies on the library being present on the target machine.

By default, when you create a new project, Cargo provides a binary crate with the root file `src/main.rs`. The `main` function inside this file serves as the entry point for the executable. If you want to create a library crate—code meant to be shared and reused—the root file is `src/lib.rs`.

7.2.1 Building Multiple Executables

A single package can contain multiple executable binaries. For instance, in a network application, you might want a `client`, a `server`, and a `tool` executable that all share the same underlying logic. By placing your shared logic in `src/lib.rs` and adding additional source files in a `src/bin/` directory, Cargo will automatically compile each file in `src/bin/` as a separate binary. All these executables share the same dependencies, have the same version, and are compiled together, making it easy to maintain related applications.

7.2.2 Versioning

Libraries and packages are versioned using Semantic Versioning (e.g., `1.2.5`), composed of three parts:

- **Major:** Incremented for incompatible API changes. As the professor notes: "The Major changes when a main function of the program changes, breaking the compatibility with the past... using file scripts in the CSV format, after I'm supporting file scripts in Excel format... the same program no longer reads those old ones."

- **Minor:** Incremented for adding new functionality in a backward-compatible manner. "The minor instead changes when our program adds a feature. I read the CSV first, then I read the Excel. The CSV continues to read them, no problem."
- **Patch (Build):** Incremented for backward-compatible bug fixes. "The structure of the program is the same. But I removed bugs... It's very normal to see high numbers."

7.3 Defining Modules and Visibility

While crates define the overarching structure, **Modules** (`mod`) allow us to organize code within a crate. Modules group related functions, structs, traits, and enums together. You can think of modules as folders within your crate's file system, and indeed, they often map directly to the actual file system directory structure.

Insight

The Folder Analogy The professor compares creating modules to organizing university files: "You already do it in part... you are often divided into polytechnic, course 1, course 2, exercise 1, exercise 2. You have divided your documents into smaller pieces... so that when you have to find things, you can get there. The same goes for the code. We want to divide the code into smaller pieces... and it makes it available to those who need it."

See Figure 7.1 for a conceptual representation of how modules branch out from the crate root.

RUST MODULE TREE ANATOMY & HIERARCHY

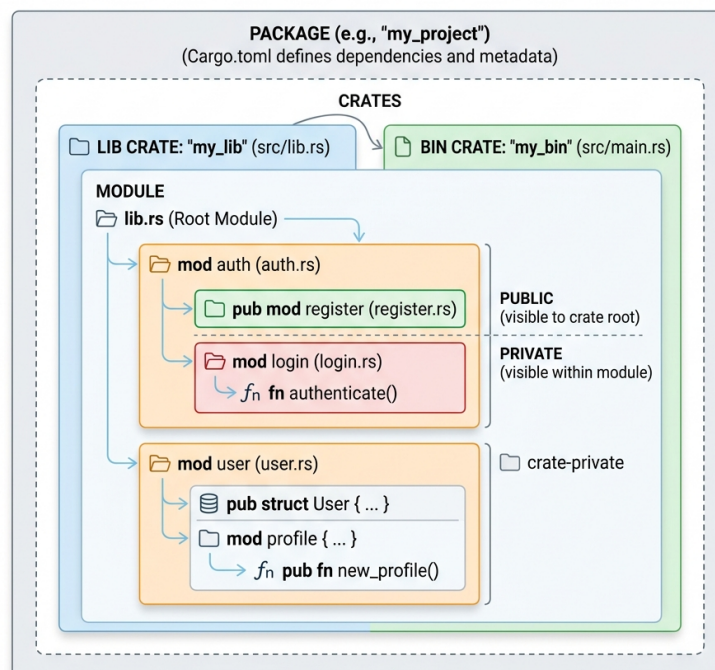


Figure 7.1: The hierarchical tree structure of modules originating from the crate root.

7.3.1 Modules and Source Files

While modules can be declared inline, writing all code in a single file quickly defeats the purpose of modularity. Rust allows mapping modules to separate source files and directories:

- **Inline Modules:** Declared within the same file using `mod name { ... }`. Useful for small modules, such as isolating unit tests.
- **Separate Files:** A module named `my_mod` can be placed in a file named `my_mod.rs` in the same directory. The parent module simply declares it with `mod my_mod;`. If this declaration is missing, the file is completely ignored by the compiler.
- **Directories for Sub-modules:** If `my_mod` contains further sub-modules, you can create a folder named `my_mod/` and place the sub-module files inside it.

Insight

Inline Modules for Testing A common use case for inline modules is grouping tests. You can use the `#[cfg(test)]` attribute on an inline module to tell the compiler to only include this code when running tests:

```
#[cfg(test)]
mod tests {
    use super::*; // Import symbols from the parent module

    #[test]
    fn test_hello() {
        // Test implementation here
    }
}
```

This keeps test code close to the implementation but prevents it from bloating the compiled release binary.

7.3.2 The `mod` and `pub` Keywords

In Rust, all items (functions, structs, fields, modules) are **private by default**. They are only visible to the module in which they are defined and its descendants. To make an item accessible from outside its module, we use the `pub` keyword. This explicit visibility model is the cornerstone of encapsulation in Rust.

Let's look at an example demonstrating module declaration and visibility rules.

```
// src/lib.rs

// Declare a module named 'network'.
// The compiler will look for its contents inline, in 'network.rs', or 'network/mod'
pub mod network {
    // A public struct
    pub struct ServerConfig {
        pub port: u16, // Public field, accessible anywhere
        ip_address: String, // Private field, only accessible within 'network'
    }

    impl ServerConfig {
        // A public constructor to instantiate the struct
        pub fn new(port: u16, ip: &str) -> Self {
            ServerConfig {
                port,
            }
        }
    }
}
```

```

        ip_address: ip.to_string(),
    }
}

// A private function, an internal implementation detail
fn log_connection() {
    println!("New connection established.");
}
}

```

Practical Usage - When and Why to Use It: Use this pattern when you want to enforce validation or initialization logic. By keeping fields private, you force clients to use the `new` constructor, preventing them from creating structurally invalid states (e.g., an invalid IP format) and controlling the instantiation process.

Deep Memory Model Analysis: In the constructor `pub fn new(port: u16, ip: &str) -> Self`, the `port` parameter is passed by value. Because `u16` implements the `Copy` trait, it is copied directly on the stack into the new `ServerConfig` instance. The `ip` parameter is a string slice (`&str`), which is a borrowed reference pointing to string data elsewhere (stack, heap, or static memory). However, the `ServerConfig` struct needs to own its data to be safely returned and dropped later. The call to `ip.to_string()` performs a heap allocation, copying the character data into a newly allocated `String`. The ownership of this heap-allocated `String` is then transferred into the `ServerConfig` struct, giving it full ownership and a lifetime independent of the original `ip` slice.

In this snippet, `network` is a public module containing the `ServerConfig` struct. While the struct itself is public, its `ip_address` field remains private. This means external code cannot directly instantiate `ServerConfig` using a struct literal because it cannot provide the private field; it must use the provided `pub fn new` constructor. This guarantees that internal invariants are maintained and users cannot bypass the intended logic.

Note

If a module is private, all of its contents are unreachable from the outside, even if those contents are marked with `pub`. The `pub` keyword only opens visibility up to the next level of the module hierarchy. A complete, uninterrupted chain of `pub` declarations is required to expose an item to the external API.

7.3.3 Navigating Paths and Re-exporting

To use an item from another module, we must specify its path. Paths can be *absolute* or *relative*:

- **Absolute paths** start from the crate root using the `crate::` prefix (e.g., `crate::network::ServerConfig`) or an external crate name (e.g., `std::mem::size_of_val`).
- **Relative paths** start from the current module. They can start directly with a module identifier, or use `self::` (representing the current module, like `./` in a filesystem) and `super::` (representing the parent module, like `../` in a filesystem).

Typing long absolute paths is tedious. We can bring a path into the current scope using the `use` keyword, functioning like an alias.

Insight

The "Friend" Analogy for `use` The professor compares the `use` keyword to how we address people: "When you're dealing with a stranger, you call him Mario Rossi [full path]. When that's your friend, you call him Mario. `use` is simply saying this, which I know. So you don't have to give it all the genealogy."

```
use crate::network::ServerConfig;

fn main() {
    let config = ServerConfig::new(8080, "127.0.0.1");
}
```

Warning

Wildcard Imports (*): You can use an asterisk to import all public items from a module (`use module::*`). The professor warns against this: "It's not necessarily a great idea... because we carry on with things that we don't control. There could be someone of whom we don't know, whose name coincides with another of our names, and we get behaviors that we don't want." Always prefer explicit imports.

Sometimes, a module's internal hierarchy is complex, but we want to provide a flat, simple API to the users of our library. We can achieve this using **re-exporting** with `pub use`. This takes an item defined in a deeply nested, possibly private module and exposes it at a higher level, effectively decoupling our internal code organization from the public interface we present to the world.

**Exam Priority**

Re-exporting for API Stability (`pub use`) The professor strongly emphasizes that designing for evolvability is the main reason to use modularity in real-world systems. Re-exporting is critical here: by exposing internals through a `pub use` facade in the library root, you can completely reorganize the internal sub-modules (e.g., switching from an old `beta` module to a `beta_v2` module) without breaking the code of any user who depends on your library.

Common Pitfall: Exposing complex, deeply nested module structures to clients (e.g., forcing them to write `crate::internal::submodule::details::MyStruct`). This tightly couples the client to your internal structure, making future refactoring impossible without breaking changes.

Expected Mastery Level: High. You must know how to refactor internal modules while keeping the public API stable using `pub use`.

7.4 External Dependencies

No system exists in isolation. Rust's package manager, Cargo, makes it incredibly easy to leverage the broader ecosystem of external crates hosted on `crates.io`.

To add an external library, we declare it in the `[dependencies]` section of our `Cargo.toml` file. For example, to include `clap`, a popular declarative library for parsing command-line interfaces:

```
[dependencies]
clap = { version = "4.5", features = ["derive"] }
```

When you next run `cargo build`, Cargo will automatically fetch, resolve, and compile the external library from `crates.io`.

Using the `derive` feature, `clap` allows us to declaratively define our command-line arguments using a struct:

```
use clap::Parser;

#[derive(Parser, Debug)]
#[command(version, about = "A simple greeting program")]
struct Args {
    /// Name of the person to greet
    #[arg(short, long)]
    name: String,

    /// Number of times to greet
    #[arg(short, long, default_value_t = 1)]
    count: u8,
}

fn main() {
    let args = Args::parse();
    for _ in 0..args.count {
        println!("Hello {}!", args.name);
    }
}
```

Practical Usage - When and Why to Use It: Use declarative macro-based parsing (like `clap`'s `derive` feature) whenever building command-line utilities. It drastically reduces boilerplate error-handling and automatically provides help menus, bounds checking, and type conversion.

Deep Memory Model Analysis: The call to `Args::parse()` parses the environment arguments and constructs the `Args` struct. The `args.name` field is of type `String`, which means the parsed string argument has been copied into a new heap allocation, and the ownership of this heap memory is given to the `args` instance living on the `main` function's stack. As `args` owns the data, there are no complex lifetime constraints tying the struct to the raw OS string arguments. When `main` completes, the `args` struct is dropped, recursively triggering the drop of the heap-allocated `name` string.

This approach leverages the builder pattern behind the scenes, automatically generating the logic required to parse incoming arguments, apply constraints, and populate the `Args` struct.

Exam Focus

The Standard Library Prelude Notice that we never have to write `use std::vec::Vec;` or `use std::string::String;`. This is because Rust automatically imports a small set of highly ubiquitous standard library items into every module. This automatic import is known as the **prelude** (`std::prelude`).

When you use library functions, "you are exploiting pieces of Rust's standard library. Vector is inside the library called `STD`. File is inside the library called `STD::IO`." Even though they are built-in, they function as external modules that are automatically linked.

7.5 Foreign Function Interface (FFI)

Rust's strict guarantees of memory safety and zero-cost abstractions make it an excellent choice for system-level programming. However, we often need to integrate Rust code with other

programming languages, such as C or Python. This interoperability is achieved through a Foreign Function Interface (FFI).

To allow other languages to call our Rust code, we must compile our library as a **dynamic library** compatible with C calling conventions.

7.5.1 Building a Dynamic C-Library

Rust supports multiple crate types for libraries, defined via the `crate-type` key in `Cargo.toml`:

- `rlib`: A static Rust library (the default).
- `dylib`: A dynamic Rust library.
- `cdylib`: A dynamic library conforming to the C Application Binary Interface (ABI), exportable to other languages.
- `staticlib`: A static library conforming to the C ABI.

To build a library compatible with other languages, we modify our `Cargo.toml` to specify the crate type as `cdylib`:

```
[lib]
crate-type = ["cdylib"]
```

If no `crate-type` is specified, Cargo defaults to building an `rlib` (Rust static library), which is highly optimized but only usable by other Rust programs. Setting it to `cdylib` generates a dynamic library specifically formatted for C-interoperability (`.dll` on Windows, `.so` on Linux, and `.dylib` on macOS).

Next, we write the Rust function we wish to expose. We must apply two crucial modifiers:

1. `extern "C"`: This tells the Rust compiler to use the C Application Binary Interface (ABI) for this function, ensuring the way arguments are passed and values are returned matches what C (and Python) expects.
2. `#[unsafe(no_mangle)]` (or `#[no_mangle]` in older editions): By default, the Rust compiler alters (mangles) function names to include type information for linking purposes (e.g., renaming `add` to `add_i32_i32` to handle parameters). This attribute disables mangling, ensuring the exported function retains exactly the name we gave it. The professor notes: "It's unsafe because it could break things" since the compiler can no longer guarantee symbol uniqueness or type safety across boundaries.

```
// src/lib.rs

#[unsafe(no_mangle)]
pub extern "C" fn add_numbers(a: i32, b: i32) -> i32 {
    a + b
}
```

! Exam Priority

FFI Modifiers: `extern "C"` and `no_mangle` The professor explicitly highlights this mechanism to show how Rust can be embedded as a high-performance engine within Python or C ecosystems.

Common Pitfall: Forgetting `#[unsafe(no_mangle)]`. If omitted, the Rust compiler renames the function to something mangled like `_ZN7libname11add_numbers17h...E`, making it impossible for the Python `ctypes` module to find it by the name `add_numbers`. Furthermore, failing to use `extern "C"` means the function will use the Rust ABI, which may pass arguments in different registers than the C ABI expects, leading to crashes or garbage data.

Expected Mastery Level: Medium. You should conceptually understand why these modifiers are mandatory for interoperability across language boundaries.

When compiled with `cargo build --release`, Cargo will produce a dynamic library file (`.dll` on Windows, `.so` on Linux, or `.dylib` on macOS).

7.5.2 Calling Rust from Python

Other languages can now load this compiled dynamic library and invoke the `add_numbers` function. Here is an example of how Python can utilize our Rust code using its standard `ctypes` module:

```
import ctypes
import os

# Load the dynamic library compiled by Rust
# Note: The extension depends on the OS (.so, .dll, or .dylib)
lib_path = os.path.abspath("target/release/libpack1.dylib")
rust_lib = ctypes.CDLL(lib_path)

# Configure the argument and return types
# The professor notes: "I am instructing Python on the interface of my function."
rust_lib.add_numbers.argtypes = (ctypes.c_int32, ctypes.c_int32)
rust_lib.add_numbers.restype = ctypes.c_int32

# Call the Rust function natively!
result = rust_lib.add_numbers(10, 32)
print(f"Rust calculated: {result}") # Output: Rust calculated: 42
```

Warning

FFI intrinsically bypasses some of Rust's safety checks because the compiler cannot guarantee the behavior of the foreign code calling into or being called by Rust. Designing robust FFI boundaries requires careful attention to memory ownership, data representation, and thread safety.

7.6 Domande ed Esercizi da Esami Passati

Nessuna domanda specifica da esami passati è stata mappata direttamente su questo capitolo.

Chapter 8

Polymorphism and Traits

When building complex software systems, we frequently encounter the need to represent diverse concepts that share common capabilities. For instance, in an e-commerce application, you might define distinct data structures to represent a `User` and an `Order`. While their internal memory layouts and business logic differ entirely, both might need to be “validated” before being processed or saved to a database.

If we were to write entirely disjoint validation routines without a unifying abstraction, our codebase would quickly become repetitive and brittle. We need a way to express that different types can exhibit a shared behavior. In programming language theory, the ability to present the same interface for differing underlying forms is known as *polymorphism*.

Note

The DRY Principle

A core tenet of software engineering is DRY (Don’t Repeat Yourself). Polymorphism is a fundamental tool for adhering to this principle, allowing us to write generalized code that seamlessly operates over a variety of concrete types.

In traditional programming paradigms, polymorphism is typically achieved through several distinct mechanisms. Object-oriented languages, such as Java or C++, heavily rely on classical **class inheritance** (subtyping polymorphism), where types descend from a common ancestor to inherit shared behavior. Alternatively, they use **interface programming** to define explicit contracts, or **generic programming** (parametric polymorphism) to write algorithms that operate on generic types without knowing their exact form ahead of time.

However, rigid object-oriented taxonomies based on inheritance often struggle to gracefully model the real world. In reality, behaviors cross-cut across completely unrelated types.

Insight

The Biology Taxonomy Analogy

The professor highlighted how rigid inheritance hierarchies fall apart in complex domains, using biology as an example. Traditional taxonomy divides the world cleanly into Animals and Plants. But what about Fungi? They are neither perfectly animal nor vegetal, sitting awkwardly in the middle. Rigid object-oriented inheritance forces developers into these difficult classification decisions because types must fit strictly into a single tree.

To avoid the pitfalls of rigid class hierarchies, Rust adopts a completely different approach based on **Traits**. Rust rejects classical inheritance entirely; instead, it separates the structural data (the `struct` or `enum`) from the behavior. By using traits to formalize shared behavior, Rust seamlessly combines the strengths of interface programming and generics. Types simply opt-in

to a behavior by implementing a trait, allowing developers to model complex, cross-cutting capabilities without being forced into a restrictive inheritance tree.

Defining and Implementing Traits

A Trait in Rust is essentially a contract. It declares a set of method signatures that a type must implement to satisfy that contract. The professor likened traits to interfaces in Java (such as `Runnable` or `Cloneable`), where a class commits to providing specific methods like `run()` or `clone()`. However, unlike Java, Rust traits are implemented without any inherent object hierarchy.

Let us build our intuition by modeling the validation scenario. We can define a `Validatable` trait that promises a single method:

```
trait Validatable {
    fn validate(&self) -> bool;
}
```

This trait acts as an interface. It does not contain data; it simply states that anything claiming to be `Validatable` must provide a `validate` method returning a boolean. Now, let us apply this behavior to two entirely different concrete types:

```
struct User {
    name: String,
    age: u8,
}

struct Order {
    total: f64,
}

impl Validatable for User {
    fn validate(&self) -> bool {
        !self.name.trim().is_empty() && self.age >= 18
    }
}

impl Validatable for Order {
    fn validate(&self) -> bool {
        self.total > 0.0
    }
}

struct PhoneNumber {
    number: String,
}

impl Validatable for PhoneNumber {
    fn validate(&self) -> bool {
        // Professor's domain rule: must start with '+' and have a maximum length
        self.number.starts_with('+') && self.number.len() <= 16
    }
}
```

This code effectively demonstrates how distinct types isolate their logic:

- **Step 1:** We define a `PhoneNumber` containing simply a `String`.
- **Step 2:** We create an `impl Validatable` for `PhoneNumber` block, uniquely associating the trait's contract with this type.

- **Step 3:** We provide a custom implementation of `validate` that executes business logic specific to phone numbers (checking for a leading `+` and enforcing a length limit of 16 characters).

Memory Model Analysis: In this snippet, `User` contains a `String`, which is a heap-allocated buffer managed by a stack-allocated structure (pointer, capacity, length). The `age` field is an 8-bit integer allocated directly on the stack. When `validate(&self)` is called, ownership of the `User` or `Order` is not transferred. Instead, an immutable borrow (`&self`) is created. This allows the method to read the fields without consuming the instance, meaning the caller retains ownership. The lifetime of the borrow is strictly tied to the duration of the method call, guaranteeing no dangling pointers can occur while accessing the data.

Notice the syntax: `impl Validatable for User`. This block is entirely separate from the standard `impl User` block where we might define the struct's native methods. By isolating the trait implementation, we explicitly declare that the `validate` method is not just a coincidental name, but a fulfillment of the `Validatable` contract.

Insight

Default Implementations

Traits can also provide default implementations for their methods. If a trait defines a method body, the implementing type can choose to either use the provided default or override it with its own custom logic. This is highly useful for providing base functionality out of the box.

One of the most powerful features of Rust's trait system is the ability to implement custom traits for types that you did not originally define, including types from the standard library. For example, we can make standard strings or integers implement our `Validatable` trait:

```
impl Validatable for String {
    fn validate(&self) -> bool {
        !self.trim().is_empty()
    }
}

impl Validatable for i32 {
    fn validate(&self) -> bool {
        *self > 0
    }
}
```

This allows developers to extend the behavior of built-in types to fit their specific domain logic seamlessly. However, to prevent conflicts, Rust enforces strict rules: you can only implement a trait for a type if either the trait or the type is defined within your current crate.

Once we have formalized this common behavior, we unlock the power of polymorphism. We can now write functions that operate not on specific structs, but on *any* type that implements our trait. Rust provides two fundamental mechanisms to achieve this: **Static Dispatch** and **Dynamic Dispatch**.

Static Dispatch and Monomorphization

The most common way to leverage traits is through generic programming. We can write a function that accepts any generic type `T`, but we constrain `T` using a trait bound:

```
fn check<T: Validatable>(item: &T) {
    if item.validate() {
        println!("The item is valid.");
    }
}
```

```

    } else {
        println!("Validation failed!");
    }
}

```

Here, `<T: Validatable>` tells the compiler that this function accepts a reference to any type `T`, provided that `T` implements `Validatable`. Because of this guarantee, the compiler allows us to safely call `item.validate()` inside the function body.

If a generic type must satisfy multiple behaviors, we can combine trait bounds using the `+` operator. For functions with complex signatures, Rust provides the `where` clause to keep the function declaration clean and readable:

```

// Compact syntax for multiple bounds
fn process<T: Validatable + Clone>(item: &T) { /* ... */ }

// Extended syntax using 'where'
fn process_cleaner<T>(item: &T)
where
    T: Validatable + Clone
{
    /* ... */
}

```

But how does this execute at the hardware level? Unlike languages like Java, which use Type Erasure and process generic types as opaque objects at runtime, Rust uses a mechanism called **Monomorphization**.

When the Rust compiler sees calls to the `check` function—perhaps once with a `User` and once with an `Order`—it acts as a code generator. It creates two entirely separate, specialized copies of the `check` function:

```

// Generated internally by the compiler
fn check_user(item: &User) { /* static call to User::validate */ }
fn check_order(item: &Order) { /* static call to Order::validate */ }

```

Because the compiler generates specialized machine code for every type used, the CPU executes a direct static call to the specific `validate` method. There is zero runtime overhead.

Insight

Rust Monomorphization vs. Java Type Erasure

The professor explicitly contrasted Rust's approach with Java's generic programming. In Java, generic lists like `ArrayList<T>` use a mechanism called *Type Erasure*. Java does not generate a specialized list for strings and another for integers; it compiles a single generic list that operates on raw `Object` references. This minimizes code duplication but sacrifices performance, as it necessitates dynamic method lookups and forces primitive types (like `int`) to be boxed into objects (like `Integer`). Rust's Monomorphization does the exact opposite: it duplicates the code for each type, avoiding object boxing and maximizing CPU performance at the cost of slightly larger binaries.

! Exam Priority

Static Dispatch vs. Dynamic Dispatch: The Embedded vs. High-Frequency Trade-off

The professor heavily emphasizes the contrasting architectural impacts of these two mechanisms. Monomorphization (Static Dispatch) duplicates code for every concrete type used. While this enables aggressive compiler optimizations (like inlining) and zero-cost runtime abstraction, it causes *code bloat*. This is critically problematic in embedded systems (e.g., microcontrollers like the Atmel 328 with just 2KB of RAM) where binary size is strictly constrained. Conversely, Dynamic Dispatch uses a single function copy and resolves methods at runtime via Fat Pointers and VTables, saving space but incurring CPU cycles for pointer indirection. This micro-delay is unacceptable in scenarios where every CPU cycle translates to massive scale, such as High-Frequency Trading, intensive LLM (Large Language Model) training, or Bitcoin mining hashing algorithms.

Common Pitfall: Assuming one dispatch method is universally superior. Overusing static generics in memory-constrained environments can lead to compilation failures due to binary size limits, whereas overusing dynamic dispatch can silently degrade performance in tight computational loops.

Expected Mastery Level: High. You must be able to confidently justify when to use `<T: Trait>` versus `&dyn Trait` based on specific hardware or performance constraints.

Dynamic Dispatch and Trait Objects

While static dispatch is fast, it is not always flexible enough. Because monomorphization requires generating distinct functions, you cannot, for example, easily store a mix of `User` and `Order` objects in a single vector, since a `Vec<T>` strictly requires all elements to be of the exact same compile-time type `T`.

When we need a truly heterogeneous collection or want to avoid code duplication entirely, we use **Dynamic Dispatch**. In Rust, dynamic dispatch is achieved using *Trait Objects*, denoted by the `dyn` keyword.

```
fn check_dynamic(item: &dyn Validatable) {
    if item.validate() {
        println!("The item is valid.");
    }
}
```

Memory Model Analysis: When passing a reference to `check_dynamic(item: &dyn Validatable)`, the standard reference is implicitly converted into a Fat Pointer on the stack. This Fat Pointer consists of two machine words: a data pointer referencing the actual instance (which may live on the stack or heap) and a VTable pointer referencing a statically allocated virtual method table. The VTable itself resides in the read-only data segment of the compiled binary. No ownership is transferred during this dynamic call; the borrow checker statically ensures that the underlying data outlives the Fat Pointer.

Instead of generating specialized copies, the compiler creates only *one* version of the `check_dynamic` function. This single function is capable of handling any type that implements `Validatable`. However, because the compiler does not know the specific size or type of the data at compile time, it must resolve the correct method to call at runtime.

The Mechanism: Fat Pointers and VTables

To make runtime method resolution possible, the reference `&dyn Validatable` is not a standard pointer; it is a **Fat Pointer**. A fat pointer occupies twice the size of a standard system pointer because it actually consists of two separate pointers packed together:

1. **Data Pointer:** Points to the actual instance in memory (e.g., the specific `User` or `Order` struct).
2. **VTable Pointer:** Points to a static Virtual Method Table (`vtable`) constructed specifically for that concrete type during compilation.

DYNAMIC DISPATCH IN RUST: FAT POINTER MECHANISM

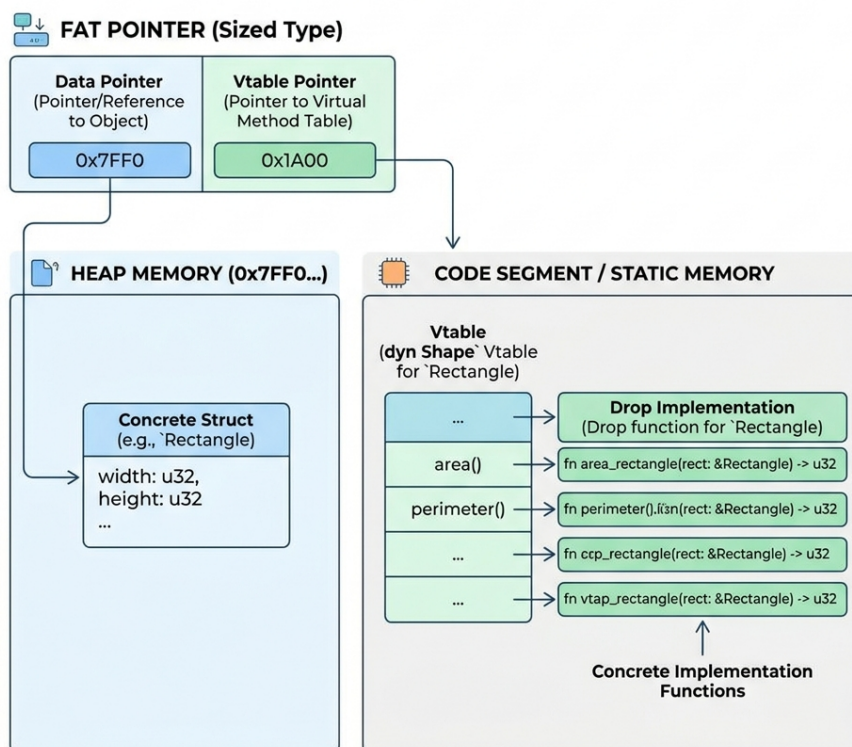


Figure 8.1: Memory layout of a Trait Object representing a Fat Pointer and its corresponding VTable.

As visualized in Figure 11.1, the `vtable` is an array of function pointers. It contains meta-data about the type (like its size and memory alignment, which are required to run the destructor via `drop`), followed by pointers to the specific implementations of the trait's methods.

When `item.validate()` is called inside `check_dynamic`, the program dereferences the fat pointer, navigates to the `vtable`, finds the index corresponding to the `validate` function, and finally jumps to that code segment, passing the data pointer as `&self`.

Exam Focus

Trade-offs of Dynamic Dispatch

Dynamic dispatch yields a smaller binary footprint because it does not duplicate code. It also allows for heterogeneous collections, such as a `Vec<Box<dyn Validatable>>`. The penalty is runtime overhead: looking up the function in the `vtable` requires extra memory jumps (pointer indirection), which is less cache-friendly and prevents compiler optimizations like inlining. While the cost is measured in nanoseconds, it becomes highly relevant in high-frequency trading or intensive numerical simulations.

Warning

Object Safety Limits

Not all traits can be turned into Trait Objects via `dyn`. A trait must be *Object Safe*. The two primary rules are: 1. The trait's methods must not return `Self` (since the compiler wouldn't know the size of the returned object dynamically at compile time). 2. The methods must not have generic type parameters. If a trait violates these rules, the compiler will safely refuse to create a `dyn Trait` fat pointer for it.

Advanced Trait Patterns

To write highly robust and expressive code, Rust provides additional constructs built on top of the base trait system. Two notable mechanisms are Associated Types and Supertraits.

Associated Types

Sometimes, a trait needs to define a relationship with another type without forcing the implementer to specify generic parameters everywhere. We achieve this using an **Associated Type**.

```
trait Container {
    type Item; // Associated type

    fn get(&self) -> &Self::Item;
}

struct IntBox {
    value: i32,
}

impl Container for IntBox {
    type Item = i32; // Concrete specification

    fn get(&self) -> &i32 {
        &self.value
    }
}
```

By declaring `type Item;`, we dictate that whoever implements `Container` must explicitly define what `Item` is. This is heavily used in standard library traits, most notably `Iterator`, which defines `type Item` to indicate the type of element the iterator will yield.

To demonstrate how traits and generics can elegantly combine, the professor showed that `Container` can also be implemented for a generic struct:

```
struct Boxed<T> {
```

```

        value: T,
    }

    impl<T> Container for Boxed<T> {
        type Item = T; // The associated type dynamically maps to the generic parameter

        fn get(&self) -> &T {
            &self.value
        }
    }
}

```

Here is how generics integrate with associated types step-by-step:

- **Step 1:** A struct `Boxed<T>` is defined to generically hold any type `T`.
- **Step 2:** When implementing the `Container` trait, we declare `impl<T> Container for Boxed<T>`, bringing the generic parameter into the trait implementation's scope.
- **Step 3:** We explicitly map the trait's associated type `Item` to the generic parameter `T`. This tells the compiler that the returned type of `get` will always match the type stored in the box.

While you could achieve somewhat similar functionality using a generic trait like `trait Container<T>`, associated types drastically improve readability and prevent the complexity of managing multiple overlapping generic implementations for a single struct.

Supertraits

Often, one trait builds logically upon another. If a type wishes to implement a sophisticated trait, it must first prove it implements a foundational one. We define this dependency using a **Supertrait**.

```

trait Printable {
    fn print(&self);
}

// AdvancedPrintable requires Printable
trait AdvancedPrintable: Printable {
    fn print_twice(&self) {
        self.print();
        self.print();
    }
}

```

Here, `Printable` is a supertrait of `AdvancedPrintable`. The compiler enforces that any type attempting to implement `AdvancedPrintable` must also provide an implementation for `Printable`. This guarantees that methods relying on the foundational behavior (such as calling `self.print()` within `print_twice()`) will always execute safely.

Standard Library Traits

A significant portion of Rust's syntax and behavior is powered by traits defined in the standard library. Implementing these traits for your own types allows them to interact seamlessly with Rust's built-in operators and language features. For many of these, Rust can automatically generate the implementation using the `#[derive(...)]` macro.

Comparison: `PartialEq`, `Eq`, `PartialOrd`, and `Ord`

Rust separates equality and ordering into distinct traits to handle mathematical edge cases accurately.

- **`PartialEq`**: Enables the use of the `==` and `!=` operators. It is “partial” because it allows for values that cannot be strictly compared to themselves (e.g., `NaN` in floating-point numbers where `NaN == NaN` is `false`). The professor highlighted a classic floating-point pitfall: checking `0.1 + 0.2 == 0.3` will yield `false` (it evaluates to `0.30000000000000004`). Because `0.1` is a periodic number in base-2, it suffers from unavoidable rounding errors. Exact equality on floats is thus inherently unreliable, which is exactly why Rust uses `PartialEq` rather than total `Eq` for floating-point types.
- **`Eq`**: A marker trait that extends `PartialEq` to indicate a total equivalence relation (every value is strictly equal to itself).
- **`PartialOrd` and `Ord`**: Enable the use of relational operators (`<`, `>`, `<=`, `>=`). Similarly to equality, `PartialOrd` is for types where some values might not be comparable, whereas `Ord` defines a strict total order.

Formatting: `Display` and `Debug`

To convert types into text, Rust uses two primary formatting traits:

- **`Display`**: Designed for user-facing output (used with the `{}` format specifier). It must be implemented manually, as the programmer needs to define the specific representation suitable for end users.
- **`Debug`**: Intended for developer-facing output, providing an internal representation of the data (used with the `{:?}` format specifier). It can be automatically implemented via `#[derive(Debug)]`.

Duplication: `Clone` vs. `Copy`

When assigning a value to a new variable or passing it to a function, Rust’s default behavior is to *move* the value, transferring ownership.

- **`Clone`**: Explicitly creates a duplicate of a value. Because cloning can involve expensive operations like heap allocations, it must be invoked explicitly via the `.clone()` method.
- **`Copy`**: A subtrait of `Clone` that changes the default assignment semantics from a *move* to a *copy*. It indicates that duplicating the value is simply a matter of copying bits, which is fast and cannot fail (e.g., primitive integers). It can only be implemented if all fields of the type also implement `Copy`.

Resource Management: `Drop`

The `Drop` trait serves as Rust’s equivalent of a destructor. It allows you to specify custom code that runs automatically when a value goes out of scope, releasing resources such as heap memory, network connections, or file handles (an implementation of the RAII pattern). If a programmer wishes to destroy a value prematurely, they can pass it to the globally available `std::mem::drop` function, forcing the value to be moved and subsequently destroyed.

Memory Model Analysis

The Mechanics of `std::mem::drop` and Container Destruction

The professor clarified that `std::mem::drop(x)` is actually a deceptively simple, entirely empty function: `fn drop<T>(_x: T) {}`. By passing a variable into it by value, ownership of the data is transferred to the function's scope. Because the function ends immediately doing nothing, the variable goes out of scope and its `Drop` implementation is triggered early.

Furthermore, when a complex data structure like a `Vec<T>` goes out of scope, its `Drop` implementation traverses the collection, guaranteeing that the `drop` method is called sequentially on every single item it contains before the vector's own backing memory is finally released.



Exam Priority

Drop and Copy Mutual Exclusivity

The professor heavily stressed that `Drop` and `Copy` are mutually exclusive. If a type implements `Drop`, it is actively managing a resource (e.g., a heap allocation or file handle). If such a type were allowed to be implicitly bit-wise copied via the `Copy` trait, you would end up with two instances holding pointers to the exact same underlying resource. When both instances go out of scope, the `drop` method would run twice, leading to catastrophic double-free vulnerabilities or resource corruption.

Common Pitfall: Trying to derive `Copy` on a struct that contains a heap-allocated field (like `String` or `Vec`) or a custom `Drop` implementation. The compiler will reject this immediately.

Expected Mastery Level: Very High. Understanding why `Drop` and `Copy` conflict is fundamental to grasping Rust's memory safety guarantees.

Type Conversions: From, Into, and FromStr

Rust prefers explicit type conversions. The `From` and `Into` traits define how to consume a value of one type to create a value of another. They are perfectly dual: implementing `From<T>` for `U` automatically provides an `Into<U>` implementation for `T`. However, conversion is not necessarily symmetric; being able to convert a `String` into a custom type does not automatically mean the reverse is possible or logical.

For parsing strings specifically, Rust offers the `FromStr` trait. Implementing it unlocks the ability to use the `.parse()` method on strings to instantiate your type, gracefully handling potential parsing failures by returning a `Result`. During the lecture, the professor gave the example of parsing a string into a custom struct by inspecting character slices, leveraging methods like `s.starts_with()` and `s.trim()` inside the custom `from_str` implementation.

Exam Relevance and Recurrence

Exam Relevance Score: Very High

Notes: Questions on polymorphism in Rust appear frequently in exams, particularly concerning the architectural trade-offs between Monomorphization (Static Dispatch) and Trait Objects (Dynamic Dispatch). You are highly likely to be asked to compare the memory implications of both (code bloat vs. VTable lookup overhead). The structure and content of a Fat Pointer (data pointer + VTable pointer) is another recurring theoretical question. Furthermore, the mutual exclusivity of `Drop` and `Copy` traits is a classic exam topic used to test understanding of RAII and memory safety.

8.1 Domande ed Esercizi da Esami Passati

Exam Focus

Domanda 1: Nel contesto della programmazione generica, si spieghi che cosa si intende per monomorfizzazione. Si scriva una porzione di codice che mostri un esempio pratico in cui la monomorfizzazione viene applicata, spiegando perché tale esempio è rilevante. Si indichino i principali punti di forza e i punti di debolezza di tale proprietà, facendo un raffronto rispetto agli approcci alla programmazione generica offerti da altri linguaggi di programmazione.

Soluzione: La monomorfizzazione è il meccanismo con cui il compilatore Rust implementa la programmazione generica (Static Dispatch). Quando una funzione accetta parametri generici vincolati da un trait, il compilatore non crea una singola funzione generica (come accade in Java tramite *Type Erasure*), ma genera internamente copie distinte e specializzate della funzione per ogni tipo concreto utilizzato nel sorgente.

Esempio Pratico:

```
trait Validatable { fn validate(&self) -> bool; }

fn check<T: Validatable>(item: &T) { item.validate(); }

struct User;
impl Validatable for User { fn validate(&self) -> bool { true } }

struct Order;
impl Validatable for Order { fn validate(&self) -> bool { false } }

fn main() {
    let user = User;
    let order = Order;
    check(&user); // Il compilatore genera check_user
    check(&order); // Il compilatore genera check_order
}
```

L'esempio è rilevante perché dimostra che `check` a livello di linguaggio macchina viene espansa in copie distinte (`check_user`, `check_order`) che effettuano chiamate dirette e statiche ai rispettivi metodi, senza alcuna ambiguità o ricerca a runtime.

Raffronto e pro/contro: Il principale punto di forza è la prestazione (Zero-Cost Abstraction): le chiamate statiche non hanno overhead a runtime e permettono ottimizzazioni spinte come l'inlining. Il punto di debolezza è il *code bloat* (aumento della dimensione del binario), particolarmente critico in sistemi embedded con memoria limitata. Al contrario, l'approccio in Java (Type Erasure) minimizza la duplicazione del codice utilizzando riferimenti a un generico `Object`, ma sacrifica le prestazioni necessitando di *boxing* per i tipi primitivi e lookup dinamici a runtime.

Exam Focus

Domanda 2: Dato il seguente programma si descriva il comportamento del programma e la mappa della memoria utilizzata dalle variabili

```
1 fn main() {  
2     trait Calculate {  
3         fn add(&self, l: u64, r: u64) -> u64;  
4         fn mul(&self, l: u64, r: u64) -> u64;  
5     }  
6     struct Modulo(u64);
```

Soluzione: Il programma definisce un trait ‘Calculate’ con due metodi, e una struct ‘Modulo’ composta da un unico intero a 64 bit. Dal punto di vista della mappa della memoria e del comportamento:

- L’istanza di ‘Modulo’ (es. ‘let m = Modulo(10);’) viene allocata direttamente sullo **stack**, in quanto le struct in Rust risiedono nello stack se non incluse in uno smart pointer (come ‘Box’). Il campo intero è allocato al suo interno.
- I metodi all’interno del trait prendono `&self`, effettuando un *borrowing* immutabile che non sposta la ownership ma crea un riferimento sicuro valido per la durata della chiamata.
- Se ‘Modulo’ implementa ‘Calculate’ e viene utilizzato dinamicamente (es. `&dyn Calculate`), Rust costruisce a runtime un **Fat Pointer** (due word di memoria). Il Fat Pointer contiene un **Data Pointer** (che punta a ‘Modulo’ sullo stack) e un **VTable Pointer**, che punta alla Virtual Method Table associata al tipo ‘Modulo’, situata nella sezione *read-only* del binario.

Exam Focus

Domanda 3: Si usi un esempio concreto per dimostrare la capacità di introdurre modularità attraverso i Tratti.

Soluzione: Rust garantisce estrema modularità separando i dati strutturali (definiti tramite ‘struct’ o ‘enum’) dal comportamento condiviso (definito tramite ‘trait’).

Esempio di modularità:

```
trait Validatable {  
    fn validate(&self) -> bool;  
}  
  
struct User { name: String, age: u8 }  
struct Order { total: f64 }  
  
// Implementazioni isolate per ciascun tipo  
impl Validatable for User {  
    fn validate(&self) -> bool { self.age >= 18 }  
}  
  
impl Validatable for Order {  
    fn validate(&self) -> bool { self.total > 0.0 }  
}
```

Questa struttura è fortemente modulare: ogni tipo dichiara esclusivamente i propri dati. L’implementazione del contratto ‘Validatable’ avviene in blocchi di codice separati. In questo modo è possibile estendere i comportamenti (implementando il trait per nuovi tipi, come ‘PhoneNumber’, o persino per tipi della standard library come ‘String’ o ‘i32’) senza alterare i tipi esistenti o inserire forzatamente comportamenti all’interno di una classe radice o in gerarchie complesse.

Exam Focus

Domanda 4: Si dimostri come sia possibile implementare il polimorfismo attraverso i tratti. Si fornisca anche un esempio concreto che faccia riferimento ad almeno due strutture diverse.

Soluzione: In Rust, il polimorfismo si ottiene scartando le classiche gerarchie di classe a favore dei **Trait**. Diverse strutture possono aderire indipendentemente a un trait fornendo una propria implementazione del contratto. Questo consente a funzioni esterne di accettare genericamente ogni tipo che soddisfa il trait (polimorfismo statico tramite *monomorfizzazione*) o a runtime usando riferimenti ai trait object (polimorfismo dinamico).

Esempio concreto (Dynamic Dispatch):

```
trait Validatable {
    fn validate(&self) -> bool;
}

struct User { name: String }
struct PhoneNumber { number: String }

impl Validatable for User {
    fn validate(&self) -> bool { !self.name.trim().is_empty() }
}
impl Validatable for PhoneNumber {
    fn validate(&self) -> bool { self.number.starts_with('+') }
}

// Funzione polimorfica: accetta a runtime QUALSIASI tipo
// che implementa Validatable usando i Fat Pointers.
fn check_dynamic(item: &dyn Validatable) {
    if item.validate() {
        println!("The item is valid.");
    }
}

fn main() {
    let u = User { name: String::from("Mario") };
    let p = PhoneNumber { number: String::from("+39333") };

    // Stessa funzione applicata a tipi strutturalmente disgiunti
    check_dynamic(&u);
    check_dynamic(&p);
}
```

In questo frammento, `check_dynamic` realizza il polimorfismo accettando tramite riferimento dinamico `&dyn Validatable` oggetti di natura strutturalmente diversa (`User` e `PhoneNumber`), la cui risoluzione dei metodi avverrà dinamicamente a runtime tramite la lettura dei puntatori dalla VTable.

Chapter 9

Testing and CI

Software development is not just about writing code that works; it is about writing code that *continues* to work as the system evolves. When building non-trivial systems, the confidence a developer has in their codebase stems directly from the safety nets they have put in place. Testing is the primary safety net.

In this chapter, we will explore the philosophy behind automated testing, the mechanisms Rust provides to test your code natively, and how writing tests can fundamentally improve the design of your software.

9.1 The Philosophy of Testing

Imagine a manufacturing plant that produces plastic water bottles. The plant does not wait until a million bottles are shipped to customers to check if they hold water. Instead, there are quality control gates at every step of the production cycle. For instance, the water source is continuously analyzed to ensure no epidemics are spread. The raw plastic pellets are sampled for their chemical and physical characteristics before entering the machine—otherwise, the plant might produce 18,000 defective bottles. During the blow-molding phase, where recycled trimmings are sometimes used, a lack of material might cause a bottle to form incompletely (becoming just a deformed piece of plastic). Camera systems immediately discard these errors.

In software engineering, testing serves the exact same purpose. Many developers fall into the trap of writing a piece of code, running it once with a temporary `main` function to see if it prints the right output, and then moving on. However, this manual verification is lost to time. When the codebase grows, or when another developer modifies that code six months later, there is no automatic way to verify that the original behavior remains intact.

Writing tests is an investment in the future. It helps prevent **regressions**—bugs that occur when new additions unintentionally break existing functionality. They are also essential during **refactoring**: you must have a solid suite of tests for version 1 of your code before you can safely optimize or rewrite it into version 2. Furthermore, test suites act as living, executable documentation. If you want to know exactly how a function is supposed to behave, especially in edge cases, you look at its tests.

! Exam Priority**Tests as an Investment and Specifications**

As the professor emphatically states, writing tests is an investment in the future that prevents **regression time**—the time wasted when new additions break existing features. Furthermore, writing tests often reveals that a system is *under-specified*. By testing edge cases (like passing zero or an empty vector), you naturally ask “what should happen here?”, turning the test into a strict supplement to the specification.

Common Pitfall: Thinking of tests as a “waste of time” instead of a long-term cost-saving measure, or writing them only *after* finishing the code rather than using them to drive the design.

Expected Mastery Level: High. You must conceptually understand how tests formalize requirements and prevent regressions, especially when modifying or refactoring code.

Insight**The Cost of Bugs**

Fixing a bug during the initial development phase is cheap. Finding and fixing that same bug after it has been integrated into a larger system is expensive. If the bug reaches the end-user in production, the cost is astronomical. As the professor notes: *“Money can maybe be recovered, but time doesn’t come back. What is burnt is burnt. And if you do something not good, you lose your face.”* Automated tests catch errors when they are cheapest to fix.

Insight**Analogy: The Inaccessible Sensor Box**

To illustrate the danger of testing only at the very end, the professor shared a personal anecdote about building a 10cm sensor box. He carefully designed the internal circuitry but left testing the wall-mounting mechanism until the very end. Once the complex box was fully assembled, he realized the mounting holes were completely inaccessible! He had focused so much on the complex internals that he missed a basic structural flaw, forcing him to throw it away and start over. Writing tests early prevents you from building a “black box” that you later realize is fundamentally flawed.

9.2 The Testing Pyramid

To build reliable software, we employ multiple layers of verification, often visualized as a pyramid.

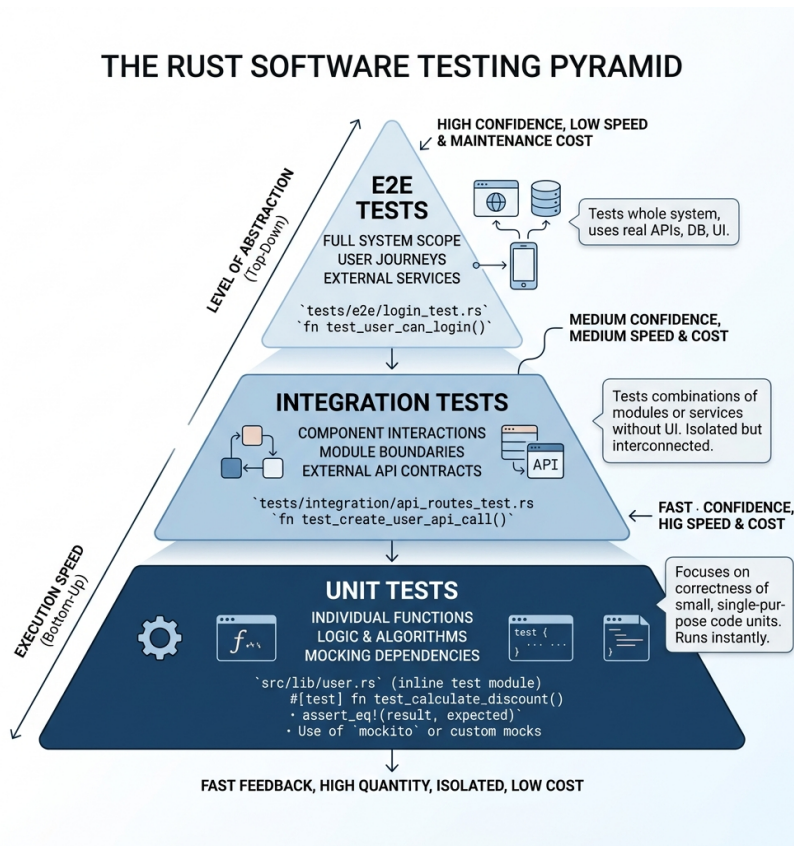


Figure 9.1: The Testing Pyramid: Balancing cost, speed, and scope.

The pyramid consists of several layers:

- **Static Analysis:** At the very foundation sits static analysis. In Rust, the compiler itself and tools like `rust-analyzer` or `clippy` catch numerous errors before the code even runs. As the professor distinguishes: “red flags” mean the code is broken, while “yellow flags” (warnings) indicate code smells or constructs written “as an intern” that should be simplified.
- **Unit Tests:** The base of the executable pyramid. These verify single, isolated elements of code. They answer the functional question: “*Did we build the software right?*” They are fast, numerous, and written by the developer who knows the internal logic (*white-box testing*).
- **Integration Tests:** The middle layer. These verify that multiple modules or systems work together correctly. They are essential when parts of a system (like a MySQL database) are out of our control and we must ensure our interactions with them are valid (*black-box testing*).
- **End-to-End (E2E) Tests / Acceptance Tests:** The tip of the pyramid. These test the entire application from the user’s perspective, answering the ultimate question: “*Did we build the right software?*” (i.e., did we build what the client actually needed?). Furthermore, they evaluate operational criteria. As an analogy, if you buy a laptop, it is not enough that the CPU functions. It must weigh less than a kilo, the battery must last 4 hours, and it must have a backlit keyboard so you can use it at night. If it lacks these operational features, it is useless to the user, even if it is functionally flawless.

9.3 Unit Testing: The Foundation

Unit tests are aimed at checking if a single unit of code behaves exactly as expected in isolation. In Rust, writing and running unit tests is built directly into the language and its package manager, Cargo.

Because unit tests are typically written by the same developer who wrote the code, they are a form of **white-box testing**. You know the internal implementation details, which allows you to intelligently choose edge cases. As the professor illustrated: if you know a watch is mechanical (made of gears), you might test what happens if a grain of sand gets inside. If it is an electronic watch, sand might not affect it, but a drop of water would. Knowing the internal structure guides your testing strategy.

When writing unit tests, you must test the normal case (the “happy path”) and the edge cases. To use another analogy: if you buy a car, you expect it to turn on. If it doesn’t, you bought a stove, not a car. But you also need to know that the car still travels safely when it rains. If the car randomly evaporates into a pile of scattered iron pieces because of a thunderstorm, that is a severe failure. However, if you have a traditional gas car and you pour whiskey into the fuel tank, you *expect* the engine to break. Testing edge cases means verifying both that the software survives realistic stress and that it fails predictably when misused.

This internal knowledge also highlights the importance of **cyclomatic complexity**—the number of independent paths through your code. If you have an `if` inside another `if`, you have 4 possible combinations. If that is nested inside a `for` loop, the number of combinations multiplies rapidly, creating what the professor calls “astral combinations” that can break everything. This is a strong incentive to keep functions small. It is far better to have two simple functions called in cascade than a single 200-line monolithic block, because smaller functions can be tested individually with ease.

Let’s look at a trivial function that divides two numbers, and see how we would test it.

```
pub fn divide(a: i32, b: i32) -> i32 {
    a / b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_divide_happy_path() {
        // Arrange
        let numerator = 6;
        let denominator = 3;

        // Act
        let result = divide(numerator, denominator);

        // Assert
        assert_eq!(result, 2, "6 divided by 3 should be 2");
    }
}
```

Memory Model and Ownership Analysis: In the `divide` snippet, both arguments `a` and `b` are primitives (`i32`). Primitives implement the `Copy` trait, meaning they are allocated entirely on the stack and copied into the function’s scope when `divide` is called. There is no heap allocation or ownership transfer of dynamically allocated memory here. The `assert_eq!` macro takes its arguments by reference behind the scenes (i.e., it borrows the `result` and the literal `2`), which is why it can compare them without consuming them, preserving their lifetimes

if needed later in the test scope.

9.3.1 Mechanism: The Test Module

Notice the structure of the code above. We place our unit tests in a child module named `tests`. This module is annotated with `#[cfg(test)]`. This is a conditional compilation flag: it tells the Rust compiler to completely ignore this module when compiling the application for release. The tests are only compiled and executed when you explicitly run the `cargo test` command.

Inside the module, we use `use super::*`; to bring the parent module's items (like the `divide` function) into scope. We then decorate individual test functions with the `#[test]` attribute. The test runner uses this attribute to automatically locate and execute all test functions.

Note

The AAA Pattern

Notice how `test.divide_happy_path` is structured into three distinct phases. This is known as the **Arrange, Act, Assert** (AAA) pattern:

- **Arrange:** Set up the initial state, variables, and conditions.
- **Act:** Invoke the specific function or behavior under test.
- **Assert:** Verify that the actual outcome matches the expected outcome.

Adhering to this pattern keeps tests readable and consistent.

Insight

Test Naming and "Furbo" Names

It is crucial to give your test functions a "smart" (*furbo*) name. Naming tests `t1`, `t2`, and `t3` is useless, because if `t3` fails, the test report tells you nothing about what broke. Instead, a name like `test_when_button_clicked_light_turns_on` gives you immediate context about what part of the application logic is failing.

9.3.2 Assertions and Usage Guidelines

Rust provides a family of macros to verify outcomes. **When and why to use them:**

- `assert!(condition)`: Use when you have a simple boolean outcome (e.g., checking if an `Option` is `some()`).
- `assert_eq!(left, right)`: Use this as your primary verification tool. It compares two values for equality and, crucially, prints both values if the test fails. This makes it far superior to `assert!(a == b)` for debugging.
- `assert_ne!(left, right)`: Use when you want to guarantee state has changed or a value is not a default/error state, without knowing its exact new value.

When `assert_eq!` fails, the error message indicates exactly what was expected versus what was found (e.g., "expected 7, found 5"). A common convention is to place the actual result on the left and the expected outcome on the right. A helpful mnemonic from the lecture is that the expected value goes on the *right*, because in English, "right" also means "correct"!

9.3.3 Advanced Testing Mechanics

Beyond simple assertions, Rust allows test functions to return a `Result<(), String>`. If the test function returns `Ok(())`, the test passes; if it returns an `Err(String)`, the test fails, and the string is printed in the failure documentation. This is extremely useful when your test involves operations that can naturally fail using the `?` operator.

Additionally, within your `mod tests`, you are not limited to writing just `#[test]` functions. You can write **auxiliary helper functions** to generate lists of inputs, construct complex objects, or generate random numbers to stress-test your domain space. For example, if you are testing floating-point math, a helper function could rapidly generate cases like `0.1 + 0.2` to check for precision issues.

9.3.4 Testing Edge Cases and Failures

Testing the “happy path”—where all inputs are correct and everything works perfectly—is necessary but insufficient. You must also test **edge cases**. What happens if we pass zero as the denominator?

```
#[test]
#[should_panic(expected = "divide_by_zero")]
fn test_divide_by_zero_panics() {
    divide(6, 0);
}
```

If a function is structurally designed to panic under certain conditions, we can verify that behavior using the `#[should_panic]` attribute. If the function *does not* panic, the test will fail!

9.4 How Tests Drive API Design

Writing tests often triggers an important psychological shift. When writing the `test_divide_by_zero_panics` test, a developer might realize: *“Wait, if my function panics, the entire program could crash if user input happens to be zero.”*

A test highlighting a panic condition is a strong signal that the function’s signature might be flawed. In mathematics, division by zero is undefined. In software, we should embrace this complexity rather than ignoring it.

Refactoring the code based on testing feedback leads to safer, more robust API design. Instead of panicking, the function should return an `Option<i32>` (or a `Result`):

```
pub fn divide_safe(a: i32, b: i32) -> Option<i32> {
    if b == 0 {
        None
    } else {
        Some(a / b)
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_divide_safe_success() {
        assert_eq!(divide_safe(6, 3), Some(2));
    }

    #[test]
```

```
fn test_divide_safe_by_zero() {  
    assert_eq!(divide_safe(6, 0), None);  
}
```

Memory Model and Ownership Analysis: By returning an `Option<i32>`, the function still operates entirely on the stack. An `Option` is an enum, and since `i32` fits within standard processor registers/stack frames, no heap allocation is required. This makes the "safe" version just as memory-efficient as the panicking version, while cleanly transferring ownership of the result (the `Option`) back to the caller.

By writing the test, we discovered that our system was under-specified. We improved the design, making it the caller's responsibility to handle the `None` case. This results in software that is fundamentally safer.

Insight

Tests as Documentation and Specification

Working with clients is often a process of negotiation. A client might ask you to build a machine that separates egg yolks. Inside that machine, there will be tiny software sub-components that the client doesn't even know exist. You cannot ask the client how those microscopic internal pieces should behave in edge cases! You have to decide.

When you make these decisions, tests become a strict supplement to the specification. Instead of just remembering a vague conversation, capturing those requirements as executable tests is incredibly powerful. A test comment saying, *"This edge case handles the specific scenario requested by Mario Rossi in his May 3rd email,"* prevents future regressions and ensures the specific requirement is never lost to time, even if someone else inherits your codebase.

9.5 Integration Testing

While unit tests are excellent for testing specific logic in isolation (*white-box testing*), they do not guarantee that the whole system works. What happens when multiple distinct components are assembled?

This is where **integration tests** come in. Integration tests evaluate how modules interact. Because they test the interactions between components, they operate as *black-box tests*. They cannot access the internal, private details of your modules; they can only use your library's public API, exactly as an external user would.

To use an analogy from the lecture, consider testing a computer monitor. A unit test might verify in isolation that individual pixels turn on or that the HDMI decoder chip processes signals correctly. An integration test, however, treats the monitor as a black box: you send a video signal (input) and verify that the correct image appears on the screen (output). You do not look at the internal circuitry. Naturally, integration tests should only be run *after* unit tests have passed; if you already know the display panel is broken, there is no point in checking the HDMI integration!

9.5.1 Structure of Integration Tests

In Rust, integration tests live outside the `src` directory, inside a dedicated `tests/` directory at the root of the project. Cargo compiles each file in the `tests/` directory as its own distinct crate.

Imagine a system with two components: an `OrderRepository` that interacts with a database to fetch prices, and an application logic module that applies discounts.

An integration test for this workflow might look like this:

```
// Inside tests/integration_test.rs
use my_store::repository::OrderRepository;
use my_store::services::DiscountService;

#[test]
fn test_applying_discount_updates_database() {
    // Arrange: Create an empty DB repository and populate it
    let mut repo = OrderRepository::new();
    repo.insert(1, 200.0); // Product ID 1 costs 200.0

    // Act: The service applies a 10% discount to product 1 (only if price > 100)
    let mut service = DiscountService::new(&mut repo);
    service.apply_discount(1);

    // Assert: Verify the repository reflects the new price (180.0)
    let updated_price = repo.find(1).unwrap();
    assert_eq!(updated_price, 180.0);
}
```

Notice how we are testing the assembled system as a **black box**. The integration test does not care *how* the `OrderRepository` is implemented internally. It could be using a transient in-memory hash table (where data is lost when the program closes), or it could be connected to a robust, persistent MySQL database. The test simply presses the “buttons” of the public API and verifies the consequences.

Memory Model and Ownership Analysis: This integration test demonstrates complex borrowing dynamics. `OrderRepository::new()` allocates a `HashMap` on the heap, while the `repo` structure itself lives on the stack. When we call `DiscountService::new(&mut repo)`, we create a mutable borrow of `repo`. The `service` instance holds this mutable reference, tying its lifetime to the mutable borrow of `repo`. Later, when we call `repo.find(1)`, we take an immutable borrow (`&repo`). Why doesn’t this cause a compiler error for having both a mutable and immutable borrow simultaneously? Because of Rust’s Non-Lexical Lifetimes (NLL)! The mutable borrow held by `service` is no longer used after `service.apply_discount(1)`, so the compiler gracefully ends the mutable borrow early, allowing `repo.find(1)` to safely take an immutable reference.

Warning

The Cost of Integration Tests

When an integration test fails, it is often more expensive to debug than a unit test. Because you are testing a complex flow across multiple modules, a failure means *something* broke along the path, but the test doesn’t explicitly tell you *which* internal piece failed. Therefore, integration tests should supplement, not replace, a comprehensive suite of unit tests.

9.6 Parametric Testing with `rstest`

Often, you want to test the same logic against many different inputs. Writing separate functions for each case can be tedious. Rust’s ecosystem offers libraries like `rstest` to simplify this via parametric tests.

Using `rstest`, you define the parameters in the macro, and the test runner executes the function multiple times with the supplied data:

```
use rstest::rstest;
```

```
#[rtest]
#[case(6, 3, 2)]
#[case(10, 2, 5)]
#[case(-4, -2, 2)]
#[case(0, 5, 0)]
fn test_divide_multiple_cases(
    #[case] a: i32,
    #[case] b: i32,
    #[case] expected: i32
) {
    assert_eq!(divide(a, b), expected);
}
```

This centralizes the logic and clearly defines the expected input-output mappings without repetitive boilerplate. Additionally, if a specific test is currently broken but you want to run the rest of the suite without failure while you work on a fix, you can temporarily annotate it with `#[ignore]`.

! Exam Priority

White-box vs Black-box Testing

The professor explicitly contrasts unit tests and integration tests. Unit tests are *white-box*: you know the internal logic and can target specific nested ‘if’ statements or edge cases. Integration tests are *black-box*: you only see the public API, ensuring the assembled components interact correctly.

Common Pitfall: Attempting to test internal implementation details within an integration test, or skipping unit tests because “integration tests will catch it.”

Expected Mastery Level: High. You must be able to clearly distinguish between these two paradigms and know that integration tests go in a separate `tests/` folder and only test the public API.

9.7 The Limits of Testing: Concurrency and Proofs

While tests are exceptional at finding bugs, they cannot definitively prove the *absence* of bugs. You would need an infinite number of tests to cover every possible state of a non-trivial program.

This limitation becomes especially critical in **concurrent programming**. Concurrent programs depend on the flow of time and thread scheduling, which are non-deterministic and outside the programmer’s control. A test might pass 99 times and fail on the 100th run simply because the threads happened to interleave differently. Therefore, for concurrency, testing is not enough.

You must rely on logical demonstrations to mathematically ensure correctness, exactly like proving a geometry theorem. To prove the Pythagorean theorem, you start with Euclid’s axioms (e.g., between two points there is a straight line) and logically deduce the truth. In concurrent programming, we use the axioms provided by the operating system (e.g., a Mutex can be possessed by only one thread at a time) and Rust’s ownership rules to logically guarantee that data races cannot occur. Tests will help you catch glaring issues, but they cannot replace a sound logical proof.

Exam Relevance and Recurrence

Score: High Questions on testing often focus on the conceptual differences between unit and integration tests (white-box vs black-box), the location of test files, and conditional compilation

flags (`#[cfg(test)]`). You may also be asked to read a simple test and identify what makes it fail, or how a test-driven approach helps refine a function's signature (e.g., migrating from returning a panic to returning an `Option` or `Result`).

9.8 Domande ed Esercizi da Esami Passati

Nessuna domanda specifica per questo capitolo negli esami passati analizzati.

Chapter 10

Collections and Persistence

10.1 Introduction to Dynamic Containers

When designing system-level applications, we frequently encounter scenarios where the volume of data is not known at compile time. Static arrays are insufficient because their size must be defined in advance. We need dynamic containers—data structures that can grow or shrink at runtime. In Rust, these containers allocate memory on the heap and provide safe abstractions for managing it.

This chapter explores Rust’s most heavily used dynamic collections: sequences (`Vec<T>`), associative arrays (`HashMap<K, V>`), and text containers (`String`). We will also investigate how to serialize and persist this data to the filesystem using `PathBuf` and `File`.

10.1.1 Shared Collection API and Complexity

Regardless of the specific collection type, Rust provides a consistent set of core APIs that are shared across most dynamic containers:

- `new()`: Creates a new, empty collection.
- `with_capacity(n)`: Preallocates space for `n` elements, avoiding reallocation costs if the size is known.
- `len()`: Returns the number of elements currently stored.
- `is_empty()`: Returns `true` if the collection contains no elements.
- `clear()`: Removes all elements, resulting in an empty collection (often preserving capacity).
- `iter()`: Returns an iterator over references to the elements.
- `into_iter()`: Disassembles the collection, transferring ownership of the elements to the caller.
- `FromIterator` / `collect()`: Constructs a collection by consuming an iterator.

Data Structure	Rust	C++	Java	Python
Dynamic Array	<code>Vec<T></code>	<code>std::vector</code>	<code>ArrayList</code>	<code>list</code>
Double-ended Queue	<code>VecDeque<T></code>	<code>std::deque</code>	<code>ArrayDeque</code>	<code>collections.deque</code>
Linked List	<code>LinkedList<T></code>	<code>std::list</code>	<code>LinkedList</code>	N/A
Hash Map	<code>HashMap<K, V></code>	<code>std::unordered_map</code>	<code>HashMap</code>	<code>dict</code>
Ordered Map	<code>BTreeMap<K, V></code>	<code>std::map</code>	<code>TreeMap</code>	N/A
Hash Set	<code>HashSet<T></code>	<code>std::unordered_set</code>	<code>HashSet</code>	<code>set</code>
Ordered Set	<code>BTreeSet<T></code>	<code>std::set</code>	<code>TreeSet</code>	N/A
Priority Queue	<code>BinaryHeap<T></code>	<code>std::priority_queue</code>	<code>PriorityQueue</code>	<code>heapq</code>

Table 10.1: Equivalent Collections Across Programming Languages

Insight

[Understanding Computational Complexity] The professor emphasizes understanding the Big-O complexity for various collection operations to make informed architectural decisions:

- **$O(1)$ (Constant time):** The operation takes the same amount of time regardless of the container size.
- **Amortized $O(1)$:** Most operations are $O(1)$, but occasionally a more expensive operation occurs (e.g., reallocating a `Vec`'s buffer). Over many operations, the average cost remains $O(1)$.
- **$O(\log N)$ (Logarithmic time):** The execution time grows logarithmically. It grows very slowly even for large datasets (e.g., operations on a `BTreeMap`).
- **$O(N)$ (Linear time):** The time scales directly with the number of elements (e.g., inserting into the middle of a `Vec`).
- **$O(N \log N)$ (Linearithmic time):** Typical complexity for efficient sorting algorithms like Quicksort or Mergesort.

10.2 Sequences: Vectors (`Vec<T>`)

10.2.1 The Problem of Contiguous Dynamic Data

Imagine you are reading lines from a log file. You do not know how many lines exist until you reach the end of the file. You need a data structure that keeps elements in a sequence, allows rapid sequential access, and can expand its capacity on demand.

10.2.2 Mechanism and Memory Layout

The standard solution in Rust is the `Vec<T>` (vector). A vector ensures that all its elements of type `T` are contiguous in memory.

Under the hood, a `Vec<T>` consists of three fields (one pointer and two integers), as illustrated below:

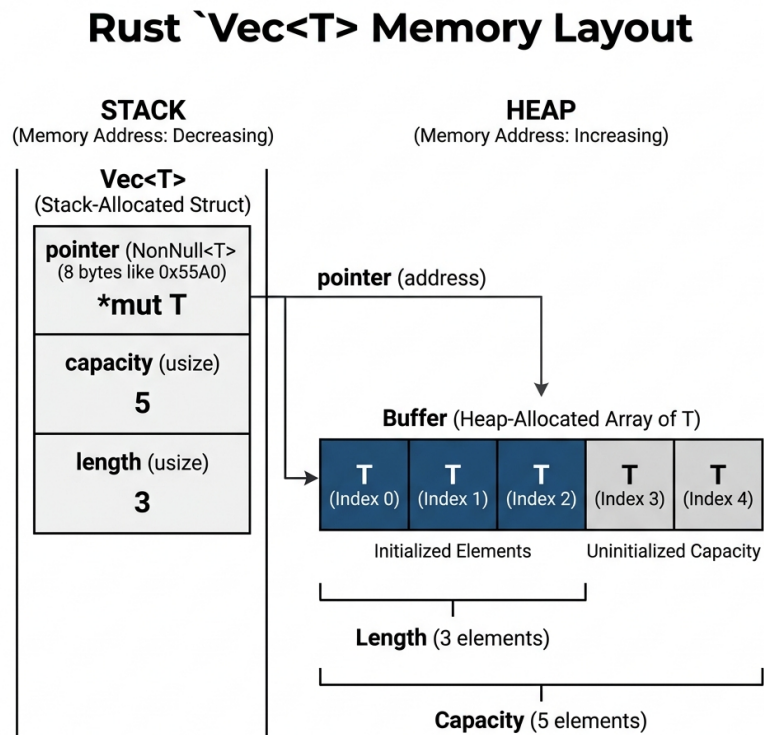


Figure 10.1: The three core components of a `Vec<T>`: pointer, capacity, and length.

- A **pointer** to the heap-allocated buffer containing the data.
- The **capacity**: how much space is currently allocated.
- The **length**: how many elements are currently instantiated.

When you push an element onto a vector, it occupies the next free slot in the pre-allocated buffer. If the length reaches the capacity, the vector must *reallocate*: it requests a larger block of memory (usually double the size), copies the existing elements over, and deallocates the old buffer.

Insight

[Sorting and Slicing Advantages] The contiguous memory layout of a `Vec` provides two major advantages:

- **Sorting:** Sorting algorithms perform exceptionally well on contiguous memory. If you store elements non-contiguously, sorting becomes significantly more expensive.
- **Slicing:** Creating a slice from a vector is an $O(1)$ operation because it simply requires extracting a pointer to the start and a length.

10.2.3 Concrete Example: Building a Vector

Let us examine the basic usage of a vector:

```
let mut numbers: Vec<i32> = Vec::new(); // Length: 0, Capacity: 0
```

```
// Pushing elements allocates memory on the heap
numbers.push(10);
numbers.push(20);
numbers.push(30);

println!("First element: {}", numbers[0]);

// Popping removes the last element (30)
if let Some(last) = numbers.pop() {
    println!("Removed: {}", last);
}
```

Memory Model Analysis: The variable `numbers` itself is allocated on the stack, consisting of three machine words: a pointer to the heap, the capacity, and the length. Initially, the heap pointer might be null or a dangling placeholder because `Vec::new()` does not allocate. When `push(10)` is called, Rust requests heap memory to store the elements. The `numbers` stack structure is mutated to update its capacity, length, and the pointer to the newly allocated heap block. As elements are pushed, ownership of the values moves into the `Vec`'s heap allocation. When `numbers` goes out of scope, its `Drop` implementation will be called, deallocating the heap buffer before popping the stack frame.

Note

[Optimizing Allocations] Reallocation is an expensive $O(N)$ operation. If you know approximately how many elements you will store, you should use `Vec::with_capacity(n)`. This allocates the necessary memory upfront, significantly amortizing the cost of insertion to $O(1)$.

10.2.4 Trade-offs

Vectors are highly efficient for inserting or removing at the *end* (`push/pop`) and provide excellent CPU cache locality due to their contiguous layout. However, inserting or removing elements at the *front* or in the *middle* forces all subsequent elements to shift, resulting in an $O(N)$ cost. In addition to `push` and `pop`, vectors provide `insert(index, element)` and `remove(index)`. However, using these methods requires shifting all subsequent elements, introducing an $O(N)$ cost per operation.

When and why to use it: Use `Vec<T>` by default for sequences when you need fast random access by index, cache-friendly iteration, and mostly append-only insertions.

10.2.5 Heterogeneous Sequences via Enums

While a `Vec<T>` strictly requires all elements to be of the same type `T`, you can elegantly store heterogeneous data by defining an `enum`. By wrapping different data types in enum variants, the vector remains technically homogeneous, but logically heterogeneous:

```
enum SpreadsheetCell {
    Int(i32),
    Float(f64),
    Text(String),
}

let row = vec![
    SpreadsheetCell::Int(3),
    SpreadsheetCell::Text(String::from("blue")),
    SpreadsheetCell::Float(10.12),
]
```

```
];
```

10.3 Sequences: Double-Ended Queues (VecDeque<T>)

When operations at both ends of a sequence are frequent (such as in a FIFO queue or a sliding window), `VecDeque` is the ideal choice.

10.3.1 Mechanism and Memory Layout

Unlike `Vec`, which only efficiently grows at the tail, a `VecDeque` is implemented as a **ring buffer**. It allocates a contiguous block of memory on the heap (just like a `Vec`), but it maintains two distinct indices: a **head** and a **tail**.

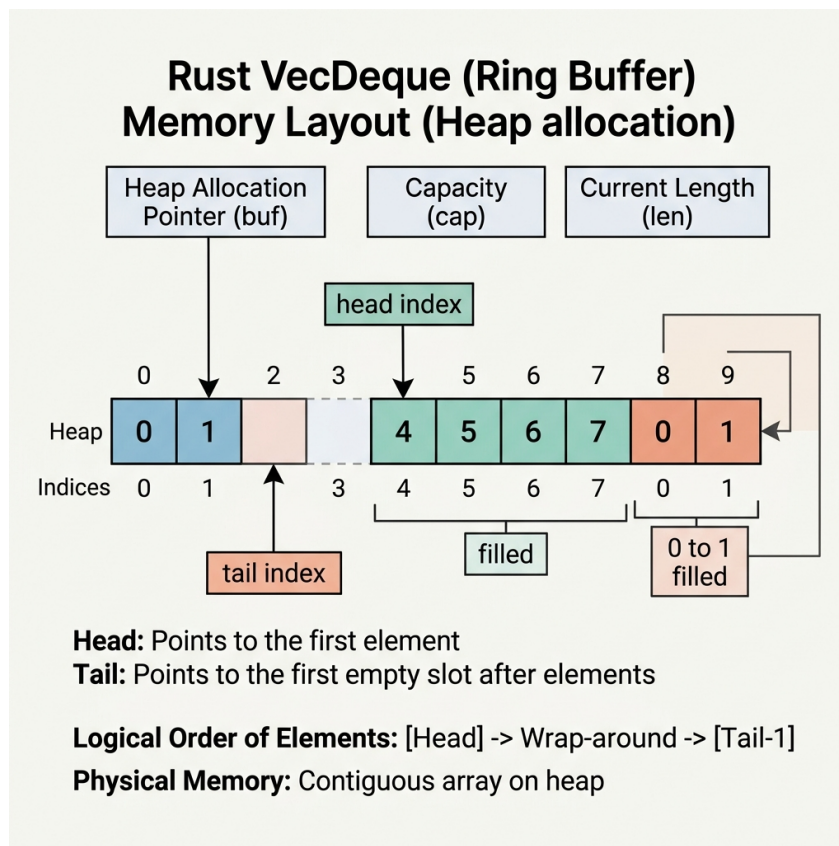


Figure 10.2: The ring buffer memory layout of a `VecDeque`. Elements can wrap around the end of the allocated heap space back to the beginning.

Because it tracks both ends, it achieves $O(1)$ **constant time insertions and removals at both ends**. If you push an element to the front, the `head` index simply decrements. If it hits the start of the allocated buffer, it mathematically “wraps around” to the very end of the buffer space. No existing elements need to be shifted in memory. Shifting and reallocation only occur if the ring buffer becomes completely full.

Since the elements might wrap around the buffer, they are not always strictly contiguous in physical memory order from the perspective of a slice. You can, however, force them to be physically contiguous using the `make_contiguous()` method.

When and why to use it: Use `VecDeque` when implementing a FIFO queue or maintaining a sliding window of recent elements where frequent insertions and removals occur at both ends.

10.4 Sequences: Linked Lists (`LinkedList<T>`)

Rust also provides a doubly linked list, `LinkedList<T>`.

10.4.1 Mechanism and Memory Layout

Unlike `Vec` and `VecDeque`, a linked list does not use a single contiguous block of heap memory. Instead, every single element inserted into the list is wrapped in a `Node` struct, which is allocated entirely on its own on the heap.

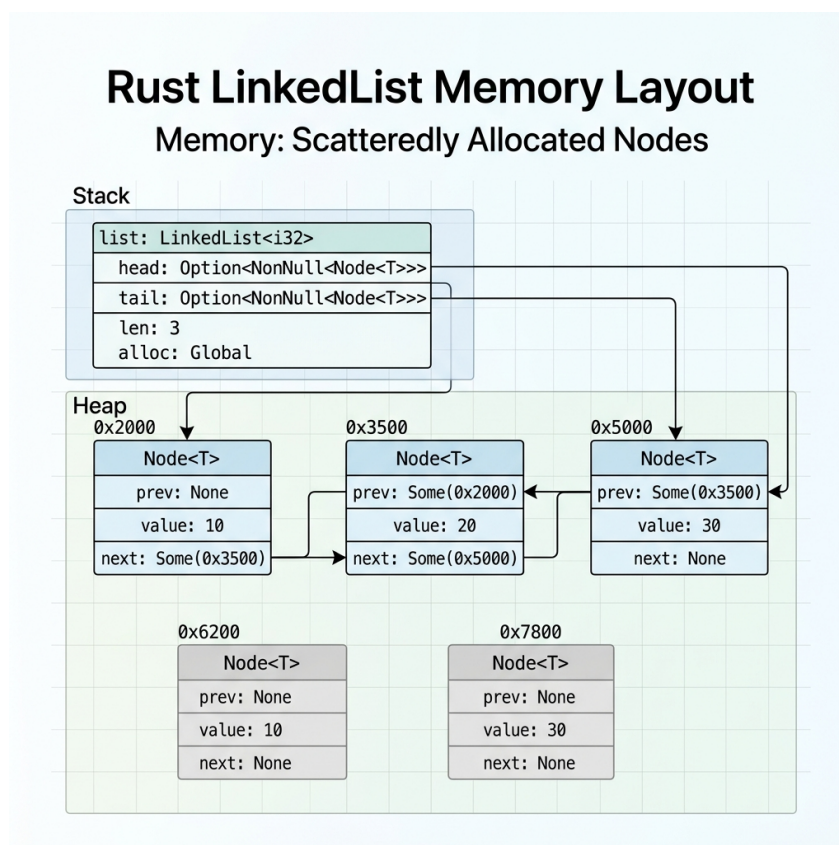


Figure 10.3: The doubly linked list memory layout. Nodes are scattered across the heap, connected by `prev` and `next` pointers.

Each node contains three components: a pointer to the previous node, the actual value, and a pointer to the next node. Because nodes are physically independent, inserting an element at the front or back guarantees a strictly $O(1)$ **constant time cost**. The system simply allocates a new node on the heap and updates the `next/prev` pointers of the existing head or tail to include the new node. Absolutely no shifting of existing elements is required.

10.4.2 Trade-offs

However, due to nodes being scattered randomly across the heap, `LinkedList` suffers from a high probability of **cache misses**. When the processor attempts to iterate through the list, fetching the next node likely requires querying the much slower main RAM rather than the ultra-fast CPU cache. Furthermore, accessing an arbitrary element by index requires traversing the list node-by-node, resulting in an $O(N)$ access time.

When and why to use it: Use `LinkedList<T>` almost exclusively when you need constant time $O(1)$ concatenation of two lists or splitting a list at a known node without allocating,

especially in real-time systems where reallocation latency is unacceptable. It is rarely used for general-purpose programming.

10.5 Text Collections: `String` vs `&str`

Strings are among the most misunderstood collections in Rust. To write robust systems code, you must build a solid mental model distinguishing between `String` and `&str`.

10.5.1 Building Intuition

Think of a `String` exactly like a `Vec<u8>`. In fact, a `String` is simply a wrapper over a vector of bytes that guarantees the bytes form a valid UTF-8 sequence. Since it owns its data on the heap, you can grow it, mutate it, and append to it.

Conversely, an `&str` (string slice) is a borrowed view into some UTF-8 data. It consists of a pointer to the start of the text and a length. It does not own the data, and it cannot grow or shrink.

! Exam Priority

Common Pitfall: Unnecessarily taking ownership by using `String` as a function parameter when a read-only `&str` view would suffice, leading to wasteful heap allocations.

Expected Mastery Level: Very High. A fundamental exam topic is choosing the correct string type:

- Use `String` when you need to **own** the text data or modify it (e.g., concatenating input, reading from a file).
- Use `&str` when you only need to **read** or view the text (e.g., passing a string as an argument to a function). This avoids unnecessary heap allocations.

10.5.2 Concrete Example: Conversions

```
// s1 is an &str pointing to read-only memory (the binary itself)
let s1: &str = "Hello, 🌐world!";

// We create an owned String allocated on the heap
let mut s2: String = String::from(s1);

// We can mutate the String
s2.push_str(" 🌐I 🌐am 🌐learning 🌐Rust.");

// We can extract a view (&str) from the String
let view: &str = &s2[0..5]; // "Hello"
```

Memory Model Analysis: `s1` is a fat pointer (`ptr`, `len`) residing on the stack, pointing to the `.rodata` segment of the compiled binary; it represents an immutable borrow with a `'static` lifetime. When creating `s2`, Rust allocates a new buffer on the heap and copies the bytes from `s1`. `s2` is an owned `String` (a stack structure with `ptr`, `len`, `capacity`). Calling `push_str` may trigger a heap reallocation if the new text exceeds the current capacity. Finally, `view` is another `&str` fat pointer on the stack, but this time its pointer points into the heap buffer owned by `s2`, and its lifetime is tied to the scope of `s2`. If `s2` is dropped or mutated further, `view` becomes invalid, which the borrow checker enforces at compile time.

10.6 Maps, Sets, and Priority Queues

10.6.1 The Problem of Fast Lookup

While vectors are excellent for sequential data, they are extremely inefficient if we need to search for an element by an arbitrary identifier (like a username or a product ID). A linear search takes $O(N)$ time. We need a structure that provides $O(1)$ expected lookup times.

10.6.2 Mechanism: Hashing and Buckets

The `HashMap<K, V>` associates keys of type `K` with values of type `V`. When a key is inserted, it passes through a *hash function*, producing a semi-random integer (the hash). This integer is used to index into a series of "buckets".

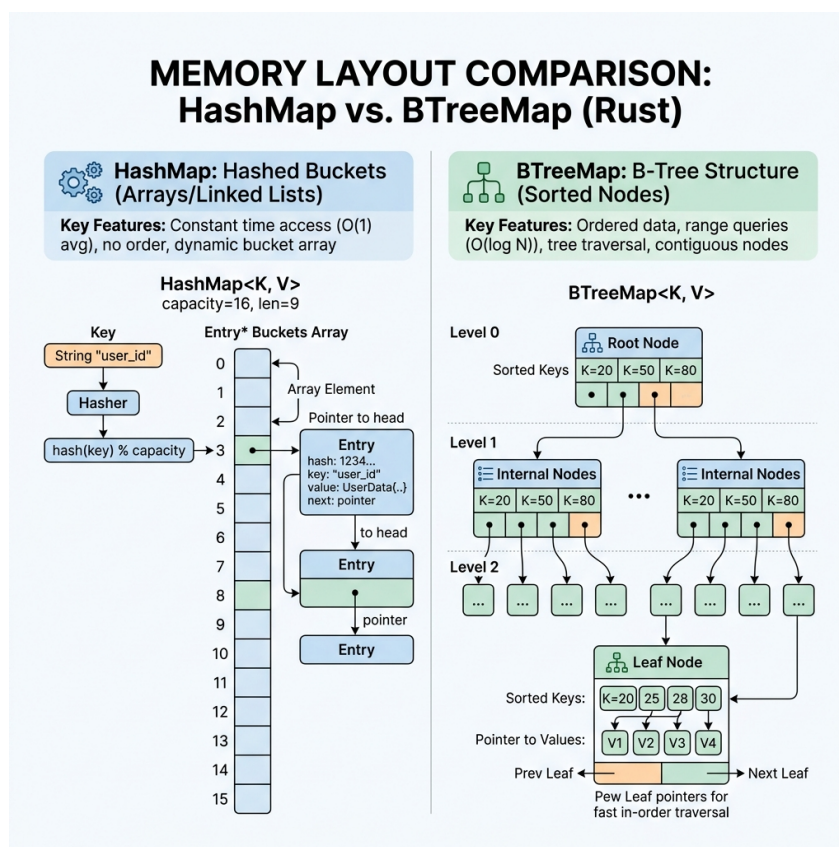


Figure 10.4: Memory Layout and Hashing Mechanism in a HashMap

As shown in Figure 10.4, if multiple keys map to the same bucket (a collision), they are stored linearly inside that bucket. For the map to function correctly, keys must implement two traits:

- **Hash:** to convert the key into a number.
- **Eq:** to precisely check for equality in case of collisions.

! Exam Priority

Common Pitfall: Assuming `HashMap` iteration order is predictable, or using types with internal mutability (e.g., `RefCell`) as keys, breaking invariants.

Expected Mastery Level: Very High. You must know when to choose `HashMap` ($O(1)$ expected, unordered) versus `BTreeMap` ($O(\log N)$, ordered). You also must know that `HashMap` requires `Hash` and `Eq`, while `BTreeMap` requires `Ord`, and that these implementations must be logically consistent.

Iterating over a `HashMap` yields keys and values in an unpredictable order. If you need ordered iteration, use a `BTreeMap` instead, which stores elements in a balanced search tree at the cost of $O(\log N)$ access time. Furthermore, keys in any map must remain immutable while they are stored.

Insight

[Consistent Traits and Internal Mutability] Just like `hashCode` and `equals` in Java, the `Hash` and `Eq` traits in Rust **must be logically consistent**. If two objects are equal (`Eq`), they must produce the same hash. Furthermore, a good hash function should evenly distribute values to avoid turning bucket lookups into $O(N)$ linear searches.

Crucially, **keys must remain stable**. You cannot use objects with internal mutability (like `RefCell`) as keys in a map, because mutating the key after insertion would corrupt the map's internal layout.

10.6.3 Map Initialization and Filtering

Unlike `Vec`, which has the handy `vec! []` macro, there is no standard macro for initializing maps (because the choice between `HashMap` and `BTreeMap` is often balanced, so the language designers preferred explicit instantiation). Instead, you initialize them using the `From` trait with an array of tuples:

```
use std::collections::HashMap;

let mut scores = HashMap::from([
    (String::from("Alice"), 80),
    (String::from("Bob"), 90),
    (String::from("Carol"), 70),
]);
```

Rust maps also provide excellent methods for filtering in-place:

- `retain(|key, &mut value| ...)`: Keeps only elements for which the closure returns `true`.
- `remove(key)`: Removes a specific key.

When iterating, you can use `into_iter()` to completely disassemble the map, taking ownership of the key-value tuples.

10.6.4 Concrete Example: The Entry API

Rust's `HashMap` exposes an incredibly elegant `Entry` API for updating values conditionally.

```
use std::collections::HashMap;

let mut scores = HashMap::new();
```

```

scores.insert(String::from("Alice"), 50);

// Using the Entry API: Insert "Bob" if he does not exist.
scores.entry(String::from("Bob")).or_insert(30);

// Update Alice's score safely
scores.entry(String::from("Alice")).and_modify(|score| *score += 10);

```

Memory Model Analysis: The `HashMap` `scores` is allocated on the stack, maintaining state about its heap-allocated hash table. When we call `String::from("Alice")`, a heap allocation occurs for the string data. The map takes ownership of this string, transferring the ownership from the caller to the heap-allocated bucket of the map. The `entry` method borrows `scores` mutably, and if the key is not found, `or_insert` consumes the `String` (moving its ownership to the map). The closure passed to `and_modify` takes a mutable borrow `&mut V` of the value residing in the map's heap, allowing us to safely mutate it in place without extracting it.

The **Entry** API avoids checking the map twice (once to see if the key exists, and once to insert/update), which improves both performance and readability.

10.6.5 Sets: `HashSet` and `BTreeSet`

When you only care about the presence of an element rather than a key-value association, you can use a set. A `HashSet<T>` is conceptually a `HashMap<T, ()>` and is perfect for deduplication or mathematical set operations (union, intersection, difference) in expected $O(1)$ time.

When and why to use it: Use `HashSet` for fast duplicate removal, fast membership checks, and set math operations where iteration order does not matter.

Its ordered counterpart, `BTreeSet<T>`, requires elements to implement `Ord` and is ideal when you need to extract ranges of values or iterate in a guaranteed order.

When and why to use it: Use `BTreeSet` when you need to iterate over unique elements in a deterministic order or need to efficiently query a range of elements.

10.6.6 Priority Queues: `BinaryHeap`

Sometimes you do not need to keep all elements perfectly sorted, but you frequently need to extract the "most important" or "heaviest" element. A `BinaryHeap<T>` provides a priority queue where the maximum element is always kept at the top. Peeking the top element is $O(1)$, while pushing or popping costs $O(\log N)$. It relies on the `Ord` trait to determine priority. To implement a Min-Heap (where the smallest/lightest element is extracted first, e.g., in Dijkstra's shortest path algorithm), you must invert the ordering logic. Rather than writing a custom `Ord` implementation, it is much easier to wrap your elements in `std::cmp::Reverse<T>`.

When and why to use it: Use `BinaryHeap` to implement task schedulers, find the top-K elements without sorting the entire dataset, or implement algorithms like Dijkstra's shortest path.

10.7 Persistence: `Path`, `PathBuf`, and `File`

Once we have accumulated data in memory via collections, we usually need to save it to disk. Rust handles filesystem abstractions via the `std::fs` and `std::path` modules.

10.7.1 Paths: `PathBuf` vs `Path`

Similar to the dichotomy between `String` and `&str`, Rust separates file paths into owned and borrowed types. Why not just use a `String` for a path? Because path separators (`/` on Unix,

\ on Windows) and encoding rules differ strictly across operating systems. A `Path` guarantees correct, cross-platform path semantics.

- `PathBuf`: The owned, heap-allocated, mutable equivalent of `String`. You use this when you need to construct or modify a path dynamically.
- `&Path`: The borrowed equivalent of `&str`. Used to pass a read-only path to functions.

```
use std::path::PathBuf;

// Building a path cross-platform
let mut config_path = PathBuf::from("/etc");
config_path.push("app");           // Joins correctly: /etc/app
config_path.push("config.json");   // /etc/app/config.json

// Displaying the path safely
println!("Config file is at: {}", config_path.display());
```

Insight

[Path Tokenization] Paths have intrinsic tokenization. A `Path` is aware of OS-specific separators (/ on Unix, \ on Windows) and treats them as boundaries between directory components. This makes navigating directories (e.g., getting the parent or filename) robust.

10.7.2 File System Utility Functions

The `std::fs` module provides various functions for interacting with the file system itself (not just the contents of files):

- `read_dir`: Returns an iterator over the entries within a directory.
- `create_dir` / `remove_dir`: Creates or removes an empty directory.
- `copy`: Duplicates a file.
- `rename`: Renames or moves a file. Note that `rename` behavior depends heavily on the OS, especially when moving files across different mounted disks.
- `remove_file`: Deletes a file.

10.7.3 File System Operations and I/O Traits

The `std::fs::File` struct represents an open file on the operating system. When a `File` goes out of scope, Rust automatically closes the file descriptor (leveraging RAII), preventing resource leaks.

Rust models Input/Output operations through four core traits in the `std::io` module:

- **Read**: Implemented by `File`, `TcpStream`, and `Stdin`. It provides methods to consume byte streams:
 - `read()`: Reads a chunk of bytes.
 - `read_to_end()`: Reads all bytes into a `Vec<u8>`.
 - `read_to_string()`: Reads the entire file into a UTF-8 `String`.

- `read_exact()`: Guarantees filling a pre-sized buffer exactly, useful for reading fixed-length binary records.
- `bytes()`, `chain()`, `take()`: Utility adapters for iteration and composing streams.
- **Write**: Implemented by `File`, `TcpStream`, `Stdout`, and `Stderr`.
 - `write()`: Attempts to write a buffer. Might be partial.
 - `write_all()`: Writes the entire buffer.
 - `flush()`: Forces the operating system to physically push its internal buffers to the persistent disk.
- **BufRead**: A sub-trait of `Read` that manages an internal buffer to minimize expensive user-to-kernel mode context switches. It is implemented by `BufReader`, `Cursor`, and `StdinLock`.
 - `fill_buf()`: Fills the internal buffer.
 - `consume(n)`: Marks `n` bytes as read from the buffer.
 - `read_line()` and `lines()`: Reads up to the next newline character.
- **Seek**: Allows manipulating the file cursor. Implemented by `File` and `Cursor`, but notably *not* by `TcpStream` (since network streams are not replayable).
 - `seek(pos)`: Moves the cursor relative to the start, end, or current position.
 - `rewind()`: Resets the cursor to the beginning (equivalent to `seek(0)`).
 - `stream_position()`: Returns the current cursor offset.

```
use std::fs::File;
use std::io::{Write, Read, BufReader, BufRead};
use std::path::Path;

// Writing to a file
let path = Path::new("data.txt");
// File::create truncates the file if it exists, or creates it
let mut file = File::create(&path).expect("Failed to create file");
file.write_all(b"Hello from Rust!\nSecond line").expect("Failed to write");

// Reading from a file with a buffer
let input = File::open(&path).expect("Failed to open file");
let buffered = BufReader::new(input);

for line in buffered.lines() {
    println!("{}", line.expect("Failed to read line"));
}
```

Memory Model Analysis: The `path` variable is a reference `&Path` (a fat pointer) on the stack, pointing to the string literal's static memory. When calling `File::create`, the OS allocates a file descriptor and returns it; the stack variable `file` takes ownership of this OS resource. The string literal `b"..."` is a slice `&[u8]` pointing to the read-only memory. In the reading phase, `BufReader` allocates a heap buffer to fetch chunks of data at once, minimizing expensive syscalls. The `lines()` iterator creates a new `String` (heap allocation) for each line parsed, transferring ownership of each line to the loop variable `line` inside the `Result`. Finally, RAII automatically closes both `file` and `input` when they go out of scope.

Note

[Advanced Open Options] For more precise control (such as appending to a file instead of truncating), you can use `std::fs::OpenOptions`, which allows explicit combinations of `read`, `write`, `append`, and `create` flags.

10.7.4 Error Handling in I/O

Virtually all I/O operations return a `std::io::Result<T>`. In case of an error, it returns a `std::io::Error`, which contains an `ErrorKind` enum. Checking the `ErrorKind` allows your application to intelligently recover. Common error kinds include:

- **Interrupted**: The OS interrupted the operation; you can safely retry it.
- **PermissionDenied**: The user lacks OS rights.
- **UnexpectedEof**: Reached the end of the file sooner than requested (e.g., during `read_exact()`).
- **InvalidData**: The data read was not valid UTF-8.
- **AlreadyExists**: Attempted to create a file with `OpenOptions` refusing to overwrite.

10.7.5 Structured Data Serialization with Serde

While manipulating raw bytes and strings is powerful, real-world applications often need to save structured data like configuration files or game states. The Rust ecosystem standardizes this with the **Serde** (Serialize/Deserialize) framework.

By annotating a struct with `#[derive(Serialize, Deserialize)]`, Serde automatically generates code to convert your data structures to and from formats like JSON, CSV, or YAML, drastically simplifying persistent storage.

```
use serde::{Serialize, Deserialize};

#[derive(Serialize, Deserialize, Debug)]
struct Config {
    username: String,
    theme: String,
}
// You can then use crates like 'serde_json' to seamlessly
// convert this struct to a JSON string and save it via File::write_all.
```

10.8 Closures and Functional Traits

In the final portion of our discussion on dynamic structures and execution, we must consider Rust's functional programming capabilities. Rust allows us to treat executable code as data, via **higher-order functions** that can accept functions as parameters or return them.

10.8.1 Function Pointers (fn)

A traditional function can be referenced using the `fn` type (a function pointer). These pointers do not carry any state.

```
fn add(a: i32, b: i32) -> i32 { a + b }
fn multiply(a: i32, b: i32) -> i32 { a * b }

// choose returns a function pointer
```

```
fn choose(sum: bool) -> fn(i32, i32) -> i32 {
    if sum { add } else { multiply }
}

let operation = choose(true);
println!("Result:␣{}", operation(2, 3)); // 5
```

10.8.2 Closures and State Capture

When we need a function to remember the context in which it was defined, we use a **closure**. A closure can capture *free variables* from its enclosing environment.

```
let offset = 10;
// Closure capturing 'offset' from the environment
let add_offset = |x: i32| -> i32 {
    x + offset
};
println!("Result:␣{}", add_offset(5)); // 15
```

The compiler automatically generates an anonymous struct to hold the captured variables (the "state"). Depending on how the closure interacts with those variables, it automatically implements one of three functional traits:

- **Fn**: The closure only reads the captured variables (captured by shared reference `&T`). It can be invoked multiple times concurrently without mutating its state.
- **FnMut**: The closure modifies the captured variables (captured by mutable reference `&mut T`). It can be invoked multiple times, but it evolves its internal state upon each call.
- **FnOnce**: The closure consumes the captured variables (captured by value, via the `move` keyword or implicit consumption). Because the environment is destroyed or moved, the closure can only be invoked exactly **once**.

These traits form a hierarchy: **Fn** is a sub-trait of **FnMut**, which is a sub-trait of **FnOnce**. This means any closure that implements **Fn** automatically satisfies bounds for **FnMut** and **FnOnce**.

Exam Relevance and Recurrence

Score: Very High. Collections and File I/O are the foundation of almost every system programming task in Rust. Expect to use `Vec`, `HashMap`, `String`, and `File` operations constantly throughout your code. Exam questions frequently probe your ability to choose the correct data structure, particularly contrasting `Vec` vs `VecDeque`, `HashMap` vs `BTreeMap`, and `String` vs `&str`. Understanding ownership transfer and correct handling of references during collection updates (e.g., using the Entry API) is systematically tested.

10.9 Domande ed Esercizi da Esami Passati

Nessuna domanda specifica da esami passati è stata mappata in questa sezione. I concetti trattati in questo capitolo sono comunque fondamentali e trasversali alla maggior parte degli esercizi di programmazione di sistema proposti negli esami.

Chapter 11

Smart Pointers

In our journey through Rust, we have seen how the ownership model, combined with borrowing rules, ensures memory safety without a garbage collector. However, these strict compile-time rules can sometimes feel restrictive. What if you need to build a complex data structure like a graph, where a single node might be pointed to by multiple edges? What if you want to extend the lifetime of a variable created inside a function so that it outlives the function call, without losing the ability to clean it up automatically?

To solve these problems, Rust relies on *smart pointers*. The concept of a smart pointer is not unique to Rust; it was born much earlier in the C++ ecosystem (with `unique_ptr`, `shared_ptr`, and `weak_ptr`) and was subsequently adopted and expanded upon by Rust. In Rust, smart pointers are data structures that act like pointers but encapsulate additional metadata and powerful guarantees.

Insight

The Magic of Traits and Operator Overloading

In C++, the concept of smart pointers relies heavily on operator overloading—extending the meaning of operators like `*` or `->` for custom types. Rust adopts a similar philosophy using traits. Under the hood, all smart pointers in Rust are just structs that implement two crucial traits:

- **Deref** (and **DerefMut**): Allows the smart pointer to behave like a standard reference, enabling you to use the `*` operator to access the underlying data seamlessly. This lets the compiler automatically perform deref coercions.
- **Drop**: Dictates what happens when the smart pointer goes out of scope, typically handling the automatic deallocation of the heap memory.

11.1 Exclusive Ownership on the Heap: `Box<T>`

The simplest and most common smart pointer is `Box<T>`. A `Box` allows you to store data on the heap rather than the stack. The box itself—which is essentially just a pointer—remains on the stack, while the data it points to lives on the heap.

Usage Guidelines: When and Why to Use `Box<T>`

- **When you have a type whose size can't be known at compile time** and you want to use it in a context that requires an exact size (e.g., recursive types, trait objects).

- **When you have a large amount of data** and you want to transfer ownership without copying the data byte-by-byte across the stack.

Building Intuition: Why use a Box?

Consider a function that creates a piece of data. Normally, local variables live on the stack and vanish as soon as the function returns. If we want the caller to own this data without copying potentially large structures byte-by-byte across the stack, we can allocate it on the heap.

```
fn produce(modify: bool) -> Box<i32> {
    let mut b = Box::new(0); // Allocates memory on the heap

    if modify {
        *b = 5; // Dereferencing allows us to modify the heap data
    }

    b // The box is returned, transferring ownership to the caller
}

fn main() {
    let b1 = produce(false);
    let b2 = produce(true);
} // b1 and b2 are dropped here, and their heap memory is safely freed
```

Deep Memory Model Analysis: When `produce` is called, the stack frame holds the `modify` parameter and the local variable `b`. `Box::new(0)` allocates exactly 4 bytes (for an `i32`) on the heap, and `b` stores the 8-byte pointer to this heap allocation. If `modify` is true, the heap data is mutated in place via dereferencing (`*b = 5`). Returning `b` transfers ownership: the 8-byte pointer is moved from `produce`'s stack frame to `main`'s stack frame (into `b1` or `b2`). The heap allocation itself remains untouched during this move. Finally, when `b1` and `b2` go out of scope at the end of `main`, their lifetimes end, triggering the `Drop` trait which deallocates the corresponding heap memory exactly once, avoiding any memory leak or double-free vulnerabilities.

! Exam Priority

Professor's Emphasis: Avoiding Leaks and Double Freeing

In C and C++, allocating memory with `malloc` requires explicitly calling `free`. Forgetting to do so causes a memory leak, while calling `free` twice causes undefined behavior. By tying the heap allocation to the lifetime of the `Box`, Rust completely eliminates both vulnerabilities.

Common Pitfall: Assuming moving a `Box` copies the heap data. It only moves the pointer on the stack, transferring exclusive ownership without copying the underlying heap data. **Expected Mastery Level:** Very High. You must thoroughly understand how the `Drop` trait is tied to the `Box`'s scope to guarantee exactly-once deallocation.

Fat Pointers vs. Thin Pointers

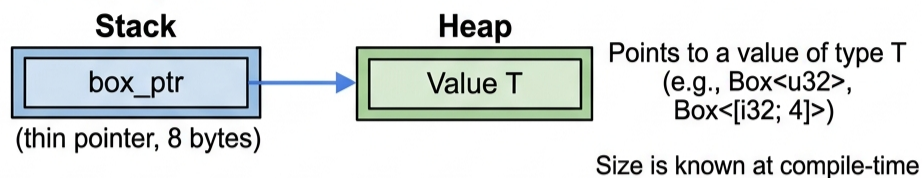
If the compiler knows the size of the data stored in a `Box` (e.g., `Box<i32>`), the pointer is simply an 8-byte memory address on a 64-bit architecture. This is called a *thin pointer*.

However, boxes are incredibly useful for storing *unsized types* (types whose size is not known at compile time), such as slices (`[T]`) or trait objects (`dyn Trait`). When a `Box` points to an unsized type, it becomes a *fat pointer* (taking up 16 bytes on a 64-bit architecture).

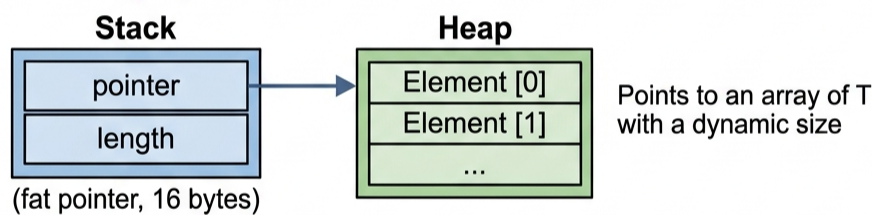
- **Slices** (`Box<[T]>`): The first 8 bytes point to the heap data, and the second 8 bytes store the length of the slice.
- **Trait Objects** (`Box<dyn Debug>`): The first 8 bytes point to the heap data, and the second 8 bytes point to a vtable used for dynamic dispatch. As the professor details, the vtable contains a sequence of entries: the first is the pointer to the drop logic, the second is the size of the concrete type, the third is its alignment, and the subsequent entries are the methods offered by the trait (e.g., the formatting method for `Debug`).

Rust Memory Layout: `Box<T>` Types

1) `Box<T>` (Sized)



2) `Box<[T]>` (Slice)



3) `Box<dyn Trait>` (Trait Object)

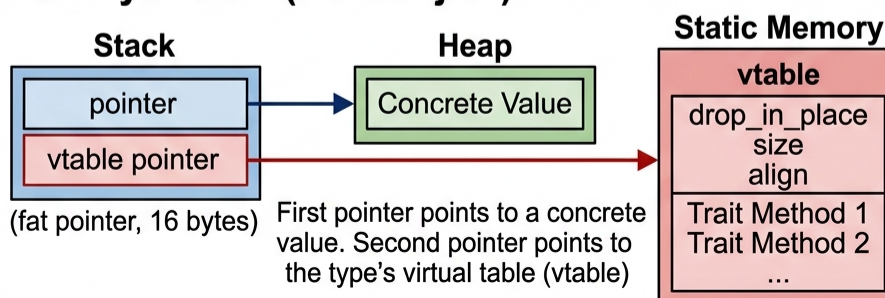


Figure 11.1: The three fundamental memory layouts of `Box<T>`: A thin pointer for sized types, a fat pointer with length for slices, and a fat pointer with a vtable for trait objects.

11.2 Shared Ownership: `Rc<T>` and `Arc<T>`

While `Box<T>` works flawlessly for exclusive ownership, there are scenarios where a single piece of data must be owned by multiple parts of your program. Consider a Directed Acyclic Graph (DAG) or a complex tree where multiple child nodes need to reference the same parent.

This brings us to the `Rc<T>` (Reference Counted) smart pointer.

Usage Guidelines: When and Why to Use Rc<T>

- **When multiple parts of your program need to read the same data** concurrently in a single-threaded context.
- **When you cannot determine at compile time** which part of the program will finish using the data last.

How Reference Counting Works

Rc<T> keeps track of the number of active references to a value. When you create an Rc, its internal counter is initialized to 1. Every time you duplicate the pointer using Rc::clone, the counter increments. Crucially, this does *not* clone the underlying data; it merely copies the pointer and increases the reference count.

When an Rc goes out of scope, the counter decrements. The actual heap memory is only freed when the counter hits zero, meaning there are no more active owners.

Insight

Reference Counting vs. Garbage Collection

Languages like Java or Python use a Garbage Collector (typically a "Mark-and-Sweep" algorithm) to manage memory. A Mark-and-Sweep GC periodically traverses the entire heap from the root variables to find active allocations, cleaning up unreferenced ones. However, this process often requires pausing the entire program (a "stop-the-world" event), making it unsuitable for real-time applications. Rust's Rc<T> provides a deterministic alternative: memory is freed the exact moment the last reference disappears, without global pauses, at the cost of requiring the programmer to explicitly handle reference cycles.

```
use std::rc::Rc;

enum List {
    Cons(i32, Rc<List>),
    Nil,
}

use List::{Cons, Nil};

fn main() {
    // We create an Rc pointing to our base list: 5 -> 10 -> Nil
    let a = Rc::new(Cons(5, Rc::new(Cons(10, Rc::new(Nil)))));

    // Both 'b' and 'c' can now share ownership of list 'a'
    let b = Cons(3, Rc::clone(&a)); // Reference count of 'a' becomes 2
    let c = Cons(4, Rc::clone(&a)); // Reference count of 'a' becomes 3
}
```

Deep Memory Model Analysis: When `a` is created, the list nodes are allocated on the heap. The variable `a` on the stack holds an 8-byte pointer to the `Cons(5, ...)` heap block, which internally contains a `strong_count` (initialized to 1). When `Rc::clone(&a)` is called for `b` and `c`, no new heap allocations for the shared list elements are created. Instead, the `strong_count` in the heap is incremented to 2, then 3. `b` and `c` are allocated on the stack with their own list head, but their Rc fields point to the exact same heap address as `a`. When `c`, `b`, and `a` go out of scope, the `strong_count` is decremented. Only when it reaches 0 (after the last

owner drops) does the heap block invoke its drop logic, cascading the deallocation down the chain.

Warning

Calling Associated Functions on `Rc`

When interacting with the reference counter itself (e.g., checking the count), use the fully qualified syntax `Rc::strong_count(&a)` rather than `a.strong_count()`. This prevents naming collisions if the underlying data `T` happens to have a method named `strong_count`. Because `strong_count` is an associated function and not a method taking `self`, the compiler avoids ambiguity. The variable `a` behaves seamlessly as the inner type for method calls, while `Rc`-specific operations require explicit functional notation.

The Cost of Synchronization: `Rc` vs `Arc`

You might wonder: is `Rc` safe to use across multiple threads? The answer is **no**. The increments and decrements to the reference counter are standard, non-atomic operations. In a multithreaded context, race conditions could cause the counter to become corrupted, leading to double-frees or memory leaks.

Insight

Professor's Insight: Unprotected Increments

If you were to write a similar unprotected reference counter in C and modify it concurrently across multiple threads, the results would be disastrous. You would observe the counter increasing randomly, missing increments entirely, or even sporadically going backward due to race conditions. This is undefined behavior at its finest. Rust prevents this entirely by disallowing `Rc` to be sent across threads.

Memory Layout of Rust Rc<T>: Shared Ownership

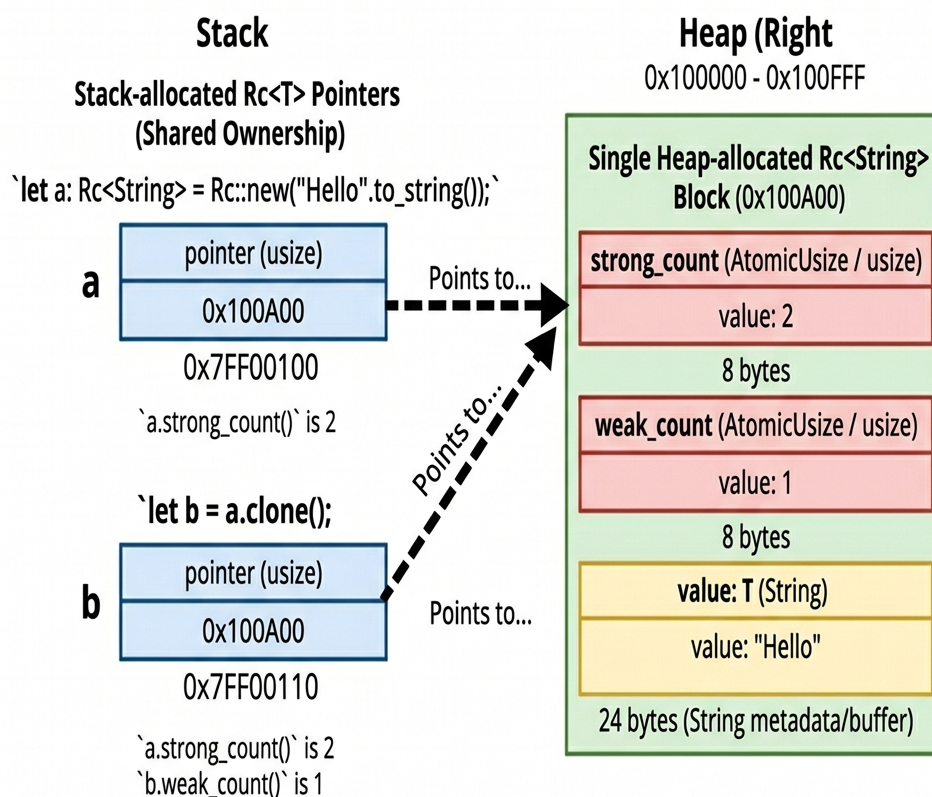


Figure 11.2: Under the hood: Memory layout of `Rc<T>`. The heap block contains a strong count, a weak count, and the actual value. Multiple stack pointers share ownership of this single heap block.

To solve this, Rust provides `Arc<T>` (Atomic Reference Counted). `Arc` uses atomic CPU instructions to update the counter, guaranteeing thread safety. However, atomic operations are slightly slower than normal arithmetic. In true Rust fashion (zero-cost abstractions), the language does not force you to pay for thread safety if you don't need it.

- Use `Rc<T>` in single-threaded contexts.
- Use `Arc<T>` in concurrent contexts.

! Exam Priority

Professor's Emphasis: The Cost of Synchronization (Rc vs Arc)

Unlike C++ which only provides the thread-safe `shared_ptr` (forcing you to pay synchronization costs even in single-threaded contexts), Rust gives you a choice. Use `Rc` for fast, non-atomic reference counting in single-threaded programs, and `Arc` for safe, atomic reference counting in concurrent ones.

Common Pitfall: Believing `Rc` is thread-safe or trying to pass it across threads. The compiler will catch this because `Rc` does not implement the `Send` trait, but you must know *why* (non-atomic counter updates lead to race conditions and memory corruption).

Expected Mastery Level: High. Be prepared to justify the choice between `Rc` and `Arc` based on concurrency needs.

11.3 Safely Mutating Shared Data

By default, an `Rc` (or `Arc`) provides *immutable* shared access. If multiple variables point to the same value, modifying it underneath them would violate Rust's core rule: you cannot have mutable and immutable aliases active simultaneously.

So, how can we mutate data hidden inside an `Rc`?

Copy-on-Write: `Rc::make_mut` and `Rc::get_mut`

If you can prove that you are the *only* owner of an `Rc` at a given time, you are allowed to mutate it. `Rc::get_mut` checks if the reference count is exactly 1 and returns an `Option<&mut T>`.

If the count is greater than 1, you can use `Rc::make_mut`. This function performs a *Copy-on-Write* (CoW): it clones the underlying data into a new heap allocation, decrements the old counter, and points your `Rc` to the new, uniquely owned copy.

```
use std::rc::Rc;

// --- get_mut example ---
let mut a = Rc::new(String::from("abc"));
// Since 'a' is the unique owner (strong_count == 1), get_mut succeeds.
if let Some(s) = Rc::get_mut(&mut a) {
    s.push('!');
}
// 'a' now contains "abc!"

// --- make_mut example ---
let mut x = Rc::new(String::from("hello"));
let y = Rc::clone(&x); // Count is now 2

// x and y point to the same string.
Rc::make_mut(&mut x).push('!');

// Now x is "hello!" and y is "hello".
// make_mut cloned the data because the count was > 1.
```

Deep Memory Model Analysis: In the `get_mut` example, `a` is the sole owner. `Rc::get_mut` safely returns a mutable reference (`Option<&mut String>`), allowing direct in-place mutation without any new allocations. If a second reference existed, `get_mut` would return `None` to prevent mutating data shared with others.

In the `make_mut` example, initially, both `x` and `y` on the stack point to the same heap allocation containing the string "hello", with a `strong_count` of 2. When `Rc::make_mut(&mut`

`x`) is called, the system detects `strong_count > 1`. It performs a new heap allocation to clone the string, decrements the `strong_count` of the original allocation (so `y`'s target now has a count of 1), and updates `x`'s pointer to point to the new, uniquely owned heap block (count of 1). The `push('!')` operation then mutates only `x`'s new heap allocation. `y` remains completely unaffected, preserving memory safety and the immutability guarantee for shared aliases.

Internal Mutability: `Cell` and `RefCell`

What if you absolutely must mutate the shared data *without* copying it? This is where the concept of **Internal Mutability** comes in. Rust provides types like `Cell<T>` and `RefCell<T>`, which allow you to mutate their inner data even if the cell itself is accessed via an immutable reference. These types are meant for single-threaded contexts.

Usage Guidelines: When and Why to Use `Cell` and `RefCell`

- **When you need to mutate data** but only have an immutable reference to it (e.g., mutating a field within an `Rc`).
- `Cell<T>` is preferred for simple types that implement `Copy` (like integers), as it moves/copies values.
- `RefCell<T>` is preferred for complex types where you need references, moving borrow checking to runtime.
- `Cell<T>`: Achieves internal mutability by copying or moving values in and out of the cell. It does not return references to the inner data. Instead, it offers methods like `set()`, `get()` (if `T` implements `Copy`), `replace()`, and `take()` to swap the contents.
- `RefCell<T>`: Allows you to obtain references to the inner data (`Ref` and `RefMut`) using the `borrow()` and `borrow_mut()` methods. By wrapping your data in a `RefCell` inside an `Rc` (i.e., `Rc<RefCell<T>>`), you achieve a powerful pattern: shared, mutable ownership. The catch? The borrowing rules are shifted from *compile-time* to *runtime*. If you try to acquire two mutable references to the data at the same time using a `RefCell`, your program will panic and crash instead of giving a compilation error.

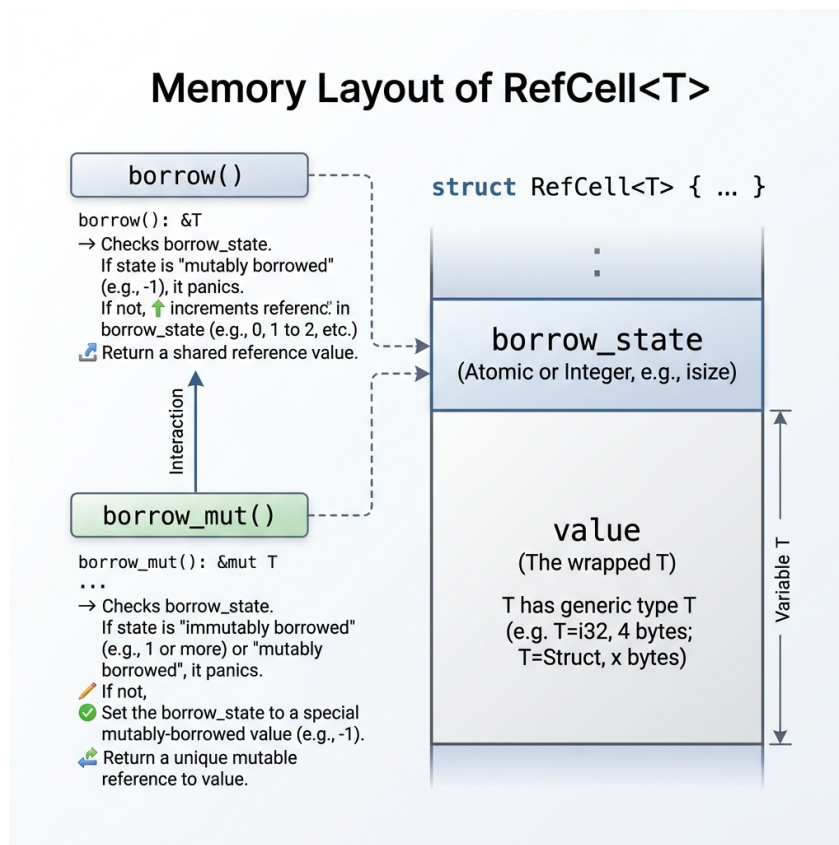


Figure 11.3: Under the hood: Memory layout of a `RefCell<T>`. It wraps the inner value alongside a `borrow_state` counter to dynamically enforce Rust's borrowing rules at runtime.

Clone-on-Write: `Cow<'a, B>`

Rust offers another unique smart pointer: `std::borrow::Cow` (Clone-on-Write). It is an enum that can hold either a borrowed reference (`Borrowed`) or an owned value (`Owned`).

`Cow` is extremely useful when you want to read data without copying it, but you might occasionally need to mutate it. If you try to mutate a `Borrowed` variant, `Cow` will lazily clone the data, become an `Owned` variant, and apply the mutation, leaving the original data untouched. If the data is already `Owned`, it modifies it in place without extra allocations. This pattern drastically reduces unnecessary heap allocations when modifications are rare.

11.4 Breaking Cycles with `Weak<T>`

Reference counting has one fundamental flaw: memory leaks caused by **reference cycles**. Imagine object A has an `Rc` pointing to object B, and object B has an `Rc` pointing back to object A. Both reference counts are at least 1. Even if all external variables pointing to A and B are dropped, their internal pointers keep each other's counts at 1. The memory will never be freed.

To fix this, Rust provides `Weak<T>`. While cloning an `Rc` increments the *strong* counter, downgrading an `Rc` to a `Weak` pointer increments the *weak* counter. Crucially, **the heap data is deallocated as soon as the strong counter reaches zero**, regardless of how high the weak counter is.

```
use std::rc::{Rc, Weak};
```

```
fn main() {
```

```

let strong_five = Rc::new(5);

// Downgrade creates a Weak pointer. Strong count remains 1.
let weak_five: Weak<i32> = Rc::downgrade(&strong_five);

// To use the value, you must upgrade the Weak pointer back to an Rc
if let Some(val) = weak_five.upgrade() {
    println!("Value is still alive: {}", val);
}

drop(strong_five); // Strong count drops to 0, memory is freed.

// Now, upgrade() will return None
assert!(weak_five.upgrade().is_none());
}

```

Deep Memory Model Analysis: The `Rc::new(5)` call allocates the value 5 on the heap alongside two counters: `strong_count` (1) and `weak_count` (1). `Rc::downgrade` increments the `weak_count` to 2, leaving `strong_count` at 1. The `weak_five` variable on the stack points to the same heap block. When `upgrade()` is called, it checks if `strong_count > 0`. Since it is, it temporarily increments `strong_count` to 2 and returns an `Rc`. Once the `if let` scope ends, `strong_count` returns to 1. When `strong_five` is explicitly dropped, `strong_count` becomes 0, and the inner value (5) is dropped. However, the heap allocation *itself* remains partially alive to hold the weak counter. As the professor emphasizes, **the entire block in the heap is actually released only when both the strong and weak counters hit 0**. When `weak_five` goes out of scope, `weak_count` drops to 0, and the entire underlying memory block is finally returned to the operating system.

! Exam Priority

Professor's Emphasis: Reference Cycles and `Weak<T>`

A tree where a parent has an `Rc` to its children, and children have an `Rc` back to their parent, creates a reference cycle. This renders the data un-deallocatable because the `strong_count` never reaches 0. In garbage-collected languages (Java, Python), a complex "Mark-and-Sweep" pause is needed to find and clear these disconnected islands. Rust avoids this overhead but forces you to break cycles explicitly by using `Weak<T>` for back-references (e.g., child to parent).

Common Pitfall: Forgetting that `Weak::upgrade()` returns an `Option<Rc<T>>`. You must handle the `None` case because the data may have already been dropped! **Expected Mastery Level:** High. You should be able to design a cyclical data structure (like a doubly linked tree or graph) using `Rc` and `Weak` correctly.

When building a tree, a parent should own its children using `Rc<T>` (or `Arc<T>`), but children should only have a `Weak<T>` pointer back to their parent. This ensures the tree structure can be correctly deallocated from the root down, avoiding infinite cycles.

Exam Relevance and Recurrence

Score: Very High

Notes: Smart pointers are a core topic that bridges memory safety, borrowing rules, and complex data structures. Exams frequently test your ability to choose the correct smart pointer for a given scenario (e.g., `Rc` vs `Arc`, `RefCell` vs `Cell`). Expect questions asking you to identify memory leaks caused by reference cycles and how to fix them with `Weak<T>`. You may also be asked to trace the reference counts of an `Rc` across various clones and drops.

11.5 Domande ed Esercizi da Esami Passati

Exam Focus

Domanda 1: Sia dato un primo valore di tipo `std::cell::Cell<T>` ed un secondo valore di tipo `std::cell::RefCell<T>` (dove `T` fa riferimento alla medesima entità). Si indichino le differenze tra i due e le modalità di occupazione della memoria (quantità, zone di memoria, ecc.).

Soluzione: Entrambi i tipi permettono la *Internal Mutability*, ovvero la capacità di modificare i dati interni pur possedendo solo un riferimento immutabile alla struttura che li contiene. Sono progettati per contesti single-thread.

- **Cell<T>:** È preferito per tipi semplici che implementano il trait `Copy` (come gli interi). Non restituisce riferimenti ai dati interni, ma sposta o copia i valori dentro e fuori dalla cella tramite metodi come `set()`, `get()`, `replace()` e `take()`.
- **RefCell<T>:** È preferito per tipi complessi. Permette di ottenere riferimenti espliciti ai dati interni (`Ref` e `RefMut`) spostando le regole di borrow checking dal tempo di compilazione al runtime. Se le regole vengono violate (ad esempio, richiedendo due riferimenti mutabili contemporaneamente), il programma va in *panic* anziché generare un errore di compilazione.

In termini di memoria, sia `Cell` che `RefCell` fungono da wrapper attorno ai dati originari e tipicamente risiedono sullo stack, a meno che non vengano esplicitamente allocati sull'heap (ad esempio, incapsulandoli in un `Rc<T>`).

Exam Focus

Domanda 2: Si definisca il concetto di Smart Pointer, quindi si fornisca un esempio (Rust o C++) che ne evidenzi il ciclo di vita.

Soluzione: In Rust, gli smart pointer sono strutture dati che si comportano come puntatori (grazie all'implementazione del trait `Deref`, che permette l'uso dell'operatore `*`), ma incapsulano metadati aggiuntivi e forti garanzie sulla sicurezza della memoria. Il loro ciclo di vita è governato dal trait `Drop`, che stabilisce come deallocare automaticamente la memoria heap quando lo smart pointer esce di scope. Un esempio classico è `Box<T>`:

```
fn produce() -> Box<i32> {
    let mut b = Box::new(0); // Alloca 4 byte sull'heap. 'b' e' sullo stack.
    *b = 5;                  // Dereferenziazione per mutare il valore.
    b                        // Trasferisce il possesso al chiamante.
}

fn main() {
    let b1 = produce();
} // b1 esce di scope: il trait Drop interviene e dealloca la memoria heap in modo sicuro
```

Quando si invoca `Box::new(0)`, il dato viene allocato sull'heap, mentre il puntatore rimane sullo stack. Al termine di `main`, la variabile `b1` esce dallo scope, innescando automaticamente `Drop` che libera esattamente una volta la memoria heap associata, evitando *memory leaks*.

Exam Focus

Domanda 3: Si spieghi il concetto di possesso in relazione agli smart pointers. Come viene gestito il ciclo delle risorse quando si utilizzano smart pointers? Infine, si espliciti il ciclo di vita delle risorse nel seguente esempio: `{ let mut i = 10; let bi1 = Box::new(i); let mut bi2 = Box::new(bi1); bi2 = 20; i = bi2; println!("{}", i, bi1, bi2); }`

Soluzione: Il **possesso (ownership)** indica chi è responsabile della deallocazione di una risorsa. `Box<T>` garantisce un possesso esclusivo sulla memoria heap: spostare un `Box` trasferisce l'ownership copiando il puntatore sullo stack, senza copiare i dati sull'heap. Smart pointer come `Rc<T>` e `Arc<T>` offrono invece un possesso condiviso tramite un contatore di riferimenti. Il **ciclo delle risorse** è gestito automaticamente tramite il trait `Drop`. Quando lo smart pointer esce dallo scope, il codice in `Drop` dealloca la risorsa (per `Box` lo fa immediatamente, per `Rc` solo quando il contatore `strong_count` raggiunge lo zero). Nel codice fornito (pur contenendo operazioni logicamente ed a livello di tipo errate in Rust, come assegnare un intero a un `Box`), il ciclo teorico delle risorse è il seguente:

- `let mut i = 10;` Alloca un valore intero sullo stack.
- `let bi1 = Box::new(i);` Crea un'allocazione sull'heap contenente il valore di `i`. `bi1` acquisisce il possesso esclusivo di questa memoria heap.
- `let mut bi2 = Box::new(bi1);` Il possesso di `bi1` viene trasferito (spostato). Viene creata una nuova allocazione heap per `bi2` (`Box<Box<i32>>`), rendendo il `bi1` originale sullo stack non più valido.

Exam Focus

Domanda 4: Si definisca il problema delle referenze cicliche nell'uso degli smart pointers. Si fornisca quindi un esempio in cui tale problema sia presente.

Soluzione: Le referenze cicliche avvengono in contesti di possesso condiviso (utilizzando `Rc<T>`). Se un oggetto A possiede un puntatore `Rc` verso un oggetto B, e B possiede un puntatore `Rc` verso A, i loro `strong_count` rimarranno bloccati ad almeno 1. Anche quando tutte le variabili esterne che puntano ad A e B vengono distrutte e i rispettivi scope terminano, i riferimenti interni reciproci impediscono ai contatori di scendere a 0. Di conseguenza, il trait `Drop` non viene mai eseguito, causando un *memory leak*. **Esempio:** Una struttura dati ciclica come un albero o un grafo in cui un nodo padre possiede i propri figli tramite `Rc<T>`, e i figli possiedono un riferimento al padre usando a loro volta un `Rc<T>`. Questo genera un ciclo infinito che non potrà essere deallocato. Per risolvere e spezzare il ciclo, il riferimento dal figlio al padre deve essere implementato usando un `Weak<T>`, il quale incrementa solo il `weak_count` e non impedisce che la memoria venga liberata quando lo `strong_count` arriva a zero.

Chapter 12

Closures and Iterators

12.1 Introduction to Closures

In system programming, there are many scenarios where we need to pass behavior as an argument to another function. Think of a sorting algorithm where you need to specify a custom comparison rule, or a thread that needs to execute a specific task. While traditional functions can be passed around via function pointers, they have a critical limitation: they cannot easily capture variables from the environment where they are defined.

This is the problem that **closures** solve. A closure is an anonymous function that can capture variables from its surrounding lexical scope. You can think of a closure as a regular function equipped with a "backpack" containing the data it captured from its environment.

12.1.1 Building Intuition: Capturing the Environment

When you define a closure in Rust, the compiler automatically creates an anonymous **struct** under the hood. This struct contains the variables that the closure captures. Depending on how the closure uses these variables, they are captured either by shared reference, mutable reference, or by value (ownership transfer).

```
let y = 10;
let z = 20;

// A closure capturing y and z by reference
let add_refs = |x| {
    x + y + z
};

println!("Result: {}", add_refs(5)); // Output: 35
```

Deep Memory Model Analysis: When the closure `add_refs` is created, the compiler allocates an anonymous struct on the stack. Because `y` and `z` are only read, this struct contains two immutable references: `&i32` pointing to `y` and `&i32` pointing to `z`. No heap allocation occurs here. The closure's lifetime is implicitly bounded by the lifetimes of `y` and `z`. When `add_refs` is invoked, it accesses these stack-allocated integers via the stored references without taking ownership.

Note

The compiler infers how variables should be captured by looking at what the closure does with them. If you only read `y` and `z`, they are captured as immutable references (`&y`, `&z`).

If the closure modifies a captured variable, it implicitly captures a mutable reference.

```

let mut sum = 0;
// Captures 'sum' by mutable reference
let mut accumulate = |x| {
    sum += x;
};

accumulate(5);
accumulate(7);
println!("Sum: {} ", sum); // Output: 12

```

Insight

Mental Model: Closures as Structs

When you write `|x| { sum += x }`, Rust generates a struct holding `&mut i32` (pointing to `sum`). The closure itself is just an instance of this struct that implements a specific trait to become callable.

Sometimes, especially when returning a closure from a function or sending it to another thread, you want the closure to take full ownership of the captured variables. You force this behavior using the `move` keyword.

Note

Lifetimes and move: Without `move`, a closure captures variables by reference, meaning the closure's lifetime is strictly tied to (and cannot outlive) the shortest-lived captured reference. Using `move` makes the closure self-sufficient. Also, note that using `move` forces capture by value, but it *does not* automatically make the closure `FnOnce`. The trait implemented still depends entirely on how the captured value is used inside the closure's body.

```

let prefix = String::from("ID_");

// 'move' forces the closure to take ownership of 'prefix'
let generate_id = move |i| {
    format!("{}", prefix, i)
};

// prefix is no longer accessible here!

```

Deep Memory Model Analysis: Here, the `String` `prefix` is a fat pointer on the stack (containing a pointer, length, and capacity) that manages a heap-allocated UTF-8 buffer. By using `move`, the ownership of the `String` is transferred entirely into the closure's anonymous struct. The struct is created on the stack containing the `String` fat pointer. As a result, the original `prefix` variable becomes uninitialized (moved). When the closure is eventually dropped, it will drop the `String`, deallocating the heap buffer. The closure now dictates the lifetime of the data, making it "self-sufficient" and safe to return or pass to other threads.

12.1.2 The Closure Traits: `FnOnce`, `FnMut`, and `Fn`

Rust formalizes the behavior of closures through three functional traits. These traits dictate what the closure is allowed to do with its captured environment and how many times it can be called.

- **`FnOnce`:** The most general closure type. It consumes its captured variables. Because it consumes them, it can be called **strictly once**. All closures implement `FnOnce`.

- **FnMut**: A closure that can mutate its captured variables. It can be called multiple times. It requires `&mut self` to be executed.
- **Fn**: A closure that only reads its captured variables (or captures nothing at all). It can be called multiple times without modifying its state. It takes `&self`.

Exam Focus

Exam Tip: Remember the hierarchy of closure traits: **Fn** is a sub-trait of **FnMut**, which is a sub-trait of **FnOnce**. A function that expects an **FnOnce** can accept any closure, but a function expecting an **Fn** is the most restrictive.

12.1.3 Passing and Returning Closures

When a function accepts a closure, it can do so using **Generics** (static dispatch) or **Trait Objects** (dynamic dispatch).

! Exam Priority

Common Pitfall: Confusing when to use generics vs. trait objects for closures, resulting in either code bloat or unnecessary runtime overhead. **Expected Mastery Level:** High. You must be able to justify your choice between monomorphization and dynamic dispatch.

Using Generics (Static Dispatch):

```
// We don't care about the specific type F, as long as it implements Fn
fn apply<F>(value: i32, f: F) -> i32
where
    F: Fn(i32) -> i32
{
    f(value)
}
```

```
let squared = apply(5, |x| x * x); // Generates apply_square
let halved  = apply(10, |x| x / 2); // Generates apply_divide
let inc     = apply(3, |x| x + 1); // Generates apply_increment
```

Trade-off: Generics generate specialized machine code for every unique closure passed to `apply` (a process called monomorphization). For instance, the compiler will generate three completely distinct copies of the `apply` function under the hood (e.g., `apply_square`, `apply_divide`, `apply_increment`). This makes it extremely fast (often inlined), but can slightly increase compile times and binary size.

Using Trait Objects (Dynamic Dispatch):

```
struct PipelineStep {
    // We use a Boxed trait object to store ANY closure of this signature
    op: Box<dyn Fn(i32) -> i32>,
}
```

Trade-off: Trait objects encapsulate the closure behind a pointer (a fat pointer containing the closure data and a vtable). **Deep Memory Model Analysis:** The vtable for a closure in this context typically contains exactly four entries: a pointer to the `drop` logic, the `size` of the captured data, its `alignment`, and the pointer to the actual anonymous method to execute. This is slightly slower due to pointer indirection (cache misses as the CPU jumps to the vtable and then the code—as the professor noted, “skipping two steps breaks the location of the cache a bit, we’re talking about nanoseconds”). However, its major advantage is **homogeneity**. It allows storing heterogeneous closures (e.g., an addition closure, a multiplication closure) in a single collection like `Vec<Box<dyn Fn(...)>>` to build flexible execution pipelines.

! Exam Priority

Common Pitfall: Forgetting the `move` keyword when returning a closure, leading to "borrowed value does not live long enough" compiler errors. **Expected Mastery Level:** Very High. Returning stateful closures (generators) is a heavily emphasized exam pattern. **Returning Closures (Generators):** When a function returns a closure, the exact type of the closure is unknown at compile time. Thus, we must use the `impl Trait` syntax (e.g., `impl Fn() -> T`).

```
// Returns a closure that yields copies of the captured value 'v'
fn value_source<T: Clone>(v: T) -> impl Fn() -> T {
    // 'move' is essential here to take ownership of 'v'
    move || v.clone()
}
```

Warning

The "Dangling" Threat: If we did not use `move`, the closure would try to capture `v` by reference. However, `v` is a parameter local to the function's stack frame. As soon as the function returns, that stack frame is destroyed. Any returned reference would become *dangling*, pointing to invalid memory. The `move` keyword forces the closure to take full possession of the value ("diventa mio, me lo copio"), making it self-sufficient.

Returning a closure is also a common pattern for creating stateful generators. This approach neatly encapsulates internal state without requiring you to define an explicit struct.

```
fn create_generator(prefix: String) -> impl FnMut() -> String {
    let mut i = 0;
    // 'move' ensures 'prefix' and 'i' are moved into the closure
    move || {
        i += 1;
        format!("{}", prefix, i)
    }
}

let mut gen = create_generator(String::from("User_"));
println!("{}", gen()); // User_1
println!("{}", gen()); // User_2
```

Deep Memory Model Analysis: When `create_generator` is called, `prefix` (a `String` with a heap allocation) and `i` (an `i32`) are created on the stack frame of the function. The `move` keyword forces the closure to take ownership. A new anonymous struct implementing `FnMut` is created, and the `String` fat pointer and the integer are moved into this struct. The function then returns this struct (its size is statically known to the compiler, so it is returned on the stack). Each time the closure is called, it mutates its internal `i` state directly in its struct memory and allocates a new `String` on the heap to return.

12.2 Iterators: The Core Concept

Most algorithms involve looping over data. The traditional `for` or `while` loops with index increments work well for simple arrays, but they expose the internal structure of the collection.

What if we want to iterate over a tree, a linked list, or a hash map?

The **Iterator** pattern solves this. An iterator is an object that provides a uniform interface to traverse a sequence of elements, regardless of how those elements are stored or generated. This explicit representation of operations over a sequence also lends itself exceptionally well to performance optimizations. For example, using dedicated libraries like **rayon**, it is trivial to transform a sequential iterator pipeline into a parallel one that exploits multi-core processors.

12.2.1 The Iterator Trait

In Rust, the Iterator mechanism is shockingly simple. While many other languages separate the check for more elements (`hasNext()`) and the retrieval (`next()`), Rust combines both into a single `next` method.

```
pub trait Iterator {  
    type Item; // The type of element being iterated  
  
    // Returns 'Some(Item)' if there is an element, or 'None' if finished  
    fn next(&mut self) -> Option<Self::Item>;  
}
```

When `next()` returns `Some(value)`, you have a value. When it returns `None`, the iterator is exhausted. Calling `next()` again after `None` will typically just return `None` forever.

12.2.2 Iteration Modes

Collections typically provide three ways to create an iterator, corresponding to the three modes of ownership:

1. `iter()`: Returns an iterator over shared references (`&T`). The original collection remains intact and immutable.
2. `iter_mut()`: Returns an iterator over mutable references (`&mut T`). Useful for modifying elements in place.
3. `into_iter()`: Returns an iterator that takes ownership of the collection, yielding owned values (`T`). The original collection is consumed and destroyed.

Note

Note that not all collections support all three modes identically. For instance, `HashMap` and `BTreeMap` do not provide a way to iterate *mutably* over their keys. Mutating a key in-place would break the underlying data structure's invariants.

! Exam Priority

Common Pitfall: Using a `for` loop directly on a collection (`for x in v`) when you meant to only read it (`for x in &v`), accidentally consuming and destroying the container. **Expected Mastery Level:** High. You must instinctively know whether an iteration yields `T`, `&T`, or `&mut T`.

```
let mut v = vec![String::from("A"), String::from("B")];

// 1. Immutable Iteration
for s in v.iter() {
    println!("{}", s); // s is &String
}

// 2. Mutable Iteration
for s in v.iter_mut() {
    s.push_str("1"); // s is &mut String
}

// 3. Consuming Iteration
for s in v.into_iter() {
    println!("{}", s); // s is String (moved)
}
// 'v' is no longer valid here!
```

Deep Memory Model Analysis:

- `v.iter()`: Creates an iterator struct on the stack containing a pointer to the vector's heap buffer and an index. It yields `&String` references. No heap data is duplicated or dropped.
- `v.iter_mut()`: Similar to `iter()`, but yields `&mut String` references. It relies on unsafe code under the hood to split the mutable borrow safely, ensuring no two mutable references point to the same element.
- `v.into_iter()`: The vector is "disassembled". The iterator takes ownership of the vector's heap buffer. As it iterates, it yields the actual `String` structs (fat pointers) to the caller. Once iteration finishes, the iterator drops the remaining un-yielded elements and deallocates the vector's heap buffer itself.

Note

The for Loop Sugar: Under the hood, Rust's `for` loop is syntactic sugar. It calls `IntoIterator::into_iter()` on the collection to get an iterator, and then repeatedly calls `next()` until it hits `None`.

12.3 The Iterator Pipeline: Adapters and Consumers

The true power of iterators in Rust comes from combinability. The `Iterator` trait has over 50 default methods (adapters and consumers) that allow you to chain operations elegantly.

12.3.1 Lazy Evaluation

Iterators in Rust are **lazy**. This means that creating an iterator and chaining adapters does absolutely nothing until a *consumer* is called.

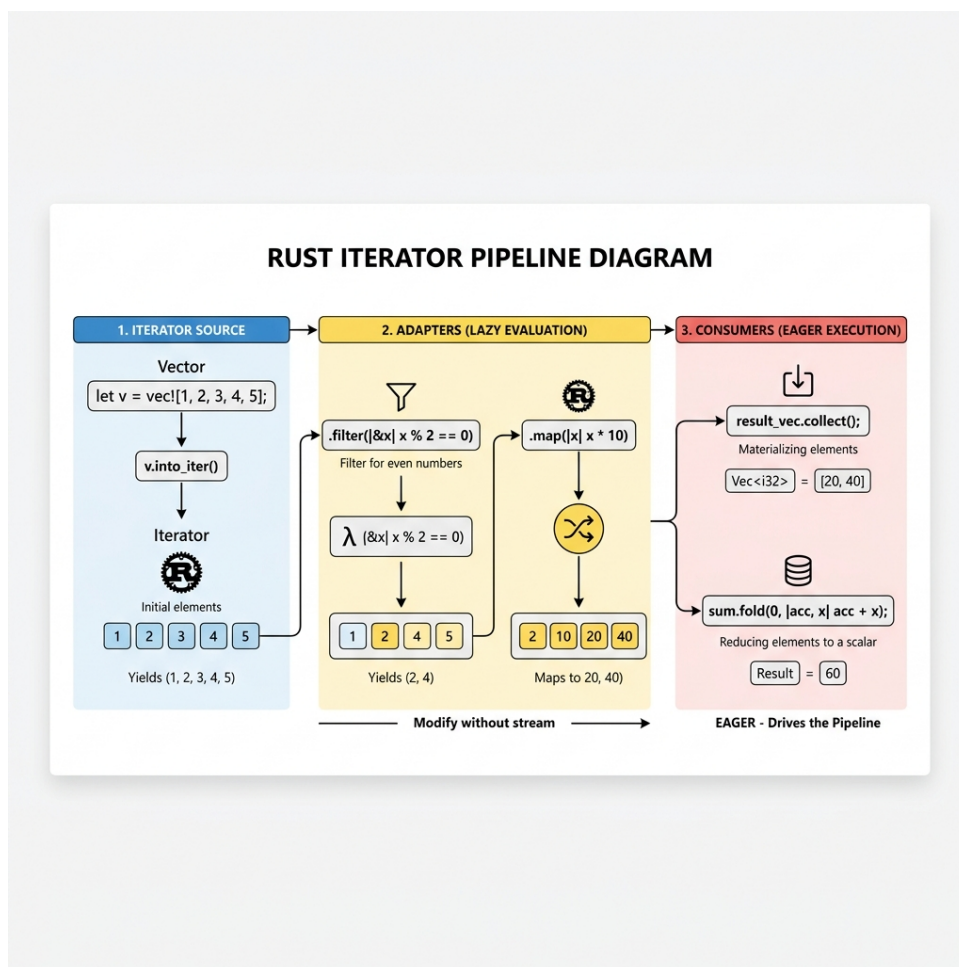


Figure 12.1: The Iterator Pipeline: Adapters chain lazily, and Consumers drive the execution.

Insight

Professor's Analogy: Ordering a Custom Car

Iterators are entirely lazy. They simply prepare the pipeline but perform no work until requested. It's like buying a custom car: you place the order and the manufacturer says, "It will arrive in six months." Why? Because car manufacturers learned it's not cost-effective to pre-build everything and risk it sitting unsold. It's much better to wait for someone to actually request it, provide the money, and *only then* start the production process. Similarly, iterators avoid spending CPU cycles on elements you might never actually use!

! Exam Priority

Common Pitfall: Creating complex iterator adapters but forgetting to call a consumer, meaning the iteration code never actually executes. **Expected Mastery Level:** High. Understand how the request for an element propagates strictly from the consumer back up to the source.

Think of an iterator pipeline as a plumbing system. You can connect hoses and valves (adapters like `map` and `filter`) all you want, but no water will flow. It's only when you open the tap at the very end of the line (a consumer like `collect`) that the system activates. The consumer asks the last adapter for an element, which in turn asks the previous one, propagating the request all the way up to the source container. Elements are processed strictly on-demand. This zero-cost abstraction ensures that we don't perform unnecessary work or allocate intermediate collections in memory.

12.3.2 Adapters

Adapters are methods that take an iterator and return a *new* iterator.

- `map(|x| ...)`: Applies a closure to every element, transforming it.
- `filter(|x| ...)`: Keeps only elements for which the closure returns `true`.
- `enumerate()`: Yields `(index, value)` pairs.
- `take(n)`: Yields the first n elements, then stops.
- `take_while(|x| ...)`: Continues yielding elements as long as the condition returns `true`, then stops.
- `skip(n)`: Ignores the first n elements, then yields the rest.
- `skip_while(|x| ...)`: Ignores elements as long as the condition is `true`, then yields all remaining elements.
- `chain(other)`: Appends another iterator to the end of this one.
- `zip(other)`: Pairs up elements from two iterators into tuples.
- `rev()`: Reverses the direction of iteration (requires the underlying iterator to support moving backwards).
- `flatten()`: Removes one level of nesting (e.g., turns an iterator of lists into an iterator of elements).
- `flat_map(|x| ...)`: Applies a function that returns an iterator, and then flattens the result.
- `filter_map(|x| ...)`: A concise combination of `filter` and `map` that yields elements for which the closure returns `Some`.
- `inspect(|x| ...)`: Allows you to examine the element (e.g., for `println!` debugging) without modifying or consuming it.
- `fuse()`: Creates an iterator that will definitively yield `None` forever after yielding `None` once, ensuring predictable termination.
- `cloned()` / `copied()`: Creates an iterator that clones or copies the elements, turning an iterator of references into owned values.

12.3.3 Consumers

Consumers are methods that call `next()` on the iterator pipeline and produce a final value.

- `collect()`: Gathers all elements into a collection (e.g., a `Vec` or `HashMap`).
- `sum()`, `product()`, `count()`, `min()`, `max()`: Perform common aggregations.
- `fold(init, |acc, x| ...)`: The mother of all consumers, applying an accumulator.
- `try_fold(init, |acc, x| ...)`: A variant of `fold` that halts immediately if an error (`Err` or `None`) is encountered.
- `reduce(|acc, x| ...)`: Similar to `fold`, but uses the first element of the sequence as the initial value, returning an `Option`.
- `partition(|x| ...)`: Splits the elements into a tuple of two collections: those that satisfy the predicate and those that don't.

```
let numbers = vec![1, 2, 3, 4, 5, 6];

// The adapters do nothing until collect() is called
let even_squares: Vec<i32> = numbers.into_iter()
    .filter(|&x| x % 2 == 0) // keeps 2, 4, 6
    .map(|x| x * x)         // turns to 4, 16, 36
    .collect();            // drives the pipeline
```

Warning

Because iterators are lazy, if you write a map operation that produces side effects (like printing to the console) but you never call a consumer at the end, the code will compile but *nothing will happen*. The compiler will issue a "must use" warning.

12.3.4 Understanding fold

! Exam Priority

Common Pitfall: Trying to use mutable external variables to accumulate a sum or product in an iterator pipeline instead of using `fold`. **Expected Mastery Level:** Very High. The professor explicitly calls this “the basis of computation” and the most powerful method of all iterators.

The `fold` method is the fundamental aggregation operation. It’s conceptually similar to the accumulator register in a CPU. It starts with an initial value, and applies a function to combine the accumulator with every element from the iterator. All other consumers could theoretically be implemented using `fold`.

```
let v = vec![1, 2, 3];
// sum starts at 0.
// step 1: acc(0) + x(1) -> 1
// step 2: acc(1) + x(2) -> 3
// step 3: acc(3) + x(3) -> 6
let total = v.iter().fold(0, |acc, &x| acc + x);
println!("Total:␣{}", total); // 6

// reduce uses the first element (1) as the initial accumulator
let total_reduce = v.iter().copied().reduce(|acc, x| acc + x);
println!("Total␣(reduce):␣{:?}", total_reduce); // Some(6)
```

Insight

Professor’s Insight: Fold variants

While `fold` requires an explicit initial value, `reduce` simply uses the first element of the sequence as the initial value. This is why `reduce` returns an `Option` (in case the iterator is empty). Another powerful variant is `try_fold`, which works just like `fold` but halts the accumulation immediately if the closure returns an `Err` or `None`, acting as a short-circuiting aggregator.

12.4 Implementing Custom Iterators

You don’t always have to rely on standard collections. Let’s create a custom iterator that splits a string into words by looking for spaces.

First, we define our collection and an auxiliary struct to hold the iterator state.

```
struct Words<'a> {
    text: ␣&'a str,
}

struct WordsIter<'a> {
    text: ␣&'a str,
    pos: usize,
}
```

Next, we implement `Iterator` for `WordsIter`.

```
impl<'a> Iterator␣for␣WordsIter<'a> {
    type Item = ␣&'a str;␣//␣Notice:␣returning␣string␣slices,␣no␣allocations!

    fn next(&mut␣self)␣->␣Option<Self::Item>␣{
```

```

        let bytes = self.text.as_bytes();
        let len = self.text.len();

        // Skip leading spaces
        while self.pos < len && bytes[self.pos] == b' ' {
            self.pos += 1;
        }

        if self.pos >= len {
            return None; // Reached the end
        }

        let start = self.pos;

        // Find the end of the word
        while self.pos < len && bytes[self.pos] != b' ' {
            self.pos += 1;
        }

        // Return a slice of the string
        Some(&self.text[start..self.pos])
    }
}

```

Finally, we hook it up to the `IntoIterator` trait, allowing our `Words` struct to be used directly in `for` loops!

```

impl<'a> IntoIterator for &'a Words<'a> {
    type Item = &'a str;
    type IntoIter = WordsIter<'a>;

    fn into_iter(self) -> Self::IntoIter {
        WordsIter {
            text: self.text,
            pos: 0,
        }
    }
}

// Usage:
let my_words = Words { text: "hello rust world" };
for word in &my_words {
    println!("{}", word);
}

```

Exam Relevance and Recurrence

Score: Very High

Notes: Closures and Iterators form the backbone of functional-style programming in Rust and are heavily tested. You will frequently be asked to write code that chains iterator adapters, to explain why a closure fails to compile due to lifetime/borrowing rules, or to choose between generic and dynamic dispatch for closures. Expect questions asking you to build stateful generators or parse data streams using custom iterators. Be exceptionally comfortable with `fold`, `filter_map`, and `into_iter`.

12.5 Domande ed Esercizi da Esami Passati

Exam Focus

Domanda 1:

Dato il seguente programma si motivi l'errore di compilazione e si corregga il programma in modo da stampare il valore aggiornato della variabile `count` dopo ogni incremento.

```
1 fn main() {  
2 let mut count = 0;  
3  
4 let mut inc
```

(Nota: il codice del testo originale d'esame è troncato, ma l'intento è illustrare l'uso errato di un borrow mutabile in concomitanza con un borrow immutabile esterno).

Soluzione e Spiegazione Teorica:

L'errore di compilazione deriva dalle regole di borrowing di Rust. Il frammento sottintende la creazione di una closure (ad esempio `let mut inc = || count += 1;`) che modifica la variabile catturata `count`. Come spiegato in questo capitolo, quando una closure modifica una variabile del suo ambiente, il compilatore genera una struct anonima che la cattura implicitamente tramite un **referimento mutabile** (`&mut count`).

Finché la closure continua a essere utilizzata successivamente nel programma, il borrow mutabile su `count` rimane attivo per tutta la sua estensione. Questo impedisce rigorosamente di prendere in prestito la variabile in modo immutabile dall'esterno (ad esempio usandola in un `println!`) per stamparne il valore. Un prestito mutabile esclusivo non può infatti coesistere con altri prestiti attivi.

Per risolvere il conflitto e soddisfare la richiesta di stampare `count` a ogni incremento, la soluzione più corretta è quella di inserire l'istruzione di stampa **direttamente all'interno del corpo della closure**:

```
fn main() {  
    let mut count = 0;  
  
    // La closure mantiene il riferimento mutabile e  
    // gestisce interamente l'accesso ai dati catturati.  
    let mut inc = || {  
        count += 1;  
        println!("{}", count);  
    };  
  
    inc(); // Stampa 1  
    inc(); // Stampa 2  
}
```

Exam Focus**Domanda 2:**

Spiegare il comportamento del seguente programma.

```
1 fn call_x(mut f: impl FnMut(), x:i32 ) {  
2   for _ in 0..x {  
3     f();  
4   }  
5 }
```

Soluzione e Spiegazione Teorica:

Questo frammento di codice illustra due concetti chiave trattati nel capitolo: l'impiego dei tratti delle closure e il **dispatch statico** (Generics).

- **Il tratto `FnMut`:** La funzione accetta un parametro `f` vincolato dal tratto funzionale `FnMut`. Ciò dichiara esplicitamente che la closure passata è autorizzata a **modificare il proprio ambiente catturato** (il proprio stato interno). Essendo in grado di mutare se stessa, la funzione deve vincolare la reference a `f` stessa come mutabile (`mut f`).
- **Generics e Dispatch Statico (`impl Trait`):** Utilizzare la sintassi `impl FnMut()` equivale a dichiarare un parametro generico. La risoluzione della chiamata avverrà a tempo di compilazione tramite *monomorfizzazione*: il compilatore genererà una versione altamente ottimizzata di `call_x` per ogni tipo unico di closure che le viene passato.
- **Flusso di Esecuzione:** Nel corpo della funzione, l'iterazione si ripete `x` volte (tramite il range `0..x`). A ogni iterazione, viene invocata la closure `f()`. Poiché la closure implementa `FnMut`, se l'ambiente "catturato" include uno stato o dei contatori, questi potranno variare stabilmente a ogni step del ciclo.

Chapter 13

Asynchronous Programming

13.1 Introduction: The Waiting Problem

When developing system-level applications, we frequently encounter situations where our program must perform multiple operations concurrently. A common instinct is to reach for threads. However, before deciding on a concurrency model, we must fundamentally categorize the nature of the work our program is doing: is it **CPU-bound** or **I/O-bound**?

A **CPU-bound** task is one where the bottleneck is the processor. Examples include training AI models, performing complex matrix multiplications, or encrypting large streams of data. In these scenarios, computation can be cleanly divided into smaller chunks, and assigning each chunk to a dedicated OS thread allows us to leverage multi-core processors. The goal here is *parallelism*: decreasing total execution time by doing mathematical work simultaneously across multiple cores.

Insight

The Professor's Matrix Analogy

Imagine an AI system multiplying two massive matrices (e.g., 10 million rows by 10 million columns). We cannot compute this sequentially. Instead, we assign the multiplication of specific rows and columns to different threads. If we use 10 threads across multiple cores, each thread handles a chunk of the problem. Because the math is purely CPU-bound, the total execution time is drastically reduced (roughly approaching $1/N$).

Conversely, an **I/O-bound** task spends the vast majority of its time waiting for an external subsystem to complete an operation. Imagine a program performing a deep file system search (like Windows Explorer searching the entire disk). The program must read chunks of bytes and compare strings. However, the time spent comparing strings is a marginal fraction of the time spent waiting for the disk to retrieve blocks and cross the OS barrier. If measured finely, the thread spends 99.9% of its time parked, doing nothing.

Insight**The Threading Paradox in the Cloud**

If we use a dedicated OS thread for every I/O-bound task, we encounter a severe inefficiency. Consider a web server like Amazon. It receives an HTTP request, decodes it, and queries a 60-year-old database. While waiting for the database, the thread sits idle. However, each thread carries a fixed cost. If you deploy your own server to the cloud, you pay for memory. If your server receives 10,000 concurrent requests and creates 10,000 threads, it consumes 10-20 Gigabytes of RAM (due to the 1-2MB stack per thread) just to *wait*. We are paying for resources that are effectively doing nothing 99.9% of the time. Benchmark data (e.g., Jim Blandy's experiments) highlights this disparity:

- **Creation Time:** $\sim 0.3 \mu\text{s}$ for an async task vs $\sim 17 \mu\text{s}$ for a thread (50x slower).
- **Context Switch:** $\sim 0.2 \mu\text{s}$ for a task vs $\sim 1.7 \mu\text{s}$ for a thread (8.5x slower, due to system calls).
- **Memory:** 300-500 bytes for a task vs $\sim 9.5 \text{ KB}$ of kernel data + 1-8MB stack for a thread.

For I/O-bound tasks, we do not need more calculation speed; we need the capacity to efficiently manage thousands of concurrent *waiting periods*. This is where **asynchronous programming** shines. We use threads to compute in parallel, but we use asynchronous execution to *wait* in parallel.

13.2 Strategies for Handling Concurrency

To intuitively understand the difference between parallelism and concurrency, consider an analogy from a cafeteria: multi-threading is like having multiple cooks in the kitchen simultaneously preparing meals (parallel execution of CPU-bound tasks). Asynchronous concurrency is like the staff serving pasta. You do not need a dedicated server for each student in the line. A single server can rapidly hand out portions, interleaving their actions. While one student grabs their plate, the server is already serving the next. This overlap of waiting times is the essence of concurrent coordination.

Let us evaluate the three primary strategies for handling multiple I/O-bound operations (e.g., reading three files, or serving three HTTP requests):

1. **Sequential Execution:** We perform Activity 1, wait for it to finish, then Activity 2, and so on. The total time is the sum of all individual times. It is easy to implement but provides unacceptable performance for the user.
2. **Multi-threading (Preemptive):** We spawn a thread for each activity. The OS preemptive scheduler manages them. The code remains simple, but as discussed, the memory and context-switching overhead scale terribly with the number of tasks.
3. **Asynchronous (Cooperative):** A single thread initiates Activity 1, tells the OS "let me know when the data is ready," and immediately moves on to initiate Activity 2. The thread coordinates thousands of tasks by cooperative switching: when a task cannot proceed without waiting, it explicitly yields control back to the application, allowing the thread to work on something else.

The asynchronous approach is essentially a cooperative, application-level scheduling system. But how do we express this in code?

13.3 From Callbacks to State Machines

Historically, asynchronous behavior was implemented using **callbacks** or continuations (similar to `.then()` in JavaScript). You would instruct a system: “Start reading this file, and when you finish, execute this closure.”

While this avoids blocking the thread, callbacks suffer from severe pathologies:

- **Wiring Complexity & Error Handling:** Suppose you want to copy a file. You invoke `read_async` and pass a continuation to invoke `write_async`. However, `write` can fail (e.g., disk full, file deleted). The `read` might also fail before starting. What should be a trivial sequence quickly explodes into 4 or 5 complex error-handling branches. Propagating these errors across deeply nested closures is notoriously difficult.
- **Control Flow limits:** Implementing a simple `while` loop that performs asynchronous reads inside is nearly impossible to represent statically with simple callbacks. You cannot statically develop the combination tree for an arbitrary number of loops. The only solution is to build a complex **state machine**.

13.3.1 The `async/await` Transformation

Rust solves this using the `async` and `await` keywords. Instead of writing nested callbacks, we write code that looks entirely synchronous and sequential.

```
async fn copy_file(src: &str, dst: &str) -> Result<(), std::io::Error> {
    let data = read_async(src).await?;
    write_async(dst, &data).await?;
    Ok(())
}
```

When the compiler encounters an `async fn`, it does something magical: it does not execute the function block sequentially. Instead, it transforms the entire function body into a **State Machine**.

Every `.await` point represents a state transition—a point where the function can be paused and later resumed. The compiler achieves this by synthesizing a state machine structured as an `enum`, where each variant stores the local variables needed for that specific state:

```
enum CopySM {
    State0 { src: String, dst: String },
    State1 { dst: String, data: Vec<u8>, read_future: ... },
    State2 { write_future: ... },
    Done
}
```

1. **State 0:** Start executing until `read_async(src).await`. The state machine saves `dst` and transitions to State 1. If the read is not ready, it yields control. 2. **State 1:** Resume once the read completes, extracting the `data`. Proceed until `write_async(dst, &data).await`, saving necessary futures and transitioning to State 2. 3. **State 2:** Resume once the write completes, transitioning to `Done` and returning the final result.

Note

Zero-Cost Abstractions

The compiler generates this state machine for you. It handles all the complex routing, variable saving (across `await` points), and looping. You get the readability of synchronous code with the performance of asynchronous execution.

13.4 Formalizing the Mechanism: The Future Trait

Because `async` functions are compiled into state machines, they do not immediately return their final value. Instead, calling an `async fn` returns an anonymous object that implements the `Future` trait.

! Exam Priority

Futures are Inert (Lazy Evaluation)

Common Pitfall: Calling an `async fn` and discarding the returned `Future` without `.awaiting` it.

Expected Mastery Level: High.

The professor explicitly contrasts Rust with JavaScript. In JS, calling an `async` function (like `fetch`) immediately places the continuation onto the browser's single global **Message Queue**. The runtime automatically executes it. In Rust, an `async` function simply returns a state machine object (a `Future`). It does *absolutely nothing* until it is actively polled by the executor (typically initiated by `.await`). If you forget the `.await`, the operation is silently never executed.

Let's examine the core of asynchronous Rust: the `Future` trait.

```
use std::task::{Context, Poll};
use std::pin::Pin;

pub trait Future {
    type Output;

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>)->Poll<Self::Output>;
}
```

The most critical part of a `Future` is its `poll` method. When a `Future` is polled, it attempts to make progress on its internal state machine. It returns an enum:

- `Poll::Ready(Output)`: The asynchronous operation has fully completed, and the final value is returned.
- `Poll::Pending`: The operation cannot proceed right now (e.g., still waiting for network data). The `Future` yields control back to the caller.

13.4.1 Understanding Pin and Context

You will notice two unusual types in the `poll` signature: `Pin` and `Context`.

- `Pin<&mut Self>`: Because the compiler transforms our `async` block into a state machine, local variables that are held across `.await` points must be stored inside this generated struct. Sometimes, these variables might contain references to other variables within the *same* struct (self-referential structs). If the `Future` were moved in memory, these pointers would instantly become invalid. `Pin` is a wrapper that guarantees the object will never be moved in memory, ensuring memory safety for the state machine.
- `Context<'_>`: When a `Future` returns `Poll::Pending`, how does the system know when to try polling it again? It would be terribly inefficient to constantly poll a waiting `Future` in a busy loop. The `Context` provides access to a **Waker**. The `Future` registers this **Waker** with the operating system's event listener. When the OS event occurs (e.g., data arrives on a socket), the **Waker** is triggered, signaling that this specific `Future` is ready to be polled again.

13.4.2 Ownership Across Time

In synchronous code, ownership is tied to the current call stack. In asynchronous code, however, a computation can suspend at an `.await` point and resume much later. Local variables that are used after the `.await` become part of the Future's internal state (as seen in our `enum` example).

If the chosen Executor is multi-threaded (like Tokio's default worker pool), it might employ *work stealing*. A Future could suspend on Thread A, but later be woken up and resumed by Thread B. Consequently, ownership in async Rust crosses not just space (threads), but time. This imposes the constraint that any data held across an `.await` point must implement the `Send` trait to migrate between threads safely.

13.5 Executors and the Runtime

Rust provides the language syntax (`async/await`) and the standard interface (`Future` trait). However, **Rust does not provide an async runtime or a built-in message queue.**

If a Future only advances when `poll` is called, who calls `poll`? This is the job of the **Executor**.

The Executor is an application-level, cooperative scheduler. It maintains a queue of all active Futures (often called *tasks*). It picks a task and calls `poll`. If it returns `Pending`, the Executor parks the task and moves on to the next one.

Insight

Runtimes in the Ecosystem

Because Rust leaves the runtime choice to the programmer, different domains have different standard runtimes:

- **Tokio:** The heavy-duty standard for server applications. It utilizes multiple worker threads, each with its own task queue. If a worker finishes its queue, it utilizes *work stealing* to take tasks from a neighbor's queue.
- **smol:** An extremely compact, single-threaded runtime. It has a tiny memory footprint, making it ideal for Embedded Systems where resources are tightly constrained.
- **async-std:** An older attempt to provide 1-to-1 asynchronous counterparts to the standard library, though it is less dominant today.

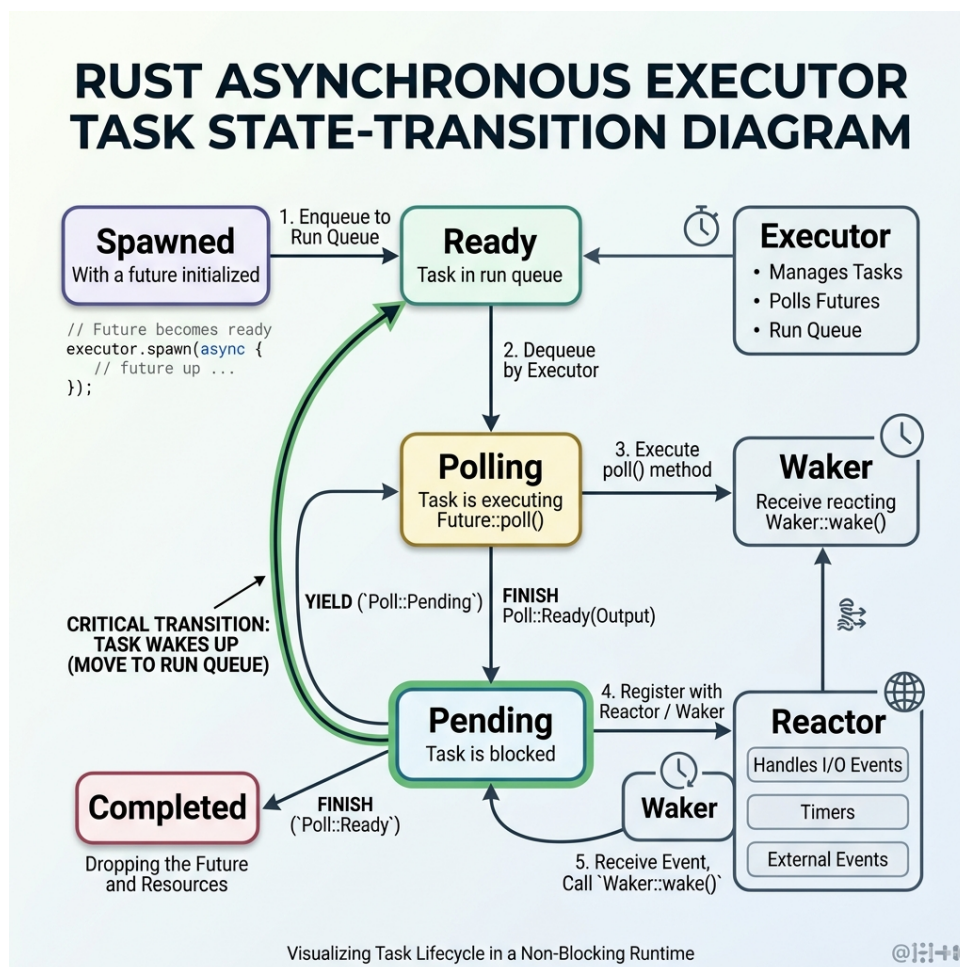


Figure 13.1: The Architecture of an Asynchronous Runtime

As seen in Figure 13.1, a typical runtime consists of two parts:

1. **The Executor:** Polls tasks. If a task is pending, it yields the thread back to the Executor.
2. **The Reactor:** Interfaces with the Operating System (e.g., `epoll` on Linux, `kqueue` on macOS). When an OS I/O event completes, the Reactor uses the **Waker** to move the parked task back into the Executor's ready queue.

13.6 Working with Tokio

Because the standard library does not include a runtime, the Rust ecosystem relies on external crates. The de-facto standard runtime is **Tokio**. It provides the **Executor**, the **Reactor**, and asynchronous versions of standard library components (like `tokio::fs` and `tokio::net`).

To start an asynchronous program, we must initialize the runtime. Tokio provides a convenient macro to transform our standard `main` function:

```
#[tokio::main]
async fn main() {
    println!("Hello from the async world!");

    let result = fetch_data().await;
    println!("Data: {}", result);
}
```

```

async fn fetch_data() -> String {
    // Simulating an asynchronous wait
    tokio::time::sleep(std::time::Duration::from_secs(1)).await;
    "Database_payload".to_string()
}

```

Under the hood, `#[tokio::main]` expands into a synchronous `main` function that builds the Tokio runtime and blocks the main thread while the Executor runs the `Future` generated by our `async fn main`.

13.6.1 Concurrent Task Spawning

To achieve true concurrency, we must spawn multiple independent tasks onto the Executor. This is analogous to spawning an OS thread, but we are spawning a lightweight *green thread* managed by Tokio.

```

use tokio::task;

#[tokio::main]
async fn main() {
    let data = String::from("Shared_payload");

    // Spawn a new independent asynchronous task
    let handle1 = tokio::spawn(async move {
        process_request(1, data).await;
    });

    let handle2 = tokio::spawn(async {
        process_request(2, String::from("Other")).await;
    });

    // We can await the JoinHandles to ensure they complete
    let _ = handle1.await;
    let _ = handle2.await;
}

async fn process_request(id: u32, payload: String) {
    println!("Processing_request_{}with_payload:{}", id, payload);
    tokio::time::sleep(tokio::time::Duration::from_millis(500)).await;
    println!("Finished_request_{}", id);
}

```

Memory Model Analysis

Memory and Ownership in `tokio::spawn`: When you call `tokio::spawn`, Tokio dynamically allocates the task's state machine (the compiled `Future`) on the **heap**. This heap allocation is necessary because the task's lifetime is now decoupled from the caller's stack—the caller could return, but the background task might still be running. Furthermore, the `async move` block forces a transfer of ownership of any captured variables (like `data`) into the `Future`'s state machine. Because Tokio utilizes a multi-threaded work-stealing executor by default, this `Future` might be polled by Thread A, paused at an `.await`, and later resumed by Thread B. Consequently, every value inside the `Future`'s state machine must implement the `Send` trait. Finally, since Tokio cannot prove how long the background task will live, any captured references would be unsafe; thus, the spawned `Future` is constrained by the `'static` lifetime (meaning it cannot contain borrowed references to short-lived stack variables).

Managing Homogeneous Tasks: JoinSet

If you need to spawn many tasks that all return the same type of result, managing dozens of `JoinHandles` manually is tedious. Tokio provides the `JoinSet` structure. You can call `set.spawn()` repeatedly to add tasks to the set, and then loop over `set.join_next().await` to process the results as they complete, in whatever order they finish.

! Exam Priority

Blocking the Cooperative Executor

Common Pitfall: Running heavy, synchronous CPU-bound work (like complex math, gzip compression, or blocking I/O such as `std::fs::read`) directly inside an `async` task.

Expected Mastery Level: Very High.

The professor repeatedly emphasizes that Tokio's scheduler is *cooperative*. An `async` task must yield control by reaching an `.await` point. If a task spends hundreds of milliseconds calculating without awaiting (e.g., compressing 100,000 rows of data with gzip), it hogs the underlying worker thread. The executor cannot preempt it, meaning all other parked tasks on that thread are starved.

For CPU-heavy work within an `async` application, you **MUST** explicitly offload it using `tokio::task::spawn_blocking`. This function takes a standard closure (not an `async` block) and executes it on a dedicated, separate thread pool specifically created for blocking operations, leaving the core asynchronous worker threads free to continue polling I/O tasks.

13.6.2 Coordinating Tasks: `join!` and `select!`

Often, we must coordinate multiple asynchronous tasks rather than just awaiting them in sequence. The golden rule is: **use sequential `.awaits` ONLY when a task explicitly requires the result of the previous task.**

```
// Sequential execution: We MUST wait for user before fetching orders
let user = fetch_user(42).await;
let orders = fetch_orders(&user).await;
println!("User has {} orders", orders.len());
```

If tasks are independent, awaiting them sequentially wastes time. Tokio provides powerful macros to coordinate independent tasks efficiently:

- `tokio::join!`: Executes multiple futures concurrently on the same task. It waits for *all* of them to complete and returns a tuple of their results. It is ideal when you need to fetch from multiple independent endpoints at once. Total execution time equals the longest of the tasks.
- `tokio::try_join!`: Similar to `join!`, but if any future returns an `Err`, it aborts the others and returns the error immediately. This embraces the "all or nothing" logic.
- `tokio::select!`: Runs multiple futures simultaneously but only waits for the *first* one to complete. The remaining futures are cancelled (dropped). It looks syntactically similar to a `match` statement.

Practical Usage Guidelines

When and why to use coordination macros:

- **join! / try_join!:** Use when tasks are *independent* (e.g., fetching user profile and user settings) but you need *all* results before proceeding. It interleaves them without spawning new worker tasks.
- **select!:** Use to handle timeouts (e.g., race a network request against a `tokio::time::sleep`) or to process whichever event arrives first on multiple channels. Canceling the slow tasks automatically cleans up resources.

13.7 Sharing State and Communicating

When multiple spawned tasks need to interact, they face the same challenges as traditional threads. Tokio offers async-aware synchronization primitives that yield control instead of blocking the underlying OS thread.

13.7.1 Shared State: `Arc<tokio::sync::Mutex<T>>`

If tasks must mutate shared data, wrap it in Tokio's asynchronous `Mutex` paired with an `Arc`. Unlike `std::sync::Mutex`, calling `lock().await` suspends the task rather than blocking the thread, allowing the worker to execute other tasks while waiting. However, sharing memory can still create bottlenecks or lead to deadlocks.

13.7.2 Message Passing: Channels

Often, it is safer and more scalable to share data by communicating (message passing) rather than sharing memory with locks. Tokio provides several asynchronous channels:

- **oneshot (1-to-1):** Used to send a single value. It is not repeatable. Ideal for an asynchronous task that needs to communicate a specific result to its spawner.
- **mpsc (Multi-Producer, Single-Consumer):** Used for pipelines or streaming data. Bounded `mpsc` channels provide *backpressure*. For example, if a channel has a capacity of 10, the 11th `send().await` will automatically suspend the sender until the consumer processes a message.
- **broadcast (1-to-N):** One producer sends messages to many consumers. Every consumer receives a copy of *all* messages.
- **watch (1-to-N):** Observes a variable and wakes up consumers when the data changes. Crucially, consumers only see the *latest* value. Useful for reacting to configuration updates.

Practical Usage Guidelines

When and why to use different channels:

- **oneshot:** When a spawned background task needs to return exactly one result to its spawner. It has zero overhead for queues.
- **mpsc:** To implement pipelines or worker queues. Crucially, bounded **mpsc** channels provide *backpressure*—if the receiver is overwhelmed, senders automatically **.await** (suspend) until buffer space frees up, preventing out-of-memory crashes.
- **broadcast:** For system-wide event notifications (e.g., "Shutdown initiated") where many independent tasks need to hear the exact same message.
- **watch:** For sharing frequently updated state (like configuration settings) where receivers only care about the *latest* value and can skip intermediate ones.

Exam Relevance and Recurrence

Score: Very High.

Notes: Asynchronous programming and Tokio are heavily tested both conceptually and practically. You must be able to explain the difference between CPU-bound and I/O-bound tasks, and why threads are inefficient for I/O. Expect questions asking you to mentally translate an **async fn** into its corresponding **enum** state machine. You will frequently be required to identify bugs where CPU-heavy code blocks the executor, or where a **Future** is dropped without being **.awaited**.

13.8 Domande ed Esercizi da Esami Passati

Exam Focus

Domanda: Si descriva il concetto di risultato differito, come esposto dalla classe C++11 `std::future<T>`, indicando tre modi alternativi offerti dalla libreria standard per creare un oggetto di tale tipo e illustrando, per ciascuno di essi, un possibile caso d'uso che ne evidenzia l'utilità.

Risposta: *Nota: Il materiale di questo capitolo affronta il concetto di risultato differito specificamente per il linguaggio Rust, dove esso viene modellato dal trait **Future**, invece che dalla classe `std::future<T>` del C++11. Di seguito viene presentata la teoria in base ai contenuti trattati nel capitolo.*

Il concetto di risultato differito esprime un valore che non è immediatamente disponibile al momento della chiamata della funzione. Nel modello asincrono illustrato, quando si invoca una funzione **async**, essa non restituisce direttamente il risultato, ma ritorna un oggetto anonimo che implementa il trait **Future**. Questo costrutto si distingue per la sua "lazy evaluation" (valutazione pigra): il **Future** è inerte e non compie alcuna operazione se non viene attivamente interrogato (*polled*) da un **Executor**, tipicamente attraverso l'uso della keyword **.await**. Sotto il cofano, il **Future** viene trasformato dal compilatore in una macchina a stati (spesso sotto forma di **enum**), permettendo l'esecuzione cooperativa senza bloccare il thread sottostante, una caratteristica essenziale per gestire efficientemente operazioni di I/O concorrenti.

Chapter 14

Asynchronous Concurrency

In the previous chapters, we explored how traditional multithreading allows us to perform multiple tasks simultaneously. Multithreading is highly effective for CPU-bound workloads, where calculations can be distributed across multiple physical cores to reduce total execution time. However, when dealing with I/O-bound workloads—such as web servers managing thousands of concurrent requests, or systems performing extensive file system searches—spawning a dedicated thread for every operation quickly becomes inefficient.

The professor emphasizes a fundamental distinction: *concurrency* is about dealing with multiple tasks overlapping in time, while *parallelism* is about executing multiple tasks exactly at the same time. Asynchronous programming in Rust is a powerful tool to achieve massive concurrency, allowing a single thread to manage thousands of I/O operations without blocking.

In this chapter, we will introduce asynchronous concurrency in Rust. We will explore how it fundamentally differs from thread-based concurrency, how Rust implements it under the hood using state machines and the `Future` trait, and how to build robust concurrent applications using the Tokio runtime, tasks, asynchronous mutexes, and channels.

14.1 The Limits of Standard Threads

When a program performs an I/O operation, such as reading from a disk or a network socket, the operating system blocks the requesting thread until the data is ready. If we have thousands of concurrent requests, a traditional multi-threaded approach would require spawning thousands of threads.

This approach has severe drawbacks:

- **Memory Overhead:** Every standard thread requires a pre-allocated stack (often 1-2 MB). As the professor noted, a typical enterprise system might start with 500 threads, immediately consuming 500 MB of RAM just for idle stacks! Having 10,000 threads means allocating gigabytes of memory just for the stacks, even if the threads spend 99.9% of their time waiting for data.
- **Context Switch Cost:** The OS preemptive scheduler must continuously switch context between threads. Context switching involves saving registers, invalidating caches, and updating the Translation Lookaside Buffer (TLB), taking microseconds per switch. With thousands of threads, these overheads sum up to significant delays.

Insight

[The True Cost of I/O: The Heartbeat Analogy and the Bakery] To understand why waiting for I/O in standard threads is so devastating to performance, the professor provides a vivid analogy mapping CPU clock cycles to human time. If a single CPU cycle is equivalent to one human heartbeat (about 1 second):

- Accessing L1 cache takes 1-2 heartbeats.
- Accessing L2/L3 cache takes around 15 heartbeats.
- Accessing main memory (RAM) takes about 4 minutes.
- Reading from a generic SSD takes from 1.5 to 4 days.
- Sending a network packet across the internet is equivalent to waiting 5 years!

When a thread blocks on network I/O, the CPU is essentially forced to wait "years" in relative time. To mitigate these memory delays, modern CPUs rely heavily on caches. The professor compares this to a *Panetteria* (bakery): a baker doesn't buy ingredients for one sandwich at a time. They fetch in bulk (like the CPU fetching a whole cache line) because it's statistically probable that adjacent data will be needed soon (spatial locality). By using small, fast stores (L1) fed by medium stores (L2) and large stores (L3), the CPU minimizes the catastrophic "4-minute" RAM penalty.

Insight

[Parallel Execution vs. Parallel Waiting] Use standard threads to *compute* in parallel (CPU-bound tasks). Use asynchronous programming to *wait* in parallel (I/O-bound tasks). Asynchronous concurrency allows a single thread to manage thousands of concurrent I/O operations without blocking.

14.2 The Rust Async Model: Cooperative Scheduling

Instead of relying on the operating system's preemptive scheduler to pause threads, asynchronous Rust uses a **cooperative scheduling** approach at the application level. This concept is closely related to what are historically known as *green threads* or *fibers*. While native threads are scheduled preemptively by the OS, green threads yield control voluntarily. In Rust, this is achieved via the async runtime, which manages these lightweight tasks without the heavy OS context switches.

14.2.1 The Future Trait and State Machines

When you define a function with the `async` keyword, the compiler transforms the function body into a state machine that implements the `Future` trait. Calling an `async` function does not immediately execute its body. Instead, it instantly returns an inert `Future` object.

```
async fn fetch_data() -> String {
    // This code does not run immediately!
    "Data_loaded".to_string()
}

// In another function:
let my_future = fetch_data(); // Returns a Future, execution has not started.
```

To actually execute a **Future**, it must be *polled*. The **Future** trait defines a `poll` method that returns either:

- `Poll::Pending`: The operation is not yet complete. The future yields control back to the executor.
- `Poll::Ready(value)`: The operation has finished, returning the result.

When you use the `.await` keyword on a future, you instruct the system to suspend the current asynchronous task if the future returns **Pending**.

Note

The `.await` syntax marks explicit **suspension points**. It tells the executor: “If this value isn’t ready yet, park this task and go execute another one. Wake me up when the data is ready.” This is the core of cooperative scheduling.

14.2.2 The Executor and the Waker

Because futures are inert, an **executor** (often part of a runtime like Tokio) is responsible for polling them in a loop. When a future returns **Pending**, it registers a **Waker** with the underlying I/O subsystem. Once the hardware or operating system signals that the data is ready, the **Waker** notifies the executor, which then re-polls the future. This mechanism ensures that the CPU does not waste cycles busy-polling empty queues.

14.3 Building Async Programs with Tokio

Rust’s standard library provides the **Future** trait and the basic building blocks for asynchronous programming, but it intentionally does not include an executor. The most widely used asynchronous runtime in the Rust ecosystem is **Tokio**.

To use Tokio, you must include it in your `Cargo.toml` with the `full` feature flag, and annotate your main function.

```
#[tokio::main]
async fn main() {
    println!("Hello from the async world!");
}
```

The `#[tokio::main]` macro rewrites your `main` function to initialize the Tokio runtime (allocating worker threads and message queues) and then executes the asynchronous body inside that runtime.

14.3.1 Spawning Independent Tasks: `tokio::spawn`

When you write `.await`, you are creating a sequential dependency: the current task cannot proceed until the awaited future completes. If you want to run independent asynchronous activities concurrently, you use `tokio::spawn`.

```
use tokio::task;

#[tokio::main]
async fn main() {
    // Spawn an independent background task
    let handle = tokio::spawn(async {
        // This block runs concurrently
        tokio::time::sleep(std::time::Duration::from_secs(1)).await;
    });
}
```

```

        "Task finished"
    });

    // The main task continues immediately
    println!("Doing other work...");

    // Optionally await the spawned task's JoinHandle to get its result
    let result = handle.await.unwrap();
    println!("Result: {}", result);
}

```

Deep Memory Model Analysis: In this snippet, `tokio::spawn` creates a new asynchronous task. The `async` block generates an anonymous state machine structure. Since the `async` block takes ownership of no variables here, the state machine size is very small and is allocated on the heap by the Tokio runtime to ensure its lifetime is independent of the current function frame. The `handle` returned is a `JoinHandle`, which is a lightweight struct placed on the current task's stack. When `.await` is called on the handle, it checks the heap-allocated state of the spawned task. If we were capturing variables, they would be moved (ownership transfer) into the heap-allocated state machine because the task must be `'static` and `Send`. There is no borrowing (`&`) allowed from the local stack since the spawned task might outlive the stack frame or execute on a different OS thread.

Warning

Futures spawned with `tokio::spawn` must be `Send` and `'static`. Because Tokio's default executor is multi-threaded, tasks can yield on one thread and be resumed on another. Therefore, any data captured by the `async` block must be safely transferable across thread boundaries (`Send`) and cannot contain references to local variables that might go out of scope (`'static`).



Exam Priority The `Send` and `'static` Constraints

The professor heavily emphasizes the requirements for background tasks: they must own their data (`'static`) and be safe to move across threads (`Send`). Forgetting to move variables or using `Rc` inside a spawned task will immediately cause a compilation error. **Common Pitfall:** Attempting to borrow local variables inside `tokio::spawn` or passing non-thread-safe types like `Rc<T>`. **Expected Mastery Level:** High. You must fluently explain why the borrow checker rejects non-`'static` references in tasks and how to fix it (e.g., using `Arc` or passing by value).

14.3.2 Combining Futures: `join!` and `select!`

Sometimes you need to coordinate multiple futures without spawning completely independent tasks:

- `tokio::join!`: Runs multiple futures concurrently on the *same task* and waits for all of them to complete. The total time taken is approximately the duration of the longest future.
- `tokio::select!`: Runs multiple futures concurrently, but waits only for the *first* one to complete. Once a future resolves, the others are immediately cancelled (aborted). This is highly useful for implementing timeouts or listening for cancellation signals.

14.3.3 Avoiding Executor Starvation

Because Tokio’s scheduler is cooperative, tasks must yield control voluntarily via `.await`. If you insert a long CPU-bound loop or a blocking standard library call (like `std::thread::sleep`) inside an `async` function, you will block the underlying worker thread. This prevents other asynchronous tasks from being executed!

Always use asynchronous alternatives like `tokio::time::sleep`. For unavoidable, heavy CPU-bound computations, offload the work using `tokio::task::spawn_blocking`, which runs the closure on a dedicated thread pool specifically designed for blocking operations.

14.4 Shared State and Synchronization

Just like in standard multithreading, concurrent tasks often need to share data. While passing ownership or using channels is preferred, sometimes a shared state is necessary.

The professor uses a practical analogy to explain shared state: imagine preparing your lunch and leaving it in the shared office refrigerator. In a sequential world (living alone), you can safely assume your lunch will be exactly as you left it. But in a concurrent world, another thread (or colleague) might modify it or eat it! The solution is putting a padlock (*catenaccio*) on the refrigerator. This lock is exactly what a `Mutex` provides, ensuring mutually exclusive access.

However, in an `async` environment, you must be careful. If you wrap data in an `Arc<std::sync::Mutex>`, blocking the mutex will block the entire worker thread, stalling the executor. To solve this, Tokio provides `tokio::sync::Mutex`.

```
use std::sync::Arc;
use tokio::sync::Mutex;

#[tokio::main]
async fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter_clone = Arc::clone(&counter);
        let handle = tokio::spawn(async move {
            // lock().await suspends the task if the lock is held by someone else,
            // allowing the thread to execute other tasks in the meantime!
            let mut num = counter_clone.lock().await;
            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.await.unwrap();
    }

    println!("Final count: {}", *counter.lock().await);
}
```

14.4.1 Synchronous Coordination: Mutex and Condvar (Bounded Buffer)

While Tokio provides asynchronous synchronization, traditional OS threads rely on `std::sync::Mutex` and `std::sync::Condvar` to coordinate complex state changes. The professor illustrates this by building a **Bounded Buffer**—a thread-safe, fixed-capacity FIFO queue where producers must wait if it’s full, and consumers must wait if it’s empty.

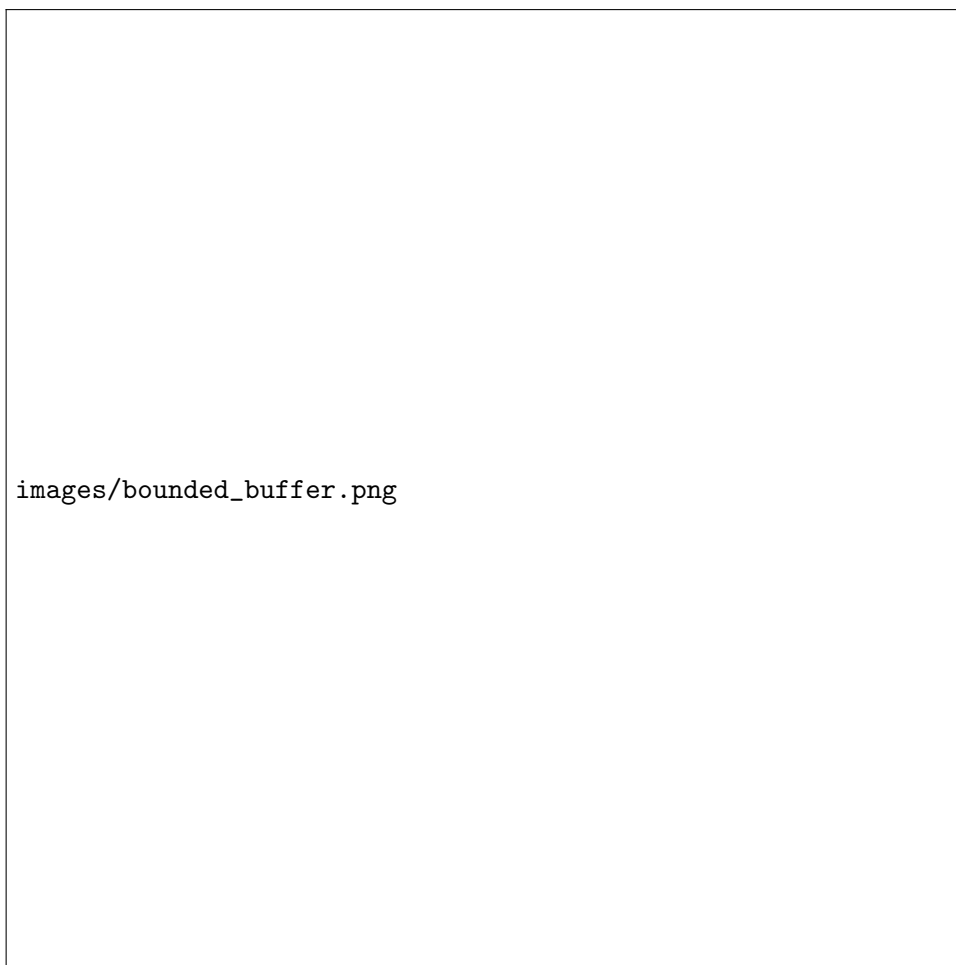


Figure 14.1: A thread-safe Bounded Buffer using Mutex and Condvar.

To implement this, we encapsulate a `VecDeque` and a boolean `closed` flag inside a *single* `Mutex`, paired with a `Condvar`:

```
use std::collections::VecDeque;
use std::sync::{Mutex, Condvar};

pub struct BoundedBuffer<T> {
    // Both the queue and the 'closed' flag MUST be in the same Mutex
    data: Mutex<(VecDeque<T>, bool)>,
    cv: Condvar,
    capacity: usize,
}

impl<T: Send> BoundedBuffer<T> {
    pub fn new(capacity: usize) -> Self {
        BoundedBuffer {
            data: Mutex::new((VecDeque::new(), true)), // true = open
            cv: Condvar::new(),
            capacity,
        }
    }

    pub fn push(&self, item: T) -> bool {
        let mut guard = self.cv.wait_while(self.data.lock().unwrap(), |(q, is_open)|
            q.len() >= self.capacity && !is_open
```

```

    }).unwrap();

    if !guard.1 { return false; } // If closed, abort push

    guard.0.push_back(item);

    // OPTIMIZATION: Drop the lock BEFORE notifying, so the woken thread
    // doesn't immediately block trying to acquire the Mutex.
    drop(guard);
    self.cv.notify_all();
    true
}

pub fn pop(&self) -> Option<T> {
    let mut guard = self.cv.wait_while(self.data.lock().unwrap(), |(q, is_open)|
        q.is_empty() && *is_open
    ).unwrap();

    if guard.0.is_empty() {
        return None; // Woke up because it was closed and empty
    }

    let item = guard.0.pop_front().unwrap();
    drop(guard);
    self.cv.notify_all();
    Some(item)
}
}

```

Insight

[Condvar Best Practices] **1. All state in one Mutex:** A common mistake is using two Mutexes for two variables. The Condvar can only unlock *one* Mutex while sleeping. All related state (like the queue and the closed flag) must be in the *same* Mutex tuple.

2. wait_while: Always use `wait_while` (or a loop) to wait. This protects against "spurious wakeups" and ensures the condition is evaluated securely.

3. Drop before Notify: By manually dropping the Mutex guard (`drop(guard);`) before calling `notify_all()`, we prevent a performance penalty where the woken thread instantly goes back to sleep because the notifier still holds the lock.

Deep Memory Model Analysis: Here, `Arc::new(Mutex::new(0))` allocates both the reference counter and the Mutex state on the heap. The variable `counter` itself is a smart pointer on the main task's stack. Inside the loop, `Arc::clone(&counter)` increments the atomic reference count and creates a new `Arc` pointer (`counter_clone`) on the stack. The `async move` closure takes ownership of `counter_clone`, moving it into the task's heap-allocated state machine. When `lock().await` yields a `MutexGuard`, this guard is placed into the state machine's state so it can be held across suspension points. Once the lock is acquired, the guard provides a mutable reference (`&mut i32`) to the heap-allocated integer, safely avoiding data races. Finally, the guard is dropped at the end of the block, releasing the Mutex lock.

Practical Mastery Focus: When and Why to Use `tokio::sync::Mutex` Use `tokio::sync::Mutex` *only* when you need to hold the lock across an `.await` suspension point. If your critical section is brief, contains no asynchronous operations, and merely updates a quick value (like a counter), use the standard `std::sync::Mutex`. The standard mutex is much faster and less memory-intensive for purely synchronous state updates.

Exam Focus

[Async Mutex vs. Standard Mutex] Exam questions frequently ask why `tokio::sync::Mutex` is needed. The key distinction is that `tokio::sync::Mutex::lock()` is an asynchronous method (`.await`). If the mutex is already locked, it suspends the *task* rather than blocking the OS *thread*. This frees the executor's worker thread to poll other pending tasks.

Warning

[Hardware Realities: Atomicity, Visibility, and Ordering] The existence of CPU caches and instruction reordering introduces three critical problems in multithreading that are purely *hardware facts*, totally independent of the programming language:

1. **Atomicity:** An operation like `A += 1` might require multiple assembly instructions (fetch, increment, store). A thread might be interrupted halfway, leaving corrupted data.
2. **Visibility:** A core writes a value to its local L1 cache. Unless explicitly flushed, other cores reading from their own L1 caches will not see this update.
3. **Ordering:** Concurrency fundamentally introduces non-determinism. As the professor's "red shirt" analogy highlights: if you tell a friend "when I put on my red shirt, I'm ready to go", a highly optimized system might reorder your actions, putting the red shirt on you *before* you are actually ready, simply because it sees no local data dependency!

To solve these, processors provide specific hardware instructions called *Memory Barriers* or *Fences* (e.g., `FENCE` on x86, `Barrier` on ARM) that force cache synchronization. High-level synchronization primitives automatically use these barriers under the hood.

**Exam Priority Execution Non-Determinism and Memory Visibility**

The professor frequently emphasizes that the main difficulties in concurrency arise from non-determinism, atomicity, visibility, and ordering. Memory barriers are required to prevent caches and CPUs from reordering instructions in ways that violate logical sequencing. **Common Pitfall:** Assuming sequential execution order is preserved globally across multiple threads without explicit synchronization. **Expected Mastery Level:** Very High. You must recognize that synchronization isn't just about mutual exclusion; it's also about enforcing memory visibility across thread caches.

14.5 Message Passing: Channels in Tokio

To coordinate tasks without shared state, Tokio offers a rich set of asynchronous channels, conceptually expanding on the standard library's channels. The architecture and relationships of these channels can be visualized as follows:

Rust mpsc (Multiple Producer, Single Consumer) Channel Diagram

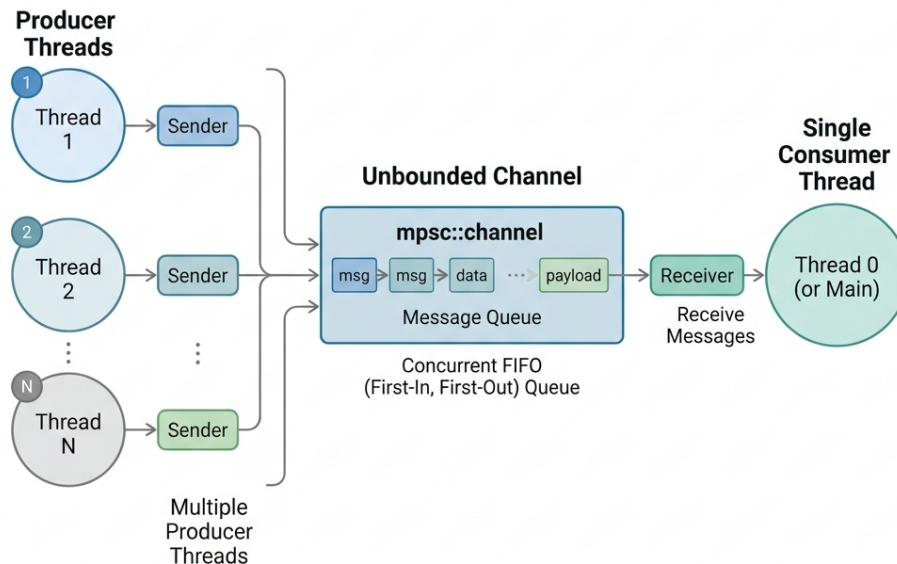


Figure 14.2: Overview of asynchronous message passing patterns.

Tokio provides four main types of channels:

1. **oneshot**: A single producer sends exactly one message to a single consumer. Ideal for sending a specific result back to an awaiting task.
2. **mpsc** (Multiple Producer, Single Consumer): Bounded channels designed for high-throughput streaming of messages.
3. **broadcast**: Multiple producers can broadcast messages to multiple consumers. Every receiver gets a copy of the message.
4. **watch**: Multiple consumers monitor a single value. When the producer updates the value, consumers are notified of the change.

14.5.1 Backpressure and the MPSC Channel

The most commonly used channel is the `mpsc` channel. Unlike the unbounded channels in the standard library, Tokio's `mpsc` channels are inherently **bounded**. You must specify a capacity when creating them.

```
use tokio::sync::mpsc;

#[tokio::main]
async fn main() {
    // Create an MPSC channel with a capacity of 10 messages
```

```

let (tx, mut rx) = mpsc::channel(10);

tokio::spawn(async move {
    for i in 0..20 {
        // If the channel is full, send().await suspends this task!
        if tx.send(i).await.is_err() {
            println!("Receiver_dropped!");
            return;
        }
    }
});

// Simulate a slow consumer
while let Some(msg) = rx.recv().await {
    println!("Received: {}", msg);
    tokio::time::sleep(std::time::Duration::from_millis(100)).await;
}
}

```

Deep Memory Model Analysis: The `mpsc::channel(10)` call allocates the channel's shared state (a queue and wait lists) on the heap. It returns `tx` (a **Sender**) and `rx` (a **Receiver**) to the main task's stack. The `async move` block transfers ownership of the **Sender** to the spawned task's heap-allocated state. When `tx.send(i).await` is called, the value `i` (allocated initially on the task's stack) is copied/moved into the heap queue. If the queue is full (size ≥ 10), the future returned by `send` returns `Poll::Pending`, parking the task and saving its state. When the main task consumes a message via `rx.recv().await`, the message is moved from the heap queue back to the main task's stack (`msg`), and the channel wakes the parked producer task so it can resume sending.

This bounding mechanism introduces **backpressure** into the system. If the producer is generating messages faster than the consumer can process them, the channel will fill up. Once the channel reaches its capacity of 10, the next `tx.send(val).await` call will return `Poll::Pending`, suspending the producer task.

Backpressure is a critical property of robust distributed systems: it prevents a fast producer from exhausting the system's memory by queuing an infinite number of messages, forcing the producer to slow down and match the consumer's pace.

14.5.2 Common Channel Patterns

The professor highlights three major concurrent architectural patterns that are easily implemented using channels (whether standard, `crossbeam`, or Tokio):

1. **Fan-In / Fan-Out:** A single producer generates tasks (e.g., parsing PDFs). Multiple worker threads compete to pull messages from the same channel (Fan-Out). The first available worker grabs the task, processes it, and sends the result to an output channel collected by a single consumer (Fan-In). This maximizes CPU usage but *does not guarantee the order* of results.
2. **Pipeline:** Data flows sequentially through discrete stages (Producer $\rightarrow$ Stage 1 $\rightarrow$ Stage 2 $\rightarrow$ Consumer). Unlike Fan-Out, a pipeline maintains the exact order of the data. It is highly effective when individual processing steps are heavy and can be offloaded to separate threads.
3. **Thread Pool (Producer-Consumer):** A specialized Fan-Out where the channel doesn't transmit raw data, but rather executable closures (`Box<dyn FnOnce() + Send>`). The worker threads act as generic executors that simply pop a function from the channel and run it.



Figure 14.3: Fan-In/Fan-Out and Pipeline channel patterns.

14.6 Choosing Between Shared State and Channels

At the end of the concurrency lecture, the professor provides a definitive checklist for deciding how to synchronize threads:

1. **Is there a simple channel-based solution?** If yes, use it. Channels are generally safer and less prone to subtle logic errors.
2. **Do you need to wait for a specific event or condition?** Use a `Condvar` (or `async` equivalent). Channels can signal completion, but `Condvar` allows waiting for complex, multi-variable state changes.
3. **Do you have complex interlocking state?** Use `Mutex` and shared state. It is the most general solution, but requires the highest discipline to avoid bugs.

Warning

[The Three Deadlocks of Shared State] When writing complex shared state, always audit for:

- **Deadlock:** Thread A waits for Thread B, and Thread B waits for Thread A. Execution halts.
- **Livelock:** Threads continuously yield to each other out of "politeness", burning CPU cycles but making no forward progress.
- **Starvation:** A low-priority thread is perpetually bypassed by faster/higher-priority threads, never acquiring the Mutex.

14.7 Domande ed Esercizi da Esami Passati

Exam Focus

[Domanda: TokenManager] **Testo:** Un applicativo software multithread fa accesso ai servizi di un server remoto, attraverso richieste di tipo HTTP. Tali richieste devono includere un token di sicurezza... Poiché la emissione richiede un tempo apprezzabile, si vuole centralizzare la gestione del token.

Soluzione basata sul capitolo: La gestione centralizzata del token può essere implementata utilizzando un pattern di stato condiviso protetto da **Mutex** e **Condvar** (come mostrato nel pattern **BoundedBuffer**), dove un singolo thread coordina il rinnovo. In alternativa, in un contesto asincrono Tokio, si applica il pattern **Producer-Consumer** usando canali: un task produttore rinnova il token e lo distribuisce via canale. Per un singolo valore aggiornato e distribuito a più task richiedenti, il testo suggerisce l'uso del canale **watch**.

Exam Focus

[Domanda: DelayedExecutor] **Testo:** La struct **DelayedExecutor** permette di eseguire funzioni in modo asincrono, dopo un certo intervallo di tempo...

Soluzione basata sul capitolo: Per eseguire un ritardo in modo asincrono senza bloccare l'executor, si usa `tokio::time::sleep` all'interno di un task generato con `tokio::spawn` (anziché il bloccante `std::thread::sleep`). Per l'accodamento dei job, l'architettura sfrutta il pattern architetturale **Thread Pool (Producer-Consumer)**, accodando chiusure eseguibili (`Box<dyn FnOnce() + Send>`) in un canale che risveglierà i task in attesa al momento opportuno.

Exam Focus

[Domanda: Aggregatore di misure] **Testo:** Un sistema di monitoraggio raccoglie misure di temperatura da più sensori. Le misure vengono raccolte in modo asincrono... Compito del sistema è aggregare le misure ricevute...

Soluzione basata sul capitolo: Questo scenario corrisponde direttamente al pattern **Fan-In** tramite canali **mpsc**. Molti thread (sensori) agiscono come *producer* che inviano i dati attraverso il canale concorrente, affidando l'esclusione mutua al meccanismo interno del canale. Un singolo thread consumatore (l'aggregatore) legge i messaggi e calcola le medie, seguendo la regola aurea presentata nel capitolo: "Is there a simple channel-based solution?".

Exam Focus

[Domanda: Race conditions e Thread] **Testo:** Si definisca un esempio in cui... si possano evitare raceconditions nel momento in cui i thread debbano accedere in scrittura alla stessa risorsa. Si distingua il caso in cui tale risorsa sia uno scalare e quella in cui sia una struttura più articolata.

Soluzione basata sul capitolo: Per risorse scalari (come un contatore) il capitolo consiglia l'uso di `std::sync::Mutex` per garantire accesso esclusivo e gestire l'atomicità hardware. Se la risorsa è complessa (es. `BoundedBuffer`), è necessario combinare `Mutex` con `Condvar`, assicurandosi che *tutto* lo stato correlato risieda nello stesso `Mutex` (evitando così le tre forme di stallo: Deadlock, Livelock e Starvation).

Exam Focus

[Domanda: MultiChannel] **Testo:** La struttura `MultiChannel` implementa il concetto di canale con molti mittenti e molti ricevitori. I messaggi inviati... vengono recapitati a tutti i ricevitori attualmente collegati.

Soluzione basata sul capitolo: Nelle astrazioni asincrone fornite da Tokio, il recapito di repliche di un messaggio da molti produttori a molti consumatori è esattamente il ruolo del canale **broadcast**. Come spiegato nel testo: "Every receiver gets a copy of the message".

Exam Focus

[Domanda: DelayedQueue] **Testo:** Una `DelayedQueue<T:Send>` è un particolare tipo di coda non limitata che offre tre metodi principali... si implementi tale struttura dati nel linguaggio Rust.

Soluzione basata sul capitolo: Si ricalca il design sincrono del `BoundedBuffer`. La coda e il suo stato devono essere incapsulati in un **singolo Mutex** condiviso, associato a una **Condvar**. Nelle operazioni di lettura, se l'elemento è presente ma non scaduto, si usa la **Condvar** per sospendere il thread (tramite `wait_while`) rilasciando temporaneamente il blocco della **Mutex** per non bloccare gli inserimenti. Al verificarsi di cambiamenti, `notify_all()` risveglia il thread dormiente.

Exam Focus

[Domanda: Utilizzo dei Mutex in Rust] **Testo:** Attraverso un esempio pratico si illustri l'utilizzo dei Mutex nel linguaggio Rust. Qual è il meccanismo che consente il loro utilizzo in un contesto thread-safe?

Soluzione basata sul capitolo: Un esempio pratico illustrato nel capitolo è la protezione di un contatore condiviso tramite `Arc<Mutex<i32>>` e l'acquisizione del `MutexGuard`. Il meccanismo di base che assicura il contesto thread-safe è garantito a livello hardware dalle **Memory Barriers** (o Fences). Queste prevengono riordini non deterministici delle istruzioni e forzano la visibilità delle modifiche nelle cache CPU (risolvendo il problema della "camicia rossa").

Exam Focus

[Domanda: Cache] **Testo:** Una cache è una struttura dati, generica, thread safe che consente di memorizzare coppie chiave/valore... Poiché il numero di richieste in lettura può essere molto maggiore delle richieste in scrittura...

Soluzione basata sul capitolo: L'implementazione deve fare uso di stato condiviso. Richiede disciplina per evitare la **Starvation** (es. infiniti lettori che affamano i thread in scrittura per l'aggiornamento o la pulizia delle chiavi scadute). Le operazioni sincrone andrebbero protette tramite `std::sync::Mutex` (mantenendo il blocco breve) mentre compiti periodici di "pulizia" possono essere delegati a un task separato con `tokio::time::sleep` per non bloccare l'executor.

Exam Focus

[Domanda: Trattati fondamentali della concorrenza] **Testo:** Si identifichino i tratti fondamentali della concorrenza. Successivamente, in riferimento alla mutabilità/immutabilità delle risorse, si delinei come questa affligga la gestione della sincronizzazione a livello thread.

Soluzione basata sul capitolo: I tre problemi fondamentali identificati dal testo sono: **Atomicità** (operazioni divise in più istruzioni interrotte a metà), **Visibilità** (valori isolati nelle cache L1 dei core) e **Ordinamento** (riordino hardware per ottimizzazione, che causa non determinismo). Se le risorse sono *immutabili*, non serve sincronizzazione in quanto il dato non cambia; con stato *mutabile* condiviso (es. padella nel frigorifero condiviso) occorrono **Mutex** e Memory Barriers per evitare race conditions.

Exam Focus

[Domanda: MpMcChannel] **Testo:** La struct `MpMcChannel<E: Send>` è una implementazione di un canale su cui possono scrivere molti produttori e da cui possono attingere valori molti consumatori... buffer circolare di "n" elementi...

Soluzione basata sul capitolo: Questo ricalca l'architettura dei canali **mpsc** analizzati nel testo (adattata a multi-consumatore). Essendo il canale limitato ("n" elementi), introduce il concetto essenziale di **backpressure**: se il buffer è pieno, i tentativi di invio si sospendono restituendo `Poll::Pending`. In questo modo i producer veloci non esauriscono la memoria del sistema, riallineando il loro ritmo a quello dei consumatori.

Exam Focus

[Domanda: SingleThreadExecutor] **Testo:** La classe `SingleThreadExecutor` implementa il concetto di `ThreadPool` basato su un singolo thread incapsulato... In assenza di richieste, resta fermo senza consumare cicli macchina...

Soluzione basata sul capitolo: Corrisponde al pattern **Thread Pool (Producer-Consumer)** su un singolo worker. Utilizzando un canale (o una `Condvar`), la `submit()` funge da produttore immettendo closure eseguibili nel canale. Il thread in background si comporta da consumatore e legge continuamente dal canale. In assenza di elementi, si bloccherà passivamente attendendo che vi siano dati senza sprecare cicli di CPU (grazie a costrutti come `Condvar` o un canale sincrono/asincrono).

Exam Relevance and Recurrence

Score: Very High Questions on the differences between concurrent execution (threads) and asynchronous concurrency (futures/Tokio) frequently appear in oral exams and written tests.

You are highly likely to be asked about:

- The difference between `std::sync::Mutex` and `tokio::sync::Mutex` and the cost of blocking an async runtime thread.
- Memory ordering, visibility issues, and the necessity of synchronization primitives (the "red shirt" problem) along with hardware cache impacts.
- The implementation details of a Bounded Buffer using `Condvar` (specifically the use of `wait_while` and dropping the mutex guard before notifying).
- Why cooperative scheduling requires non-blocking I/O and how starvation occurs if you use `std::thread::sleep` inside a Tokio task.
- The memory implications (stack overhead) of thousands of native threads vs lightweight async tasks.

Chapter 15

Unsafe Rust and FFI

15.1 Breaking the Rules (Carefully): The Need for Unsafe Rust

Throughout this book, you have seen how Rust’s borrow checker and strict ownership rules prevent a myriad of memory safety issues at compile time. Safe Rust is designed to systematically eliminate **undefined behaviors (UBs)**. This does not merely refer to memory mismanagement (such as dereferencing a pointer that has already been freed), but also extends to concurrency issues, like preventing two threads from incrementing the same shared variable without synchronization (data races).

By default, the compiler is incredibly conservative: if it cannot guarantee with absolute certainty that a piece of code is safe, it will reject it. While this conservatism is the cornerstone of Rust’s reliability, it occasionally poses a problem. Sometimes, you know a specific memory manipulation is valid, but you cannot express those invariants in a way the compiler understands.

Furthermore, systems programming does not happen in a vacuum. Operating systems are often written in C, and you will frequently need to interact with hardware directly or call existing C libraries. Hardware and C compilers do not understand Rust’s borrowing rules.

To solve these problems, Rust provides an escape hatch: **Unsafe Rust**. By using the `unsafe` keyword, you temporarily take over the responsibility of ensuring memory safety from the compiler. You are telling the compiler, “Trust me, I know what I am doing.”

! Exam Priority

As the professor often highlights, the language intentionally forces you to be verbose when bypassing its protections. When you need to deliberately bypass the borrow checker (for example, to demonstrate a data race manually), you find yourself having to write `unsafe` multiple times, wrapping structures like `UnsafeCell` in `unsafe` blocks. This verbosity is by design: it serves as a repeated, explicit acknowledgment where you declare, “Yes, I am violating the rules; yes, I know I am overriding the safeguards built to prevent undefined behaviors; I take full responsibility.”

Common Pitfall: Thinking that a single `unsafe` block turns off the borrow checker globally. It does not; it only allows specific unsafe operations, and you still have to manually prove to yourself that you aren’t causing undefined behaviors like data races.

Expected Mastery Level: High. You must understand exactly why the language forces this verbosity and what responsibilities you take on when using `unsafe`, especially when manipulating shared state.

Insight**What Unsafe Rust Does NOT Do**

A common misconception is that `unsafe` turns off the borrow checker entirely. This is false. Inside an `unsafe` block, references are still checked, and ownership rules still apply. The `unsafe` keyword simply unlocks five specific "superpowers":

- Dereference a raw pointer.
- Call an unsafe function or method.
- Access or modify a mutable static variable.
- Implement an unsafe trait.
- Access fields of unions.

Everything else remains strictly checked.

15.2 Raw Pointers: Bypassing the Borrow Checker

The most common reason to use Unsafe Rust is to manipulate **raw pointers**. While Rust's standard references (`&T` and `&mut T`) are guaranteed to be valid and non-aliasing, raw pointers (`*const T` for immutable and `*mut T` for mutable) are much more akin to C pointers.

Raw pointers have very different rules compared to standard references:

- They are allowed to ignore borrowing rules by having both mutable and immutable pointers to the same memory location, or multiple mutable pointers.
- They aren't guaranteed to point to valid memory (they can be null or dangling).
- They don't implement automatic cleanup (no `Drop`).

15.2.1 Creating and Dereferencing Raw Pointers

You can *create* raw pointers in safe code. The compiler does not mind if you simply hold a raw pointer; the danger only arises when you try to read from or write to the memory it points to. Therefore, *dereferencing* a raw pointer requires an `unsafe` block.

Let's look at a concrete example of creating raw pointers from standard references:

```
fn main() {
    let mut num = 5;

    // Creating raw pointers is SAFE
    let r1 = &num as *const i32;
    let r2 = &mut num as *mut i32;

    // Dereferencing raw pointers is UNSAFE
    unsafe {
        println!("r1 is: {}", *r1);
        *r2 = 10;
        println!("r2 modified num to: {}", *r2);
    }
}
```

When and Why to Use It: Use raw pointers when you are interfacing with C code that expects pointers, or when you are building foundational data structures (like a doubly-linked list or a custom `Vec`) where you must bypass the strict aliasing rules of safe Rust.

Deep Memory Model Analysis: In the snippet above, `num` is a local variable allocated on the stack. Because it is an `i32`, it occupies 4 bytes of stack space. When we create `r1` and `r2`, no new heap allocations occur, and no ownership is transferred; we are merely creating stack-allocated raw pointers (memory addresses) that point back to the stack location of `num`. Safe Rust borrowing rules usually prevent the existence of simultaneous mutable and immutable references. However, by casting to raw pointers (`*const i32` and `*mut i32`), we circumvent the borrow checker's aliasing guarantees. The lifetimes of raw pointers are not tracked by the compiler, meaning they could outlive `num` and become dangling. Here, it is safe because the operations occur within `main` where `num` is guaranteed to be valid, but the `unsafe` block is necessary for the compiler to allow dereferencing.

In this code, we cast a valid reference to a raw pointer. Because we know `num` lives for the entire duration of `main`, we can safely assure ourselves that dereferencing `r1` and `r2` will not result in a segmentation fault. However, the compiler cannot prove this via the raw pointers, so it requires the `unsafe` block.

Note

Visualizing Memory Layout

To better conceptualize how unsafe code maps to system memory, it is helpful to look at how raw pointers operate under the hood. Refer to **Figure 15.1** (`images/unsafe_rust.png`), which illustrates how raw pointers can bypass the typical stack guarantees and point directly to arbitrary heap segments or memory-mapped I/O registers.

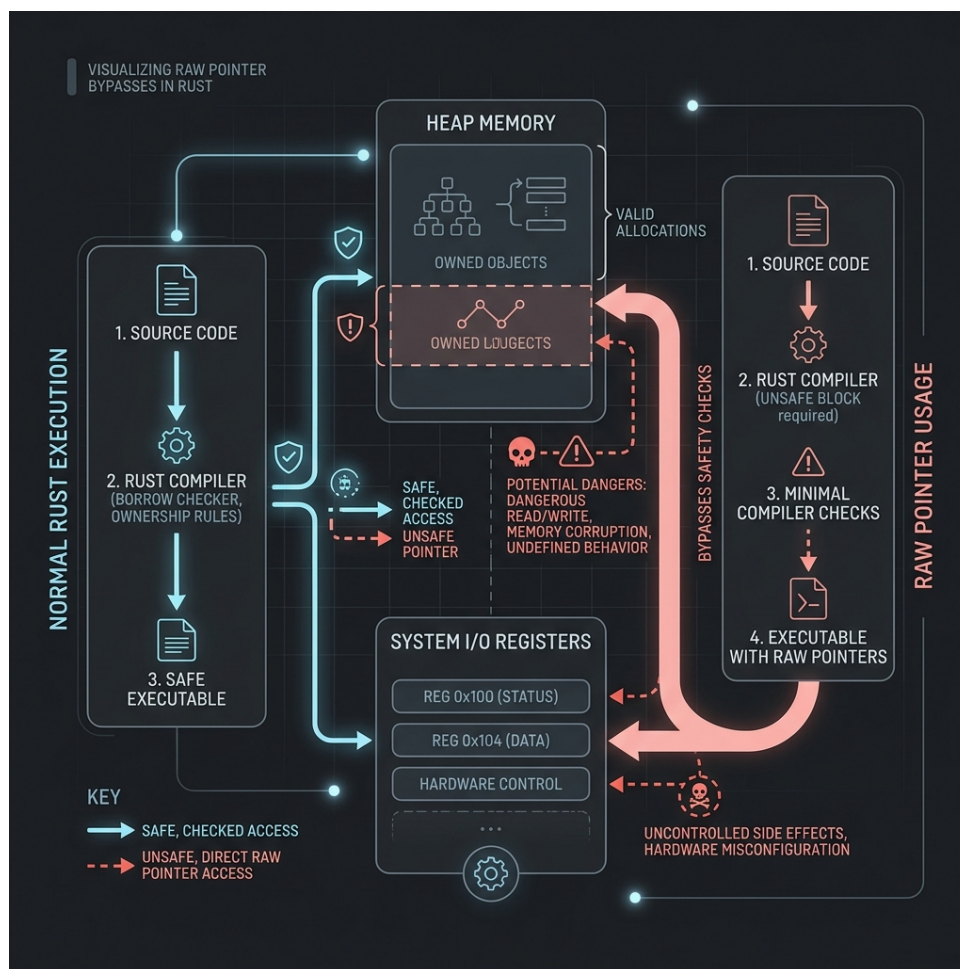


Figure 15.1: Visualizing raw pointer bypasses in Rust

15.3 Unsafe Functions and Safe Abstractions

You can declare an entire function as `unsafe` by prefixing it with the `unsafe` keyword. This means the function has internal requirements that the compiler cannot verify, and the caller must ensure these conditions are met before calling it. Calling an `unsafe fn` must always be done inside an `unsafe block`.

```
unsafe fn dangerous_operation() {
    // This function performs unchecked memory operations
}

fn main() {
    unsafe {
        dangerous_operation();
    }
}
```

15.3.1 The Philosophy of Safe Abstractions

One of the most important architectural patterns in Rust systems programming is building **safe abstractions over unsafe code**. Just because your library uses `unsafe` internally doesn't mean your users should have to write `unsafe` blocks. If you can wrap the unsafe operations

in a safe function API that meticulously enforces the necessary invariants, you can expose a completely safe interface.

A classic example is the standard library's `split_at_mut` method for slices. It takes a mutable slice and splits it into two non-overlapping mutable slices. The compiler's borrow checker isn't smart enough to know that the two resulting slices don't overlap, so the implementation uses `unsafe` pointer math. However, the method signature itself is completely safe, meaning users can call it without `unsafe` blocks.

Another crucial example of this philosophy is **interior mutability**. Types like `Cell`, `RefCell`, and `Mutex` provide a safe API to mutate data through a shared reference. Under the hood, however, they all rely on `std::cell::UnsafeCell`, the core primitive in Rust that legally bypasses the borrow checker to allow mutating data across an immutable reference. By wrapping `UnsafeCell` and enforcing runtime checks or thread synchronization, these types expose a safe interface over an inherently unsafe operation.

When and Why to Use It: Create safe abstractions over unsafe code when you are writing a library or a core component that needs maximum performance or hardware access, but you want to expose a foolproof, safe API to the rest of the application.

15.3.2 Unsafe Traits: Send and Sync

One of the five "superpowers" of Unsafe Rust is the ability to implement an `unsafe trait`. The most prominent examples of unsafe traits in Rust are the concurrency marker traits: `Send` and `Sync`.

- `Send` indicates that ownership of a value of this type can be safely transferred to another thread.
- `Sync` indicates that it is safe for multiple threads to hold immutable references (`&T`) to a value of this type simultaneously. A type `T` is `Sync` if and only if `&T` is `Send`.

These are **marker traits**, meaning they have no methods. The Rust compiler automatically implements them for most types if all their constituent parts are also `Send` and `Sync`. However, if you are building a custom concurrency primitive (or intentionally bypassing safety), you might need to implement them manually. Because the compiler cannot verify thread-safety guarantees on its own when raw pointers or interior mutability are involved, `Send` and `Sync` are marked as `unsafe traits`. Implementing them requires an `unsafe impl` block, whereby you, the programmer, take full responsibility for ensuring the absence of data races.

15.3.3 Demonstrating Undefined Behavior: Forcing a Data Race

To truly appreciate the protections Safe Rust offers, we can look at what it takes to deliberately create a data race. As the professor detailed in the lecture, if you want to demonstrate a classic read-modify-write data race (e.g., two threads incrementing the same variable), you have to fight the compiler every step of the way.

You must manually bypass the protections by creating a custom structure, wrapping the data in an `UnsafeCell`, and explicitly forcing the `Sync` trait implementation.

```
use std::cell::UnsafeCell;
use std::thread;

// 1. Create a custom structure wrapping UnsafeCell
struct MyUnsafeCounter {
    value: UnsafeCell<i32>,
}
```

```

// 2. Implement an unsafe trait. We force the compiler to trust us!
// Without this, we cannot share MyUnsafeCounter across threads.
unsafe impl Sync for MyUnsafeCounter {}

impl MyUnsafeCounter {
    fn process(&self) {
        // 3. Unsafe block required to dereference the raw pointer from UnsafeCell
        unsafe {
            let before = *self.value.get();
            *self.value.get() = before + 1;
            let after = *self.value.get();

            let delta = after - before;
            if delta != 1 {
                // This will print due to the data race!
                println!("Delta is negative or strange: {}", delta);
            }
        }
    }
}

fn main() {
    // Variable instantiated in main, shared by reference
    let counter = MyUnsafeCounter { value: UnsafeCell::new(0) };

    thread::scope(|s| {
        // Spawn two threads, each doing 10 million iterations
        s.spawn(|| {
            for _ in 0..10_000_000 { counter.process(); }
        });
        s.spawn(|| {
            for _ in 0..10_000_000 { counter.process(); }
        });
    });
}

```

Step-by-step Walkthrough:

1. **Custom Structure and UnsafeCell:** The compiler normally prevents mutating shared data. By using `UnsafeCell`, we get a raw pointer to the inner data, enabling interior mutability.
2. **unsafe impl Sync:** Because `UnsafeCell` is expressly *not* thread-safe, it does not implement `Sync`. To pass our `MyUnsafeCounter` across threads, we must implement `Sync` manually. We use `unsafe impl`, effectively telling the compiler: "I take full responsibility for thread safety."
3. **The unsafe Block:** Inside the `process` method, calling `self.value.get()` returns a raw pointer (`*mut i32`). Dereferencing this raw pointer to read `before`, increment, and read `after` requires an `unsafe` block.

Insight**The Professor's Analogy: "I violate all the rules"**

When demonstrating this data race, the professor highlights how much effort it takes to create undefined behavior in Rust: *"You see how many times it has to say: 'I violate all the rules.' Because it's built to prevent us from doing it. So I had to say it a million times: 'Yes, Unsafe, Unsafe, I know, Unsafe is wrong. Yes, I know, I know, I know.' Patience, right?"*

The verbosity of Unsafe Rust is a feature, not a bug. It ensures that undefined behaviors, such as read-modify-write data races that lead to lost updates or bizarre negative deltas, are never introduced accidentally.

Warning**Undefined Behaviors and Concurrency**

Undefined behaviors (UBs) do not just mean dereferencing a freed pointer (use-after-free). As the professor stressed, **two threads incrementing the same shared variable** is an intrinsic concurrency problem that triggers UB. In a data race, a thread might be suspended mid-operation (e.g., after reading 5 but before writing 6). When it wakes up, it overwrites the progress made by other threads, resulting in lost increments and a final counter well below the expected 20 million.

Insight**The Professor's Analogy: The Fridge and TOCTOU**

The core issue behind data races is **TOCTOU** (Time Of Check to Time Of Use). The professor uses a vivid analogy: *"Before, in the fridge, there was my lunch. When I needed it, there was no more."*

You cannot assume that shared things remain exactly as they were when you last checked them. If you read a value (Time of Check) and then are suspended by the scheduler, by the time you wake up and write the modified value (Time of Use), another thread may have already altered the state, making your original read obsolete.

15.4 Foreign Function Interface (FFI)

A primary motivation for Unsafe Rust is interacting with other languages, particularly C. This is known as a Foreign Function Interface (FFI). Because C does not respect Rust's borrow checker or memory models, crossing the boundary between Rust and C is inherently unsafe.

15.4.1 Calling C from Rust

Suppose you are building an application and need to rely on a heavily optimized, pre-existing C library. Rewriting it in Rust would be too costly and error-prone. Instead, you can define the C functions in Rust using an **extern** block and call them directly.

```
// We define the signature of the C standard library's 'abs' function
extern "C" {
    fn abs(input: i32) -> i32;
}

fn main() {
    let x = -3;
    // Calling an external function is inherently unsafe
```

```

unsafe {
    println!("Absolute value of {} according to C is: {}", x, abs(x));
}
}

```

When and Why to Use It: Use FFI when you need to leverage existing C/C++ libraries (e.g., standard libraries, system calls, optimized math routines) without rewriting them in Rust.

Deep Memory Model Analysis: When calling `abs(x)`, the Rust compiler places `x` (an `i32`) in a CPU register or on the stack according to the C Application Binary Interface (ABI) conventions for the target platform. Ownership is implicitly handled by copying, since `i32` implements `Copy`. The C function executes entirely outside of Rust’s memory model; it manages its own stack frame. Once the C function returns, the integer result is placed back into the expected return register (like `rax` or `eax` on x86) and read by the Rust runtime. Because the Rust compiler cannot track what the C function does with memory, the call is intrinsically `unsafe`.

The "C" specifies the **Application Binary Interface (ABI)**. The ABI defines how the function is invoked at the assembly level (how arguments are passed, how the stack is managed). The C ABI is the *lingua franca* of systems programming.

Warning

The Danger of FFI

When calling C code, the Rust compiler completely trusts the signature you provide in the `extern` block. If you incorrectly define a C function as taking an `i32` when it actually takes an `i64`, or if the C function corrupts the stack, the Rust compiler cannot save you. Your program will crash or exhibit undefined behavior.

15.4.2 Calling Rust from C

The relationship goes both ways. You can expose Rust functions so they can be compiled into a shared library and called by C, Python, Node.js, or any language that supports the C ABI.

To do this, you must instruct the Rust compiler not to "mangle" the function name. Name mangling is a technique compilers use to add extra information to function names to support features like namespaces and generics. Since C does not support mangling, we disable it using the `#[no_mangle]` attribute.

```

#[no_mangle]
pub extern "C" fn call_from_c() {
    println!("Just called a Rust function from C!");
}

```

Deep Memory Model Analysis: By marking the function with `#[no_mangle]` and `extern "C"`, we instruct the Rust compiler to export the symbol exactly as `call_from_c` and to expect arguments and return values using the C stack/register conventions. When a C program calls this function, the transition crosses the ABI boundary: the C stack frame is prepared, and execution jumps into the Rust compiled code. Inside the function, standard Rust stack allocation and ownership rules resume. The `println!` macro allocates temporary structures on the stack and safely borrows the string slice.

This function can now be exported as a standard C symbol. Notice that `unsafe` is not required on the Rust side here, as the Rust code itself obeys all safe rules. The unsafety is offloaded to the C code that calls it.

Exam Focus**Exam Tip**

Be prepared to explain *why* FFI operations require the `unsafe` keyword, and how the `extern "C"` block dictates the ABI. Also, be sure to understand the purpose of `#[no_mangle]` when exposing Rust code to other languages.

15.5 Trade-offs: Performance vs. Correctness

Using Unsafe Rust is always a trade-off. By manually managing pointers or hooking into FFI, you gain performance, extreme flexibility, and interoperability. However, you completely forfeit the compiler's safety net for those specific operations. A single mistake in an `unsafe` block can introduce use-after-free vulnerabilities, buffer overflows, or data races—the exact bugs Rust was designed to eliminate.

Therefore, the golden rule of System Programming in Rust is: **Keep unsafe blocks as small as possible**. Encapsulate them behind safe, thoroughly-tested abstractions so that the rest of your codebase can remain entirely safe.

Exam Relevance and Recurrence

Score: High

Notes: FFI and Unsafe Rust are frequent exam topics, particularly the justification for *why* an `unsafe` block is needed and what it enables. You will likely be asked to explain the relationship between Safe Rust abstractions (like `UnsafeCell` and `Mutex`) and Unsafe Rust, and how FFI violates the borrow checker's assumptions. The verbosity of Unsafe Rust in demonstrating race conditions is a recurring conceptual question.

15.6 Domande ed Esercizi da Esami Passati

Nessuna domanda d'esame specifica presente per questo capitolo.

Chapter 16

Exam Strategies and Past Exercises

16.1 Mastering the Final Exam

Based on an analysis of past exams, the professor consistently evaluates your understanding of Rust systems programming across three main dimensions:

1. **Theoretical Concepts:** Your deep understanding of Rust's ownership model, memory management, and concurrency traits.
2. **Memory Layout Analysis:** Your ability to map Rust's high-level abstractions to their exact byte-level representations on the Stack and the Heap.
3. **Concurrency Implementations:** Your capability to architect and implement thread-safe, concurrent data structures using synchronization primitives.

This chapter breaks down each type of exercise and provides a repeatable, step-by-step strategy for solving them, complete with examples from past exams.

16.2 Type 1: Memory Layout & Sizing Analysis

These exercises test your understanding of how Rust stores data under the hood. You will typically be asked to define the memory footprint of specific structs or smart pointers, and specify whether their components reside on the Stack or the Heap.

Exam Focus

Past Exam Question Example

Explain the differences between stack and heap. Furthermore, indicate in detail where the data composing the following Rust constructs is located: `Box<T>`, `RefCell<T>`, and `&[T]`.

Step-by-Step Solution Strategy

Step 1: Define Stack vs. Heap

Start with a concise definition. The **Stack** is used for fixed-size data known at compile time, featuring fast allocation/deallocation via the stack pointer. The **Heap** is used for dynamic data whose size might change at runtime, requiring slower allocation mechanisms and pointer indirection.

Step 2: Deconstruct the Types

For each requested type, mentally deconstruct it into its stack-allocated pointer/metadata and its heap-allocated payload.

Step 3: Detail the Locations

- **Box<T> (Owned Slice on the Heap):**

- *Stack*: A "fat pointer" containing the memory address pointing to the heap allocation (8 bytes on 64-bit), and a `usize` representing the length of the slice (8 bytes). Total Stack: 16 bytes.
- *Heap*: A contiguous array of elements of type `T`, occupying `size_of::T() * length` bytes.

- **RefCell<T> (Interior Mutability):**

- *Stack*: Assuming the `RefCell` itself is declared as a local variable, it resides entirely on the stack. It contains a borrow counter (usually an `isize` tracking active mutable/immutable borrows) and the actual payload data `T`.
- *Heap*: None, unless `T` itself allocates on the heap.

- **&[T] (Borrowed Slice):**

- *Stack*: A "fat pointer" containing the memory address of the first element (8 bytes) and the length of the slice (8 bytes). Total Stack: 16 bytes.
- *Heap/Stack*: The actual data elements reside wherever the borrowed array was originally allocated (could be stack, heap, or static memory).

Insight

Exam Tip

Always explicitly state that "fat pointers" (like slices and trait objects) take up exactly two machine words (16 bytes on a 64-bit system). Do not just say "it's a pointer." Mention the length/capacity metadata!

16.3 Type 2: Theoretical Concurrency Concepts

These questions test your understanding of Rust's "Fearless Concurrency" guarantees and how the compiler prevents data races.

Exam Focus

Past Exam Question Example

Identify the fundamental traits of concurrency in Rust. Subsequently, with reference to the mutability/immutability of resources, outline how this affects the management of thread-level synchronization.

Step-by-Step Solution Strategy

Step 1: Identify the Fundamental Traits

Immediately name the two core marker traits: `Send` and `Sync`.

- **Send**: Indicates that ownership of a value of this type can be safely transferred to another thread.
- **Sync**: Indicates that it is safe for multiple threads to hold immutable references (`&T`) to a value of this type concurrently. (Formally, `T` is `Sync` if and only if `&T` is `Send`).

Step 2: Connect Mutability to Synchronization

Explain how Rust's borrowing rules scale to multi-threading:

- *Immutability*: If multiple threads only need to read a resource, no explicit synchronization (like `Mutexes`) is needed. You can wrap the resource in an `Arc` (Atomic Reference Count) to share ownership, and the compiler guarantees safety because immutable references (`&T`) cannot cause data races.
- *Mutability*: If even one thread needs to modify the resource, Rust's rule of "aliasing XOR mutability" kicks in. You cannot have multiple threads holding mutable references. To mutate shared state safely, you must use synchronization primitives like a `Mutex` or `RwLock`. These primitives enforce mutually exclusive access at runtime, essentially turning a shared immutable reference (`&Mutex<T>`) into an exclusive mutable reference (`&mut T`) for the duration of the lock guard.

16.4 Type 3: Low-Level Concurrency Implementation

This is typically the most challenging part of the exam. You will be asked to architect a thread-safe data structure from scratch.

Exam Focus

Past Exam Question Example

Implement a `ConcurrentCache` trait. The cache must be thread-safe, allow storing key-value pairs, and automatically expire entries after a specified duration. The cache must employ a background thread to periodically clean up expired entries.

Step-by-Step Solution Strategy

Step 1: Problem Analysis & Struct Design

Identify the shared state. A cache needs a `HashMap`. Because it must be thread-safe and handle expiration, the map must store values alongside their expiration timestamps. Since it will be accessed concurrently by multiple threads, the `HashMap` must be wrapped in a `Mutex` (or `RwLock`), and shared via an `Arc`.

```
use std::collections::HashMap;
use std::sync::{Arc, Mutex};
use std::time::Instant;

struct CacheEntry {
    value: Arc<String>,
    expires_at: Instant,
}

pub struct ConcurrentCacheImpl {
    map: Arc<Mutex<HashMap<String, CacheEntry>>>,
    // Need a way to signal the background thread to stop when dropped
}
```

Step 2: Implement the Core Logic

Implement the `get` and `set` methods. Remember to handle lock acquisitions cleanly and check for expiration during `get` operations.

```
impl ConcurrentCacheImpl {
    pub fn set(&self, key: &str, value: &str, duration: std::time::Duration) {
        let mut map = self.map.lock().unwrap();
        map.insert(
            key.to_string(),
            CacheEntry {
```

```

        value: Arc::new(value.to_string()),
        expires_at: Instant::now() + duration,
    },
);
}

pub fn get(&self, key: &str) -> Option<Arc<String>> {
    let mut map = self.map.lock().unwrap();
    if let Some(entry) = map.get(key) {
        if Instant::now() < entry.expires_at {
            return Some(Arc::clone(&entry.value));
        } else {
            // Lazy cleanup: remove if expired
            map.remove(key);
        }
    }
    None
}
}

```

Step 3: Background Worker & Cleanup

The question demands a background thread for active cleanup. When spawning background threads that share state, you must ensure the thread terminates properly when the main structure is dropped, preventing memory leaks.

```

impl Drop for ConcurrentCacheImpl {
    fn drop(&mut self) {
        // Signal the background thread to terminate using a boolean flag
        // or a channel, and wait for it to join.
    }
}

```

Warning

The Drop Trait Trap

When an exam asks you to spawn a background thread inside a constructor (`new`), the professor is explicitly testing whether you remember to implement the `Drop` trait. If you spawn a thread and never terminate/join it in `Drop`, you have created a resource leak, which will result in heavily penalized points!